

SCRIPTUM

* Whitepaper v16 *

DrFunkenstein — May 2026

“Then let us leave behind a chain worthy of remembrance, as history forgets most men. The chain will not.”

— DrFunkenstein —

Table of Contents

PART I — WHY	15
Introduction	15
What Scriptum Is	15
Executive Summary	16
The Safety Control Window (NEXUS)	19
What NEXUS Does Not Protect	19
Kavaca — Post-Quantum Cryptographic Security	21
Real Problems. Real Solutions.....	21
Key Compromise.....	21
Malicious or Unintended Transactions	22
Automated Draining Attacks	22
Coercion and Forced Transactions	22
Quantum Threats.....	22
Definitions	22
Protocol vs. Unit	22
Core protocol vocabulary.....	23
State vocabulary.....	23
Named mechanisms	23
Economic vocabulary.....	29
Privacy and Disclosure Vocabulary	30
Parameter references	31
System Intent.....	31
Core Principle.....	32
Problem Statement.....	32
Structural Weakness of Immediate Execution.....	32
Consequences.....	33
Limitation of Existing Mitigations.....	33
The Core Problem.....	33
Design Requirement.....	33
Scriptum Approach.....	33
Core Transition.....	33
Design Philosophy.....	33
The Hard Layer — Ownership and State.....	34

The Soft Layer — Value and Fees	34
PART II — HOW	36
Ilmarinen — System Architecture.....	36
Block Parameters.....	36
Monetary Policy	37
Coinbase — The Miner Reward.....	38
Emission Formula.....	38
Coinbase Transaction.....	38
Maturity	39
Supply Accounting.....	39
Constraints	39
Mining Algorithm — SHA3-512	39
Node Tiers.....	40
Multi-Miner Protocol.....	41
Post-Quantum Cryptography — Kavaca.....	41
Why a Single Primitive, Not Pluggable Crypto	42
Address Format.....	43
Key Architecture — Spend, View, Recovery	44
The Three Questions	45
Spend Tree.....	45
View Tree	46
Recovery Tree	46
The Separation Doctrine.....	46
Cross-References	47
Public Key Binding at First UNLOCK.....	47
Multi-Address Consolidation	48
Stealth Settlement	49
Validation	49
Design Properties	50
Rationale.....	50
Architectural Foundation — The UTXO Model.....	50
The Universal Parcel Principle	51
Every Parcel is the same	51
Visibility, ownership, authorisation: three separate mechanisms.....	52

What this excludes	52
Why this matters.....	53
The Parcel Data Model.....	53
Universal Parcel Structure.....	53
Class-State Validity.....	55
Spend Commitment Construction	55
Spending Validation — Uniform Path.....	56
Constitutional Slot Table	56
Field-Level Invariants.....	58
The Universal Time Model.....	58
Memo Lifecycle	59
Relationship to Downstream Documents	59
Stealth Settlement and Spend Authorisation	59
The Cryptographic Construction.....	59
Recipient Discovery	60
The Mallory Defence.....	60
Spending Transaction Structure	61
Privacy and Linkage.....	61
Cost and Properties.....	62
Inscription	62
What an inscription does	63
Fee model.....	63
Witness pattern (application-layer).....	64
Discoverability	64
Temenos — The Parcel Model.....	64
Attribute Model	64
CLASS	65
ROLE.....	66
STATE	67
Migration States	68
Migration Authority.....	68
State Definitions	69
Default State.....	69
LOCKED	69

NEXUS.....	69
ACTIVE	70
PERMA_LOCKED (MARKER).....	71
OPEN (VAULT).....	71
ARC.....	71
TREASURE Vesting.....	71
The Box Mechanism.....	72
State Transitions.....	73
CIRCULATING Lifecycle	73
TREASURE Lifecycle.....	75
VAULT Lifecycle	75
Lifecycle Reference Diagram	75
NEXUS — The Safety Control Window.....	76
NEXUS — The State-Transition Crucible.....	76
CANCEL — Two Authorisation Paths	78
ACTIVE State — Refresh and Default-to-LOCK.....	79
Transaction Model	79
Transaction Definition.....	79
User-Initiated Transactions	79
Coin Selection and Destination Rules	79
Stationary Transactions.....	81
Consolidation Transactions.....	82
Transfer Transactions	82
Composite Settlement Transactions.....	84
Ragnarök Participation Transactions	85
Inscription Transactions	85
Protocol-Driven Transactions.....	86
Time-Driven State Transitions.....	86
Block-Level Mechanics	87
Composite Settlement Resolutions	87
Ragnarök Events	88
Issuance Side-Effects	89
Value Transfer Sequence.....	89
No Hidden Execution	89

The Covenant — Fees and Miner Incentives	90
Fee Philosophy.....	90
Fee Model: Operation-Based, Not Input-Based	91
Fee Schedule	91
f_S — Progressive Tier Model	92
Examples	92
Design Objectives	93
Dust Threshold	93
Miner Reward.....	94
Menuha — Sunday Rest.....	94
Urðarbrunnr — Valuation Integrity and Temporal Pricing.....	94
Valuation Coefficients.....	94
Economic Consequence	96
The UIS — Three Temporal Layers	96
The Four Mechanisms.....	96
V_A Proposer Architecture	97
Proposer Eligibility	97
Submission and Signing.....	97
Proposer Identity and Node Coordination	98
Oracle Source Diversity.....	98
Aggregate Computation	98
Hugin and Munin Integration.....	99
Hugin and Munin — V_A Anomaly Detection.....	99
Detection Without Intervention.....	99
Protected Mode — Aurum.....	100
Mining Speed and Hardware	100
What Determines Mining Speed.....	100
51% Attack at Low Hashrate — Economics	100
Block Validation by Hardware	101
Minimum Block Time.....	101
Block Architecture and Capacity Model.....	101
The Capacity Formula.....	101
Fee Cascade — Block Aggregation, Menuha, Quarterly Distribution	102
Per-Block Aggregation (every block, 10 seconds).....	102

Menuha Separation (every 60,480 blocks, ~weekly)	103
Quarterly TITHE Distribution (every 13th Menuha, ~90 days)	103
TITHE Ratio Adjustment.....	103
Properties.....	104
In Practice	104
HICP Indexing — Annual Real-Value Calibration	105
EUR-denominated values subject to HICP indexing.....	105
The cumulative HICP factor	106
Application of the cumulative factor	106
HICP Oracle and Annual Ratification.....	106
Why two stages.....	107
Source identification	108
HICP Transaction Types	108
Transaction Sizes	109
Genesis Block Parameters	111
Block Size — The Single Governance Variable	111
Protocol-Driven Transitions and Limbo.....	112
Storage Formula.....	113
The Scaling Path	113
Horkos	113
Privacy Model.....	114
The Privacy Stack.....	115
Architectural Note: Validity Versus Anonymity.....	116
What the Chain Provides and Does Not Provide.....	117
Adoption Is Privacy.....	118
Ratatoskr — Chain-Traversal Enumeration	119
Audit via Ratatoskr	119
Wallet Recovery via Ratatoskr.....	119
Selective Disclosure	120
Epoch-Bounded View-Key Disclosure	121
Master Rotation	122
What the Holder Sees, What the Chain Sees, What the Delegate Sees.....	122
Boundary Honesty.....	123
On-Chain Settlement vs Off-Exchange Trading	123

Forward Reference to CM	124
Wallet-Layer Encryption	124
Address Space and Cluster Behaviour	125
Node Types — Genesis Requirements	126
Home Mining Node	126
Dedicated Mining Node	126
Full Validation Node	126
Sanctum — Archive Node	127
Consensus and Finality	128
Chain Selection	128
Confirmations	128
Fork Handling	129
Terminal Block Height — RAGNAROK_HEIGHT	129
Open Source and Community Review	130
Network Behaviour and Mempool Policy	130
PART III — USE	132
Aurum — Wallet and User Experience	132
Core Flow	133
UX Requirements (Summary)	133
Markets and Aurum	133
Scriptum as a Traded Asset	134
The Exchange Model	134
What the NEXUS Prevents	134
What the NEXUS Does Not Prevent	135
Menuha and Volatility	135
Menuha Sweep — Two Tiers (Goldmarie and Frau Holle)	135
Goldmarie — Chain Primitive (Tier 1)	135
Frau Holle — Wallet-Driven Cross-Address Consolidation (Tier 2)	136
How the Two Tiers Interact	137
Eligibility and Binding Rules (Both Tiers)	138
Composite Settlement Primitives	138
Horkos	138
Participants	139
Negotiated Terms and Structural Floors	139

The Five-Stage Lifecycle	140
Outcomes.....	140
Menuha Behaviour	141
Why This Design	141
Horkos Resolution Sequence.....	142
Ás-brú — Fast-Rail Exchange Bridge.....	143
Participants	143
Fixed Parameters and Trade Terms	144
The Five-Stage Lifecycle	145
Menuha Interaction	146
Edge Cases and Failure Modes.....	146
Why This Design	147
Kiva — The Discovery Hall.....	149
Participation Model	149
The Hall — Presence	150
The Board — Ás-brú Offers	150
Handoff to Settlement.....	151
Privacy Posture.....	151
Why This Design	152
Price Discovery — The Bootstrap Path.....	152
Phase 1 — Aurum Market.....	153
Phase 2 — Community Exchange Listing.....	154
Phase 3 — Global Exchange Listing.....	154
Operational Modes and Practical Use	154
Secure Storage — The Default	155
Immediate Use — The ACTIVE State.....	155
Market Interaction and Liquidity	155
System Coherence	156
Operational Principle.....	156
PART IV — INSTITUTION	157
Governance — WOC, Founders, and Curia	157
Constitutional Roles	157
Council Registry	160
VAULT-Class Authorisation.....	162

Decision Tiers — Four-Level Multi-Sig	164
Level 1 — Curia Operational.....	164
Level 2 — Founders Strategic	164
Level 3 — Cross-Cohort Significant	165
Level 4 — Constitutional	165
Emergency Operations.....	165
Decision Classification.....	165
No-Quorum Rules	166
WOC Emergency Authority	166
WOC Succession	166
Fountain Governance.....	166
The Upgrade Ceremony	167
Cut and Splice, Not Branching.....	167
The Ten Gates.....	168
Procedural Commitments.....	169
Welcomed Forkability	169
Limits of Procedural Protection	170
The Tithe — Protocol Revenue.....	170
The Split — Stage One	170
The Tithe Allocation — Stage Two.....	171
WFC Compensation Pool — Chain Primitive.....	172
Why the caps exist.....	174
Underflow as bootstrap reality.....	174
What Funds Pays For	174
What the RESERVE Holds	174
What the Charity Pot Holds.....	175
Tithe at Genesis	175
Fountain — Genesis Liquidity Provisioning	175
Allocation	175
Price.....	176
Mechanism — Horkos covenants via Aurum	176
Eligibility — public	176
Authority	177
Fiat proceeds	177

Closure.....	177
Exchange-Choice Architecture.....	177
Protocol Hard Caps — Curia Capture Defence.....	179
What Fountain is Not	179
The Donation Mechanism — Dana	180
Aurum Implementation	180
Supply Accounting.....	181
PART V — GENESIS	181
Rebirth and the Founding Manifest.....	181
Block 0 — The Rebirth Block	181
Block 0 — Deterministic Construction.....	181
DISEMBARK — Protocol-Driven Instantiation.....	182
ARC 0 — The Founding Manifest.....	182
Allocation	183
Vesting Schedules.....	184
PART VI — LONGEVITY	186
The Hard Ceiling — Supply, Scarcity, and the Answer to Everything	186
The Coin and its Unit.....	186
The River of Forgetting — Lethe and the Lost Coins	186
The Sacred Fire — Burn Economics and Gehenna.....	187
The Mayflower Parcel.....	188
The Centennial Renewal — Ragnarök, the Arc, and the Successor Phoenix	189
TREASURE — Automatic Boarding	189
Protocol Enforcement.....	189
The Arc Procedure — Enrollment and Boarding.....	189
Enrollment.....	189
Scheduled Boarding.....	189
Cancellation.....	190
PACKING Phase.....	190
STRAGGLER_WINDOW.....	190
BOARDING	190
ANOINTED and CONDEMNED	191
After Boarding.....	191
Ragnarök and Departure	191

ARRIVAL — Disembarkation in the Successor Phoenix	192
Finality.....	192
The Number That Matters	192
Canonical Continuity and Forks	192
The Oldest Truth.....	192
Ain Soph — System Philosophy	193
Scriptum is not:	193
Scriptum is:.....	193
PART VII — SECURITY & REGULATORY	194
The Security Model.....	194
Structural Safety vs.Economic Safety.....	194
What a 51% Attacker Can Do	196
What a 51% Attacker Cannot Do	197
The Three Attack Scenarios	197
Key Theft Scenario	198
Oracle Manipulation Scenario	198
Timestamp Gaming Scenario.....	199
NIST Alignment Statement	199
Regulatory Position	200
Invariants and Assurance	201
Conclusion — A System Built for Failure	202
Final Statement.....	203
Appendix A. Privacy Architecture	203
The Three-Layer Model	203
Stealth Wallets — How They Work.....	203
Sending to a Stealth Address	203
Receiving — Chain Scanning.....	204
The Audit Key	204
Protocol Integration	205
Appendix B. Aurum UX Reference.....	205
Aurum UX Reference	205
A.1.1. Warning Levels	206
A.1.2. Warning Placement.....	207
B.1. Boarding Status Display	207

B.2. UIS Protected Mode	208
Extended Protected Mode (Prolonged Oracle Failure)	208
B.3. Parcel State Display	208
B.4. Fee Preview	209
Colophon	209

PART I — WHY

Introduction

Scriptum is a general-purpose settlement chain architected around custody-grade safety assumptions.

It is general-purpose in scope: a chain that handles small everyday payments, fast-rail exchange settlement, attested off-chain delivery, private peer-to-peer transfers, institutional bulk settlement, and zero-knowledge anonymous transitions being planned (Scriptum V2, post-Genesis) — all through the same consensus rules, the same fee schedule, the same primitives. It is not a specialised privacy coin, not a specialised payments coin, not a specialised settlement layer. It is a settlement chain that accommodates the full range of legitimate use.

It is custody-grade in posture: every consensus rule, every state transition, every primitive is designed first for the case where the value at stake matters and the holder cannot tolerate loss to theft, key compromise, or quantum decryption. The Safety Control Window treats every transaction as if it might be unauthorised until the 24-hour reversal window closes. Post-quantum signatures and key exchange are baseline from genesis, not a future migration. Selective disclosure is cryptographically bounded, not policy-bounded. Custody discipline shapes the chain's defaults; everyday use inherits the discipline at no operational cost.

This is the reconciliation of two design priorities that other chains treat as opposed. A holder transferring €10 of household value operates the same chain a custodian transferring €200 million operates. Both inherit the same safety assumptions. The chain does not have separate modes, separate fee schedules, separate cryptographic baselines for different holder profiles. The chain has one set of rules and one set of safety assumptions, and the safety assumptions are tuned for the case where the holder cannot afford to be wrong.

What Scriptum Is

You walk into the bank.

Someone is already there robbing it. He takes you into the safe. Your credentials are valid. But the safe has a time lock. It does not open because you asked. It opens when the mechanism has confirmed — after the observation window has passed — that no alarm was raised.

That is Scriptum's Safety Control Window.

Every other blockchain is a bank where a valid key opens the safe immediately. Scriptum is the bank where a valid key starts the clock.

Bitcoin's security rests on hashrate. The more miners compete, the more expensive an attack becomes. This works. But at low hashrate — in the early life of any chain — the cost of a 51% attack falls, and with it the cost of reversing recent settlements.

Scriptum adds a second layer. The Safety Control Window means that reversing a finalised settlement requires not merely a short reorganisation but sustained majority control for the full 24-hour window. An attacker who achieves 51% hashrate for a short burst can attempt to reorganise recent Bitcoin history. To reverse a finalised Scriptum settlement, that attacker needs 51% for a full day — continuously, without interruption.

Hashrate is the vault door. The SCW is the time lock. Both must be defeated. Neither alone is sufficient.

Scriptum builds on Bitcoin's proven foundation — proof-of-work, Parcel accounting, fixed supply — and adds what Bitcoin lacks: protocol-level time separation between authorization and settlement, post-quantum cryptography from genesis, and a fee model that is fair at every scale. From the person sending €10 to the institution settling €200 million. The same rules. The same chain.

Executive Summary

Scriptum's design rests on two core architectural commitments: post-quantum cryptography (Dilithium3 signatures, Kyber key exchange) and the Safety Control Window — the protocol-enforced delay between transaction instruction and irreversible settlement during which an unauthorised movement can still be cancelled. From these commitments flow the chain's structural properties — view-key delegation for holder-controlled audit, address binding for efficient per-address operations, atomic settlement primitives, transparent reserve verifiability — and the applications the design enables: regulated CASP custody, institutional treasury operations, peer-to-peer settlement, atomic bilateral trades via Horkos, payments, exchanges, donations, and Foundation reserve management within a single coherent protocol. Settlement-grade custody is one application of the design, not its purpose. The chain is general-purpose; the post-quantum + SCW foundation gives every application access to properties that other chains cannot offer. Five structural layers: a cancellable Safety Control Window (the NEXUS dwell-state) on protected paths; post-quantum cryptography on every address and signature from genesis; mandatory stealth-compatible output construction (every SPEND derives a fresh one-time output via Kyber/ML-KEM key encapsulation against the recipient's published view key) with delegated-audit view keys; sealed sender memos for per-transaction recipient-visible provenance disclosure and selective audit; and a weekly Menuha rest that closes execution on Sundays. Two constants: honest supply accounting including acknowledged Lethe, and centennial Ragnarök renewal via Arc migration. **Scriptum is built for safety, security, controlled transfer, usability, auditability, fairness, and societal reroute.** It provides reasonable privacy as a property of its architecture — privacy in the bank-wire sense, where ordinary commercial transactions are not subject to broadcast public surveillance — but not stealth as a chain-level commitment. Stronger privacy postures are achievable through wallet-side composition of the chain's existing primitives (multiple home addresses, stealth-on-receive, view-key delegation with epoch-bounded scope); the chain provides the cryptographic primitives without claiming maximum stealth as a default property. The chain is designed to outlive its founders, operate under any price regime, and meet institutional compliance requirements without surrendering reasonable user privacy.

Every existing blockchain assumes that keys remain secure, that users act without error, and that authorization always reflects intent. When those assumptions fail — and they do fail — the result is immediate and irreversible loss. There is no recovery window. There is no intervention point. The system was not designed to survive failure.

Scriptum was. It separates authorization from execution by protocol. Value must be explicitly unlocked, observed, and confirmed over time before it can move. Compromise does not equal immediate loss. Error does not equal permanent loss. The window exists precisely to survive failure — not to prevent it.

In a Bitcoin attack, a valid signature moves value immediately and irreversibly.

In Scriptum, a valid signature begins an observation window. An attacker with a stolen key cannot extract value without also holding sustained hashrate control long enough to suppress the holder's CANCEL across the full 8,640-block Safety Control Window — approximately twenty-four hours of continuous dominance in plain sight.

Every coin in motion returns to LOCKED if the attack fails at any point. Every ACTIVE coin expires back to LOCKED by protocol-enforced EXPIRE. The attacker who holds the key but cannot sustain control for a full day extracts nothing; the attacker who holds sustained control but lacks the key extracts nothing.

This is not a claim that theft is impossible. It is a claim that theft requires BOTH cryptographic compromise AND sustained mining dominance, detectable and cancellable throughout.

Bitcoin spends tens of billions of dollars a year on hashrate to raise the cost of an instant attack. Scriptum raises it structurally, by replacing instant finality with a cancellable observation window and requiring any attacker to maintain both prerequisites across the window.

As Scriptum accumulates adoption, it accumulates hashrate the same way every successful proof-of-work chain does — on top of the structural protection it already has.

Bitcoin has hashrate. Scriptum has hashrate and SCW, and will for as long as the chain runs.

The architectural decision that creates this advantage must be made at block zero. Scriptum made it. Every address is post-quantum from genesis. Stealth addressing and holder-controlled audit keys are live at launch, built on NIST-standardised CRYSTALS primitives. The cryptographic legacy problem that every existing blockchain will eventually face — the day quantum computers can break ECDSA — Scriptum does not have. The chain is born current.

Scriptum introduces two foundational security innovations with no direct equivalent in existing blockchain systems.

The first is the Safety Control Window (SCW). In every existing blockchain, authorization and execution are effectively the same act. A valid signature moves value immediately and irreversibly. Scriptum separates these two events by protocol. Value must be explicitly unlocked, enter a mandatory 8,640-block observation window, become Active, and remain

valid before it can move. This introduces time as a fundamental security dimension. Compromise does not equal immediate loss. The window exists precisely to survive failure, not to prevent it.

The second is post-quantum cryptography. Scriptum uses CRYSTALS-Dilithium3 as its native signature scheme, standardised by NIST in 2024. Every address, from the genesis block forward, is secured against quantum attack. **The chain is born post-quantum**, with constitutional upgrade capacity through the Upgrade Ceremony if cryptographic primitives need amendment in future.

Beyond security, Scriptum introduces a monetary model of genuine novelty. The maximum supply is **protocol-fixed under current canonical rules at 42,000,000 Scriptum** — the answer to the ultimate question of life, the universe, and everything — and is **amendable only through the full Upgrade Ceremony** (a constitutional hard fork requiring all ten gates of governance ratification). Under canonical operation the cap is hard; the chain itself enforces it on every block. Amendment is a constitutional act, not an algorithmic adjustment. The base unit is the Nugget: one Scriptum equals one hundred million Nuggets. The block reward is a continuous function of net circulating supply: as supply grows toward 42 million, the reward asymptotically approaches zero. It never reaches it. The Scriptum system continues through successive Phoenix cycles

Burning is formally supported and economically designed. All burns are directed to Gehenna, a protocol-defined address provably unspendable by mathematical construction. Burned Scriptum reduces net circulating supply, expands the mineable space below the 42M ceiling, and increases the block reward — attracting miners, strengthening security, and restoring equilibrium without intervention. Burning is the network breathing.

Scriptum draws deliberately on mythological names from many cultures and traditions. Gehenna from the Abrahamic faiths. Lethe and Aegis from ancient Greece. Ragnarök from the Norse. Kavaca — divine armour from the Hindu tradition — names the post-quantum security layer. Ilmarinen, the eternal Finnish smith who forged the sky, names the system architecture. Ma'at, the Egyptian goddess of truth and balance, names the transaction model. Ain Soph, the Kabbalistic infinite, names the system philosophy. Temenos, the sacred Greek enclosure, names the vault. Urðarbrunnr, the well of fate from Norse cosmology, names the valuation integrity system. Irkalla, the Sumerian land of no return, anchors the final chapter. This is not decoration. These names carry weight. They give Scriptum a language that anyone, anywhere, across any tradition, can feel before they understand the mathematics.

Scriptum acknowledges what other chains ignore: that coins are lost permanently through accident, death, and circumstance. These lost coins — designated Lethe — Lost Scriptum — are categorised, acknowledged, and explicitly excluded from all protocol calculations. Scriptum does not pretend to know what it cannot know.

At Year 100 plus one day, the protocol triggers Ragnarök — the centennial renewal. A new chain is born. Every holder must actively board the Arc within a defined migration window. One hundred years of accumulated Lethe is left behind. The new chain departs clean, with known supply, purified circulation, and a fresh 42 million ceiling.

Scriptum is a centennial protocol — designed to outlive everyone who builds it, everyone who mines it, and everyone who holds it, and to begin again for those who come after.

The Safety Control Window (NEXUS)

Scriptum introduces the Safety Control Window (NEXUS) as a protocol-level control mechanism for value movement. Value is not immediately spendable. It must first be explicitly unlocked, which opens a defined control window during which the intended action becomes visible, verifiable, and reversible.

ACTIVE is a post-dwell state. Every Parcel in ACTIVE has already completed NEXUS dwell during its entry to hot status (via UNLOCK or via EJECT from a NEXUS-protected transition). Subsequent direct SPEND from ACTIVE creates new dwelling Parcels at the recipient; the consumed ACTIVE Parcel itself does not re-enter NEXUS because it already passed through. NEXUS protects entry to ACTIVE; direct SPEND from ACTIVE is operational use of value the holder has already brought through the protection.

The NEXUS allows detection of unintended or malicious actions, user intervention and cancellation, and controlled execution of value transfer. At completion of the NEXUS, the asset becomes ACTIVE and may be spent, locked, or burned.

The NEXUS is mandatory at the entry to ACTIVE state, enforced at the protocol level. Every UNLOCK passes through the NEXUS dwell window before value reaches ACTIVE. Once a holder has actively brought value into ACTIVE state, they may choose to keep it there through periodic REFRESH actions; if they do so, they have continuously and deliberately reaffirmed their consent to hot status, and may use that hot value through direct SPEND.

Direct SPEND from ACTIVE is not a bypass of safety — it is the operational use of value the holder has actively maintained in hot state through deliberate ongoing action. When ACTIVE expires through REFRESH inaction, the value returns to LOCKED automatically, returning to NEXUS-protected status without holder intervention. The chain's safety architecture is the NEXUS at entry to ACTIVE, the REFRESH discipline that requires active maintenance to preserve hot status, and the automatic return to LOCKED when active maintenance lapses. Holders also have the option of private SPEND modes (from ACTIVE or directly from LOCKED) that re-enter NEXUS for an additional cancellation window when they want it.

What NEXUS Does Not Protect

The NEXUS is a powerful safety primitive but it is bounded. Honest framing matters: holders, custodians, and regulators reading this Whitepaper must understand the boundary between what the NEXUS protects against and what it does not.

The NEXUS protects: value moving from LOCKED to ACTIVE (every UNLOCK passes through the 8,640-block NEXUS dwell window during which the holder may CANCEL); value at rest in LOCKED state (theft requires UNLOCK, which surfaces the attack in the dwell window); recovery of value from LOCKED state when a private key has been compromised (the holder retains CANCEL authority during dwell, independent of whether the attacker has the spend key).

The NEXUS does not protect: value already in ACTIVE state. Once a holder has actively brought value into ACTIVE through completed UNLOCK, that value may be SPENT directly without re-entering NEXUS. A direct SPEND from ACTIVE is final at consensus inclusion; there is no dwell window, no CANCEL opportunity, no reversal. This is the operational trade-off the holder accepts when choosing to keep value in ACTIVE through REFRESH: hot value is operationally ready and spendable immediately, but it is not NEXUS-protected against compromise during the period it remains hot. Holders who want NEXUS protection on outward movement from ACTIVE choose private SPEND modes, which re-enter NEXUS for an additional cancellation window. The chain provides both options; the holder selects per transaction based on the value at stake and the operational urgency.

The NEXUS applies to every value movement in Scriptum without exception. Holder-initiated SPENDs, UNLOCK consolidations, VAULT disbursements authorised by Council multisig (FUND, RESERVE, CP), and any other authorised value transfer all pass through the NEXUS_DWELL state for the 8,640-block safety window. The protection is universal.

Protocol-driven transitions that move value without human authorisation — block aggregation (creating miner and protocol Parcels per block), Menuha separation (splitting accumulated VAULT into FUND/RESERVE/TITHE), quarterly TITHE distribution, vesting Parcel maturation, COINBASE Parcel maturation — do not pass through NEXUS_DWELL. These are deterministic protocol mechanics with no signer to coerce or mistake; the NEXUS exists to protect against mistakes and coercion of signing authority, which protocol-driven transitions lack.

The distinction is: who signed? If a human or governance body signed, the value movement goes through NEXUS_DWELL. If the protocol itself moves value as part of its deterministic mechanics, NEXUS_DWELL does not apply.

The NEXUS does not protect: off-chain value commitments. Horkos covenants, Ás-brú trades, and other settlement primitives have their own protection structures (multi-sig agreement, stake-at-risk, attestor verification, window timeouts). The NEXUS is not a substitute for these; it operates only on the Scriptum side of value movement.

The NEXUS does not protect: value moved by holder intent. If a holder genuinely intends to SPEND directly from ACTIVE, the chain executes the SPEND; the NEXUS does not second-guess intent. A holder who is tricked into signing a SPEND transaction (phishing, social engineering, hostage scenarios) has a Dilithium3 signature on the transaction; consensus treats this as authorisation. The NEXUS protects against unauthorised key use; it does not protect against deceived holders.

The NEXUS does not protect during Limbo. CANCEL validity is height-based: CANCEL is valid only while `current_block_height < eject_height`. Once the NEXUS dwell window has technically closed (block height has reached `eject_height`), CANCEL is rejected at consensus validation. The protocol-driven EJECT transition then executes in the next block under normal load, completing the lifecycle within one block. Under congested load (mempool fuller than block capacity), EJECT may not be included immediately; the Parcel enters *Limbo* — the gap between the formal end of the cancellation window and the actual EJECT inclusion. During Limbo the NEXUS provides no further protection: CANCEL no

longer works (height-based rule), and EJECT has not yet executed. This is a deliberate consensus design choice. Limbo is typically one block under normal load and rare in practice; under sustained congestion it may extend. Holders should be aware that the final block of the NEXUS dwell is not the latest moment a CANCEL can be submitted — CANCELS intended to be honoured should be submitted with at least one or two blocks of margin before `eject_height`.

The honest framing: **the NEXUS is the chain's structural defence against unauthorised UNLOCK, not a universal safety guarantee for every transaction.** The holder's responsibility is to choose the operational mode (LOCKED with full NEXUS protection, or ACTIVE with operational readiness and direct-SPEND availability) appropriate to the value at stake and the use pattern. The chain provides the primitives; the holder chooses the discipline.

Kavaca — Post-Quantum Cryptographic Security

Scriptum uses CRYSTALS-Dilithium3 (ML-DSA, NIST standard 2024) as its native signature scheme. This provides resistance against quantum attacks, publicly verifiable security, and long-term robustness. Every address from genesis is post-quantum secured. **The chain is born post-quantum, with constitutional upgrade capacity through the Upgrade Ceremony if cryptographic primitives need amendment in future.**

Post-quantum signatures are substantially larger than classical signatures. A typical SPEND transaction carries:

Transaction base (~3,400 bytes): header, sender's `home_address`, one Dilithium3 signature covering the whole transaction, metadata

Per input (~100 bytes): outpoint reference plus `commitment_opening`

Per output (~1,400 bytes): universal Parcel structure including `kyber_ciphertext`, `spend_commitment`, and other universal fields

A typical user SPEND consumes one input and creates two outputs (stealth payment + change). Total: ~6,300 bytes.

At the genesis 2 MB block size, this yields approximately 25 sustained settlements per second after block overhead. This is settlement-grade infrastructure where deliberateness is the design property, not a limitation. Scriptum is not designed for high-frequency small transactions.

For comparison, Bitcoin transactions are typically ~250 bytes; Scriptum transactions are roughly 25× larger. This is the deliberate trade-off for post-quantum security: the Dilithium3 signature alone is ~3,293 bytes, accounting for over half of every transaction.

Real Problems. Real Solutions.

Key Compromise

In conventional systems, possession of a private key enables immediate and irreversible transfer. In Scriptum, an attacker can initiate an unlock but cannot immediately transfer

value. The NEXUS creates a visible and controllable period during which the legitimate owner can detect and cancel.

Malicious or Unintended Transactions

User errors or manipulated transactions typically result in irreversible loss. In Scriptum, value must pass through the NEXUS before transfer. Unintended actions can be identified and reversed before execution.

Automated Draining Attacks

The most devastating attacks on existing wallets succeed because they execute in milliseconds. Smart contracts drain entire holdings before any human can respond. In Scriptum, instantaneous draining is impossible by protocol; any automated attack must survive the 8,640-block NEXUS before value leaves the chain. Even with valid signing capability, value cannot move without surviving 8,640 blocks of observable time. Speed is not a weapon here.

Coercion and Forced Transactions

Forced actions still require entry into the NEXUS, creating a temporal buffer for intervention or cancellation.

Quantum Threats

Scriptum mitigates future cryptographic weakness through Dilithium3. Authorization alone is insufficient — execution requires surviving the NEXUS.

Definitions

Scriptum uses precise terms. This section defines them once, for quick reference.

Protocol vs. Unit

Scriptum — the name of the protocol and the chain. *Scriptum* is the name of the system as a whole: the consensus rules, the NEXUS architecture, the governance Charter, the economic structure, the body of work documented in the WP, CM, SPX, and the rest of the corpus.

FUNK — the unit and ticker symbol of the chain's monetary asset. *FUNK* is the unit in which amounts are denominated and the ticker symbol used on exchanges. Bitcoin's analogue: *Bitcoin* is the protocol, *BTC* is the ticker; for Scriptum, the protocol is *Scriptum* and the ticker is *FUNK*. When the WP refers to "5 FUNK," it means five units of the protocol asset; when it refers to " $f_FUNK = f_EUR / V_A$," it means the fee amount denominated in FUNK. The chain's hard cap is $42,000,000\ FUNK = S_MAX$. Where corpus prose uses *Scriptum* as a unit ("0.1% of principal in Scriptum"), the intended meaning is the unit FUNK; usage will tighten over time to use FUNK consistently where unit denomination is meant.

Core protocol vocabulary

Parcel — Unspent Transaction Output. A discrete unit of value, identified by (txid, output_index), that has been created but not yet consumed as input to a subsequent transaction. Every Parcel carries three attributes: CLASS, ROLE, and STATE.

CLASS — the economic identity of a Parcel. Four classes exist: CIRCULATING (ordinary user value), TREASURE (governed allocations with vesting), VAULT (protocol-controlled institutional value), and MARKER (non-economic identity records).

ROLE — the semantic purpose within a class. Examples: WARDEN, FOUNDER, and CURIA under TREASURE; WFC, FUNDS, RESERVE, and CP under VAULT; MAYFLOWER and DANA under MARKER.

STATE — the current position in the lifecycle. The principal states are LOCKED, COINBASE, NEXUS_DWELL, ACTIVE, SEALED, OPEN, PACKING, ARC, and PERMA_LOCKED.

State vocabulary

LOCKED — the default state of every CIRCULATING Parcel. Protected by the Safety Control Window. Cannot be spent without passing through UNLOCK and EJECT.

NEXUS — a Parcel in the controlled-transfer dwell-state. Entered by UNLOCK or SPEND. The Safety Control Window is open during dwell; the holder can CANCEL the transition via signature or via hash-preimage submission. Eject fires at unlock_height + 8,640, executing the entry's pre-committed destination state transition. Still not spendable during dwell.

ACTIVE — post-EJECT state for self-spends following UNLOCK. The Parcel is now spendable via SPEND, BURN, DONATE, or LOCK. The ACTIVE window is bounded; the holder may REFRESH to extend it; if no action is taken, EXPIRE returns the Parcel to LOCKED.

PERMA_LOCKED — the state of MARKER-class Parcels. Cannot be unlocked, spent, or burned. Persists across every Ragnarök.

Named mechanisms

NEXUS — Safety Control Window. The dwell period within NEXUS during which CANCEL remains valid. The NEXUS has a fixed dwell duration of 24 hours (8,640 blocks) defined by the protocol (see SPX-2) and is not subject to dynamic adjustment. The core protective primitive of Scriptum.

NEXUS — the state-transition crucible. The protocol-level machinery that provides cancellable controlled-transfer windows for the chain's three protected paths: UNLOCK to HOME, private SPEND from hot, private SPEND from cold. Value enters NEXUS at the source action; protocol-driven EJECT delivers it to its committed destination at unlock_height + 8,640; CANCEL during dwell returns the value to source. Direct SPEND from ACTIVE operates outside NEXUS as the chain's fast operational path — the value has already passed through NEXUS at UNLOCK and the holder has been actively maintaining its

hot status through periodic REFRESH; direct SPEND is the operational use of actively-maintained ACTIVE Scriptum, not a bypass of safety. NEXUS is not a mixer, churn pool, or provenance-fog layer; it is the named architecture of the Safety Control Window.

Kavaca — the post-quantum cryptographic protection, combining Dilithium3 signatures and Kyber key encapsulation, active from genesis.

Ilmarinen — the node software that runs Scriptum. The mythic Finnish smith who forged the Sampo; here, the consensus engine that maintains the chain’s state machine, validates transactions, and propagates blocks. Ilmarinen runs on every participating node — home miners, dedicated miners, full validators, and Sanctum archives.

Hayyānu — the standalone Scriptum miner. Named for the Ugaritic divine craftsman Hayyānu the Skilled (ḥyn d-ḥrš yd-m, “Hayyān of the deft hands”). Performs SHA3-512 proof-of-work, assembles candidate blocks, includes Ás-brú API confirmations, and submits the block to the network when a valid hash is found. Architecturally a separate binary from Aurum: Hayyānu connects to a running Ilmarinen node over TCP (default localhost:8333) and exchanges mining jobs and solutions via the scriptum-rpc wire protocol. The separation is structural — App Store and Play Store policies prohibit mining in consumer wallet applications, so Aurum ships mining-free across all distribution channels; Hayyānu is direct-download only from dfqr.org. The mining engine is the same whether invoked as a standalone binary at industrial scale or surfaced via Aurum’s home-mining UI on platforms where mining is permitted (the UI shell wraps the same Hayyānu binary as a subprocess). One engine, two surfaces, three distribution paths.

Aurum — the reference Scriptum wallet. Bundles Ilmarinen internally so every Aurum user runs a validating node (depth varies by device tier — see §Node Tiers). Provides the user-facing layer for all chain operations: receiving, spending, NEXUS routing, Horkos negotiation, Ás-brú trade construction, view-key delegation, Sealed Sender Memo handling, Frau Holle Menuha sweeps, Aurum Market (the public price-and-offer data viewer), and Kiva (the opt-in discovery hall — see §Kiva). Aurum is mining-free by design — Hayyānu is a separate binary (see §Hayyānu) so Aurum can ship cleanly through App Store and Play Store distribution channels. The “golden wallet” — the bright, refined surface over the chain’s machinery. Open-source under Apache 2.0; the full Aurum specification is published as a separate technical document. Other wallet implementations conforming to WP / CM / SPX are welcomed and unprivileged.

Kiva — the discovery hall above the settlement primitives. Named for the Pueblo ceremonial chamber where members of a community become visible to one another for collective business — distinct from private dwellings, distinct from the central plaza. Opt-in and symmetric: a holder enters Kiva by explicit choice, becomes visible to other members by their home Scriptum address, and is invisible to and from the hall without entering. Two surfaces operate inside Kiva — the hall (presence list with optional Buy/Sell tags and on-chain reputation indicators inline) and the board (signed Ás-brú offers with 24-hour TTL ceiling, active withdrawal at any time, maximum five active offers per address). Kiva does not execute trades, does not hold funds, does not become a counterparty, does not adjudicate disputes. After two holders find each other through Kiva, the interaction

becomes direct Aurum-to-Aurum and settlement proceeds through the chosen primitive — Aurum Market, Horkos, or Ás-brú. A public hall by intent: holders are identified by home address so that on-chain reputation is the trust layer. Holders who require privacy of the fact-of-trading do not enter Kiva and use the chain's private primitives directly. Kiva is not consensus — no transaction validity depends on it; it is an Aurum-side overlay using the existing Scriptum P2P network for gossip transport. The full Kiva specification appears in §Kiva (this document) and in the Aurum Technical Document.

Krakophone — the Crowglass implementation for the Kraken exchange. Ships with Aurum at launch as Aurum's bundled Crowglass observer. Queries Kraken's public API for committed Ás-brú trade fingerprints; reports matches to the miner running Krakophone within Ilmarinen. Other Crowglass implementations (for additional exchanges in the Foundation's eventual recommended Crowglass operational configuration) may be developed post-Foundation-incorporation; Krakophone is the first.

cumulative HICP factor — A Tier 1 chain-state value initialised to 1.00 at genesis and updated annually through the HICP_RATIFY transaction. Compounds annual HICP adjustments per Charter §4.2 (monotonic non-decreasing; deflation year contributes 1.00). Applied to all EUR-denominated values at fee calculation and cap check time. See §HICP Indexing — Annual Real-Value Calibration.

HICP Indexing — The chain's mechanism for adjusting EUR-denominated values annually to preserve real purchasing power across the chain's century-scale lifetime. Annual cycle: 30-day Observation Window (1 January – 31 January) with open HICP_PROPOSE submissions; median aggregation at window close; 60-day Countersign Window (1 February – 31 March) requiring 5-of-9 Curia ratification; cumulative factor update on successful ratification. See §HICP Indexing — Annual Real-Value Calibration.

HICP_PROPOSE — Open-submission transaction for the annual HICP Observation Window. Any chain participant may submit observed HICP factor with source identifier. Median aggregation across submissions produces the candidate factor for Council ratification. Zero fee.

HICP_RATIFY — Council multisig transaction (5-of-9 Curia, L1) ratifying the candidate HICP factor produced by Observation Window median aggregation. Commits the cumulative HICP factor update. Zero fee.

CAP_AMENDMENT_RATIFY — L4 multisig transaction (WOC + 3-of-5 Founders + 7-of-9 Curia) authorised during the HICP Countersign Window (1 February – 31 March annually). Commits new base values for FUND authorisation cap parameters (monthly cap, annual cap, per-tier boundaries). HICP indexing continues to compound on the new base. Zero fee.

HORKOS_COMMIT — Party A's transaction in Horkos bilateral settlement: commits the principal value (the asset being sold or settled) to the protocol-controlled ESCROW address. See Horkos section for full settlement flow.

HORKOS_STAKE — Party B's transaction in Horkos bilateral settlement: commits the cryptographic stake bond (the forfeit guarantee) to the protocol-controlled ESCROW address. See Horkos section for full settlement flow.

Council Registry — The Tier 1 chain-state object holding the constitutional authority structure for VAULT-class Parcel authorisation. Holds three locks (WOC Lock, Founder Lock, Curia Lock) each with their respective slots. Each slot stores a Dilithium pubkey, missed-vote counter, dormant flag, and bootstrap one-shot flag. Mutated by signed transactions per the slot operation threshold table. Parallel to Bundle Registry and SlotBox Registry.

Dormant slot — A Council Registry slot whose missed-vote counter has exceeded 3, marking it as dormant. Dormant slots do not contribute to the lock's effective N for threshold calculation. Dormant slots are self-healing: when they sign a transaction again, their counter resets to 0 and they return to active.

WOC_INSTALL_SLOT — A bootstrap-period transaction signed by WOC alone, replacing a placeholder pubkey in a non-WOC Council Registry slot with the real council member's pubkey and setting the slot's one-shot flag to true permanently. Each non-WOC slot can be WOC-installed exactly once. After all 14 WOC_INSTALL_SLOT events, the chain's constitutional sovereignty is fully constituted.

REMOVE_SLOT_KEY / ADD_SLOT_KEY — Post-bootstrap Council Registry mutation transactions. REMOVE_SLOT_KEY removes a pubkey from a slot (leaving it empty); ADD_SLOT_KEY installs a pubkey into an empty slot. Thresholds vary per slot's body (Curia, Founder) and operation type (REMOVE asymmetric with ADD).

WOC_REMOVE / WOC_ADD — Council Registry mutation transactions specifically for the WOC slot. Both require 5-of-5 Founders + 7-of-9 Curia signatures with WOC explicitly excluded from participation.

Council app — A genesis-critical Aurum component providing the user interface for council members to coordinate multisig transactions. Key-gated by the council member's Dilithium key. Specifications in the Aurum Technical Document.

Horkos — the bilateral atomic settlement primitive (Greek Ὅρκος, the personification of sworn oath in Hesiod's *Works and Days*; son of Eris, punisher of perjurers). Three-party covenant: A commits Scriptum to ESCROW, B delivers off-chain (fiat, goods, services), C attests to the delivery. The chain enforces structural floors (stake $\geq 0.1\%$ of principal as a nominal protocol minimum — commercial use cases will typically negotiate substantially higher stakes; window ≥ 360 blocks, window $<$ remaining time to Ragnarök) and procedural rules; within those, parties shape the contract. Resolution flows through standard UNLOCK $\rightarrow$ EJECT $\rightarrow$ SEND. Earlier specifications named this primitive Delayed Atomic Settlement (DAS); the rename unifies the chain's mythological vocabulary.

Ás-brú — the fast-rail exchange bridge primitive (Old Norse for Bifröst, the bridge of the Æsir connecting Midgard to Asgard). Two-party covenant: A locks Scriptum in ESCROW, B transfers off-chain on a centralised exchange. Window fixed at 60 blocks (~10 minutes); no

third-party attester; miners run **Crowglass** — the exchange observation daemon — which queries the named exchange’s public API for the committed off-chain transfer fingerprint and includes ASBRU_RELEASE in the building miner’s block when a matching transfer is confirmed. Half-atomic by design — cryptographic enforcement on the Scriptum side, miner-consensus + reputation backing on the off-chain side. The bridge that connects the on-chain economy to exchange-rail liquidity without requiring formal listing.

The Crowglass naming is intentionally exchange-agnostic. The daemon observes whichever exchange the trade’s commit fingerprint specifies; the protocol does not maintain an admitted-exchange list, and miners choose which exchanges they observe. The Foundation publishes a recommended Crowglass operational configuration (see GOV §12.1 and §11.3); the recommendation is informational, not protocol-binding. The corvid imagery (Odin’s ravens Huginn — “thought” — and Muninn — “memory” — fly out across the world and return with what they have seen) names the function: distributed observation returning structured testimony. Glass is the window onto external state.

Scriptum V2 (forward reference). Cryptographic anonymity for specific transactions — sender hiding via STARK-based ring proofs over fixed-denomination trees — is planned for a future Scriptum V2 specification. This is out of scope for this document. The Scriptum chain at Genesis ships pseudonymity and recipient unlinkability via stealth derivation and selective disclosure (see Stealth Settlement and Selective Disclosure). Stronger privacy postures will be specified separately when ready.

Menuha — the weekly rest. A 24-hour window each Saturday 23:00 UTC to Sunday 23:00 UTC during which user-initiated outward value transactions (SPEND, BURN, DONATE, LOCK, and all user-initiated Horkos stages including agreement, loading, confirmation, and HORKOS_ATTEST) are invalid. Protocol-driven transactions continue, including the protocol-driven resolution sequence of Horkos contracts whose attestation finalised before the rest day.

Frau Holle — the cross-address Menuha sweep. Wallet-driven and holder-signed; consolidates LOCKED Parcels across the holder’s stealth-derived addresses into a single LOCKED Parcel at the holder’s HOME address when the cross-address fuse fires (holder’s address count exceeds three). Aurum’s default behaviour is automatic sweep at every Menuha when the fuse fires; opt-out is available per-Menuha or persistently; persistent opt-out triggers reaffirmation prompt every Menuha to confirm continued skip or opt back in (no silent cluster growth). Non-Aurum wallets follow their own policies. Frau Holle is the holder’s explicit cluster-exposure act.

Goldmarie — the per-address Menuha sweep. Chain primitive operating at every Menuha block; for every address (HOME or stealth) holding three or more LOCKED Parcels of the same (class, role), the chain consumes those Parcels and produces a single LOCKED Parcel at the same address. No view-key needed (chain sees per-address state without holder identity); no opt-out (consensus rule; missing required Goldmarie consolidations invalidate the block); fee zero. Named for the diligent good daughter in the Frau Holle fairy tale; pairs with Frau Holle at the cosmological level — Goldmarie keeps every room tidy in place, Frau Holle does the whole house.

HOME — the holder's published-identity address. The address where senders look up the holder's epoch Kyber pubkey for stealth derivation. The destination for Frau Holle's cross-address consolidation (the holder's explicit cluster-exposure act). HOME holds Parcels only after explicit consolidation or when it is the tipping input's address of a self-directed transaction. HOME is not an operational hub; value flows through the holder's address cluster.

Tipping input — under smallest-first coin selection, the last input consumed in a transaction; the input that pushed the accumulated sum over the target value. All smaller inputs are fully consumed by the transaction; the tipping input is the one with leftover value. The change Parcel goes to the tipping input's address under the universal change destination rule. For single-input transactions, the one input is trivially its own tipping input.

Ratatoskr — the chain-traversal enumeration primitive run **wallet-side and auditor-side, not as protocol consensus machinery**. Given a holder's key pair (or a holder-delegated view key), Ratatoskr traverses the chain and reports the address set derivable from those keys (i.e. the set of stealth-derived addresses currently containing Parcels the holder controls). Has one function: report. The primitive is computationally identical to what every wallet does when scanning the chain for its holder's Parcels; Ratatoskr names this scanning operation as a first-class concept and specifies its interfaces (wallet-internal scan, archive-served scan, auditor-delegated scan via view-key handoff). Used by wallets, auditors, and the wallet-side construction of cross-address SWEEP transactions. The squirrel running up and down Yggdrasil carrying messages.

PI — Personal Identity, the operational sovereign in the Identity Tree architecture. The holder's persistent on-chain identity where value lives, where counterparty interactions accumulate, where the on-chain behavioural pattern develops.

Sealed Sender Memo — the encrypted address disclosure carried by every SPEND transaction. Encrypts the sender's published address (and any optional additional content within the size budget) to the recipient using a key derived from the same Kyber/ML-KEM-encapsulated shared secret that derives the stealth output address. Recipient decrypts via their Kyber view key. External observers cannot read the memo. Mandatory in form (every SPEND carries a well-formed encrypted memo); content is holder-chosen but the minimum is the sender's published address. The Sealed Sender Memo provides **recipient-visible sender-provenance disclosure** as a deliberate trade-off: recipient-side unlinkability is not absolute (the recipient learns which published address sent them value), but external observers without view-key access still cannot link senders to recipients. The chain's per-transaction audit primitive — holders can selectively disclose individual memos to auditors or counterparties to prove specific transactions without revealing their broader on-chain footprint.

Hugin and Munin — the chain's mechanisms for detecting anomalous V_A readings. If a sudden spike or crash in observed exchange prices would result in V_A deviating significantly from recent values (e.g., flash crash or market manipulation), Hugin and Munin trigger protective behaviour: V_A is held to a smoothed value during the anomaly

period, preventing the chain's fee schedule from being distorted by transient price action. Hugin observes per-block V_P/V_A deviation; Munin holds the cumulative drift and raises the alarm. The exact thresholds and detection algorithms are specified in SPX.

Gehenna — the burn address. Provably unspendable. All BURN transactions send value here.

Chesed — the semantic name for voluntary donation to the Vaults (DONATE).

Dana — the permanent MARKER issued on confirmation of a donation.

Mayflower — the permanent MARKER issued at Genesis to Founders and WOC, re-instantiated unchanged at every Rebirth.

Phoenix — a single complete chain from Genesis to Ragnarök; one hundred years of chain life.

Arc — the manifest that carries state across Ragnarök to the successor Phoenix.

Ragnarök — the terminal block of every Phoenix. Block RAGNAROK_HEIGHT.

Economic vocabulary

V_A — the chain's market price reference for FUNK against EUR, used to compute EUR-denominated fees in FUNK at transaction time (e.g., $\text{fee_FUNK} = \text{fee_EUR} / V_A$). The chain maintains V_A as a smoothed reference price drawn from regulated exchange feeds. Defined and protected by Urðarbrunnr (UIS); the exact smoothing and oracle selection rules are specified in SPX.

$f_U, f_A, f_S, f_{\text{refresh}}$ — the four fee components. f_U is paid at UNLOCK (calculated on summed unlocked value). f_A is reserved at UNLOCK using the V_A at the UNLOCK block, and paid to the miner at the EJECT block (flat instruction fee, irrespective of amount). f_S is the progressive tier fee paid at SPEND (calculated on amount or payload) and at Horkos load operations (HORKOS_COMMIT on principal plus attestation fee, HORKOS_STAKE on stake). f_{refresh} is the small fee paid for ACTIVE-state REFRESH (governance-set, expected near-zero).

Tithe — the protocol's share of fees. 90% goes to the miner who includes the transaction; 10% is the Tithe, split internally as 48% WFC (compensation pool for the fifteen constitutional position-holders), 32% Funds (Foundation operational allocation), 10% RESERVE (strategic long-term reserve), 10% CP (Charity Pot, the structural humanitarian allocation, additionally receiving all WFC cap-overflow). See §15 for full mechanics.

UIS — the Urðarbrunnr Integrity System. The machinery that maintains V_A honestly in the presence of oracle noise, network churn, and attempted manipulation.

Horkos — the protocol primitive for bilateral trade settlement with third-party attestation. The name (Greek Ὁρκος, the personification of sworn oath in Hesiod) reflects the contract's nature: a sworn bilateral covenant whose breach incurs cryptographic forfeit. Earlier specifications referred to this primitive as "Delayed Atomic Settlement" (DAS); the rename

in this version reflects mature naming consistent with the chain’s mythological vocabulary (Kavaca, Hayyānu, Ilmarinen).

Privacy and Disclosure Vocabulary

The chain uses six related but distinct terms when describing what recipients, observers, and delegates can and cannot see. These terms have specific technical meanings in this corpus and must not be used interchangeably.

Pseudonymity. The default property of every Scriptum address. An on-chain address is a 64-byte hash; it is not bound to a legal identity at chain level. An observer can see what an address holds, sends, and receives, but cannot derive the legal-person counterparty from the chain alone. Pseudonymity is the baseline; it is what every cryptocurrency provides. *Pseudonymity comparable to ordinary banking records* is the operational standard Scriptum claims: the transaction record exists, is observable to participants in the transaction and to legitimate investigators with proper authority, and is not visible to the general public except where parties choose to publish.

Recipient unlinkability. The property that distinct outputs sent to the same recipient address are not visibly tied to a common recipient by an on-chain observer. Stealth derivation provides recipient unlinkability by default: every SPEND output lands at a fresh, single-use stealth address; an observer cannot determine that two outputs go to the same recipient without auxiliary information.

Audit visibility (selective disclosure). The property that a holder can deliberately reveal a bounded portion of their own activity to a chosen delegate (auditor, accountant, custodian, heir, legal counterparty) without revealing anything outside the bound. Scriptum provides audit visibility per UTC calendar year through the epoch-bounded view-key construction (CM-16): the holder exports a specific year’s epoch view secret; the delegate decrypts that year’s outputs and nothing else. The boundary is cryptographic; the holder does not depend on the delegate’s good behaviour.

Privacy (umbrella term). When used without qualification, “privacy” in this corpus refers to *the holder’s control over what is revealed to whom, when, and how*. It is the aggregate effect of pseudonymity, recipient unlinkability, audit visibility, and (where chosen) cryptographic anonymity. “Privacy” is the noun for the design property; the specific mechanisms above are the means by which it is realised. “*Reasonable privacy*” in operational prose means pseudonymity comparable to ordinary banking records plus recipient unlinkability, not cryptographic anonymity.

Discipline for cross-document use. Other corpus documents (CM, SPX, GOV, CON, IA, EA, Highlights, ATD) defer to these definitions. Claims framed as “private” in body prose mean pseudonymous + unlinkable. Claims framed as “audit visibility” or “selective disclosure” reference the CM-16 epoch-bounded view-key construction only.

Parameter references

Specific numeric values — block time, Safety Control Window duration, fee tier percentages, genesis allocation amounts, emission constants — are defined in the Scriptum System Parameters document (SPX). This whitepaper references named parameters where relevant without reproducing their values, so that governance changes to SPX do not require whitepaper revisions.

System Intent

Scriptum is a general-purpose settlement chain architected around custody-grade safety assumptions. It serves the full spectrum of value-transfer needs — from small everyday payments to large institutional settlements — with the same chain, the same primitives, and the same fee schedule that scales fairly across all of it. The chain does not specialise; it provides the foundation, and the user picks the appropriate primitive for each transaction. The safety assumptions are constant across the spectrum: every transaction inherits the Safety Control Window, post-quantum cryptography, and the consensus disciplines that custody-grade use requires.

Trading in both directions, on-chain primitives across the whole rail spectrum. Scriptum provides bidirectional liquidity through cryptographically enforced primitives without requiring custodial intermediaries. Horkos handles attested off-chain settlement for fiat wires, physical goods, services, and complex deliveries — three-party covenant with stake-backed enforcement. Ás-brú handles fast-rail exchange bridges — two-party covenant, miner-monitored, no exchange cooperation required. Standard SPEND (in three modes — direct, private-from-hot, private-from-cold) handles peer-to-peer transfers across all sizes. Holders find each other through Kiva — the opt-in discovery hall — and observe prices through Aurum Market — the public price-and-offer data viewer; neither is itself a settlement primitive, both surface counterparties and information for the settlement primitives below. Holders move Scriptum in or out of fiat, crypto, and goods/services through whichever primitive fits the rail; the protocol is versatile by composition rather than by special-casing.

Scriptum's privacy posture is pseudonymity plus recipient unlinkability. Senders construct stealth-derived destination addresses via ML-KEM-768 encapsulation against recipients' published epoch Kyber public keys (see Stealth Settlement). Holders may delegate visibility via the View-Key Registry to auditors with cryptographically-bounded scope (see Selective Disclosure). Memos between sender and recipient are encrypted with key material derived from the KEM shared secret. This privacy posture is sufficient for ordinary commercial settlement: chain-only adversaries cannot link senders to recipients, while holders retain full audit control through view-key disclosure. Stronger privacy postures — including cryptographic anonymity for specific transactions — are planned for a future Scriptum V2 specification. See the V2 forward-reference glossary entry.

Safety as a default, usability as a discipline. Every cancellable path passes through NEXUS — value is observable for 24 hours and recoverable via CANCEL until eject. Theft of a key does not mean theft of value; the holder, an observer, or an automated monitor can

submit CANCEL during the dwell window. The Aurum wallet streamlines safety into the workflow rather than against it: one-click sends from ACTIVE for fast operational use, automatic cross-address consolidation via Frau Holle on Menuha, in-flight transaction monitoring that surfaces unauthorised UNLOCKS immediately. Safety is not a friction tax. The protocol enforces structural protection; the wallet makes the protection invisible to compliant users while remaining mandatory at the consensus layer.

Institutional-grade auditability with cryptographic boundaries. View-key delegation gives holders cryptographic control over who can see their on-chain history. Custodians, accountants, regulators, and counterparties can be given precisely the audit scope the holder chooses — per-transaction (via Sealed Sender Memo disclosure), per-epoch (via epoch-bounded view-key handoff per CM-16), or full-portfolio (via multi-epoch handoff covering the periods of interest) — with the chain's primitives enforcing the boundary cryptographically. Selective disclosure is the design pattern. Subpoena power is acknowledged as outside the chain's threat model; holders compelled to disclose can do so with cryptographic proof of completeness, and parties not in the audit scope cannot read the disclosed material. Scriptum's compliance posture is pseudonymity plus bounded selective disclosure. The chain is compatible with MiCA, FATF Travel Rule, and equivalent frameworks while preserving holders' privacy against parties outside the audit scope. Holders' epoch view keys provide auditors with cryptographically-bounded visibility into the holder's chain activity over a specific generation-year scope. The chain commits to clear architectural separation between pseudonymous-with-selective-disclosure operations (the present baseline) and any future cryptographic-anonymity operations specified in future protocol versions, keeping the regulatory framing tractable as the chain evolves.

Every movement of value in Scriptum is intentional, observable, and — until confirmed — reversible. This is not a limitation; it is the design. Scriptum is built for the secure storage and deliberate movement of value in environments where failure is expected and must be survivable. It is general-purpose; the post-quantum and Safety Control Window foundation gives every application access to structural properties — cancellable settlement where useful, audit-scoped disclosure where required — that other chains cannot offer.

Core Principle

Ownership does not imply immediate control. Control requires intent, security, and time.

Problem Statement

Existing blockchain systems are built on the assumption that value transfer should be immediate upon authorization. This model assumes private keys remain uncompromised, users act without error, authorization is always intentional, and cryptographic primitives remain permanently secure. These assumptions do not hold in adversarial conditions.

Structural Weakness of Immediate Execution

In current systems, authorization and execution are coupled into a single step. Any valid signature results in immediate exposure of value. There is no separation between intent

and execution, no opportunity to observe or intervene. Security depends entirely on preventing failure — not surviving it.

Consequences

Compromised keys result in instantaneous and irreversible loss

Phishing or user error leads to permanent loss of funds

Automated exploits can drain wallets in seconds

Coercion scenarios offer no recovery window

Future cryptographic breakthroughs may invalidate existing protections without mitigation

Limitation of Existing Mitigations

Timelocks, multisignature schemes, smart contract vaults, and custodial withdrawal delays share a common limitation: they are optional, locally defined, externally enforced, and can be bypassed or misconfigured. None alters the fundamental property of the system: that valid authorization enables immediate execution.

The Core Problem

The core problem is not insufficient cryptography. The core problem is that authorization is treated as equivalent to execution. This creates a system where failure is binary, recovery is impossible, and time plays no role in security.

Design Requirement

A secure value system must assume that authorization can be compromised, users can make errors, and attackers can obtain valid signing capability. Security must not rely on preventing failure, but on controlling execution.

Scriptum Approach

Scriptum separates authorization from execution. A valid signature is no longer sufficient to move value. Value must be explicitly unlocked, enter the NEXUS, become ACTIVE, and remain valid throughout the ACTIVE period before it can be transferred.

Core Transition

Traditional systems ask: was the transaction authorized?

Scriptum asks: did the transaction remain valid over time?

Design Philosophy

Scriptum is built on three principles:

Security over convenience

Determinism over flexibility

Control over immediacy

Through its time-gated execution model, abrupt market reactions cannot be acted upon instantaneously. This results in natural dampening of short-term volatility without imposing price control. As the ecosystem grows, progressively larger capital flows are required to produce comparable price movements — increasing market inertia as the system matures. Price changes remain possible at all times, but their immediate impact on executed transactions is reduced.

The Hard Layer — Ownership and State

The hard layer consists of Parcel ownership, state transitions, and the Safety Control Window. These are governed exclusively by deterministic consensus rules. No external system is involved.

Consensus depends only on Parcel state, NEXUS rules, and deterministic protocol logic

No exchange, oracle, price feed, or external system is required for protocol operation

Ownership is either correct or not — there is no partial correctness

The hard layer must always be correct. The system does not tolerate wrong ownership.

Complexity containment:

Scriptum protocol surface is broad — it includes the NEXUS, Limbo, ACTIVE windows, PACK/ARC/BOARD, CLASS/ROLE/STATE, V_A, and the UIS. A senior engineer will recognize this as a large state machine. The critical reassurance is this: all complexity is confined to deterministic state transitions. There is no dynamic execution, no contract logic, no Turing-complete layer, and no emergent behavior. Every valid state and every valid transition is enumerated in this document. An implementation is correct if and only if it reproduces these transitions exactly. The system is wide but not deep.

The Soft Layer — Value and Fees

The soft layer consists of the V_A valuation coefficient used exclusively for fee calculation. V_A is consensus-derived — computed from signed V_P submissions and constrained by deterministic bounds — but it is not externally dictated.

V_A affects only fee magnitude — not ownership, not state, not balances

Any registered HOME may submit V_P proposals via signed VA_PROPOSE transactions, regardless of whether the HOME operates an Aurum wallet, a mining node, or a full validation node

No source is authoritative. No oracle is hardcoded

Even if V_A is wrong or manipulated: all balances remain correct, all ownership is unaffected, system integrity is fully intact. Only fee magnitude is temporarily off — users may wait for convergence.

Manipulation of V_A yields no meaningful economic advantage. To materially influence fee calculation, an attacker would need sustained control over block production across the full smoothing window — approximately 12 hours. The only possible gain is a marginal reduction in transaction fees on a single transaction. This cost-benefit profile renders such attacks economically irrational.

Meaningful fee effects appear only at very large transaction sizes. At that scale, the network is already functioning as intended: it is being used for high-value, deliberate transfers. The cost of manipulating V_A still exceeds any achievable benefit.

Furthermore, the incentive is self-defeating in both directions. A miner who drives V_A upward increases fees, triggering the UIS warning system in Aurum. Users observe the anomaly and defer transactions. Transaction volume falls. Total fee revenue for all miners — including the attacker — decreases. A miner who drives V_A downward reduces fees, increasing volume but reducing per-transaction income. Neither direction produces net gain. The economic equilibrium is honesty.

The system tolerates wrong prices. It does not tolerate wrong ownership.

PART II — HOW

Ilmarinen — System Architecture

Scriptum is a native blockchain system with Proof-of-Work consensus, SHA3-512 proof-of-work mining, Parcel-based Parcel accounting, and a fixed maximum supply of 42,000,000 Scriptum.

Block Parameters

Block time: 10 seconds (target)

Block size: 2 MB at genesis (governance variable, scales with bandwidth)

Capacity: ~17 canonical atomic cycles per second at 2 MB genesis (2,097,152 / ~11,900 bytes per first-touch UNLOCK-NEXUS-EJECT cycle $\approx$ 176 cycles per block $\approx$ 17.6 cycles per second; for bound-address operation at ~8,800 bytes per cycle, throughput rises to ~24/sec). This exceeds the 5 cycles/second design minimum by more than three times. Approximately 1.5 million cycles per day at full block utilisation at genesis. Scriptum provides deliberate value transfer at settlement grade. A private bank, a family office, or a sovereign wealth fund does not need 1,000 TPS. It needs certainty.

Difficulty adjustment: every 2,592 blocks (~7.2 hours). The interval encodes the Great Year — the 25,920-year axial precession of Earth — at second resolution: 2,592 blocks $\times$ 10 seconds = 25,920 seconds per retarget period. Tracked by Babylonian, Egyptian, Hindu, Greek, and Mayan astronomy, the Great Year is the longest cyclic rhythm civilisations have continuously observed. Retargeting at this cadence gives 2,592 samples per period (29% more than Bitcoin's 2,016) — better noise smoothing — with adjustment fires ~3.3 times per day, frequent enough to track real hashrate changes without being twitchy.

Retargeting is asymmetric: the harder direction (target shrinks; difficulty rises) is clamped to 2 $\times$ per period, the easier direction (target grows; difficulty falls) is clamped to 4 $\times$ per period. The asymmetry is anti-timewarp defence. A coalition can poison the retarget math by drifting block timestamps backward across the retarget window (within MTP and future-bound limits), making the actual elapsed time appear longer than it really was — the math then infers low hashrate and produces an easier target (difficulty falls). With tight harder-direction clamping and looser easier-direction clamping, the attacker's tool (manufacturing a difficulty drop) is capped at 4 $\times$ per period; the chain's recovery tool (raising difficulty when honest hashrate returns) is preserved through the 2 $\times$ harder-direction clamp applied at subsequent retargets.

Block timestamps are u64 milliseconds since Unix epoch. The resolution prevents retarget direction inversion at sub-second block production rates (bootstrap phase, Phoenix renewal cycles, pathological hashrate spikes). Two timestamp consensus rules apply to every accepted block beyond genesis:

- **Future bound.** $\text{block.timestamp} \leq \text{local_clock} + 7,200,000 \text{ ms}$ (2 hours). Accommodates expected miner/validator clock skew (NTP drift, geographic distribution). A block claiming a far-future timestamp is rejected.
- **Median Time Past (MTP).** $\text{block.timestamp} > \text{median}(\text{last } 2,592 \text{ block timestamps})$. For chains shorter than the window, uses all available history. The window matches the difficulty retargeting interval — same number, same Great Year, design coherence. Prevents miners from drifting timestamps backward to game retargeting.

Absolute target bounds: MIN_TARGET floors the difficulty (prevents chain halt at unmineable difficulty) and MAX_TARGET ceilings it (sanity bound at trivial difficulty). Retargeting math is overflow-safe — multiplication that would overflow u128 clamps to MAX_TARGET rather than producing wrong-direction results.

Coinbase reward bound: the coinbase output must exactly equal $R_0 \times (1 - S_{\text{net}} / S_{\text{max}})$ per the emission formula (subject to block-fee additions). Miners cannot claim arbitrary rewards; the chain enforces the exact protocol formula on every block.

Block size and transaction count limits: BLOCK_SIZE_MAX is the Curia-governance variable (2 MB at genesis); MAX_TX_PER_BLOCK is a consensus-enforced upper bound preventing resource-exhaustion attacks even if block size were governance-raised. Both are enforced at block validation; violations reject the block.

Ragnarok block height: RAGNAROK_HEIGHT = 315,360,000. Defined in blocks, not calendar time: 8,640 blocks/day x 365 days x 100 years. Leap years are not counted. Block count is the protocol primitive. This is a deliberate choice — the chain counts blocks, not seconds.

Monetary Policy

Maximum supply: 42,000,000 Scriptum (S_{max}). The cap is firm in normal operation but is amendable through the Upgrade Ceremony if governance and the community ratify such a change. The cap is a constitutional commitment, not algorithmic immutability. See §[Cap Amendability] below.

Formula constant: $R_0 = 1$ (fixed forever; no Phoenix-specific recalibration)

Block reward: $r = R_0 \times (1 - S_{\text{net}} / S_{\text{max}})$, where $S_{\text{net}} = S_{\text{mined}} - S_{\text{burned}}$ (all quantities in Scriptum)

S_{net} at Phoenix 1 genesis: 6,720,000 Scriptum (16% of S_{max} — the founding allocations created at Block 0; see §Allocation in §21).

Genesis emission of Phoenix 1: $r_1 = 1 \times (1 - 6,720,000 / 42,000,000) = 1 \times 0.84 = \mathbf{0.84 \text{ Scriptum/block}}$.

Reward asymptotically approaches zero within each Phoenix and never reaches it. Phoenix 1 mines approximately 35.26 million Scriptum across its 100-year lifespan, ending at 99.95% of S_{max} .

Over time within each Phoenix, transaction fees become the primary miner incentive. By Year 50 of Phoenix 1, block rewards are economically marginal compared to settlement-fee revenue. All successor Phoenixes operate as predominantly fee-driven economies from genesis. This transition from mining-driven to fee-driven economics is by design — the chain's institutional revenue mechanism, the 10% Tithe routed to WFC, Funds, RESERVE, and CP, is designed to scale with settlement volume, not with mining emission.

The Scriptum system continues through successive Phoenix cycles. Each Phoenix begins with whatever S_{net} disembarks from the Arc; R_0 remains fixed at 1.

Coinbase — The Miner Reward

The COINBASE transaction is the protocol-driven mechanism by which new Scriptum enters circulation. Exactly one COINBASE output is created per block, paid to the address specified by the successful miner. COINBASE has no signature, no inputs, and is enforced at the consensus layer: any block whose COINBASE output violates the emission formula is rejected by all nodes.

Emission Formula

The block reward r is computed deterministically from current chain state:

$$r = R_0 \times (1 - S_{\text{net}} / S_{\text{max}})$$

$R_0 = 1$ (formula constant, fixed forever, identical across all Phoenixes)

$S_{\text{max}} = 42,000,000$ (cap on circulating supply; amendable only through the Upgrade Ceremony)

$$S_{\text{net}} = S_{\text{mined}} - S_{\text{burned}}$$

At Phoenix 1 genesis, $S_{\text{net}} = 6,720,000$ (the 16% founding allocations). Genesis emission $r_1 = 1 \times (1 - 6,720,000 / 42,000,000) = 0.84$ Scriptum/block.

The formula is asymptotic: as S_{net} approaches S_{max} , r approaches zero but never reaches it. r is always strictly positive, however small. Mining never exhausts within a Phoenix; it slows to asymptote, and BURN events reopen headroom by reducing S_{net} . Successor Phoenixes inherit a different starting S_{net} (whatever disembarks from the Arc manifest), but R_0 does not change. Each successor Phoenix has its own emission curve shaped by the same formula and the same R_0 , scaled to whatever inherited S_{net} is.

Coinbase Transaction

For each block, the protocol creates:

COINBASE:

inputs: none

outputs: one Parcel — amount = r , state = LOCKED, class = CIRCULATING,

role = NONE, address = miner address

fee: none (COINBASE does not pay fees)

The miner claims r + any block-space fees (f_U , f_A , and the 90% miner share of f_S) in the same block.

Maturity

COINBASE outputs must reach 100 confirmations before they can enter the standard CIRCULATING lifecycle (UNLOCK → NEXUS dwell → EJECT → ACTIVE → SPEND). During this maturity period the COINBASE Parcel exists on-chain but cannot be unlocked. This matches the Bitcoin convention and protects against coinbase-reorg exploitation: a miner cannot spend a reward that could be orphaned by a longer competing chain.

Supply Accounting

Every confirmed COINBASE output increments S_{mined} by its amount. S_{mined} is unbounded over the lifetime of a Phoenix — it continues to grow as long as mining continues. $S_{\text{net}} = S_{\text{mined}} - S_{\text{burned}}$ is the quantity subject to the 42M circulating cap; S_{mined} itself is not capped.

At the start of every successor Phoenix (following Ragnarök and Arc migration), S_{mined} and S_{burned} reset to zero for fresh accounting. R_0 remains fixed at 1; the formula $r = R_0 \times (1 - S_{\text{net}} / S_{\text{max}})$ continues to apply unchanged. The starting S_{net} of the successor Phoenix is the total Scriptum value disembarked from the Arc manifest — typically a substantial fraction of S_{max} , since most of Phoenix 1's mined supply has survived as committed Parcels that boarded the Arc. Successor Phoenixes therefore mine smaller absolute quantities than Phoenix 1, with emission curves shaped by the same formula and the inherited starting state. This is the chain's mature steady-state: mining bootstraps Phoenix 1; subsequent Phoenixes are predominantly fee-driven economies, with the 10% Tithe (routed to WFC, Funds, RESERVE, and CP) providing the chain's institutional revenue.

Constraints

A block is invalid if it omits COINBASE, creates more than one COINBASE output, pays an amount other than the protocol-defined r , pays COINBASE to a non-miner address, or attempts to spend the COINBASE output before 100 confirmations. These checks are consensus-enforced and not subject to miner discretion.

Mining Algorithm — SHA3-512

Scriptum uses SHA3-512 as its proof-of-work algorithm. SHA3 (Keccak) is the NIST-standardised successor to SHA-2, designed with a different internal structure (sponge construction) that provides defense in depth against structural cryptanalytic advances. SHA3-512 produces a 512-bit output and is computationally efficient on commodity CPUs, GPUs, and dedicated hardware alike.

This is deliberate. Scriptum does not pursue ASIC-resistance through algorithmic obscurity. The chain accepts that mining will professionalise over time as the asset gains value — that is the normal trajectory of any proof-of-work cryptocurrency. The protocol's commitment is to a transparent, well-studied, standardised primitive rather than to a perpetually-shifting algorithmic moving target. Anyone with a computer can mine Scriptum at launch. As mining hardware specialises, the chain's security scales with hash-rate; the asset gains the cryptoeconomic guarantees that hash-rate buys. The Commodore 64 starts the chain. Industrial miners sustain it.

Node Tiers

Scriptum nodes are not monolithic. The protocol supports four validation tiers, each appropriate to a different operator profile, storage budget, and trust posture. Every tier validates new blocks against consensus rules; what they differ on is what historical state they retain and what queries they can serve.

Tier 1 — Light client. Verifies new block headers and follows the chain tip via SPV-style proofs. Does not maintain a Parcel set. Trusts a remote validating node for state queries (balance lookups, transaction history). Footprint: kilobytes. Suitable for very constrained environments (watch devices, embedded surfaces). The light client is the *only* tier that involves trusting a remote party for state. Aurum does not surface light-client mode by default — the architectural claim “every Aurum user is a node operator” depends on running at least Tier 2.

Tier 2 — Phone node. Maintains the full Parcel set (live Parcel equivalent), full block-header chain, and a sliding window of recent block bodies (typically the last NEXUS depth — 8,640 blocks $\approx$ 1 day). Validates every new block against consensus rules using the Parcel set plus headers. Cannot serve historical queries beyond the recent-bodies window — for older transactions, queries fall back to Tier 3 or Tier 4 peers. Footprint: target 1–5 GB depending on chain age and Parcel-set density. This is the Aurum default on mobile devices. The phone user is a real validator, not a light client.

Tier 3 — Desktop node. Maintains the full Parcel set, full block-header chain, and a longer pruned window of block bodies (typical: weeks to months of recent history). Validates every new block; can serve recent-history queries to lighter peers. Footprint: $\sim$ 145 GB at chain maturity, scaling with adoption. This is the Aurum default on desktop and laptop devices.

Tier 4 — Sanctum (full archival). Maintains the full Parcel set, full block-header chain, and *every block body since genesis*. Validates every new block; can serve any historical query at arbitrary depth. Required for compliance research, regulatory archival, deep audit, and protocol forensics. Footprint: grows linearly with chain age. This is the institutional / Foundation / research operator tier. Master Aurum builds and standalone Sanctum binaries run at this tier.

Tiers 2–4 are validating tiers: they enforce consensus rules independently and accept or reject every new block on their own determination. Tier 1 is a trusting tier: it follows the consensus declared by the validating peers it queries. The chain's decentralisation property

holds across Tiers 2–4 — every Aurum user on a phone or desktop is a real participant in consensus, not a client of someone else’s validation.

Pruning rules are protocol-relevant. A Tier 2 or Tier 3 node must be safe to drop block bodies older than its retention window — meaning the Parcel set must be self-sufficient for validating new blocks without re-scanning ancient history. Scriptum’s Parcel-set architecture (every output’s current state is fully represented in the Parcel set; spent outputs are removed; the set is the live consensus state) makes this safe. The full block-header chain is always retained — headers are tiny and required for re-validating chain weight under reorganisation.

Multi-Miner Protocol

The Ilmarinen node and the Hayyānu miner communicate over TCP from genesis — there is no in-process channel, no shared-memory shortcut, no Unix-domain socket. The architecture supports many Hayyānu instances connecting to a single Ilmarinen, and many Ilmarinen instances forming the chain network. The TCP-from-genesis decision is deliberate: it means the same wire protocol that scales from a single laptop miner pointed at localhost in development scales unchanged to thousands of industrial miners pointing at validator pools in production. No protocol rewrite at adoption scale.

The wire protocol (scriptum-rpc) defines six message types: JOB_REQUEST, JOB_RESPONSE, SOLUTION_SUBMIT, SOLUTION_ACCEPTED, SOLUTION_REJECTED, and ERROR. Framing is u32 length-prefix with u8 message tags. All transitions and types serialise via canonical to_bytes/from_bytes — the consensus hash commits to wire-format bytes, not to language-runtime Debug strings.

The protocol defines one architecture. Operators scale by running multiple Hayyānu instances against one Ilmarinen (the validator distributes jobs, each with its own miner-bound coinbase address; the first valid solution wins) or by building pool software on top. Pools are application-layer constructs — coordinating job distribution and reward sharing across participating miners — not protocol features. The chain does not know whether a given Hayyānu is solo, part of a small operator’s fleet, or part of an industrial pool. It validates blocks the same way regardless.

Crowglass — the exchange observation daemon used in Ás-brú trade validation — is a component of the Ilmarinen node, not the Hayyānu miner. Miners pull jobs from the validator that have already had Ás-brú considerations included by Crowglass; Hayyānu’s responsibility is solving the proof-of-work, not observing exchanges.

Post-Quantum Cryptography — Kavaca

Scriptum uses the full CRYSTALS suite as its native cryptographic foundation, both components standardised by NIST in 2024:

CRYSTALS-Dilithium3 (ML-DSA) — the signature scheme. Every user-authorized transaction is signed with Dilithium3. Every address is secured against quantum attack

from genesis. Protocol-driven transitions (EJECT, EXPIRE, VEST, RESEAL, SWEEP, COINBASE, DISEMBARK) are consensus-enforced and require no signature.

CRYSTALS-Kyber (ML-KEM) is the key encapsulation mechanism. Present in every Scriptum address from genesis. Kyber provides post-quantum key encapsulation — the cryptographic foundation for stealth addresses, holder-controlled audit keys, and the sealed sender memo carried by every SPEND transaction (see §Privacy Model). The sender encapsulates an ephemeral shared secret to the recipient’s published Kyber view key (a single KEM operation, the post-quantum analog of a Diffie-Hellman handshake but technically a one-pass encapsulation rather than a two-party key exchange); the resulting shared secret simultaneously derives the stealth output address and keys an authenticated symmetric encryption of the sender’s published address. Two independent privacy properties — receive-side stealth and address-level provenance disclosure to the recipient — are produced from a single Kyber KEM operation. All CRYSTALS components are active at launch; there is no dormant cryptography, no reserved fields awaiting activation, no coordinated upgrade to wait for. Scriptum ships complete.

CRYSTALS-Dilithium3 and CRYSTALS-Kyber eliminate all known classical and quantum attack vectors against Scriptum’s signing and key-encapsulation primitives. Like any cryptosystem, they remain subject to future cryptographic advances. If a fundamental flaw is discovered in either primitive, the Governance Charter’s hard-fork mechanism is available to migrate Scriptum to successor standards.

SHA3-512 is the chain’s single hash function. Every chain-level hash operation — address derivation, Merkle trees, terms hashes, proof-of-work, CANCEL preimages, exchange identifier hashes in Ás-brú trade commits — uses SHA3-512. There is no second hash function, no special-case carve-out for “identifier hashes” or “auxiliary hashes.” The motivation is coherence over micro-optimisation: the byte cost of 64-byte outputs versus 32-byte outputs is rounding error against Dilithium3 signature overhead (signatures are ~3 KB each), while the security and audit story is dramatically cleaner with one function used everywhere. NIST Level 5 post-quantum collision resistance (256-bit against Grover-birthday) applies to every chain-level hash. The CANCEL hash in particular — the safety-critical hash protecting the NEXUS against an attacker holding signing keys but not the user’s CANCEL preimage — benefits most from full Level 5 security, which is exactly the surface where attack concentration is highest.

Why a Single Primitive, Not Pluggable Crypto

A reasonable reader may ask: why does Scriptum commit to a single cryptographic primitive set rather than supporting runtime-pluggable crypto? Modern systems often offer cryptographic agility — TLS negotiates cipher suites at connection time; libraries like OpenSSL and liboqs are explicitly designed to let applications swap algorithms via configuration. Scriptum deliberately rejects this pattern. The reasoning is structural, not ideological.

Consensus chains require single canonical primitives. A blockchain is a system in which thousands of independent nodes must reach the same answer about every transaction’s validity. If one node verifies a signature using Dilithium3 while another

verifies using ECDSA, the two nodes disagree on whether the signature is valid — which means they disagree on whether the transaction is valid — which means they disagree on whether the block containing it is valid. Consensus fails. Pluggable crypto is appropriate for systems where parties negotiate bilaterally (TLS clients and servers establishing a shared cipher). It is structurally incompatible with a system where every party must reach the same answer without negotiation.

Cryptographic agility creates attack surface that single-primitive design eliminates.

Pluggability is sometimes framed as future-proofing — “if Dilithium3 is broken later, swap it out at runtime.” In practice, supporting multiple crypto suites enlarges the attack surface substantially. Downgrade attacks become possible: an adversary forces the system to negotiate down to a weaker algorithm. Implementation bugs multiply across each supported algorithm. The TLS ecosystem has been bitten repeatedly by exactly this — historical attacks such as FREAK, Logjam, and POODLE exploited cipher-suite negotiation rather than weaknesses in any single algorithm. Scriptum’s structural defense is the opposite: one signing algorithm, one key-encapsulation algorithm, fully committed, single audit surface, no negotiation, no downgrade.

The real upgrade path for a blockchain is governance-coordinated hard fork through Menuha, not runtime configuration. If Dilithium3 is ever broken and Scriptum must migrate to a successor primitive, the chain does not survive by individual nodes flipping configuration flags. The chain survives because the constitutional governance specified in the Governance Charter ratifies the upgrade through Level 4 multi-sig (Warden of the Chain (WOC) + 3-of-5 Founders + 7-of-9 Curia), the new node software is published, and the network deploys coordinately during a Menuha window. Menuha is the chain’s weekly 24-hour rest period (§31) — no outward transactions execute, the mempool drains, the protocol performs its own housekeeping via SWEEP. It is the cleanest possible deployment moment: no transactions in flight, no consensus risk during the swap, every operator synchronized to the same boundary, 24 hours of buffer for rolling deployment. When Menuha closes and the next operational week begins, every up-to-date node enforces the new rules simultaneously; nodes that have not upgraded fall out of consensus and stop validating until they do.

This is genuine cryptographic agility — slow, deliberate, governance-mediated, explicit, and structurally appropriate for a consensus system. Pluggable crypto would let individual nodes upgrade unilaterally, which is precisely what consensus systems must prevent. Single-primitive commitment combined with the Menuha-windowed hard-fork mechanism gives Scriptum the agility it actually needs without any of the attack surface or consensus risk that runtime pluggability would introduce. The chain ships complete, with a single canonical post-quantum primitive set, and an upgrade path designed for the rare necessity rather than the routine convenience.

Address Format

Every Scriptum address commits cryptographically to two bundle components:

- **Version** — single byte. Genesis value is 0x01. Future bundle format evolutions use a new version; addresses derived under different versions cannot collide.
- **Dilithium3 public key** — signature verification.

Kyber768 keypairs for stealth derivation and per-epoch ML-KEM keypairs for selective disclosure are not part of the bundle. They live in the View-Key Registry, a chain-state object parallel to the Bundle Registry, populated by PUBLISH_EPOCH_PUBKEY transactions. See Key Architecture — Spend, View, Recovery.

The Bech32 address encoding adds a five-character checksum derived from the hash for transcription safety; the checksum is not part of the bundle.

Addresses are stored on chain as 64-byte address hashes — the full SHA3-512 hash of the key bundle, no truncation. NIST Level 5 collision resistance (256-bit against Grover-birthday) applies to address identity, matching the chain-wide hash function commitment per Post-Quantum Cryptography. Users see addresses as Bech32 strings with HRP “s1q” per SPX-0 — the user-facing encoding is Bech32 wrapping the 64-byte address with the standard Bech32 BCH checksum. The full key bundle is revealed by the holder at the first UNLOCK at their HOME address and recorded in the Bundle Registry, a consensus-state object keyed by HOME address. Parcels themselves carry only owner_address; the bundle lives in chain state, queried by validating nodes when signature verification is required. Stealth-derived addresses do not register independent bundles; their spending is authorised via the holder’s HOME bundle plus a commitment_opening, per Stealth Settlement and Spend Authorisation. The mechanism is described in detail in Public Key Binding at First UNLOCK.

Stealth addresses, holder-controlled audit keys, and post-quantum key encapsulation (Kyber/ML-KEM) are available from block zero. No activation event. No reserved fields. No coordinated upgrade. Scriptum ships with the full cryptographic stack live.

The address format is fixed from genesis. Any future structural change requires a hard fork under the Governance Charter.

ESCROW addresses are protocol-controlled wallets used for Horkos. They have no user private key. Funds at ESCROW addresses move only through the protocol-driven resolution sequence triggered by attestation outcome or timeout — UNLOCK followed by EJECT followed by SEND, executed by the protocol rather than by user signatures. Neither the parties (A, B), nor the attestor (C), nor the Foundation, nor Curia can move an ESCROW Parcel discretionarily. Routing is determined entirely by the outcome of the trade per the agreement signed at the start of the contract. ESCROW addresses carry full post-quantum binding structure identical to user addresses; they are first-class Scriptum addresses, differing only in that no one holds the private key.

Key Architecture — Spend, View, Recovery

Scriptum’s cryptographic identity is not a single key. A holder controls three independent authorities, each derived from the same mnemonic via a distinct HKDF domain separator, each governing a separate question.

Spend Tree governs authorization — who may move value. It derives the holder's Dilithium3 signing keys and the bundle that binds the holder's HOME address. The chain-visible footprint of the Spend Tree is the Bundle Registry, consulted only when value moves.

View Tree governs visibility — who may see. It derives a Master View Secret per HOME, then per-generation roots, then per-epoch ML-KEM-768 keypairs. The chain-visible footprint of the View Tree is the View-Key Registry, consulted only when receiving, scanning, auditing, or selectively disclosing.

Recovery Tree governs continuity — how a holder recovers from device loss or passes authority to heirs. It derives multi-tier recovery material backing the Aurum Lifeboat. The Recovery Tree's specifics are wallet-layer concerns specified in the Aurum technical paper, not consensus-layer concerns.

The three trees are cryptographically independent. Compromise of one does not compromise the others. Disclosure of one does not imply disclosure of another. Possession of one grants no rights over the other two.

The Three Questions

The architecture answers three operational questions with three separate authorities:

- **Who may spend?** — Spend Tree, via Bundle Registry
- **Who may see?** — View Tree, via View-Key Registry
- **Who may recover?** — Recovery Tree, via Lifeboat archive

The three questions can be answered for the same holder by different parties. A holder may delegate visibility to an auditor without granting spend authority; may share recovery material with trustees without granting visibility; may rotate visibility material without affecting spend authority. Each delegation is bounded by the cryptographic structure of its tree.

Spend Tree

The Spend Tree derives Dilithium3 signing material:

```
Spend_Material = HKDF-SHA3-512(mnemonic_seed, "scriptum.spend")
```

The bundle composition is authorisation material only:

- **version (1 byte)** — bundle format version; Genesis 0x01
- **Dilithium3 public key (~1,952 bytes)** — signature verification

The HOME address is the SHA3-512 hash of the bundle. The Bundle Registry stores the bundle keyed by HOME address. See Public Key Binding at First UNLOCK.

A spending transaction is authorised by a Dilithium3 signature verified against the bundle in the Bundle Registry. The signature verifies the spender controls the HOME's private key. The accompanying spend_commitment binding on the Parcel verifies the Parcel was

destined for this specific Dilithium pubkey. See Stealth Settlement and Spend Authorisation.

View Tree

The View Tree derives the holder's per-HOME visibility hierarchy:

```
Master_View_Secret = HKDF-SHA3-512(mnemonic_seed, "scriptum.view.master")
```

```
Generation_Root(g) = HKDF-SHA3-512(Master_View_Secret,  
"scriptum.view.generation" || g)
```

```
Epoch_Secret(g, Y) = HKDF-SHA3-512(Generation_Root(g), "scriptum.view.epoch"  
|| Y)
```

The Epoch_Secret seeds ML-KEM-768 key generation, producing the per-(generation, year) Kyber keypair. The public key is published on-chain via PUBLISH_EPOCH_PUBKEY and registered in the View-Key Registry. The secret remains with the holder.

The View-Key Registry stores per-HOME view-authority state — the sequence of generations (advancing with each CANCEL_VIEWKEY rotation) and per-(generation, year) published epoch Kyber pubkeys. It is a chain-state object parallel to the Bundle Registry, keyed by HOME address, populated by different transactions, governing different authorities.

See Selective Disclosure for the full specification.

Recovery Tree

The Recovery Tree governs continuity:

```
Recovery_Material = HKDF-SHA3-512(mnemonic_seed, "scriptum.recovery")
```

The Recovery Tree backs the Aurum Lifeboat: a multi-tier sovereign recovery archive supporting personal recovery, trustee/inheritance recovery, institutional recovery, and governance recovery, with optional enhancements including Shamir Secret Sharing, geographic share distribution, and configurable deadman-switch protocols.

Recovery Tree specifics are wallet-layer concerns. The chain neither sees nor stores recovery material. Full specification is in the Aurum technical paper. See also Wallet-Layer Encryption later in this document.

The Separation Doctrine

If the Spend Tree leaks, the attacker can spend. This is the highest-security tier; it must be protected accordingly.

If the View Tree leaks, the attacker can see — read history, decrypt memos, audit activity within the disclosed scope. The attacker cannot spend, cannot derive spend keys, cannot move funds.

If the Recovery Tree leaks, the attacker may gain recovery rights depending on the holder's chosen tier structure. The attacker cannot directly spend or view without also obtaining the corresponding Spend or View material, unless the holder's recovery policy explicitly couples them.

Each authority can be delegated, rotated, or revoked according to its own rules:

- **Visibility** can be delegated per-year (PUBLISH_EPOCH_PUBKEY scope) or per-generation (master rotation via CANCEL_VIEWKEY)
- **Recovery** can be split across multiple trustees via Shamir Secret Sharing, with configurable thresholds
- **Spend authority** is permanent at the HOME address: the bundle is hash-bound to the address by SHA3-512, and rotating Dilithium keys means abandoning the address

This is a stronger architecture than a single monolithic bundle attempting to perform all three roles. An earlier design carried Kyber and view-key material in the bundle, coupling visibility to authorisation in ways that made selective disclosure cryptographically awkward and key rotation across visibility/spend boundaries impossible. The three-tree separation is the principled correction.

Cross-References

- **Spend Tree → Bundle Registry** — see Public Key Binding at First UNLOCK
- **View Tree → View-Key Registry** — see Selective Disclosure
- **Recovery Tree → Lifeboat** — see Wallet-Layer Encryption and the Aurum technical paper

Public Key Binding at First UNLOCK

Post-quantum cryptography has a throughput problem. NIST-standardised lattice signatures (Dilithium3) and lattice key-encapsulation (Kyber768) provide the security guarantees the next century requires, but at a cost: a Dilithium3 public key is approximately 1,952 bytes, a Kyber768 public key approximately 1,184 bytes, and a Dilithium3 signature approximately 3,293 bytes. A naïve lattice-based chain that reveals the full public-key bundle in every signed transaction pays roughly five to six kilobytes of cryptographic overhead per transaction — an order of magnitude more than ECDSA-based systems.

Without an architectural answer, this overhead dominates bandwidth consumption and constrains throughput. The architectural answer in Scriptum is public-key binding via address identity.

The address is the cryptographic commitment.

The bundle is authorisation material only. Two components:

- **version (1 byte)** — bundle format identifier. Genesis 0x01. Constitutional; future bundle format evolutions use a new version value and addresses derived under different versions cannot collide.
- **Dilithium3 public key (~1,952 bytes)** — signature verification material.

Total bundle size: ~1,953 bytes.

The bundle does NOT contain Kyber material or view-key material. Stealth derivation and visibility are governed by the View Tree (View-Key Registry); see Key Architecture — Spend, View, Recovery and Selective Disclosure. The bundle’s sole purpose is to bind a HOME address to its Dilithium3 signing authority.

`Address = SHA3-512(bundle) = SHA3-512(version || Dilithium3 public key)`

First-touch UNLOCK at a HOME address reveals the bundle for entry into the Bundle Registry. The chain verifies that the revealed bundle hashes to the claimed HOME address. From that block forward, the bundle is in chain state and is consulted on every subsequent transaction signing for that HOME.

An alternative design would carry the full bundle through every descendant Parcel.

While functional, that approach would inflate the Parcel set with duplicated bundle storage — every Parcel at a first-touched address carrying a full ~3,200-byte copy of the bundle. For an active holder with many Parcels at a single address, the duplication grows quadratically with use.

Scriptum instead stores the validated bundle once in the Bundle Registry and thereafter references it through the address identity.

The result is:

Reveal Once → Verify Once → Store Once → Reference Thereafter

which localises the post-quantum bandwidth cost to the first-touch event and keeps the Parcel set lean.

Multi-Address Consolidation

An UNLOCK transaction may consume input Parcels originating from multiple distinct addresses simultaneously.

This property is essential for stealth-compatible consolidation. Without it, a holder accumulating value across many one-time stealth-derived addresses would be unable to consolidate those Parcels efficiently.

The chain verifies the holder’s authority via the HOME bundle in the Bundle Registry. The transaction names the holder’s HOME address; the chain looks up HOME’s bundle and verifies a single Dilithium3 signature against HOME’s Dilithium3 public key. For each input Parcel at a stealth-derived address, the transaction includes a `commitment_opening` (per Stealth Settlement and Spend Authorisation) that the chain verifies against the input

Parcel's `spend_commitment`. Inputs at the holder's HOME address still require the universal `spend_commitment` opening; the 'every Parcel is the same' invariant applies uniformly. For HOME-class inputs the opening value is a domain-separated public constant identifying the origin context. No Parcel class is exempt from layer-1 commitment verification.

The `commitment_opening` is the per-input authorisation value that lets the chain reconstruct and verify the input's `spend_commitment`. For stealth-derived inputs, the opening is the `ss_derivative` (a secret value the recipient derives at receive time from the KEM shared secret via `HKDF-SHA3-512(ss, "scriptum.spend-binding")`). For HOME-class inputs (change outputs, coinbase, consolidation, vesting release, etc.), the opening is a domain-separated public constant identifying the input's origin context. There is one `commitment_opening` per input. The transaction's single Dilithium3 signature covers the whole transaction; per-input authorisation is established by the per-input `commitment_opening` combined with the bundle's Dilithium pubkey via the `spend_commitment` hash reconstruction.

The transaction is valid if and only if every source address represented in the inputs provides a valid signature.

The resulting NEXUS-dwelling Parcel is delivered to the holder's HOME address.

Any LOCKED remainder is likewise delivered to HOME.

The fee remains a single `f_U` calculated on the total unlocked value, independent of input count or source-address count.

Stealth Settlement

Every SPEND in Scriptum is stealth. The sender performs a Kyber/ML-KEM encapsulation against the recipient's Kyber public key (obtained from the recipient's View-Key Registry entry (the Tier 1 chain-state object holding Kyber and view-side material; the bundle holds only Dilithium signature material)), produces a stealth-derived destination address from the shared secret, and includes the encapsulation ciphertext in the transaction. There is no non-stealth mode; every settlement is private by default. The full cryptographic construction, including domain-separated KDF derivation of the message and spend-binding keys, the `spend_commitment` placed on each stealth-derived output Parcel, and the `commitment_opening` required at spend time, is specified in Stealth Settlement and Spend Authorisation.

Validation

When validating an UNLOCK transaction, a node:

- 1. Groups input Parcels by source address.
- 2. Retrieves the authoritative bundle from the Bundle Registry for each address.
- 3. Verifies a Dilithium signature for each represented address.
- 4. Verifies that the destination HOME address is correctly committed.

- 5. Applies the state transition.

No external registry, oracle, cache, or third-party service is required.

Validation is entirely deterministic from the transaction, the consumed Parcels, and the Bundle Registry.

Design Properties

- **Immutability.** The address is the hash of the bundle. Substituting a different bundle necessarily produces a different address. Once recorded in the Bundle Registry, a bundle is immutable; replacement is rejected by consensus.
- **One-time revelation.** The bundle is revealed once per address and thereafter referenced through the Bundle Registry.
- **No identity registry.** The Bundle Registry contains only cryptographic key material required for verification. No names, accounts, KYC information, or external identity records exist within the protocol.
- **Deterministic validation.** Every validating node derives the same bundle for a given address from the Bundle Registry.
- **Reorg safety.** The Bundle Registry is consensus state and reorganises automatically with the chain. If a first-touch UNLOCK is rolled back, the corresponding binding is rolled back with it via the standard reorg machinery.
- **Settlement efficiency.** The chain pays the post-quantum public-key cost once at address introduction rather than on every future movement of value.

Rationale

Without address-bound public-key registration, every signed operation on a post-quantum Parcel chain would repeatedly transmit the same large public-key material. Throughput would become dominated by redundant cryptographic data rather than settlement activity.

Scriptum therefore chooses a third path between repeated public-key revelation and stateful one-time signatures.

Large public keys are revealed once, cryptographically bound to the address, recorded in the Bundle Registry, and thereafter referenced through that address identity.

The chain pays the post-quantum tax once at introduction, not every time value moves.

Architectural Foundation — The UTXO Model

Scriptum's structural foundation is the UTXO architecture inherited from Bitcoin's transaction model. Value is held in discrete, value-carrying entries — Parcels — each with its own owner address and amount. The chain's consensus state is the live Parcel set. Transactions consume Parcels as inputs and create new Parcels as outputs. Every transition is consume-and-create: input Parcels cease to exist, output Parcels are created fresh.

This foundation is inherited from Bitcoin without modification at the architectural level. The terminology was renamed (UTXO → Parcel) when Scriptum was migrated to its current canonical vocabulary, but the underlying model is the same:

- **Parcel identification.** Every Parcel is uniquely identified by its Outpoint — the pair (transaction hash, output index) of the transaction that created it.
- **Transaction references.** Transactions reference inputs by Outpoint. The chain looks up each referenced Parcel in the Parcel set during validation.
- **Consume-and-create.** A transaction consumes its referenced input Parcels (removing them from the Parcel set) and creates new output Parcels (adding them to the Parcel set). No Parcel is mutated in place; transitions always produce new Parcels.
- **Holder balance.** A holder's value at an address is the sum of amount over every Parcel in the Parcel set whose owner_address equals that address. There is no account ledger; value is held by the Parcels themselves.
- **Consensus state.** The Parcel set is the authoritative chain state. Every validating node maintains it. Block validation is the application of a block's transactions to the Parcel set as a sequence of consume-and-create operations.

This foundation supplies the storage model, transaction structure, and validation flow. What follows in subsequent sections — the (CLASS, ROLE, STATE) attribute system, state-specific fields per state, multi-state lifecycle, protocol-driven transitions, the sealed sender memo, and the post-quantum binding model — are Scriptum's specific extensions to this UTXO foundation. Every Parcel is a UTXO; what distinguishes Scriptum from Bitcoin is what each Parcel carries and how its state evolves.

The Universal Parcel Principle

Scriptum's protocol recognises only Parcels. Not HOME Parcels, not stealth Parcels, not constitutional Parcels, not vault Parcels — only Parcels. Differences in handling emerge from protocol rules applied at verification time, not from differences in the underlying object.

Likewise, Scriptum's protocol recognises only addresses. Not HOME addresses, not stealth addresses, not constitutional addresses, not vault addresses — only addresses. Differences emerge from how the protocol routes authorisation based on the addresses' role in chain state, not from differences in the addresses themselves.

This is the chain's foundational architectural commitment.

Every Parcel is the same

Every Parcel carries the same consensus structure:

A class enum (CIRCULATING, TREASURE, VAULT, MARKER)

A role enum (semantic purpose within the class)

A state (PERMA_LOCKED, LOCKED, ACTIVE, depending on class)

An address (64-byte SHA3-512, Bech32 with HRP "s1q")

An amount
A creation height, offset, expiry height
A spend commitment
A Kyber ciphertext (when stealth-encrypted)
An epoch year (when stealth-encoded)
A payload (when carrying additional data)
Other fields as specified in the Parcel Data Model

Every spendable Parcel carries a `spend_commitment`. Every spend opens its commitment via the appropriate Dilithium signature. This is universal — no exceptions for HOME, stealth-derived, constitutional, or any future Parcel class.

Visibility, ownership, authorisation: three separate mechanisms

The protocol's verification machinery distinguishes three questions, each answered by a different mechanism:

Who may see? — View Tree — View-Key Registry; Kyber768 key derivation
Which authority is this Parcel bound to? — `spend_commitment` — per-Parcel commitment binding the authority
Does that authority approve the transaction? — Dilithium signature — witness over the commitment, verified per Dilithium3

These three mechanisms operate independently. View access does not imply ownership; ownership binding does not imply authorisation; authorisation requires fresh proof at each transaction.

The clean separation means:

A holder may grant visibility (view key delegation) without granting spend authority
A constitutional position-holder may be replaced (slot key swap) without invalidating prior delivered TREASURE
VAULT Parcel authority can live in the Council Registry's locks rather than in the bundle's private key — the bundle is a layer-1 ceremony, the locks are the layer-2 authority gate

What this excludes

The protocol intentionally avoids:

Separate consensus pathways per Parcel class
Privileged Parcels that bypass standard verification
"Magic" addresses with special powers at the verification layer
Class-specific cryptographic primitives
Special-case ownership rules

The chain validates every Parcel through the same machinery. Differences in handling (vesting via Box mechanism for TREASURE class; lock-based authorisation via Council Registry for VAULT class; standard bundle-based authorisation for CIRCULATING class;

non-spendable identity for MARKER class) are protocol rules that apply at verification time based on class and role. They are not exceptions to the verification machinery; they are the verification machinery acting on each Parcel's declared identity.

Why this matters

Systems survive decades through engineering. Systems survive centuries through simplicity.

The chain's century-scale lifetime — Year 100 + 1 day Ragnarök, multiple Phoenix transitions, generations of validators, decades of cryptographic evolution — requires that every component be simple enough to remain understood, audited, and trusted across that span.

Each architectural exception is a long-term liability: code paths to maintain, edge cases to test, ambiguities to resolve, and surfaces where mistakes can hide. The Universal Parcel Principle is the chain's commitment to keep these surfaces minimal.

Fewer exceptions. Fewer special cases. Fewer consensus pathways. One coherent machine — recognisable across the decades, auditable in its entirety, and small enough that a single mind can hold its full shape.

The principle binds the chain's other architectural commitments into one coherent shape: source-retention (UNLOCK output lands at the source address, not at HOME); class-driven authorisation (VAULT Parcels pass layer-1 uniformly, with layer-2 lock-based authority via the Council Registry); HICP indexing applied uniformly to every EUR-denominated value across the protocol; and the universal spend_commitment that closes the last HOME exception. Each commitment compounds the same principle. Now the principle has a name.

The Parcel Data Model

This section specifies the Parcel as a data structure. Every Parcel in Scriptum is an instance of this specification.

Universal Parcel Structure

Every Parcel carries the same 16 fields. There are no conditional field categories. Fields not applicable for a given Parcel hold empty or sentinel values (zero-filled byte strings, empty strings, NONE-valued enums).

The universal fields are:

- **class** — enum: CIRCULATING / TREASURE / VAULT / MARKER. Defines the economic identity of the value.

Class drives authorisation routing. The chain reads each Parcel's class at signature-verification time to determine the appropriate authorisation path. CIRCULATING Parcels use standard bundle-based authorisation (spend_commitment + Dilithium signature against the HOME's bundle). VAULT Parcels add class-driven lock-based authorisation: chain consults the Council Registry per the role's hardcoded rules in addition to the

standard signature check. TREASURE Parcels use the SlotBox Registry's Box mechanism per the vesting transition. MARKER Parcels are non-spendable and do not pass through standard authorisation. The Parcel's 16-field structure is uniform across classes; the chain's routing logic differs based on class. The 'every Parcel is the same' constitutional primitive applies at field-level uniformity; per-class behaviour applies at validation time.

- **role** — enum: NONE / WOC / FOUNDER / CURIA / RESERVE / WFC / FUNDS / CP / MAYFLOWER / DANA / INSCRIPTION. Defines governance affiliation or record type.
- **state** — enum: LOCKED / COINBASE / NEXUS_DWELL / ACTIVE / SEALED / OPEN / PACKING / ARC / PERMA_LOCKED. Defines current operational behaviour.
- **address** — 64 bytes. The location at which this Parcel sits in the chain's address space. Either a registered HOME address, a stealth-derived address, or a constitutional namespace identifier (slot HOME, vault HOME).
- **amount** — u64. Value in Nuggets. Cleartext.
- **memo** — variable bytes. Encrypted blob; may be empty. Carries sender-to-recipient communication or protocol-set origin labels.
- **creation_height** — u64. Block at which this Parcel came into existence.
- **offset** — u64. Blocks from creation_height to expiry, entered at creation time. 0 for immediately-spendable Parcels; MAX_U64 for never-expiring Parcels (PERMA_LOCKED).
- **expiry_height** — u64. The absolute block at which the Parcel's time-bounded behaviour fires. Equal to creation_height + offset.
- **spend_commitment** — 64 bytes. SHA3-512 hash binding the Parcel to a specific Dilithium pubkey via context-specific opening. Universal across all Parcel classes.
- **kyber_ciphertext** — ~1,088 bytes. ML-KEM-768 ciphertext, encrypted to the recipient's epoch Kyber pubkey from the View-Key Registry. Provides discovery mechanism for the recipient and audit visibility for delegated viewers.
- **epoch_year** — u16. The year (in UTC) of Parcel creation. Recovery hint for the View Tree.
- **payload** — variable bytes. Role-specific content; empty for role = NONE. Carries INSCRIPTION metadata or other role-defined data.
- **path** — u8. Protocol routing flag. Values: NONE (0x00) / UNLOCK_DWELL (0x01) / SPEND_PRIVATE_HOT (0x02) / SPEND_PRIVATE_COLD (0x03); 0x04-0xFF reserved.
- **cancel_hash** — 32 bytes. Hash for the hash-preimage CANCEL path; zero-filled if absent.
- **recipient_address** — 64 bytes. Future destination address for protocol-routing transitions (slot 2); zero-filled if absent.

The “Two Address Slots” Pattern

Every Parcel carries two address-typed fields: address (where the Parcel currently sits) and recipient_address (where it is destined to go after a protocol-routing transition). The path field tells the protocol which slot to use:

- **path = NONE** : no routing; Parcel remains at address until consumed by user or protocol
- **path = UNLOCK_DWELL** : at expiry, EJECT delivers to address (slot 1) as ACTIVE
- **path = SPEND_PRIVATE_HOT** : at expiry, EJECT delivers to recipient_address (slot 2) as ACTIVE
- **path = SPEND_PRIVATE_COLD** : at expiry, EJECT delivers to recipient_address (slot 2) as LOCKED

Class-State Validity

Each class supports specific states:

- **CIRCULATING:** LOCKED, COINBASE, NEXUS_DWELL, ACTIVE, PACKING, ARC
- **TREASURE:** LOCKED (with vesting offset until matured), PACKING, ARC
- **VAULT:** SEALED, OPEN, PACKING, ARC
- **MARKER:** PERMA_LOCKED

Spend Commitment Construction

Every Parcel's spend_commitment is computed at creation time as:

```
spend_commitment = SHA3-512(holder.Dilithium_pubkey || opening_value ||
address || outpoint)
```

In this formula, holder.Dilithium_pubkey is the Dilithium pubkey of the entity who will spend the Parcel — for stealth SPEND outputs, this is the recipient's Dilithium pubkey (the "future holder"); for HOME-class outputs (change, coinbase, consolidation, etc.), this is the originating holder's own pubkey. The address is the address the Parcel sits at — for stealth SPEND outputs, the stealth-derived address derived from the recipient's Kyber key via KEM; for HOME-class outputs, the holder's HOME address. The outpoint is the unique (txid, index) tuple identifying this specific Parcel.

The holder.Dilithium_pubkey is the public key of the entity authorised to spend the Parcel — either a real holder's pubkey (from their bundle in the Bundle Registry) or a constitutional constant (for protocol-only Parcels at slot HOMES).

The opening_value is context-specific:

For value-carrying stealth outputs (from SPEND transactions):

- **opening** = ss_derivative = HKDF-SHA3-512(ss, "scriptum.spend-binding")
- **secret to recipient** ; recipient derives from KEM shared secret

For protocol-driven creations (public openings):

- **COINBASE** → "scriptum.coinbase"
- **Change output** → "scriptum.change"
- **Consolidation (UNLOCK, SWEEP, Menuha)** → "scriptum.consolidation"
- **Vesting release** → "scriptum.release"

- **Genesis allocation** → "scriptum.genesis"
- **ATTEST Parcel** → "scriptum.attest"
- **INSCRIBE Parcel** → "scriptum.inscribe"
- **User-initiated time-lock** → "scriptum.timelock"

For constitutional Parcels at slot HOMES:

- **opening** = "scriptum.constitutional"
- **holder.Dilithium_pubkey** is a constitutional constant (the chain knows no real Dilithium privkey corresponds to it; these Parcels are protocol-vest-only)

Spending Validation — Uniform Path

A spending transaction contains:

- **input outpoint(s)**
- **claimed home_address**
- **Dilithium3 signature**
- **commitment_opening per input**

The chain validates each input identically:

- 1. bundle = Bundle_Registry[home_address]
- 2. verify Dilithium signature against bundle.Dilithium_pubkey
- 3. for each input:
 - reconstructed = SHA3-512(bundle.Dilithium_pubkey || commitment_opening || input.address || input.outpoint)
 - require reconstructed == input.spend_commitment
- 4. verify state-based spendability (current_block_height >= input.expiry_height for matured-Parcel states, or appropriate state rule)
- 5. verify conservation: sum(input.amount) == sum(output.amount) + fees
- 6. accept

The same code path handles all Parcels — those at HOME addresses, those at stealth-derived addresses. The spend_commitment's binding to the holder's Dilithium pubkey is the universal cryptographic anchor.

Constitutional Slot Table

Fourteen addresses are reserved at Genesis for the constitutional vesting positions. These are NOT registered in the Bundle Registry. They are deterministic namespace identifiers derived from public constants:

slot_HOME_FOUNDER_n = SHA3-512("scriptum.slot.founder_" || n) for n in [1..5]

slot_HOME_CURIA_n = SHA3-512("scriptum.slot.curia_" || n) for n in [1..9]

These 14 addresses have the following properties enforced by the chain:

- **No Bundle Registry entry** exists for them
- **No first-touch UNLOCK** can register a bundle (preimage attack against SHA3-512 is infeasible)
- **Parcels at these addresses** are consumable ONLY by protocol-driven vest transitions
- **The Council Registry** governs allocation decisions for Parcels at these addresses (via the Box mechanism — see §TREASURE Vesting → The Box Mechanism)

TREASURE Parcels at slot HOMEs carry spend_commitments bound to constitutional constants (not to any real Dilithium privkey). No user signature can spend them. The vest transition is the sole consumption path.

In addition to the 14 constitutional slot HOMEs, one further address in the same namespace family serves as the BURN destination — the value sink for permanently removing value from circulating supply:

Gehenna_HOME = SHA3-512("scriptum.gehenna")

Gehenna_HOME shares the same structural properties as the constitutional slot HOMEs: no Bundle Registry entry exists for it; no first-touch UNLOCK can register a bundle (SHA3-512 preimage attack against the namespace constant is infeasible); no Dilithium privkey corresponds to it. Parcels sent to Gehenna_HOME are permanently unspendable.

A BURN transaction is structurally a SPEND with destination = Gehenna_HOME. The output Parcel carries the burned amount, lives at Gehenna_HOME indefinitely (offset = MAX_U64; never matures, never expires), and contributes to the chain's S_burned supply accounting. The Parcel's existence at Gehenna_HOME is the on-chain audit record that this value was deliberately removed from circulation.

BURN is distinct from Lethe. BURN is a deliberate, signed, on-chain act with a recorded amount and timestamp; the chain's S_burned counter tracks it exactly. Lethe is unknowable value loss — keys lost, holders deceased, Parcels never claimed — and is not directly tracked by the chain. The supply identity is: circulating supply = S_mined – S_burned – S_lethe, where S_burned is known exactly and S_lethe is asymptotically estimable but never exactly computed.

The fee cascade and protocol-driven value movements use additional constitutional namespace addresses, all constructed by the same SHA3-512 pattern:

VAULT_HOME = SHA3-512("scriptum.vault") // per-block 10% fee aggregation

FUND_HOME = SHA3-512("scriptum.fund") // operating budget destination

RESERVE_HOME = SHA3-512("scriptum.reserve") // deep reserve destination

TITHE_HOME = SHA3-512("scriptum.tithe") // pre-quarterly tithe pool

CHARITY_POT_HOME = SHA3-512("scriptum.charity_pot") // charity distribution

WFC_HOME = SHA3-512("scriptum.wfc") // WFC programmatic allocation

These constitutional namespace addresses share all the structural properties of the slot HOMEs and Gehenna_HOME: no Bundle Registry entry, no first-touch UNLOCK possible, no Dilithium privkey, protocol-only consumption. Parcels at these addresses are consumed by protocol-driven transitions (block aggregation, Menuha separation, quarterly distribution) as specified in the Fee Cascade section. The Constitutional Slot Table thus contains 14 vesting slot HOMEs + Gehenna_HOME + 6 fee-cascade HOMEs = 21 constitutional namespace addresses in total.

Additional constitutional namespace addresses may be added by constitutional amendment if future governance mechanics require them.

Field-Level Invariants

The following invariants hold at every block boundary for every Parcel:

- **amount is non-negative** (u64 by type)
- **state is valid for the Parcel's class** (per Class-State Validity)
- **creation_height** <= current block height
- **expiry_height** == creation_height + offset
- **For Parcels at registered HOME addresses (in Bundle Registry):**
spend_commitment was constructed using the bundle's Dilithium pubkey
- **For Parcels at stealth-derived or slot HOME addresses:** spend_commitment was constructed using the appropriate holder's pubkey or constitutional constant
- **path** is a valid enum value
- **recipient_address** is non-empty when path != NONE
- **cancel_hash** is 32 bytes (zero-filled if not in use)
- **kyber_ciphertext** is well-formed (decapsulable by the intended recipient's epoch secret)

The Universal Time Model

Every Parcel knows when it was created (creation_height) and when something happens to it (expiry_height). The relationship between these is fixed by the offset entered at creation:

$$\text{expiry_height} = \text{creation_height} + \text{offset}$$

The interpretation of “what happens at expiry_height” depends on the state:

- **LOCKED with offset > 0** : becomes spendable at expiry_height
- **COINBASE** : matures to LOCKED at expiry_height (creation_height + 100)
- **NEXUS_DWELL** : EJECTs at expiry_height (creation_height + 8,640) per the path field
- **ACTIVE** : EXPIRES to LOCKED at HOME if not consumed by expiry_height (creation_height + ACTIVE_WINDOW_BLOCKS)
- **OPEN** : RESEALS at expiry_height (creation_height + OPEN_WINDOW_BLOCKS)
- **PERMA_LOCKED** : never; expiry_height = MAX_U64

Memo Lifecycle

The memo field on every Parcel is encrypted with the `message_key` derived from the KEM shared secret at creation. For protocol-driven creations (COINBASE, vesting release, consolidation, etc.), the protocol encapsulates against the recipient's epoch Kyber pubkey and constructs a protocol-set memo string identifying the origin.

Protocol-set memo strings:

- **Coinbase reward** — “mī Hayyānu b’ahabā” (Ugaritic: “from Hayyānu with love”)
- **Vesting release** — “Donum de DrFunkenstein” (Latin: “Gift from DrFunkenstein”)
- **Menuha consolidation** — “Frau Holle lässt grüßen” (German: “Frau Holle sends greetings”)
- **EJECT output post-NEXUS** — “Nexus dedit” (Latin: “Nexus has given”)
- **Change output** — “left over”

For sender-initiated SPEND transactions, the sender's wallet encrypts the user-provided memo content with `message_key` derived from the same KEM shared secret.

The memo is consumed when the Parcel is consumed. It does not propagate to successor Parcels at the same address unless explicitly copied (e.g., EJECT may inherit the memo from the consumed NEXUS_DWELL Parcel to its output).

Relationship to Downstream Documents

The Canonical Machine specifies transitions in terms of these fields. The System Parameters specify the time constants (COINBASE_MATURITY_BLOCKS, NEXUS_DWELL_BLOCKS, ACTIVE_WINDOW_BLOCKS, OPEN_WINDOW_BLOCKS, MIN_TIMELOCK_BLOCKS). The Invariants and Assurance document specifies testable properties grounded in this data model.

Stealth Settlement and Spend Authorisation

Every SPEND in Scriptum produces an output at a stealth-derived destination address. The recipient receives privately; external observers without the recipient's view key cannot link the output to the recipient's HOME identity. When the recipient later acts on the value — by spending it or having it swept by Menuha to HOME — the linkage between the stealth Parcel and HOME becomes public. This section specifies the cryptographic construction by which stealth outputs are created, how recipients identify them, and how they are subsequently authorised for spending.

The Cryptographic Construction

When Alice sends value to Bob, Alice performs an ML-KEM-768 encapsulation against Bob's current-epoch Kyber768 public key. This pubkey is retrieved from the View-Key Registry — a chain-state object parallel to the Bundle Registry, holding per-(HOME, generation, year) Kyber pubkeys. Alice's wallet looks up the appropriate epoch pubkey for the inclusion year and Bob's current view-authority generation.

```
(ss, ct) = ML-KEM-Encaps(Bob.current_epoch_kyber_pubkey)
```

The encapsulation produces a shared secret `ss` and a ciphertext `ct`. Both are derived from Alice's ephemeral randomness combined with Bob's public material — neither party alone can predict the result.

From the shared secret, Alice derives via HKDF-SHA3-512:

```
stealth_address = derived from ss per the address derivation function
```

```
message_key = HKDF-SHA3-512(ss, "scriptum.message")
```

```
ss_derivative = HKDF-SHA3-512(ss, "scriptum.spend-binding")
```

Alice also looks up Bob's Dilithium pubkey from the Bundle Registry. Combining these, she constructs the `spend_commitment`:

```
spend_commitment = SHA3-512(Bob.Dilithium_pubkey || ss_derivative ||  
stealth_address || outpoint)
```

The Parcel Alice creates carries:

- **address** = `stealth_address` (one-time, derived from `ss`)
- **spend_commitment** (binds to Bob's Dilithium pubkey + `ss_derivative` + Parcel identity)
- **kyber_ciphertext** = `ct` (lets Bob discover the Parcel)
- **epoch_year** = current year
- **memo** = `encrypt(message_key, sender's note)` — may be empty
- **amount** = the sent value
- **... other universal fields** populated normally

Recipient Discovery

Bob's wallet scans the chain via the View-Key Registry. For each SPEND output, Bob's wallet attempts ML-KEM-Decaps against the output's `kyber_ciphertext` using Bob's epoch secret for the year of creation:

```
ss = ML-KEM-Decaps(ct, Bob.epoch_secret_for_year)
```

On successful decapsulation, Bob recovers the shared secret. He verifies the recovered `ss` derives the correct stealth address (matching the Parcel's address field). He derives `message_key` and decrypts the memo. He derives `ss_derivative` and stores it locally — he will need it when later spending this Parcel.

If decapsulation fails or the derived address mismatches, the Parcel is not Bob's; Bob's wallet skips it.

The Mallory Defence

Mallory observes Bob's stealth-derived Parcel on chain. She sees:

- **The stealth address** (a 64-byte hash)
- **The kyber_ciphertext** (KEM ciphertext)
- **The spend_commitment** (a 64-byte hash)
- **The other public fields**

She wants to spend the Parcel.

Attack 1: Mallory submits a spending transaction naming her own HOME.

The chain looks up Mallory's bundle (with her Dilithium pubkey), reconstructs spend_commitment using Mallory.Dilithium_pubkey + the opening she provides + the Parcel's address + the outpoint. The reconstructed commitment does NOT match the Parcel's spend_commitment (which was computed with Bob's pubkey). The chain rejects.

Attack 2: Mallory submits a spending transaction naming Bob's HOME.

The chain looks up Bob's bundle, reconstructs spend_commitment using Bob.Dilithium_pubkey. If Mallory provided the correct opening (ss_derivative), reconstruction matches. But signature verification fails — Mallory's signature was made with her private key, not Bob's. The chain rejects.

Mallory cannot extract ss_derivative without Bob's epoch private key (which is in Bob's wallet, never on chain). She cannot forge Bob's Dilithium signature. Both attacks fail.

The defence is symmetric: the spend_commitment is a hash binding to BOTH the holder's Dilithium pubkey (proves "Bob is the authorised spender") AND a per-Parcel opening (proves the specific Parcel was constructed for Bob). Each input requires both proofs.

Spending Transaction Structure

A spending transaction targeting stealth-derived inputs contains:

- **Input outpoint(s)**
- **home_address** : Bob's HOME (explicit; consumed via Bundle Registry lookup)
- **Dilithium3 signature** over the transaction
- **For each input** : commitment_opening (ss_derivative for stealth inputs; or the appropriate public constant for HOME-class inputs)

The chain validates per the uniform path documented in the Parcel Data Model section. There is no special-case path for stealth versus HOME-class inputs; the same Bundle Registry lookup + signature check + commitment reconstruction applies uniformly.

Privacy and Linkage

The privacy property is address-unlinkability before spending: an observer cannot link a stealth-derived address to a specific HOME without breaking ML-KEM (a cryptographically hard problem).

The privacy property ends at spending: when Bob spends the Parcel, his transaction names his HOME publicly. The chain learns that the previously-private stealth address was indeed Bob's. Observers see "Bob's HOME spent the Parcel that was at stealth address X."

This is intentional. The chain's privacy model is "private receipt, private accumulation, public linkage at spend." Holders who require stronger privacy (e.g., post-spend unlinkability) must compose additional primitives at the wallet layer — multiple home addresses, view-key delegation.

Cost and Properties

The construction adds minimal overhead relative to the cryptographic baseline:

- **spend_commitment on each stealth Parcel:** 64 bytes (one SHA3-512 output)
- **commitment_opening** per stealth input in spending transactions: 64 bytes (the `ss_derivative` value, computed by the wallet as `HKDF-SHA3-512(ss, "scriptum.spend-binding")` from the KEM shared secret at receipt and retained locally; presented at spend time as the `commitment_opening` field that the chain verifies against the Parcel's `spend_commitment`)
- **home_address in spending transactions:** 64 bytes

Total per spend from a single stealth input: approximately 128 bytes of stealth-authorisation overhead beyond what a standard signed transaction already carries. This is negligible compared to the ~3,300-byte Dilithium3 signature already present in every spend.

The construction preserves these properties simultaneously:

- **HOME-centric Bundle Registry.** Bundle state scales with identities (holders), not with transactions. One million SPENDs per day do not create one million new Bundle Registry entries.
- Message encryption survives spend. The `message_key` is derived from a different domain-separated KDF branch (`HKDF-SHA3-512(ss, "scriptum.message")`) than the `ss_derivative` (`HKDF-SHA3-512(ss, "scriptum.spend-binding")`). Revealing the `commitment_opening` does not enable decryption of the encrypted memo.
- **Mallory cannot steal stealth Parcels.** The witness is derivable only from the shared secret, which is derivable only from the recipient's Kyber private key.
- **Storage cost is bounded.** Stealth Parcels carry an extra ~1,152 bytes (`kyber_ciphertext + spend_commitment`) compared to HOME-address Parcels. This is the post-quantum tax for stealth-by-default settlement.

This is the cryptographic foundation on which Scriptum's settlement-grade privacy rests.

Inscription

Inscription is the chain's primitive for permanently recording arbitrary content with author identity and block-height timestamp.

What an inscription does

An INSCRIPTION-role Parcel commits to a payload of arbitrary bytes (up to 1.5 MB), permanently locked at the inscriber's HOME. The transaction-level Dilithium signature implicitly authenticates the author: the holder whose HOME signed the transaction creating the inscription. The block height records when it was committed.

The chain stores the encrypted payload as opaque bytes. The chain does not parse the payload's internal structure; what the wallet puts in the payload is the wallet's concern. Common patterns include:

Just a content_hash (chain-only timestamp of arbitrary off-chain content; payload ≤ 64 bytes)

A content_hash plus the author's signature over it (self-authenticated assertion)

A content_hash plus a witness's signature over it (off-chain notarisation pattern)

The full content directly (publishing-style inscription)

Any combination the wallet constructs

Fee model

The fee is determined by payload size. The chain reads the payload's byte size directly and applies a continuous formula:

$\text{fee_EUR} = \max(20, 10 \times (\text{payload_size_KB})^{0.85})$ if $\text{payload_size} > 64$ bytes

$\text{fee_EUR} = 10$ if $\text{payload_size} \leq 64$ bytes

The two-floor structure distinguishes hash-only attestation (10 EUR minimum) from content-bearing inscription (20 EUR minimum). For payloads of approximately 3 KB and above, the formula governs directly. The formula is sub-linear in size (exponent 0.85) — larger inscriptions get a bulk-discount relative to many small ones, encouraging consolidation.

Reference points along the curve:

≤ 64 bytes (hash only): 10 EUR

5 KB: ~40 EUR

10 KB: ~70 EUR

50 KB: ~280 EUR

100 KB: ~500 EUR

200 KB: ~905 EUR

220 KB: ~980 EUR (approx. WP size)

500 KB: ~1,970 EUR

1 MB: ~3,550 EUR

1.5 MB: ~5,000 EUR (maximum)

Maximum payload per inscription: 1.5 MB. Larger documents must be split into multiple inscriptions or referenced via external storage with on-chain hash commitment.

The fee is paid in FUNK at V_A at the inscription block: $\text{fee_FUNK} = \text{fee_EUR} / V_A$.

The fee is split: 50% to Charity Pot, 25% to miner, 25% to FUND.

Witness pattern (application-layer)

A common workflow is for content authors to obtain a third-party witness signature off-chain before creating the inscription:

1. Author creates content; computes $\text{content_hash} = \text{SHA3-512}(\text{content})$
2. Author sends content (or just content_hash) to witness
3. Witness reviews and produces $\text{witness_signature} = \text{Dilithium.sign}(\text{witness_privkey}, \text{content_hash})$
4. Witness returns $\text{witness_signature} + \text{witness_address}$ to author
5. Author constructs inscription payload including content_hash , witness_address , witness_signature , and any other metadata
6. Author broadcasts the inscription transaction

The witness signature lives in the payload as bytes. The chain stores it but does not validate it during inscription creation — the chain only validates the transaction-level author signature and the fee. Future verification of the witness signature is performed by readers (with payload decryption capability) using the witness_address to look up the witness's Dilithium pubkey in the Bundle Registry.

Multi-party attestation chains can be constructed by including multiple witness signatures in the payload. Aurum provides workflow support for common attestation patterns.

Discoverability

Inscription Parcels are filterable by $\text{role} = \text{INSCRIPTION} + \text{state} = \text{PERMA_LOCKED}$. Anyone scanning the chain can build a discovery service: indexing inscriptions by author, retrieving payloads via the $\text{inscription_outpoint}$ field, providing search and verification interfaces. The chain provides the immutable primitive; the application layer builds the user-facing services.

Temenos — The Parcel Model

Attribute Model

Scriptum represents all value as Parcels defined by three orthogonal attributes:

CLASS — defines the economic identity of the value

ROLE — defines its affiliation or governance context

STATE — defines its current operational behaviour

CLASS

CIRCULATING

Ordinary user-controlled value. The standard economic class. Outward sends follow the lifecycle: LOCKED → NEXUS dwell → LOCKED (at recipient stealth address). The migration lifecycle is: LOCKED → NEXUS dwell → ACTIVE → PACKING → ARC.

TREASURE

Governed allocations assigned to constitutional roles (Warden, Founders, Curia). Defined by address-bound ownership, a deterministic vesting schedule with no cliff (linear monthly from month 1), and lifecycle persistence across Phoenix transitions. Not immediately spendable until the Parcel's milestone block. At each vesting milestone the protocol consults the slot's Box: if filled, the LOCKED (TREASURE) Parcel at the slot HOME is consumed and a LOCKED Parcel with CLASS = CIRCULATING, ROLE = NONE is created at the keyholder's HOME with offset = 0; if empty, the LOCKED (TREASURE) Parcel is re-created at the slot HOME with offset advanced by 12 months (deferred retry). TREASURE Parcels are created at genesis at the slot HOMEs with offsets matching each milestone block (one TREASURE Parcel per future milestone). See §TREASURE Vesting → The Box Mechanism. The holder then proceeds through the standard CIRCULATING lifecycle.

At Ragnarok, all Parcels that remain CLASS = TREASURE board the Arc automatically — protocol-driven, regardless of vesting position. LOCKED (TREASURE) Parcels with vesting-encoded offsets are carried into the successor Phoenix with their remaining schedules intact. No TREASURE-class Parcel is lost at Ragnarok.

Important distinction: TREASURE that has fully vested and converted to CLASS = CIRCULATING is no longer TREASURE-class. Those Parcels are the holder's ordinary circulating value and must be enrolled manually like any other CIRCULATING Parcel. The constitutional vesting mechanism's responsibility ends at the LOCKED (CIRCULATING) transition.

VAULT

Protocol-controlled institutional value assigned to RESERVE, WFC, FUNDS, and CP roles. Migration is protocol-driven and automatic, not user-initiated. Follows: SEALED → OPEN → SEALED (if unused) or SEALED → OPEN → governance action → SEALED (remainder). OPEN lasts 8,640 blocks (~24 hours). Any unspent value re-seals automatically. For migration: SEALED → OPEN → PACKING → ARC.

MARKER

A non-economic, non-spendable class used for permanent protocol records — identity, donation, evidence, and preservation. Cannot be unlocked, spent, or burned. Re-instantiated unchanged at every Rebirth. Excluded from all economic calculations. Four types exist:

Mayflower — assigned to founding participants at Genesis. Represents permanent proof of participation in the founding event (ARC 0). Exactly one per founding participant. Cannot be created after ARC 0.

Dana — issued by the protocol on confirmation of a DONATE transaction. Represents permanent proof of a voluntary donation to the protocol. Exactly one per donation transaction. Private by default. Survives every Ragnarök.

Inscription — issued by the protocol on confirmation of an INSCRIBE transaction. The MARKER is a permanent pointer to inscribed content held in the originating transaction's block body, retained indefinitely by Sanctum-tier nodes. Represents permanent preservation of an artifact in the Scriptum Bibliotheca. See §Inscription.

ROLE

Roles define the governance affiliation of a Parcel:

NONE — standard circulating value, no governance context

WOC — Warden of the Chain (governance defined in Scriptum Governance Charter)

FOUNDER — founding team allocation (governance defined in Scriptum Governance Charter)

CURIA — governing council allocation (governance defined in Scriptum Governance Charter)

RESERVE — strategic long-term protocol reserve (10% of Tithe; release requires Constitutional governance)

WFC — Warden / Founders / Curia compensation pool (48% of Tithe; distributed quarterly across the fifteen constitutional position-holders by per-person share-units 10/9/5 per Scriptum Governance Charter §3.3; HICP-indexed personal compensation caps per Charter §4.1 — WOC €2M, Founder €1.8M, Curia €1.3M annual, 2026-reference; cap-overflow 100% to Charity Pot per Charter §4.4)

FUNDS — protocol-defined stewardship allocation within the VAULT class (32% of Tithe). The protocol role is not itself a legal person; governance directs FUNDS allocations to one or more suitable entities in accordance with the Charter and applicable law. Current intended recipient is the Scriptum Foundation upon incorporation at the Bootstrap T1 milestone (see §The Foundation is an intended future entity); no specific legal entity is locked at the consensus layer.

CP — Charity Pot (10% of Tithe + all WFC cap-overflow; humanitarian allocation)

MAYFLOWER — founding participant marker (CLASS = MARKER only)

DANA — voluntary donor marker (CLASS = MARKER only)

INSCRIPTION — protocol-level record of a preserved artifact stored in the Scriptum Bibliotheca (CLASS = MARKER only). See §Inscription.

STATE

State defines the operational behaviour of a Parcel. States are class-dependent.

CIRCULATING:

For consolidation (UNLOCK to source):

NEXUS doctrine. Every authorised value movement creates dwelling Parcels at the destination. ACTIVE is a post-dwell state — Parcels in ACTIVE have already completed their NEXUS dwell. A subsequent SPEND from ACTIVE consumes the post-dwell Parcel and creates a new Parcel at the recipient; the new Parcel enters NEXUS dwell at the recipient before finalising. NEXUS applies to the new Parcel created by SPEND, not the consumed ACTIVE Parcel. There is no value-movement category that bypasses NEXUS. Protocol-driven transitions (block aggregation, Menuha separation, quarterly TITHE distribution, VEST monthly milestones) bypass NEXUS dwell because there is no human signer to coerce or to make a mistake; the NEXUS exists to protect against mistakes and coercion of signing authority, which protocol-driven transitions lack.

LOCKED (stealth-derived addresses, smallest-first) → UNLOCK → NEXUS (dwell at tipping input's address) → EJECT → ACTIVE at tipping input's address (source-retention; not HOME)

For direct outward sends:

ACTIVE (at tipping input's address) → SPEND → LOCKED at recipient's stealth-derived destination + ACTIVE change at tipping input's address (immediate, no second NEXUS — the ACTIVE input already passed NEXUS during UNLOCK)

For private outward sends from hot:

ACTIVE → SPEND-private → NEXUS (dwell, both recipient and change) → EJECT → LOCKED at recipient stealth + ACTIVE at tipping address (change)

For private outward sends from cold:

LOCKED (across stealth-derived addresses, smallest-first) → SPEND-private-from-cold → NEXUS (dwell, both outputs) → EJECT → LOCKED at recipient's stealth-derived destination + LOCKED change at tipping input's address

Or, terminating without onward send:

ACTIVE → LOCK / BURN / EXPIRE → LOCKED at source / Gehenna / LOCKED at source

Migration path:

LOCKED → user enrollment → PACK → BOARD → ARC (vesting offset and address preserved in successor Phoenix)

NEXUS provides the cancellable NEXUS window for any transition the holder wants protected: UNLOCK to HOME (consolidation), private SPEND from ACTIVE (cancellation)

safety on hot outward sends), private SPEND from LOCKED (institutional cold-spend). EJECT is the protocol-driven transition out of NEXUS, firing at `unlock_height + 8,640`. Direct SPEND from ACTIVE bypasses NEXUS — ACTIVE Scriptum has earned its hot status through the unlock-side NEXUS. The choice between direct and private SPEND modes is the holder's per transaction.

TREASURE:

LOCKED (TREASURE, offset > 0) at slot HOME → [protocol Box check at `expiry_height`] → CIRCULATING LOCKED at Box recipient_address (if Box FILLED) OR LOCKED (TREASURE, offset += end-of-queue + 1 month) at slot HOME (if Box EMPTY — FIFO recycle per The Box Mechanism)

VAULT:

SEALED → OPEN (8,640 blocks) → SEALED (no action taken)

SEALED → OPEN → [governance action] → SEALED (remainder re-seals)

SEALED → OPEN → PACKING → ARC (migration only)

MARKER:

PERMA_LOCKED (permanent, no transitions)

Migration States

PACKING and ARC are protocol-defined migration states that apply across all classes. They suspend normal economic behaviour, preserve class and role identity, and exist exclusively for inter-Phoenix transport.

PACKING

A consolidated pre-migration state representing the sum of all wallet Parcels at the time of consolidation. Non-spendable. May be reversed prior to boarding. Created by the PACK transaction approximately 24 hours before the selected boarding slot.

ARC

A terminal migration state. Non-spendable. Cannot transition to any other state within the originating Phoenix. Reserved exclusively for migration into the successor Phoenix. Reached through the BOARDING transaction. Irreversible.

Migration Authority

VAULT — protocol-driven (automatic). All VAULT boards at Ragnarok regardless of state.

TREASURE — protocol-driven (automatic). All CLASS = TREASURE Parcels board at Ragnarok regardless of vesting position. The LOCKED-with-offset state preserves the remaining schedule in the successor Phoenix.

CIRCULATING — enrollment is user-initiated. PACK is user-initiated. BOARD is protocol-driven after CANCEL_THRESHOLD.

State Definitions

Default State

All outputs are LOCKED by default. This applies universally to mined coins, received funds, and change outputs.

LOCKED

Non-spendable. Can be used as input in one transaction only: UNLOCK.

NEXUS

Initiated by UNLOCK or by a private-mode SPEND (from ACTIVE or from LOCKED). Value enters the NEXUS dwell-state. Dwell duration: 8,640 blocks (24 hours at 10-second block time) — the NEXUS. Non-spendable and observable. CANCEL is valid throughout the dwell window via either signature-based CANCEL (signed by the source address's Dilithium key) or hash-preimage CANCEL (preimage of an optional hash $X = H(S)$ committed in the entering transaction, see §[Transaction Model]).

Hard cutoff: once the dwell window closes and the protocol-determined eject slot is reached, NEXUS ejects the value to its committed destination — the source/tipping input's address for UNLOCK entries (per source-retention), or the recipient's stealth-derived destination for private-SPEND entries. CANCEL submitted after eject is permanently invalid.

The eject for each entering Parcel fires exactly 8,640 blocks after entry (24 hours at 10-second block time). Every validating node computes the same eject_height deterministically from chain state. There is no per-entry randomness; eject timing is uniform across all NEXUS-dwelling Parcels at the dwell duration.

The eject is a protocol-driven state transition (in the same category as ACTIVATE in earlier specifications). It does not require the originating holder's signature; it executes deterministically based on the parameters committed in the original signed UNLOCK transaction and the chain's consensus rules. The security model is identical to ACTIVATE: the holder's UNLOCK signature commits to all parameters; the protocol-driven transition produces ACTIVE Parcel at the holder's HOME address at unlock_height + 8,640.

CANCEL race: if an attacker initiates UNLOCK on a compromised key, the legitimate owner has the full dwell window to respond. CANCEL consumes the NEXUS Parcel — there is only one. The first confirmed CANCEL wins. Once confirmed, the NEXUS Parcel is destroyed. There is no cancel-the-cancel. The attacker position is dead.

Limbo is the waiting interval between eject eligibility and actual execution — milliseconds under normal load (the time until the next block slot accommodates the protocol-driven eject), seconds to minutes under congested load as the priority queue drains across

subsequent blocks (see §Protocol-Driven Transitions and Limbo)*. During Limbo, CANCEL is invalid because the dwell window has technically closed. Wallets must display LIMBO — **AWAITING EJECT · NO ACTION POSSIBLE. Aurum surfaces an additional reassurance notification if Limbo persists beyond approximately ten seconds.*

Protocol-driven EJECT carries priority F2 = 1 in the chain's transaction priority queue (the highest priority class, alongside CANCEL, EXPIRE, PACK, and other protocol-driven or safety-critical transactions; see §Transaction Priority). Miners building blocks during the EJECT window include protocol-driven EJECTs per the priority queue or risk having their blocks orphaned by competing miners who follow priority discipline. Persistent omission of protocol-driven EJECTs is operationally visible on chain (delayed EJECTs are chain-state-detectable) and economically irrational (omitting EJECTs forfeits the EJECT-associated fees and risks block orphan). It is not consensus-invalid per se — a block containing non-priority transactions ahead of pending EJECTs is technically valid — but it is heavily disincentivised by honest network policy and competitive miner economics. Institutional readers should treat Limbo as a bounded operational risk: protocol-driven transitions execute deterministically in expected time, with brief congestion-driven delays under stress but no path to indefinite stall.

ACTIVE

Spendable for 8,640 blocks (~24 hours) from activate_height — the actual block of NEXUS eject for self-spends. The holder always receives a full 8,640-block ACTIVE window regardless of any Limbo duration.

Five outcomes are available for an ACTIVE Parcel:

SPEND(x) — transfer x Scriptum to external recipient. Fee = f_S on amount.

LOCK(x) — re-lock x Scriptum to the originating address. Fee = 0. Coin selection. Remainder stays ACTIVE.

BURN(x) — send x Scriptum to Gehenna permanently. Structurally SPEND with destination = BURN_ADDRESS; standard SPEND fee per SPX-4; coin selection per CM-3.3; remainder returns to HOME as change. The amount sent to Gehenna increments S_burned and is permanently removed from spendable circulation.

REFRESH — extend the ACTIVE window. The holder submits a small signed refresh transaction; the protocol resets the 8,640-block clock to start from refresh_height. The holder may refresh as many times as they want; each refresh starts a fresh 24-hour window. Fee = f_refresh (governance-set, expected to be near-zero to encourage active engagement without economic burden).

WAIT — do nothing. Protocol applies EXPIRE at activate_height + 8,640 (or refresh_height + 8,640 if a REFRESH was the most recent action). Fee = 0.

WAIT is the default. **The protocol always returns funds to LOCKED if the user takes no action.** The chain rewards active engagement; passive sitting returns to safe storage automatically. Inactive holders' ACTIVE balances do not stay hot indefinitely, limiting the

window during which a compromised wallet could drain them. Active holders can keep balances ACTIVE as long as they need by refreshing. The chain is not a luxury hotel.

For SPEND, LOCK, and BURN: coin selection applies across available ACTIVE Parcels. Remainder Parcels are created at the originating address in the ACTIVE state with expiry = minimum of all consumed inputs. No operation other than REFRESH resets or extends the ACTIVE validity period of any remainder output.

PERMA_LOCKED (MARKER)

A permanent non-spendable state exclusive to CLASS = MARKER Parcels. No transitions are possible. PERMA_LOCKED Parcels are excluded from all economic calculations, never included in S_net, and always re-instantiated unchanged at every Rebirth.

OPEN (VAULT)

Released from SEALED by protocol authority following a valid governance action. OPEN lasts 8,640 blocks (~24 hours). During OPEN, the VAULT may execute a governed disbursement or enter PACKING for migration.

At expiry of the 8,640-block window: if no disbursement was executed, the VAULT re-seals automatically (OPEN → SEALED). If a partial disbursement was executed, the remainder re-seals. OPEN is never permanent. Value cannot remain in an OPEN VAULT indefinitely.

This mirrors the ACTIVE window for CIRCULATING Parcels — the protocol always returns value to a protected state if no action is taken.

ARC

A deterministic terminal state representing value committed to migration. Non-spendable. Cannot transition to any other state within the originating Phoenix. Reached exclusively through BOARDING. Irreversible. ARC defines the boundary between value that continues and value that remains.

TREASURE Vesting

TREASURE class Parcels at constitutional slot HOMEs use the LOCKED state with offset > 0 to encode vesting schedules. The Parcel sits at the slot HOME from Genesis (creation_height = 0) until current_block_height >= expiry_height (= offset). At that block, the protocol consumes the TREASURE Parcel and creates a new Parcel per the Box mechanism (see §TREASURE Vesting → The Box Mechanism):

If the slot's Box has a keyholder address: a new CIRCULATING LOCKED Parcel is created at that address, immediately spendable. If the Box is empty: a new TREASURE Parcel is created at the same slot HOME with offset = 12 months (deferred retry).

The Box Mechanism

The Box mechanism implements protocol-driven TREASURE vesting for constitutional slot holders. Each constitutional slot (1 Warden of the Chain (WOC) + 5 Founder + 9 Curia = 15 slots) has a corresponding Box in the SlotBox Registry — a Tier 1 chain-state object parallel to the Council Registry and Bundle Registry.

Box state:

EMPTY — no recipient address. At genesis, every non-Warden of the Chain (WOC) Box is EMPTY (placeholder pubkeys occupy those slots; no real holders installed). The WOC Box may be EMPTY at genesis or FILLED depending on WOC installation timing.

FILLED with recipient_address — a holder is installed; the recipient address is derived from the slot's installed Dilithium pubkey (SHA3-512(pubkey).truncate(64)); the Box is ready to receive vesting Parcels.

Box population — atomic with Warden of the Chain (WOC)_INSTALL_SLOT. When a slot is filled via the Round 14 WOC_INSTALL_SLOT transaction (WOC swaps the placeholder pubkey for the real holder's Dilithium pubkey), the chain atomically populates the corresponding Box with the holder's address derived from the installed pubkey. Slot-fill and Box-fill happen in the same transaction; the Box is FILLED from the moment the slot is live.

Box address update on slot key change. When a slot's holder changes via REMOVE_SLOT_KEY + ADD_SLOT_KEY (or WOC_REMOVE + WOC_ADD for the WOC slot), the chain automatically updates the Box's recipient address to match the new pubkey's address. Vesting Parcels delivered prior to the change remain with the prior holder (vested = paid; no clawback). Vesting Parcels delivered after the change go to the new holder.

VEST behaviour on monthly milestone. At each vesting milestone block (every ~259,200 blocks ≈ 30 days at 10-second block times), the chain processes the next-due TREASURE Parcel at each constitutional slot HOME (the Parcel at the head of the FIFO queue):

If Box is FILLED — the chain consumes the TREASURE Parcel and creates a CIRCULATING Parcel at the Box's recipient_address with the matured monthly AMOUNT. Standard NEXUS dwell applies at the recipient's address before finalisation.

If Box is EMPTY — the chain consumes the matured TREASURE Parcel and creates a new TREASURE Parcel at the slot HOME with the same AMOUNT but with offset set to the current end-of-queue position (the largest offset currently in the queue, plus one month). The matured Parcel cycles to the back of the FIFO queue. No CIRCULATING Parcel is created; no value is delivered; no value is forfeited.

Strict one-Parcel-per-month FIFO. At each milestone the chain delivers (or recycles) exactly one Parcel per slot. When the Box transitions from EMPTY to FILLED mid-schedule, subsequent months deliver one Parcel per month at the next milestone; there are no catch-up bursts. The FIFO recycling preserves the constitutional commitment exactly: the slot's holder receives the full vesting schedule. Empty-Box months extend the vesting completion date by an equal duration but do not reduce total TREASURE delivered.

Worked example. WOC slot vesting at genesis: 36 monthly TREASURE Parcels at the slot HOME, scheduled for delivery at months 1, 2, 3, ..., 36. Initial queue: Parcels at offsets 1-2-3-4-5-6-7-...-36.

Months 1-5: Box FILLED — Parcels at offsets 1-5 deliver normally. After month 5: queue at offsets 6-7-8-...-36.

Months 6-8: Box EMPTY (WOC successor not yet installed). At each month's vest milestone, the head Parcel cycles to the back: month 6 — Parcel at offset 6 consumed, new Parcel at offset 37; month 7 — offset 7 → offset 38; month 8 — offset 8 → offset 39. Queue after month 8: 9-...-36-(orig 6 @ 37)-(orig 7 @ 38)-(orig 8 @ 39).

Month 9: Box FILLED with new holder. Parcel at offset 9 delivers normally.

Months 9-36: each month delivers the head Parcel (offsets 9, 10, ..., 36).

Months 37-39: original Parcels 6, 7, 8 (now at offsets 37, 38, 39) deliver in FIFO order.

Total: 36 Parcels delivered. Calendar months elapsed: 39. Total TREASURE delivered: full vesting schedule. Holder's full constitutional compensation preserved despite 3 empty-Box months.

Boundary case — slot still empty at Ragnarök. TREASURE Parcels remaining at any slot HOME at Ragnarök (Year 100 + 1 day) auto-board to the successor Phoenix as TREASURE Parcels — same class, same offset structure (relative to the successor Phoenix's block height counter), same eventual delivery to whoever fills the slot in the successor Phoenix. The constitutional commitment carries across the centennial transition.

No new TREASURE creation post-genesis. The 318 TREASURE Parcels exist at genesis as a one-time constitutional allocation (per the Constitutional Allocation section). The chain has no mechanism to create new TREASURE Parcels post-genesis. If governance ever wishes to compensate a position-holder outside the genesis TREASURE allocation (e.g., a new constitutional seat created via L4 amendment), the compensation flows from ongoing FUND distributions, not from new TREASURE creation. The TREASURE pool is fixed at genesis and immutable thereafter.

User-locked CIRCULATING with offset is unrelated. Users may LOCK their own CIRCULATING FUNK with a chosen offset (a separate protocol primitive for time-release of personal value). User-LOCKED Parcels remain CLASS = CIRCULATING; they are not TREASURE; they do not enter the constitutional vesting machinery. The Box mechanism applies only to CLASS = TREASURE Parcels at constitutional slot HOMEs.

State Transitions

CIRCULATING Lifecycle

The chain provides four operational paths for value movement:

Path 1 — UNLOCK to source (activation): LOCKED (across stealth-derived addresses, smallest-first) → UNLOCK → NEXUS (dwell) → EJECT → ACTIVE at tipping input's address

Path 2 — Direct SPEND from hot: ACTIVE (at tipping input's address) → SPEND → LOCKED at recipient stealth-derived destination + ACTIVE change at tipping input's address (immediate, no second NEXUS — the ACTIVE input already passed NEXUS during UNLOCK)

Path 3 — Private SPEND from hot: ACTIVE → SPEND (private mode) → NEXUS (dwell, both outputs) → EJECT → LOCKED at recipient stealth + ACTIVE at tipping address (change)

Path 4 — Private SPEND from cold: LOCKED (across stealth-derived addresses, smallest-first) → SPEND (private mode, from-cold) → NEXUS (dwell, both outputs) → EJECT → LOCKED at recipient stealth + LOCKED at tipping address (change)

Cross-address consolidation (explicit): Frau Holle — wallet-driven sweep of cluster Parcels to HOME at holder's invocation (default at Menuha when cross-address fuse fires; reaffirmable opt-out; see Menuha Sweep section)

Per-address consolidation (automatic): Goldmarie — protocol-driven consolidation of ≥ 3 LOCKED Parcels of same (class, role) at any address, in place, at every Menuha

Other ACTIVE-state transitions:

ACTIVE → LOCK(x) → LOCKED at source + ACTIVE remainder at source

ACTIVE → BURN(x) → GEHENNA + ACTIVE remainder at source

ACTIVE → REFRESH → ACTIVE at source (clock reset)

ACTIVE → WAIT → EXPIRE → LOCKED at source

NEXUS → CANCEL → LOCKED at source (returns to pre-entry state)

ACTIVE → PACK → PACKING at source

PACKING → BOARD → ARC (value committed to migration)

UNLOCK and private SPEND both pass through NEXUS dwell at the entry to their respective destinations. Direct SPEND from ACTIVE is the fast operational path — the holder has actively brought value into ACTIVE state through UNLOCK (which passed through NEXUS dwell at the entry to hot status) and has been continuously maintaining the hot status through periodic REFRESH actions. Direct SPEND is the operational use of value the holder has deliberately maintained in active management; it is not a bypass of NEXUS so much as the natural outflow from value that has already earned and continuously reaffirmed its hot status. The recipient Parcel created by direct SPEND is LOCKED at destination immediately — no second NEXUS — because the consumed ACTIVE input already cleared its NEXUS during the UNLOCK that brought it into hot status. Holders choose between direct SPEND (operational use of actively-maintained value) and private SPEND (re-enter NEXUS for an additional cancellation window) per transaction. EJECT is the protocol-driven transition out of NEXUS, executed at `unlock_height + 8,640`. See §NEXUS — The State-Transition Crucible in the Privacy Model section for the full specification.

TREASURE Lifecycle

LOCKED (TREASURE, offset > 0 at slot HOME) → [protocol Box check at expiry_height] → LOCKED (CIRCULATING, offset = 0 at keyholder's HOME) OR LOCKED (TREASURE, offset += 12 months at slot HOME)

LOCKED → UNLOCK → NEXUS (dwell) → EJECT → ACTIVE → SPEND / LOCK / BURN / REFRESH / WAIT

Arc procedure (protocol-driven — ALL TREASURE boards automatically):

LOCKED (TREASURE) → [protocol] → PACKING → BOARD → ARC (vesting offset preserved in successor)

LOCKED → user enrollment → PACK → BOARD → ARC (now CIRCULATING, user-initiated)

VAULT Lifecycle

SEALED → OPEN (protocol, 8,640 blocks) → SEALED (no action — auto re-seals)

SEALED → OPEN → [disbursement] → SEALED (remainder re-seals)

SEALED → OPEN → PACKING → ARC (migration)

Lifecycle Reference Diagram

The CIRCULATING lifecycle in condensed form, showing user-initiated transactions and protocol-driven transitions:

LOCKED — UNLOCK —  NEXUS — EJECT —  ACTIVE

▲ | |

| | |

| CANCEL SPEND

| | BURN

| ▼ DONATE

| LOCKED LOCK

| REFRESH

| |

| EXPIRE

| |

Each transition fires in one direction only. UNLOCK enters the NEXUS dwell-state and opens the Safety Control Window; EJECT closes the dwell-state and produces ACTIVE at the holder's HOME address. SPEND, BURN, DONATE, and LOCK consume ACTIVE Parcels at HOME. Direct SPEND lands as LOCKED at the recipient's stealth-derived address with no NEXUS dwell. Private SPEND modes (from ACTIVE or from LOCKED) route through NEXUS for the cancellation window. REFRESH extends the ACTIVE window. EXPIRE returns unused ACTIVE Parcels to LOCKED at HOME. CANCEL reverses a NEXUS-dwelling Parcel back to its source state within the window (via signature or hash-preimage). All protocol-driven transitions (EJECT, EXPIRE) fire deterministically — EJECT at `unlock_height + 8,640`, EXPIRE at `activate_height + 8,640` (or `refresh_height + 8,640`).

TREASURE and VAULT classes follow related but distinct lifecycles. TREASURE advances LOCKED (TREASURE, offset > 0) → LOCKED (CIRCULATING, offset = 0) via monthly Box-mediated vest transitions (no cliff; first milestone at month 1; see §TREASURE Vesting → The Box Mechanism). VAULT advances SEALED → OPEN (by governance action) → CIRCULATING LOCKED through disbursement, with RESEAL returning unused VAULT to SEALED after the open window.

The full formal state machine — every valid (CLASS, ROLE, STATE) tuple, every transition, every precondition — is specified in the Canonical Machine.

NEXUS — The Safety Control Window

The Safety Control Window (NEXUS) is the protocol-enforced delay between transaction instruction and irreversible settlement during which an unauthorised movement can still be cancelled. It is the chain's core protective primitive. NEXUS is the named state-transition machinery that implements the SCW.

This section specifies the NEXUS mechanics: the NEXUS dwell-state, the CANCEL authorisation paths (signature and hash-preimage), the ACTIVE state with REFRESH switch and default-to-LOCK semantics, the protocol-driven EJECT, and the four operational paths the chain provides.

NEXUS — The State-Transition Crucible

NEXUS is the protocol-level state-transition machinery that provides cancellable controlled-transfer windows for the chain's four operational paths. It is the named architecture of the Safety Control Window. NEXUS handles three of the chain's four paths (UNLOCK to HOME, private SPEND from hot, private SPEND from cold). The fourth path — direct SPEND from ACTIVE — operates outside NEXUS as the operational use of value that has already passed through NEXUS at UNLOCK and is being actively maintained in hot status by the holder through periodic REFRESH.

NEXUS is not a mixer, not a churn pool, and not a provenance-fog layer. The chain's privacy comes from stealth derivation, identity compartmentalisation, the holder's disciplined wallet behaviour, and the structural property that NEXUS-driven EJECTs are signed by the protocol rather than by Alice — not from NEXUS as an anonymising primitive. NEXUS provides three specific properties on every cancellable transition:

Cancellable dwell-time. During the NEXUS dwell window, the originating holder may CANCEL the transition, returning value to its source addresses. The dwell duration is exactly 24 hours (8,640 blocks at 10-second block time). After the dwell window closes, NEXUS ejects the value to its committed destination.

NEXUS provides pseudonymity comparable to ordinary banking records — counterparties and lawful investigators may obtain transaction visibility while the general public does not. Every NEXUS-dwelling Parcel carries on-chain references that link it to its origin and (after EJECT) to its destination. The Safety Control Window protects holders from theft and provides a cancellation window for genuine mistakes; it does not provide cryptographic unlinkability of sender to recipient. Stronger privacy postures — including cryptographic anonymity — are planned for a future Scriptum V2 specification (see the V2 forward-reference glossary entry). The chain does not claim properties it does not deliver.

Deterministic eject at unlock_height + 8,640. The protocol fires EJECT exactly 8,640 blocks (24 hours) after the entering transaction's confirmation block. There is no randomness beacon, no slot selection, no per-entry timing variation. Every validating node computes the same eject block deterministically from chain state. The eject_height is fixed at entry and changes only under chain reorganisation deeper than the entry's confirmation, which is a catastrophic event handled by standard reorg rules. The simplicity of the rule is intentional: timing obfuscation does not break on-chain lineage (graph analysis follows input/output references regardless of when each edge is recorded), so the temporal-randomisation machinery of earlier specifications served no real privacy purpose. NEXUS achieves its safety property — cancellable dwell — through the dwell duration itself, not through eject-slot uncertainty.

Protocol-driven eject without user signature. NEXUS eject is a protocol-driven state transition, in the same category as the previous ACTIVATE primitive, EXPIRE, RELEASE, RESEAL, SWEEP, and COINBASE. The eject does not require the originating holder's Dilithium signature; it executes deterministically based on the parameters committed in the original signed UNLOCK or SPEND transaction and the chain's consensus rules. Every validating node enforces the eject identically. The on-chain record of the eject is structurally clean — no holder signature points to the destination at the eject moment; the destination was committed at entry, and the protocol delivers there.

The state lifecycles through NEXUS:

UNLOCK to source: LOCKED (stealth, smallest-first) → UNLOCK → NEXUS (dwell) → EJECT → ACTIVE at tipping input's address (with optional LOCKED remainder also at tipping address)

Private SPEND from hot: ACTIVE (at tipping address) → SPEND-private → NEXUS (dwell, both recipient and change) → EJECT → LOCKED at recipient stealth + ACTIVE at tipping address (change)

Private SPEND from cold: LOCKED (stealth) → SPEND-private-from-cold → NEXUS (dwell, both recipient and change) → EJECT → LOCKED at recipient stealth + LOCKED at tipping address (change)

In all three cases NEXUS is the same machinery — same dwell duration, same cancellation primitives (signature or hash-preimage), same protocol-driven EJECT at `unlock_height + 8,640`, same audit memo on the entering transaction. The paths differ only in source state and destination.

Direct SPEND from ACTIVE — operational use of actively-maintained hot value. The fourth operational path. The holder has actively brought value into ACTIVE state through UNLOCK (which passed through NEXUS dwell on the way to ACTIVE) and has been continuously maintaining its hot status through periodic REFRESH actions. ACTIVE state requires deliberate active management — without REFRESH actions every 24 hours, the value automatically EXPIRES back to LOCKED and re-enters NEXUS-protected status.

Direct SPEND from ACTIVE is the operational use of value the holder has actively maintained;** it is not a bypass of safety but the natural outflow from a state the holder is continuously consenting to maintain.** Direct SPEND lands as LOCKED at the recipient's stealth-derived destination immediately (subject to confirmation latency only), with no second NEXUS dwell on the SPEND side. The holder's choice to use direct mode reflects their active management of the source Parcel; private mode is the alternative when the holder wants the additional cancellation window of a second NEXUS pass.

NEXUS is the SCW machinery, named and reframed. The cancellable observation window is preserved exactly as specified — 24 hours during which the holder can CANCEL via signature or via hash-preimage submission (see §[Transaction Model]). The deterministic eject schedule simplifies analysis without compromising the safety property. The state transitions, the signature requirements, and the consensus enforcement are all preserved.

CANCEL — Two Authorisation Paths

CANCEL during NEXUS dwell is available via two authorisation paths:

Signature-based CANCEL. The originating holder signs a CANCEL transaction with the Dilithium key of the source address. This is the standard path. The fee for CANCEL is permanently zero — the chain's safety primitive must never have economic friction.

Hash-preimage CANCEL. The originating holder, when constructing the entering transaction, optionally embeds a hash $X = H(S)$ where S is a secret known to the holder. During dwell, anyone submitting S to the chain triggers cancellation: the chain verifies $H(S) = X$; if yes, the value returns to the source address. The hash-preimage path provides delegable cancellation authority — the holder can give S to a designated party (a security service, a backup trustee, anyone they choose) who can cancel on the holder's behalf without holding the holder's spend key. The hash function is SHA3-512, the chain's single post-quantum hash function (NIST Level 5, 256-bit collision resistance against Grover-birthday); the construction does not require Dilithium signing.

Both paths are independent. A holder may use neither (no cancellation possible during dwell — unusual but allowed), either alone, or both simultaneously. The chain accepts either form; the first valid CANCEL during dwell consumes the entering Parcel, and there is no cancel-the-cancel.

Hash-preimage CANCEL is post-quantum compatible: hash functions retain preimage resistance against quantum adversaries (Grover's algorithm provides only a quadratic speedup, leaving 256-bit hashes with 128-bit effective security against quantum attack — well above any practical threshold).

ACTIVE State — Refresh and Default-to-LOCK

The ACTIVE state, after emerging from NEXUS for self-spends, has a default 24-hour validity window. Within that window the holder may SPEND, LOCK, or BURN. Inaction triggers EXPIRE, returning the value to LOCKED at the same address.

The holder may extend the ACTIVE window by submitting a refresh signature — a small signed transaction that resets the 24-hour clock. The holder can refresh as many times as they want; each refresh starts a fresh 24-hour window. Refresh fee is small (governance-set; expected to be near-zero to encourage active use without economic burden).

The architecture is deliberate. The default is safety: inactive holders' ACTIVE balances do not stay hot indefinitely, limiting the window during which a compromised wallet could drain them. Active holders can keep balances ACTIVE as long as they need by refreshing. The chain rewards active engagement; passive sitting returns to safe storage automatically. Each REFRESH costs f_{refresh} , the governance-set near-zero fee for ACTIVE-state re-anchoring. Holders who keep balances ACTIVE pay f_{refresh} per refresh action. The fee is small enough not to impose meaningful economic burden on legitimate operational use, but non-zero — keeping value hot is a chargeable convenience, consistent with the chain's per-operation fee model. The chain is not a luxury hotel.

Transaction Model

Transaction Definition

Scriptum transactions are atomic instructions that modify the state of a single value path. Each transaction consumes one or more inputs and produces one or more outputs. Scriptum does not support multi-recipient batching within a single transaction. Each transfer is executed as an independent instruction.

User-Initiated Transactions

Coin Selection and Destination Rules

The chain uses smallest-first coin selection across all primitives that consume holder Parcels (UNLOCK, all SPEND modes, LOCK, BURN, DONATE, Frau Holle). The selection algorithm and the destination rule are unified.

Smallest-first selection. When a primitive needs to consume Parcels to produce an output of value V , the wallet sorts the holder's eligible Parcels by amount ascending and consumes them in order until the accumulated sum reaches or exceeds $V + \text{fees}$. Smallest-first naturally consolidates dust through normal use, keeps the holder's Parcel set leaner over

time, and does not penalise the user economically because the operation-based fee model charges per operation on summed value rather than per input.

Tipping input. Under smallest-first, exactly one input is the 'tipping input': the last input consumed, the one that pushed the accumulated sum over the target. All smaller inputs were fully consumed (their full value contributed to reaching the target); the tipping input is the one with leftover value. Single-input transactions are trivially their own tipping input.

Change destination rule. Change (the remainder after value and fees) goes to the tipping input's address. The rule is universal across all primitives that produce change: SPEND (all modes), UNLOCK, LOCK, BURN, DONATE. The rule falls out of the coin selection algorithm — no separate multi-input rule is needed because the algorithm itself identifies the tipping input.

Recipient destination rule. Recipient outputs (for SPEND, DONATE, BURN) go to the recipient's destination address: for SPEND, the recipient's stealth-derived address derived from the sender's ephemeral DH against the recipient's view key; for DONATE, the constitutional DONATE destinations; for BURN, Gehenna_HOME.

State preservation. Change inherits the source state. ACTIVE inputs → ACTIVE change; LOCKED inputs → LOCKED change. The state-preservation rule applies uniformly across all paths. Recipients always LOCKED per the universal LOCKED-arrival invariant.

UNLOCK output destination. UNLOCK activates value at the tipping input's address — not HOME. The output (ACTIVE-destined Parcel of target value V) goes to the tipping input's address. Remainder (if any) is a second output, LOCKED-destined, also at the tipping input's address. UNLOCK is no longer a consolidate-to-HOME primitive; it is an activate-at-source primitive.

Cross-address consolidation to HOME. Holders who want to consolidate their cluster at HOME use Frau Holle (wallet-driven, see Goldmarie and Frau Holle section). HOME is the published-identity anchor and the explicit-consolidation destination — not an operational hub for automatic consolidation.

Why source-retention. The earlier model used HOME as a uniform consolidation destination for UNLOCK and Frau Holle's cross-address sweep. That model traded privacy for operational simplicity: every UNLOCK exposed the holder's HOME on chain, linking the source stealth addresses to HOME publicly. Source-retention preserves the holder's privacy through the operational lifecycle: receive at stealth, activate at stealth (via UNLOCK landing at source), spend from stealth (change at source). HOME appears on chain only when the holder explicitly consolidates via Frau Holle — making cluster exposure a deliberate act rather than a side effect of normal operation. The operational cost is real (wallets track value at many addresses; the cluster grows until Frau Holle is invoked) but is bounded by the tier-1 chain primitive (Goldmarie) that consolidates per-address fragmentation at every Menuha and by Aurum's default-prompt-then-sweep behaviour for cross-address consolidation. Privacy is the default; consolidation is the explicit choice.

Stationary Transactions

Stationary transactions move value within the holder's own address space without external recipients. Inputs are consumed at one address; outputs are created at the same address per the universal change destination rule (outputs go to the tipping input's address).

LOCK(x)

Re-locks x Scriptum to the originating address. Creates a LOCKED output. Applies coin selection. Remainder stays ACTIVE with expiry = minimum of consumed inputs. Fee = 0. Symmetric with SPEND except recipient = self.

CANCEL

Reverses a NEXUS-dwelling output to LOCKED. Valid throughout the dwell window (until the protocol-determined eject slot is reached). Recovers reserved f_A . Fee = 0.

CANCEL is available via two authorisation paths:

Signature-based CANCEL. The originating holder signs a CANCEL transaction with the Dilithium key of the source address. Standard path. The fee is permanently zero — the chain's safety primitive must never have economic friction.

Hash-preimage CANCEL. The originating holder, when constructing the entering UNLOCK or SPEND, may optionally embed a hash $X = H(S)$ where S is a secret known to the holder. During dwell, anyone submitting S to the chain triggers cancellation: the chain verifies $H(S) = X$; if yes, the value returns to the source address. The hash-preimage path provides delegable cancellation authority — the holder can give S to a designated party (a security service, a backup trustee) who can cancel on the holder's behalf without holding the holder's spend key. The hash function is SHA3-512, the chain's single post-quantum hash function (NIST Level 5, 256-bit collision resistance against Grover-birthday); the construction does not require Dilithium signing.

Both paths are independent. A holder may use neither, either alone, or both simultaneously. The first valid CANCEL during dwell consumes the entering Parcel; there is no cancel-the-cancel.

REFRESH

Extends an ACTIVE Parcel's validity window. The holder signs a small refresh transaction referencing the ACTIVE Parcel; the protocol resets the 8,640-block expiry clock to start from refresh_height. The holder may refresh as many times as they want; each refresh starts a fresh 24-hour window. Fee = f_{refresh} (governance-set, expected to be near-zero to encourage active engagement). The default behaviour without REFRESH is EXPIRE → LOCKED at activate_height + 8,640: the chain rewards active engagement; passive sitting returns to safe storage automatically.

Consolidation Transactions

Consolidation transactions consume Parcels from one or more addresses and bring value together. Under source-retention (Round 11), consolidation is per-address (smallest-first selection within a single address) or cross-address (Frau Holle's explicit cluster sweep to HOME).

UNLOCK

Initiates the NEXUS. Takes a target value V (declared by the holder) as input rather than a fixed Parcel set. The wallet, using the holder's key pair, identifies LOCKED Parcels across all stealth-derived addresses derivable from that key pair and applies smallest-first coin selection to assemble inputs whose sum reaches or exceeds $V + f_U + f_A$. The UNLOCK transaction consumes the selected LOCKED Parcels and produces NEXUS-dwelling outputs destined for the tipping input's address: one ACTIVE-destined Parcel of value $V + f_A$, and (if the inputs sum to more than $V + f_U + f_A$) one LOCKED-destined remainder Parcel of value $\text{SUM}(\text{inputs}) - V - f_U - f_A$. Both outputs land at the same destination — the tipping input's address — and differ only in destination state. The transaction signs with the relevant addresses' Dilithium3 keys (one signature per distinct source address), each verified against that source address's authorisation via the `spend_commitment` + bundle reconstruction. The destination address is the tipping input's address (not HOME), preserving the holder's source-state privacy through UNLOCK. If HOME has not yet been first-touched in chain history, an UNLOCK with HOME as the tipping address is itself the first-touch event; the holder reveals the HOME bundle in the transaction. For all other tipping addresses (stealth-derived), no bundle reveal is required; spending authority is established via `spend_commitment` per Stealth Settlement and Spend Authorisation. Fee = f_U (flat instruction fee on summed value V , not per input). f_A is reserved and fixed at UNLOCK-time V_A ; it is deducted automatically at EJECT using the V_A value recorded at the UNLOCK block. The UNLOCK transaction may optionally embed a hash $X = H(S)$ to enable hash-preimage CANCEL during dwell; see §CANCEL.

Frau Holle

Frau Holle — the cross-address Menuha sweep. Wallet-driven and holder-signed. Consolidates LOCKED Parcels across the holder's stealth-derived addresses into a single LOCKED Parcel at the holder's HOME address. Aurum's default behaviour is automatic sweep at every Menuha when the cross-address fuse fires (holder's address count exceeds three); the holder may opt out per-Menuha or persistently, with reaffirmation prompt every Menuha. Cross-address fuse only (per-address fragmentation handled by Goldmarie, the chain primitive). Full specification in the Menuha Sweep section.

Transfer Transactions

Transfer transactions move value to an external recipient. The recipient output goes to the recipient's destination address; any change goes to the tipping input's address per the universal change destination rule.

SPEND(x)

Transfers x Scriptum to a recipient address, creating a LOCKED output at that recipient's stealth-derived one-time address (derived via Kyber/ML-KEM encapsulation against the recipient's published Kyber view key). SPEND has two modes; the holder selects at SPEND time:

Direct mode. SPEND consumes ACTIVE Parcels (which live at whatever addresses they were activated at — typically the tipping addresses of prior UNLOCKS). The recipient output is LOCKED at the recipient's stealth-derived destination immediately (subject to confirmation latency only). No NEXUS dwell on this path. If the inputs sum to more than the spent amount + fee, the change is an ACTIVE Parcel at the tipping input's address (state preservation: ACTIVE source $\rightarrow$ ACTIVE change). Direct SPEND is the operational use of value the holder has actively maintained through REFRESH. Fee = f_S on amount.

Private mode from hot. SPEND consumes ACTIVE Parcels and enters NEXUS with both recipient output and change output dwelling together. After 24-hour dwell, EJECT delivers each to its committed destination: recipient LOCKED at recipient's stealth-derived address; change ACTIVE at the tipping input's address (state preservation: ACTIVE source $\rightarrow$ ACTIVE change). Both outputs are cancellable together via signature or hash-preimage CANCEL during dwell; CANCEL reverses the whole transaction (all inputs return to ACTIVE at their source addresses; recipient output and change output both reverse). Fee = f_S on amount.

Private SPEND from cold. SPEND consumes LOCKED Parcels from the holder's stealth-derived addresses (smallest-first across the cluster, like UNLOCK) and enters NEXUS with both recipient output and change output dwelling together. After 24-hour dwell, EJECT delivers each to its committed destination: recipient LOCKED at recipient's stealth-derived address; change LOCKED at the tipping input's address (state preservation: LOCKED source $\rightarrow$ LOCKED change). Both outputs cancellable together during dwell. The institutional path for treasurers moving cold storage directly to counterparties without value visiting hot state. Fee = $f_U + f_S$ (the f_U covers the chain's unlock-side work; f_S covers the spend-side amount). No intermediate ACTIVE state; no exposure of the holder's HOME at any point in the transaction.

The SPEND includes the encrypted Sealed Sender Memo carrying the sender's published address (and any additional content within the size budget), encrypted to the recipient's view key. f_S is calculated at SPEND-time V_A — the V_A of the block in which the SPEND transaction is included. V_A is fixed at decision time. Bob receives a LOCKED Parcel at his stealth-derived destination; to use it, Bob does his own UNLOCK (which enters his own NEXUS dwell on his side).

DONATE(x)

Sends x Scriptum to the protocol-governed donation mechanism. The donated amount is split 50% to Funds and 50% to RESERVE. Chesed is the semantic name for this act of giving — the spirit of the disbursement — not a single discrete address; value flows directly to the two vaults at the 50/50 ratio. On confirmation, the protocol issues a DANA MARKER Parcel to the sending address: CLASS = MARKER, ROLE = DANA, STATE = PERMA_LOCKED,

AMOUNT = 1 Nugget. The DANA Parcel is the permanent private record of the donation, re-instantiated unchanged at every Rebirth. Fee = standard f_S per Fee Philosophy.

BURN(x)

Sends amount x Scriptum to Gehenna — the protocol-defined burn address, provably unspendable by mathematical construction. The Parcel continues to exist at Gehenna in state LOCKED, class CIRCULATING. It is not destroyed. It is permanently subtracted from the circulating supply via the S_{burned} counter: $S_{\text{net}} = S_{\text{mined}} - S_{\text{burned}}$. Burned value is a permanent red card: the coin exists, but it can never return to circulation. Even under a hypothetical reopening of Gehenna, protocol rules would reject any reintroduction pushing $S_{\text{net}} > S_{\text{max}}$; the emission formula makes this mathematically impossible, because $r \rightarrow 0$ as $S_{\text{net}} \rightarrow S_{\text{max}}$. Coin selection applies across ACTIVE Parcel inputs. Remainder stays ACTIVE at the originating address, carrying any existing binding. Fee = standard f_S per Fee Philosophy.

Composite Settlement Transactions

Composite settlement transactions are multi-stage protocols where the chain enforces the on-chain leg of a trade while a third party (Horkos attester) or external system (Asbru exchange API observation) handles the off-chain leg. The two primitives are distinguished by their off-chain verification mechanism — attester-driven (Horkos) versus miner-driven (Asbru).

HORKOS_AGREE

The Stage 1 multi-sig transaction signed by A, B, and C committing to negotiated trade terms (principal P , stake S , attestation fee z , attester identity, lock-to-approval window length W). The protocol verifies all three signatures and validates structural floors: $S \geq 0.001 \times P$; $W \geq 8,640$ blocks; $W \leq$ remaining time to next Ragnarök. Recording the agreement is fee-free at this stage; no funds have moved. Acceptance opens the 24-hour load window.

HORKOS_COMMIT

A's funding transaction during the Stage 2 load window. Consumes one or more LOCKED Parcels at A's address. Uploads A's principal P plus attestation fee z into the protocol-controlled ESCROW address associated with this Horkos contract. Fee = f_S calculated on $(P + z)$. The load is tracked by the chain against the agreement; if A loads the full required amount within the 24-hour window, A's load is complete.

HORKOS_STAKE

B's funding transaction during the Stage 2 load window. Consumes one or more LOCKED Parcels at B's address. Uploads B's stake S into the ESCROW address. Fee = f_S calculated on S . B's load is complete when B has loaded the full agreed stake. If either A or B fails to fully load within 24 hours, the contract lapses and loaded principals/stakes return to their loaders; the chain fees paid at load are not refunded.

HORKOS_CONFIRM

The Stage 3 multi-sig transaction signed by A, B, and C after both loads are complete. Final binding signature; transitions the contract from FUNDED to LOCKED. Fee-free at the protocol level. After HORKOS_CONFIRM, the safety is off and the lock-to-approval window of length W begins.

HORKOS_ATTEST

C's attestation transaction during Stage 4. The registered attestor signs either YES (B's off-chain delivery occurred) or NO (B's delivery did not occur) referencing the specific contract identifier. Must arrive within the lock-to-approval window W. C pays f_U as a minimal anti-spam fee. If C does not submit HORKOS_ATTEST within W, the outcome is TIMEOUT.

ASBRU_COMMIT

ASBRU_COMMIT — the user-initiated stage of an Asbru trade. Locks principal P in protocol-controlled ESCROW; declares commit fingerprint (exchange identifier + counterparty + amount + deadline). The miner who observes the matching off-chain transfer via Crowglass includes ASBRU_RELEASE in their block; absent observation within the window, ASBRU_TIMEOUT fires. Full specification in the Ás-brú section.

Ragnarök Participation Transactions

Ragnarök participation transactions enroll value for the centennial Phoenix transition. User-initiated; followed by protocol-driven BOARD or DISEMBARK at the Ragnarök boundary.

PACK

Arc procedure only. User-initiated. Consolidates all enrolled wallet Parcels into a single PACKING output. Once PACK is confirmed, the protocol takes over: BOARD fires automatically at the enrolled boarding slot, 10 blocks after CANCEL_THRESHOLD. Fee = 0.

STRAGGLER_ADD

STRAGGLER_ADD — late enrollment by a holder for the Arc manifest. Permitted within a defined window after the standard PACK deadline for holders who missed the regular cycle. Fee non-zero (per SPX) to discourage routine reliance. Full specification in the Arc / Ragnarök section.

Inscription Transactions

Inscription transactions create permanent on-chain records of off-chain content. The inscription model spans a continuous spectrum from hash-only attestation (ATTEST) to full payload inscription up to 1.5 MB (INSCRIBE). Both transaction kinds produce PERMA_LOCKED MARKER Parcels; the fee scales with payload size per the inscription fee formula.

ATTEST

ATTEST — the lowest tier of the inscription model. Hash-only payload (≤ 64 bytes). Records a content_hash on chain with optional witness signatures stored as payload bytes. Fee = 10 EUR minimum per the continuous inscription fee formula. Full specification in the Inscription section.

INSCRIBE

INSCRIBE — content-bearing inscription. Payload of arbitrary bytes up to 1.5 MB. Records the content (or content_hash plus metadata) on chain as an INSCRIPTION-role MARKER Parcel. Fee = $\max(20, 10 \times \text{payload_KB}^{0.85})$ EUR per the continuous formula. Full specification in the Inscription section.

Protocol-Driven Transactions

Time-Driven State Transitions

Time-driven transitions fire when a Parcel reaches its scheduled height (eject_height, expiry_height, open_height + window, or vesting milestone). The chain executes these automatically; missing required transitions invalidate the block.

EJECT

EJECT — protocol-driven transition from NEXUS dwell to its committed destination. For UNLOCK entries, destination is the holder's HOME address (produces ACTIVE Parcel at HOME); deducts f_A (reserved at UNLOCK at the V_A recorded in the UNLOCK block); the holder receives a full 8,640-block ACTIVE window from eject_height. For private SPEND entries, destination is the recipient's stealth-derived address (produces LOCKED Parcel at the recipient). EJECT fires deterministically at $\text{unlock_height} + 8,640$; every validating node computes the same eject_height from chain state. Fee = 0 at eject (the chain's f_A and f_S were committed at entry). At eject_height, the dwell window closes and the Parcel enters Limbo (CANCEL is invalid from this point forward). Block inclusion follows the priority queue rules in §Protocol-Driven Transitions and Limbo: under normal load, EJECT executes in the next block (Limbo = milliseconds); under congested load, EJECT executes in a subsequent block as block capacity allows (Limbo extended).

EXPIRE

EXPIRE — transitions ACTIVE to LOCKED at $\text{activate_height} + 8,640$. Fee = 0.

VEST

VEST — protocol-driven transition fired at each vesting milestone block height. Reads the SlotBox Registry for the slot's current Box state. If the Box is filled, consumes the LOCKED (TREASURE) Parcel at the slot HOME and creates a LOCKED Parcel with CLASS = CIRCULATING, ROLE = NONE at the keyholder's HOME with offset = 0. If the Box is empty, the LOCKED (TREASURE) Parcel is re-created at the slot HOME with offset advanced by 12

months. Fee = 0. The transition consumes no signature material; the protocol enforces it at block height. See §TREASURE Vesting → The Box Mechanism.

RESEAL

RESEAL — fires at `open_height + 8,640` if VAULT remains OPEN with no PACKING initiated. Returns VAULT to SEALED. Fee = 0. Analogous to EXPIRE for CIRCULATING.

Goldmarie

Goldmarie — the per-address Menuha sweep. Chain primitive operating at every Menuha block. For every address (HOME or stealth-derived) holding three or more LOCKED Parcels of the same (class, role), the chain consumes those Parcels and produces a single LOCKED Parcel at the same address with the summed amount, preserved class and role, and inherited bindings. No view-key needed (chain sees per-address state without holder identity); no opt-out (consensus rule; missing required Goldmarie consolidations invalidate the block); fee zero. Full specification in the Menuha Sweep section.

Block-Level Mechanics

Block-level mechanics fire at every block as part of block construction. Per the fee cascade, two protocol-driven Parcels are created per block: the COINBASE at the miner's HOME and the VAULT aggregate Parcel at VAULT_HOME.

COINBASE

COINBASE — creates new Scriptum for the miner. Output is LOCKED. Subject to standard unlock rules.

VAULT Aggregate

VAULT Aggregate — the protocol-driven Parcel created at every block at VAULT_HOME, carrying 10% of the block's transaction fees (the tithe portion). Created by protocol mechanics; does not pass through NEXUS_DWELL. Accumulates per block until Menuha separation into FUND_HOME, RESERVE_HOME, and TITHE_HOME. Full specification in the Fee Cascade section.

Composite Settlement Resolutions

Protocol-driven resolution transitions for the composite settlement primitives. Fire deterministically based on attestation outcome (Horkos) or window expiry / miner observation (Asbru).

HORKOS_RELEASE

HORKOS_RELEASE — protocol-driven settlement of a Horkos contract on YES attestation by C. Delivers A's principal to B; returns B's stake to B; pays attestation fee z to C. Fee = 0. Full specification in the Horkos section.

HORKOS_TIMEOUT

HORKOS_TIMEOUT — protocol-driven timeout of a Horkos contract if no HORKOS_ATTEST has been received within the lock-to-approval window. Returns A's principal to A; splits B's stake 50/50 (50% returned to B; 50% routed through the standard tithe split); refunds attestation fee z to A; C receives nothing. Fee = 0. Full specification in the Horkos section.

HORKOS_SLASH

HORKOS_SLASH — protocol-driven slashing of B's stake on NO attestation by C. Returns A's principal to A; routes B's stake through the standard tithe split (90% miner, 10% tithe per the fee cascade); pays attestation fee z to C. Fee = 0. Full specification in the Horkos section.

ASBRU_RELEASE

ASBRU_RELEASE — protocol-driven release of escrowed Scriptum on Crowglass-observed off-chain transfer confirmation. The miner who observes the matching transfer includes ASBRU_RELEASE in their block. Delivers P from ESCROW to B. Fee = 0. Full specification in the Ás-brú section.

ASBRU_TIMEOUT

ASBRU_TIMEOUT — protocol-driven timeout of an Asbru trade if no ASBRU_RELEASE has been observed within the window (60 blocks per SPX). Returns principal P from ESCROW to A. Fee = 0. Full specification in the Ás-brú section.

Ragnarök Events

Protocol-driven transitions at the Ragnarök boundary. BOARD fires at the boarding slot of the dying Phoenix; DISEMBARK fires at Block 0 of the successor Phoenix.

BOARD

BOARD — executes at the boarding slot, 10 blocks after CANCEL_THRESHOLD closes. Consumes PACKING Parcel and accepted stragglers. Produces a single ARC output. Fee = 0. Consensus-enforced: any block omitting a required BOARD output is invalid and rejected by all nodes. The miner cannot exclude it. ESCROW Parcels with pending Horkos trades are also boarded, preserving those trades across the Ragnarök boundary.

At Ragnarök (every 100 years), every TREASURE Parcel must execute its BOARD transition: the Parcel transitions from its current state into the successor Phoenix's genesis allocation, preserving its vesting schedule and constitutional role. BOARD is a protocol-driven transition; the chain executes it automatically at the Ragnarök boarding block.

Because BOARD is mandatory and protocol-driven, the block at the boarding slot must include a BOARD output for every eligible TREASURE Parcel. Miners constructing the boarding block do not have discretion to omit BOARDS. Any block omitting a required BOARD output is invalid and rejected by all nodes. This consensus rule ensures the Ragnarök transition is total and orderly — every constitutional vesting allocation crosses into the Phoenix successor, none are left in the dying generation's chain state.

DISEMBARK

DISEMBARK — protocol-driven at Block 0 of every Rebirth. Reads the complete ARC manifest and instantiates all valid ARC Parcels as LOCKED outputs in the successor Phoenix. No user action required. Executed once per Phoenix cycle.

Issuance Side-Effects

Issuance side-effects fire as protocol-driven side effects of confirmed user transactions. These are not standalone transitions; they accompany the triggering user transaction's confirmation.

DANA ISSUE

DANA ISSUE — fires on confirmation of a DONATE transaction. Creates DANA Parcel at the sending address: CLASS = MARKER, ROLE = DANA, STATE = PERMA_LOCKED, AMOUNT = 1 Nugget. Fee = 0.

Value Transfer Sequence

To execute a value transfer:

UNLOCK (user) → EJECT (protocol) → SPEND (user)

SPEND creates an ACTIVE remainder for the sender and a LOCKED output for the recipient. Every SPEND derives the recipient's one-time output address via Kyber/ML-KEM key encapsulation against the recipient's published view key; the sender includes the encapsulation ciphertext (the ephemeral key material) in the transaction for recipient-side scanning. The LOCKED change output returning to the sender's own address carries the sender's key binding forward (see §Address Binding); the one-time LOCKED output at the recipient derives from the recipient's stealth keys and does not carry the sender's binding.

No Hidden Execution

Scriptum deliberately excludes Turing-complete execution. All protocol behaviour is finite, deterministic, and explicitly defined through enumerated state transitions. This eliminates hidden execution paths and ensures that value movement remains transparent, auditable, and controllable at all times.

There are no smart contracts. There is no dynamic execution. There is no emergent behaviour. Every state a Parcel can occupy, and every transition between states, is defined in this document. An implementation is valid if and only if it reproduces these transitions exactly.

This is a security property, not a limitation. Hidden execution paths are attack surfaces. Scriptum has none.

The Covenant — Fees and Miner Incentives

Fee Philosophy

Scriptum is not the cheapest chain for large transfers. It is the only chain where the fee is fair for everyone — from a €1,000 personal transfer to a €200M institutional settlement.

Every competing blockchain charges a flat fee per transaction, independent of the amount transferred. Bitcoin charges approximately €4 regardless of whether you send €10 or €200,000,000. At €10 that is 40% of the transfer. At €200M it is essentially free. There is no proportionality. There is no fairness. There is only a flat toll that punishes small users and subsidises large ones.

Scriptum charges proportionally. The fee is always a percentage of the transfer value, calculated in € using progressive tiers, then converted to FUNK via V_A . At €1,000 you pay approximately €5 — the same as Bitcoin. Above that, you pay more than Bitcoin. But above that, you also receive what Bitcoin cannot give you: the Safety Control Window, post-quantum security, 24/6 settlement, an immutable permanent record, and a trading environment designed to protect every participant equally.

Scriptum is not fast food. It is a restaurant. The meal costs more. The quality is incomparable. And crucially, the menu is priced fairly — the same percentage for the person sending €1,000 and the institution moving €200M. Nobody is subsidised. Nobody is punished. The tiers ensure that larger transfers pay a lower percentage, not a higher one — making Scriptum progressively more competitive as transfer size grows.

The fee in FUNK is always: $f_{FUNK} = f_{EUR} / V_A$. The € amount is fixed by the progressive tiers. V_A converts to FUNK at a smoothed, tamper-resistant rate. The user pays a **protocol-calibrated euro-equivalent fee** — the conversion uses V_A as the chain's calibrated valuation coefficient, which is deliberately smoothed and lagged for tamper-resistance and is fixed at 1.0 during pre-market operation (see §Urðarbrunnr — Valuation Integrity and Temporal Pricing); the euro-equivalent is therefore the chain's authoritative valuation, not a real-time market quote. Every fee is split 90% to the miner in FUNK and 10% to the Tithe, allocated across four streams (48% WFC, 32% Funds, 10% RESERVE, 10% CP) per §15. The split applies uniformly to f_U , f_A , f_S , and $F_{REFRESH}$. There is no carve-out for flat instruction fees. No additional multiplier is required.

The chain charges its proportional commission on every value transfer, including BURN and DONATE. Burning value is a holder's active decision and the chain processes it like any other SPEND — same machinery, same fee, same payment to miners and to the tithe stream that funds Foundation operations and charitable activities. Donating to a chain VAULT works the same way: the donation pays its standard commission and the donor's act of giving simultaneously feeds the chain that will continue to safeguard their remaining holdings.

The only exception is CANCEL — the holder's safety mechanism against unauthorised UNLOCK. Charging for CANCEL would create economic friction on the chain's primary safety guarantee, which is a structural design error. CANCEL remains permanently fee-free.

Other transitions that do not transfer value to a non-holder address — LOCK (voluntary return-to-LOCKED-state, value stays with the holder), EXPIRE (protocol-driven, no holder initiation), PACK and BOARD (Arc machinery, holder sovereignty during Ragnarök) — are also fee-free, as they are not value transfers but housekeeping or protective primitives.

The principle is consistent: the chain is paid a small commission for processing every value transfer between parties or destinations, regardless of motive. The chain does not subsidise particular kinds of transactions through fee carve-outs. Holders who want their value to move pay the chain to move it; holders who want safety primitives are not taxed for using them.

Fee Model: Operation-Based, Not Input-Based

Scriptum's fees are charged per operation on the total value being moved, never per input or per Parcel. f_U is one fee per UNLOCK transaction, calculated on the sum of unlocked value. f_A is a flat instruction fee per EJECT (irrespective of amount), reserved at UNLOCK using the V_A at the UNLOCK block and paid to the miner including the EJECT block. f_S is one fee per SPEND, calculated on the sum of value being sent. A holder consolidating value from many one-time stealth-derived addresses into a single UNLOCK pays one f_U on the consolidated total — not one f_U per source address. The wallet handles Parcel aggregation behind the scenes; the user pays for value transfer, not for accounting overhead. This means stealth-driven address fragmentation does not impose a fragmentation tax on holders — the privacy property is structurally fee-neutral. The chain charges per-operation per-summed-value at every transition, regardless of how many addresses or Parcels participate as inputs. INSCRIBE follows the same operation-based model: one fee per inscription, calculated via the inscription fee formula on payload size (see Inscription section). The chain charges per inscription, not per byte of payload metadata; the formula applies to the entire payload size as a single computation.

Fee Schedule

All flat instruction fees (F_{UNLOCK} , $F_{ACTIVATE}$, $F_{REFRESH}$) are 2026-reference and HICP-indexed annually per §HICP Indexing — Annual Real-Value Calibration.

f_U — flat instruction fee, irrespective of amount, paid at UNLOCK block

f_A — flat instruction fee, irrespective of amount; reserved at UNLOCK using the V_A at the UNLOCK block, paid to the miner at the EJECT block (the protocol-driven transition out of NEXUS dwell)

f_S — proportional to amount or payload, progressive tiers, no flat component, paid at SPEND block

CANCEL, LOCK, EXPIRE, PACK, BOARD — free ($f = 0$). BURN and DONATE pay the standard SPEND fee per SPX-4 (see §Fee Philosophy).

At base reference (€1 per Scriptum, for calibration):

$f_U = 0.04$ Scriptum = €0.04

$f_A = 0.01$ Scriptum = €0.01

f_S = progressive tier % of € amount at V_A , or amount calculated via payload formula

f_S — Progressive Tier Model

All EUR-denominated values in this fee schedule are 2026-reference and HICP-indexed annually per $\$HICP$ Indexing — Annual Real-Value Calibration. The brackets shown are the 2026 base values; the chain applies the cumulative HICP factor to each boundary at fee calculation time.

Calculated cumulatively across tiers on the transferred amount in €, at current V_A . Each portion of the amount is charged at its tier rate:

Tier 1 €1 – €1,000 → 0.50%

Tier 2 €1,001 – €10,000 → 0.45%

Tier 3 €10,001 – €50,000 → 0.40%

Tier 4 €50,001 – €100,000 → 0.35%

Tier 5 €100,001 – €200,000 → 0.30%

Tier 6 €200,001 – €500,000 → 0.25%

Tier 7 €500,001 – €1,000,000 → 0.20%

Tier 8 €1M – €2,000,000 → 0.15%

Tier 9 €2M – €10,000,000 → 0.10%

Tier 10 >€10,000,000 → 0.05%

f_S has no flat instruction component. It is purely proportional to amount.

Examples

All fees are defined in €. V_A converts to FUNK at the smoothed market rate. The € amount is always identical regardless of FUNK price.

Transfer €1,000:

$f_U = €0.04$ $f_A = €0.01$ $f_S = €5.00$ Total = €5.05 (0.505%)

At €1/F: 5.02 FUNK | At €1,000/F: 0.00502 FUNK

User pays €5.05. Miner receives €4.545 (90% of every fee). Protocol Tithe receives €0.505 (10% of every fee), allocated as: WFC €0.242 (48%), Funds €0.162 (32%), RESERVE €0.051 (10%), CP €0.051 (10%).

Transfer €10,000: $f_S = €45.50$ Total = €45.52 (0.455%)

Transfer €1,000,000: $f_S \approx €2,430$ Total $\approx €2,430$ (0.243%)

Transfer €200,000,000: $f_S \approx €107,000$ Total $\approx €107,000$ (0.054%)

Comparison on €1,000 transfer:

Bitcoin: €4 flat = 0.40% (comparable cost, no NEXUS, no PQ)

Ethereum: €0.47 flat = 0.047% (cheaper, no protection)

Solana: €0.00025 flat (near free, no protection)

Scriptum: €5.02 = 0.50% (NEXUS + post-quantum + 24/6 + immutable record)

Comparison on €10 transfer:

Bitcoin: €4 flat = 40% (punishing for small users)

Ethereum: €0.47 flat = 4.7% (expensive)

Solana: €0.00025 = 0.0025% (cheap)

Scriptum: €0.05 = 0.50% (fair — same % as €1,000)

Scriptum is the only chain where the fee percentage is consistent regardless of transfer size. Every other chain charges a flat amount per transaction that punishes small users and effectively rewards large ones. Scriptum charges proportionally. The same percentage applies at €10 and at €200,000,000.

Design Objectives

Identical transfers incur identical costs, regardless of Parcel fragmentation

Internal representation of funds does not affect pricing

Users are not penalized for fragmentation or the structure of their holdings

High-value transfers remain economically efficient

Fee levels remain meaningful at any Scriptum price

Dust Threshold

The dust threshold is 2026-reference and HICP-indexed annually per SHICP Indexing.

A Parcel is dust if its fiat-equivalent value at current V_A is less than €0.01. Such Parcels are economically irrational to unlock — fees exceed value. Transactions creating dust outputs are invalid and rejected. In Nugget terms: at Scriptum = €1, dust threshold $\approx 1,000,000$ Nuggets.

Miner Reward

Miner compensation = block reward + transaction fees (f_U, f_A, f_S). As block rewards decrease toward zero, fees become the primary incentive. The fee model and V_A mechanism ensure fee sustainability across all Scriptum price levels. The € value of fees remains proportional to transfer amounts regardless of FUNK market price.

Menuha — Sunday Rest

SPEND (all three modes), BURN, DONATE, LOCK, HORKOS_COMMIT, and HORKOS_STAKE are invalid during the Menuha Window. The Menuha Window is any block whose timestamp falls on Saturday 23:00 UTC through Sunday 23:00 UTC, evaluated via Median Time Past (MTP) of the most recent 11 blocks to defeat single-miner timestamp gaming. Transactions of these types submitted during Menuha are rejected. UNLOCK, CANCEL, REFRESH, HORKOS_ATTEST, PACK, and STRAGGLER_ADD remain valid; SWEEP is valid only during Menuha. Full Menuha context in Section 30. The agreement phase of Horkos is suspended during Menuha: Stage 1 (negotiation multi-sig), Stage 2 (loading transactions HORKOS_COMMIT and HORKOS_STAKE), and Stage 3 (final confirmation multi-sig) cannot be submitted, since each commits new outward value to the contract. Stage 4 (HORKOS_ATTEST) and Stage 5 (resolution) run through Menuha unaffected — HORKOS_ATTEST is the recording of a delivery decision by attestor C, not an outward value transfer, and Stage 5 is protocol-driven (UNLOCK → EJECT → SEND with no user signatures). A Horkos contract whose attestation finalises just before or during Menuha completes its resolution sequence on schedule; only new agreements cannot be initiated during the rest day.

Urðarbrunnr — Valuation Integrity and Temporal Pricing

Scriptum introduces a deterministic valuation integrity layer governing how market price translates into executed economic cost. The Urðarbrunnr Integrity System (UIS) is not designed to determine price. It is designed to protect the translation of price into fees.

In conventional systems, fee determination based on instantaneous price creates an attack surface: any participant capable of temporarily influencing valuation can execute high-value transfers at artificially reduced cost. Scriptum eliminates this vector by introducing temporal separation between price observation and fee application. Valuation must persist over time before it can influence execution.

Price may change instantly. Its economic effect cannot.

Valuation Coefficients

The V_A mechanism is source-agnostic by design. Any registered HOME — operating an Aurum wallet, a Hayyānu mining node, or an Ilmarinen full validation node — may submit V_P proposals derived from any price source the operator chooses: an exchange, an aggregator, a private feed, or any combination. Submissions are signed transactions of kind $VA_PROPOSE$, rate-limited per HOME (see V_A Proposer Architecture). The protocol validates the signature, the rate limit, and the drift constraints — not the source. This

ensures the system remains functional across any time horizon, regardless of which exchanges or data providers exist at any given moment. No external oracle is hardcoded. No single source is privileged.

Critical distinction: V_A governs fee calculation only. It does not determine the market price of FUNK. The market price of FUNK is set by buyers and sellers on exchanges — by supply and demand — entirely independently of V_A . A proposer who misreports V_P affects only the fiat-equivalent fee calculation, not the real trading value of FUNK. This makes V_A manipulation economically irrational: the attacker cannot profit from it, and users can simply wait for V_A to converge before transacting.

V_C — current market price (external, instantaneous, outside consensus)

V_P — proposer submission: the valuation value a registered HOME submits via a signed $VA_PROPOSE$ transaction (see V_A Proposer Architecture)

V_A — accepted protocol valuation: the consensus value derived from aggregated $VA_PROPOSE$ submissions and used for all fee calculations

V_A evolves gradually based on a rolling statistical window of 4,320 blocks (~12 hours), using geometric mean and standard deviation, with bounded drift per block. This introduces temporal lag between V_C and V_A . Fees are derived exclusively from V_A .

V_A is constrained by two hard consensus rules enforced by all nodes:

Constraint 1 — Per-block drift limit:

V_P must satisfy: $0.99 \times V_{A_previous} \leq V_P \leq 1.01 \times V_{A_previous}$

V_A cannot move more than 1% from the previous block in a single step.

Constraint 2 — Trailing average band:

V_P must satisfy: $0.85 \times V_{avg(4320)} \leq V_P \leq 1.15 \times V_{avg(4320)}$

V_A cannot deviate more than 15% from the 4,320-block (~12 hour) rolling average.

Enforcement: if a submitted V_P violates either constraint, nodes clamp the V_P to the nearest valid bound (the closer of the two values defined by the inequality) before it enters the aggregation. A clamped submission is still counted; it does not cause the submitting transaction to be rejected. The aggregated V_A and the block carrying it are never rejected on grounds of an individual V_P . Execution always proceeds. This prevents chain stalls while eliminating manipulation.

Genesis V_A : $V_A = 1.0$ at block 0 of every Phoenix. A base reference of €1 is used for initial fee calibration only. Market price is established by the first trade. V_A evolution begins when proposers begin submitting V_P values derived from real market data.

Pre-market V_A : before Scriptum has an established external market price, V_A remains at 1.0. V_A evolution begins only when proposers begin submitting V_P values derived from real market data. Until then, all fees are calculated at base reference.

Economic Consequence

Rapid price increases → V_A lags V_C upward → temporarily higher effective fees

Rapid price decreases → V_A lags V_C downward → temporarily lower effective fees

Users may execute immediately or wait for V_A to converge toward V_C. Since the NEXUS enforces an 8,640-block window before value becomes transferable, V_A will typically have converged substantially before execution is possible in standard operation. Under extreme market volatility, some divergence may persist.

The UIS — Three Temporal Layers

Verðandi — Instantaneous State

Current valuation proposals (V_P) represent the present. Registered HOMEs submit signed VA_PROPOSE transactions; the accepted submissions in the current aggregation window are the present-tense V_P inputs feeding into V_A (see V_A Proposer Architecture).

Urðr — Accumulated Reality

CUSUM-based anomaly accumulation records persistent directional drift over time. If V_P consistently deviates from V_A, evidence accumulates across blocks.

Skuld — Inevitability

Threshold-based escalation determines when accumulated anomaly evidence must be acknowledged and surfaced to users.

The Four Mechanisms

The three temporal layers above describe the conceptual architecture. The Scriptum System Parameters document (SPX-8) specifies the four operational mechanisms that implement the UIS: Mechanism A performs oracle input validation — cross-source consistency and statistical outlier rejection against live oracle feeds. Mechanism B requires consecutive-cycle confirmation before V_A moves, preventing single-cycle noise from disturbing the coefficient. Mechanism C is the CUSUM-based tampering alarm, raising a public flag when the aggregated V_A drifts sustainedly from oracle reference. Mechanism D is the fast-price-movement warning, raising a separate public flag when the instantaneous oracle-vs-V_A gap exceeds threshold — a legitimate market event, not tampering.

Mechanisms A and B operate within the Verðandi layer, governing what enters V_A in the present cycle. Mechanism C lives in the Urðr layer, accumulating evidence of sustained pressure. Mechanism D reaches across both Verðandi and Skuld: it surfaces fast events immediately (present) while flagging their implications for downstream consensus observation (inevitability). Both alarms (C and D) are informational. Neither halts the chain. Neither freezes V_A. Their work is transparency — making manipulation publicly visible, not blocking it by force.

During early network formation the UIS operates in bootstrap mode: oracle redundancy is intentionally reduced (`ORACLE_ACTIVE_COUNT_MIN` = 1 at genesis, ramping toward the steady-state target as the oracle network matures). Conforming wallets and observer nodes display a standing public warning during this period. If active oracle coverage falls below the minimum threshold at any time, `V_A` holds its last valid value rather than drifting on insufficient data — `INSUFFICIENT_ORACLE_MODE` = `HOLD_LAST_VALID_V_A` in SPX-8. The protocol degrades gracefully under oracle failure; it does not halt, and the coefficient cannot be forced to an attacker-favourable value through oracle denial.

V_A Proposer Architecture

`V_A` is the chain's smoothed market price reference (see `V_A` glossary entry). The integrity of `V_A` depends on the integrity of the `V_P` (per-block proposed valuation) inputs that feed into the smoothing computation. This section specifies who may propose `V_P`, how proposals are signed and rate-limited, how node coordination handles co-located instances, how oracle source diversity is preserved, how aggregation is computed, and how the architecture interacts with Hugin and Munin.

Proposer Eligibility

Every registered HOME is automatically eligible to propose `V_P`. There is no separate proposer registry, no `PROPOSE_VA_REGISTER` transaction, no admission process. The Bundle Registry — the chain-state object that records every HOME's post-quantum key bundle — is the proposer registry. If a HOME has a bundle in the Bundle Registry, that HOME may sign and submit `VA_PROPOSE` transactions.

Implicit registration is the right framing because it costs nothing to add — every HOME is already in the Bundle Registry for spend authority — and it widens the proposer base structurally. Any holder operating a HOME may participate in the `V_A` integrity subsystem; the chain does not gate participation.

Submission and Signing

`V_P` submissions are transactions of kind `VA_PROPOSE`. The proposer signs the `V_P` value plus relevant metadata (timestamp, oracle source identifier, etc.) with the registered HOME's Dilithium3 spend key. The chain validates:

The signature against the HOME's bundle in the Bundle Registry

The drift constraints — `V_P` within acceptable range of recent `V_A` history, per SPX-specified thresholds

The rate limit — at most one `VA_PROPOSE` per HOME per 42-block window (per SPX). A second submission from the same HOME within the window is rejected at consensus

Accepted submissions enter the `V_A` aggregation per the rules specified in Aggregate Computation below. Rejected submissions are dropped without state change.

Proposer Identity and Node Coordination

Proposer identity is keyed on the HOME, not on the node process. A single HOME may be operated by an Aurum wallet instance, a Hayyānu mining node, or an Ilmarinen full validation node — or by several of these co-located on the same machine. All three node types are equally permitted to submit VA_PROPOSE on behalf of their HOME.

When multiple node processes share a HOME on the same machine (e.g., an Aurum wallet co-located with an Ilmarinen full node), only one VA_PROPOSE per 42-block window is accepted at consensus regardless. Local node coordination resolves which process submits: Ilmarinen takes precedence over Aurum on co-located machines. This is a local node-coordination detail specified in the Aurum Technical Document (ATD), not a chain consensus rule. The chain enforces only the per-HOME rate limit; what the operator's machine does internally is the operator's concern.

Oracle Source Diversity

Each proposer's V_P is drawn from whatever oracle source the operator chooses — an exchange API, a market aggregator, a private feed, or any combination. The protocol does not hardcode oracle sources. The chain validates drift constraints and the rate limit; it does not validate or rank the source.

Source diversity across the proposer pool is the architectural objective. If every proposer pulled from the same upstream feed, a single oracle compromise would distort V_A across the whole pool. Independent operators selecting independent sources produces aggregate robustness: a manipulated feed shows up as an outlier in the per-block aggregate, where Hugin and Munin detect it. The protocol's role is to keep the surface open and let diversity emerge from operator choice.

Aurum ships with at least one default oracle source pre-configured (a publicly accessible market data aggregator suitable for genesis-era operation), with operator UI to add, change, or replace sources. Operators with no preference get sensible defaults; operators with specific source preferences override them. The default source is not protocol-privileged; it is a wallet-level convenience for fresh installations.

Aggregate Computation

Each block's V_A is computed from the VA_PROPOSE submissions accepted in the relevant temporal window. The aggregation function — median, trimmed mean, weighted, or other — is governance-set and specified in SPX. The same SPX section specifies the smoothing horizon, the drift constraint thresholds, and the rate-limit window.

Locking the aggregation rule in SPX rather than in the WP keeps the chain's economic-stability surface tunable by governance without requiring WP amendment for every parameter refinement. The WP specifies the architecture (who proposes, how proposals are validated, what the chain accepts); SPX specifies the numerical machinery.

Hugin and Munin Integration

Hugin and Munin (the anomaly detection mechanisms; see Hugin and Munin section below) operate over the signed proposer pool. They extend naturally from the prior miner-only framing to per-proposer anomaly tracking: each proposer's V_P history is observable, and consistent deviation from the smoothed aggregate becomes attributable to a specific HOME.

During detected anomalies, V_A is held to a smoothed value rather than tracking the anomalous aggregate. Proposers whose submissions consistently contribute to anomaly detection become visible in the per-proposer tracking surface, which governance can examine when deciding whether to recommend operator remediation. There is no on-chain deregistration mechanism — the chain accepts every signed submission within rate-limit and drift constraints; governance and the operator community handle reputational consequences off chain. The chain's role is to authenticate submissions and aggregate honestly; judgement about whose submissions to trust over time is a human concern, surfaced by the per-proposer tracking but not actioned by consensus.

The architectural separation is: V_A Proposer Architecture specifies who may propose and how submissions are authenticated and rate-limited; Hugin and Munin specify how the chain detects and responds to anomalous aggregate behaviour and surfaces per-proposer attribution. Together they form the V_A integrity subsystem under Urðarbrunnr.

Hugin and Munin — V_A Anomaly Detection

Two complementary mechanisms watch for tampering of V_A. Hugin observes the immediate per-block deviation between V_P (signed proposals from registered proposers; see V_A Proposer Architecture) and V_A (accepted valuation): for each block, Hugin records the delta. Munin maintains the cumulative running sum of those deltas — the CUSUM accumulator — and raises an informational tampering alarm in Urðr when the accumulated drift exceeds threshold in either direction. Hugin sees what just happened; Munin holds the persistent record and triggers the alarm. Together they detect sustained one-sided pressure from a dominant proposer or coordinated group of proposers attempting to silently steer V_A over time. Named for Odin's two ravens, thought (Hugin) and memory (Munin), the pair embodies the surveillance function: immediate observation feeding into long-memory accumulation. The Ratatoskr name has been redeployed elsewhere in the architecture as the chain-traversal enumeration primitive (see §Privacy Model / Ratatoskr); it no longer refers to the V_A anomaly detector.

Detection Without Intervention

UIS does not modify protocol behaviour. It does not alter fees, block transactions, or override valuation. It detects abnormal conditions, accumulates evidence, and exposes anomalies. The protocol remains fully deterministic.

Scriptum does not enforce truth. It ensures that distortion becomes visible before it becomes decisive.

Economically meaningful manipulation requires sustained control over block production across time, making such attacks capital-intensive, observable, and self-limiting.

Protected Mode — Aurum

Aurum provides native UIS integration. When abnormal conditions occur — significant divergence between V_C and V_A, or persistent proposer pressure on the V_A aggregate — Aurum activates Protected Mode: an explicit warning is displayed and additional confirmation is required before execution. Execution itself remains fully deterministic.

Mining Speed and Hardware

Scriptum uses SHA3-512 as its proof-of-work algorithm. SHA3 is the NIST-standardised sponge-construction hash family, deliberately structured to provide defense in depth against the kinds of cryptanalytic advances that have historically chipped away at SHA-2 derivatives. Scriptum does not seek ASIC-resistance through algorithmic obscurity; it commits to a transparent, well-studied primitive and accepts that mining will professionalise as the asset gains value. Anyone with a computer can mine Scriptum at genesis.

What Determines Mining Speed

Mine speed is determined entirely by difficulty adjustment, not hardware. The difficulty adjustment maintains BLOCK_TIME = 10 seconds regardless of how many miners compete or what hardware they use. More miners means higher difficulty, not faster blocks. Mine speed affects security — the cost to attack the chain. It does not affect throughput.

Retargeting fires every 2,592 blocks — the Great Year (25,920 seconds, encoding Earth's axial precession). The cadence balances responsiveness and stability: ~3.3 retargets per day track hashrate dynamics; 2,592 samples per period smooth out per-block timing noise. The retarget math is asymmetric (2× harder, 4× easier) to resist timewarp manipulation while preserving the chain's ability to recover from hashrate collapse. See §Block Parameters for the full consensus rules.

1 miner → baseline difficulty 10s blocks low security 100 miners → 100× difficulty 10s blocks growing security 1,000 miners → 1,000× difficulty 10s blocks strong security 1,000,000 → 1,000,000× difficulty 10s blocks industrial security

51% Attack at Low Hashrate — Economics

At genesis, Scriptum's hashrate is low. A sustained 51% majority can be rented. A private chain assembled from commodity GPU time — on the order of \$50,000 for several hours of block production — can reorg recent Scriptum history in the first weeks of the chain. The whitepaper concedes this openly.

What a reorg means for holder value is treated in §34. In short: a reorg in Scriptum is not theft. Bitcoin's approximately \$30 billion per year of hashrate protects holder value because Bitcoin's architecture equates authorization with execution and has no other layer

of protection. Scriptum separates authorization from execution by protocol, and therefore protects holder value independently of hashrate. The economic incentive to rent hashrate for attack collapses when the attack cannot move money: an attacker who spends \$50,000 to rent a reorg achieves a reorg, not a theft.

Hashrate still matters in Scriptum. It establishes chain ordering, confirms transaction sequence, and hardens the chain against censorship over time. It is not, however, the mechanism by which holder value is protected. For the full security analysis — including the normalized-axis comparison to Bitcoin and the complete enumeration of what a 51% attacker can and cannot do — see §34.1 through §34.4.

Low hashrate alone does not enable theft. A low-hashrate reorg attacker, without possession of a legitimate holder's Dilithium3 private key, extracts nothing. Theft requires combined control of hashrate and key material sustained across the full 8,640-block NEXUS. See §34 for the full security model.

Block Validation by Hardware

Dilithium3 verification on modern hardware (AVX2-optimised): 0.045ms per signature. A 2 MB genesis block contains approximately 100 SPEND cycles = 200 signed transactions (UNLOCK + SPEND; EJECT carries no signature). 200 verifications x 0.045ms = 9ms total. This is negligible. Propagation time (transfer + network hops) dominates, not verification.

Home laptop (CPU) → validates 2 MB genesis block in ~9ms

Desktop CPU (Ryzen 9) → validates 2 MB genesis block in ~4ms

M4 Pro (10 cores) → validates 2 MB genesis block in ~2ms

GPU rig → sub-millisecond

Propagation at 100Mbps (2 MB genesis block): 88ms transfer + 50ms hops = 138ms total. Block time 10s. Safety margin 72x. Orphan rate ~2% at genesis block size. Comfortable.

Minimum Block Time

The absolute minimum safe block time is determined by propagation, not hardware. At 100Mbps with maximum 16MB blocks: propagation = 1.28s + 0.05s = 1.33s. We choose 10s for 13% orphan target. The minimum achievable block time at 100Mbps is approximately 1.5 seconds. Block time is a protocol constant and does not change.

Block Architecture and Capacity Model

Scriptum defines block capacity through a single formula. Throughput, storage, and orphan rate are all derived from three fixed constants and one governance variable.

The Capacity Formula

Unit definitions. Scriptum uses three levels of counting and distinguishes them in this section: (i) an atomic transaction is any single protocol message — UNLOCK, EJECT,

SPEND, CANCEL, LOCK, BURN, REFRESH, EXPIRE, COINBASE, DONATE, and so on — each with its own size and rules; (ii) a signed transaction is an atomic transaction carrying a user signature (UNLOCK, SPEND, CANCEL, LOCK, BURN, DONATE, REFRESH); EJECT, EXPIRE, and COINBASE are protocol-driven and unsigned; (iii) a SPEND cycle is the canonical settlement sequence UNLOCK → NEXUS dwell → EJECT → SPEND, a complete value transfer from a holder's LOCKED Parcel to a recipient's LOCKED Parcel at a stealth-derived destination. The NEXUS dwell sits on the unlock side only; SPEND of ACTIVE Scriptum is direct. Throughput figures in this section are given in settlements per second (= SPEND cycles per second). A 'settlement' and a 'SPEND cycle' mean the same thing. Raw atomic transaction counts include protocol-driven entries and are not used as the headline throughput metric.

SPENDs per second is determined exclusively by:

$$\text{SPENDs/sec} = \text{OR} \times \text{MBPS} / \text{CYCLE_SIZE}$$

OR = orphan rate maximum ACCEPTED (13% — protocol hard ceiling, not a target)

MBPS = bandwidth in bytes/sec (external variable)

CYCLE_SIZE = UNLOCK + EJECT + SPEND bytes (~7,100 first-touch, ~5,500 bound)

Block time cancels out of the throughput formula. Only bandwidth, orphan rate, and signature size determine SPENDs/sec. Block time affects only how capacity is chunked, not total throughput.

The capacity formula describes the theoretical throughput ceiling at the bandwidth-limited maximum block size (approximately 16 MB at 100 Mbps, yielding ~150 settlements/sec). At the genesis block size of 2 MB, realistic settlement throughput ranges from ~18/sec at fresh HOMEs (first-touch UNLOCK reveals the full post-quantum key bundle to the Bundle Registry) to ~25/sec at HOMEs already registered (subsequent UNLOCKS reference the Registry by address; no bundle reveal). Steady-state settlement throughput on a mature chain with address-reuse history is approximately ~22/sec at 2 MB. Every SPEND includes the sender's ephemeral Kyber public key for stealth derivation; spends consuming stealth-address inputs each include a 64-byte commitment_opening; both are factored into these figures. See §18.6 for the scaling path.

Fee Cascade — Block Aggregation, Menuha, Quarterly Distribution

Scriptum collects and distributes fees through a three-cadence cascade. Each user transaction declares its fee implicitly (input total – output total – fee field = fee paid to chain). The chain aggregates these fees at three timescales.

Per-Block Aggregation (every block, 10 seconds)

At block construction, the chain creates two protocol-driven Parcels:

Miner aggregate Parcel at the miner's HOME: carries the block reward (per the emission formula) plus 90% of the sum of all transaction fees in this block.

Protocol aggregate Parcel at VAULT_HOME (a constitutional namespace address): carries 10% of the sum of all transaction fees in this block (the tithe portion).

These two Parcels are created by the protocol; they do not pass through NEXUS_DWELL. They are protocol-mechanics, not signed value movements.

Menuha Separation (every 60,480 blocks, ~weekly)

At each Menuha block (every 60,480 blocks, approximately weekly), the chain protocol-drives a separation transaction:

Consumes all VAULT Parcels accumulated since the previous Menuha

Creates new Parcels at the three constitutional destinations:

- FUND_HOME (operating budget; Curia-disbursed via Box mechanism)
- RESERVE_HOME (deep reserve; L4 governance gated)
- TITHE_HOME (the constitutional tithe pool)

The exact allocation ratios are governance parameters specified in SPX (the System Parameters document).

Quarterly TITHE Distribution (every 13th Menuha, ~90 days)

The TITHE distribution cadence is quarterly, but the actual distribution transaction fires at the nearest Menuha to the quarterly boundary — specifically, every 13th Menuha. Distribution piggybacks on the chain's natural weekly rhythm rather than requiring a separate quarterly-block cadence.

At every 13th Menuha, in addition to the standard weekly Menuha separation (VAULT → FUND + RESERVE + TITHE), the chain protocol-drives the TITHE distribution:

Consumes all TITHE Parcels accumulated since the previous quarterly distribution (approximately 13 Menuha's worth of TITHE Parcels)

Creates new Parcels at the constitutional charity destinations: Charity Pot, WFC, and other destinations specified in the Governance Charter

The allocation ratios are governance variables, adjustable through the annual TITHE ratio adjustment mechanism (see SPX-specified ratio-adjustment window, opens once per year, multisig-authorized at L3 or L4 per Governance Charter).

TITHE Ratio Adjustment

TITHE allocation ratios (Charity Pot share, WFC share, other destinations' shares) are governance variables, not protocol constants. The adjustment mechanism is a structurally rate-limited annual window.

Once per year — at a specific block height specified in SPX — the TITHE ratio adjustment window opens for a defined duration (per SPX). During the window, a Curia L3 or L4 multisig transaction (threshold per Governance Charter) may submit a new ratio set. The submission is validated against:

Ratios sum to 100% of TITHE

No individual ratio exceeds a structural ceiling (per SPX, to prevent runaway concentration)

The proposing multisig satisfies the L3 or L4 threshold

If a valid ratio submission lands in the window, the new ratios take effect at the next quarterly distribution. If no submission lands, the previous ratios persist for another year.

The annual cadence provides:

Predictability — charity recipients can plan against stable ratios for at least a year

Protection — governance cannot drift ratios continuously; rebalancing requires deliberate annual action

Adaptability — the constitution can learn over time as the chain matures and charity priorities evolve

The mechanism mirrors the Box mechanism's architectural pattern: a window opens, a multisig acts within the window, the chain accepts or ignores based on whether the window was used. No continuous governance pressure on ratios; no hard-fork-grade inertia either.

The detailed parameters — the annual block height of the window opening, the window duration, the structural ceiling, the L3 vs L4 threshold — are specified in SPX and the Governance Charter.

Properties

The three-cadence cascade preserves these properties:

User transactions stay small. Fees are declared in metadata; no per-tx fee Parcel is materialised.

Per-block overhead is bounded (~2,800 bytes for two aggregate Parcels per block).

Constitutional destinations (FUND, RESERVE, TITHE, Charity Pot, etc.) receive consolidated value at meaningful cadences rather than per-transaction trickle.

Protocol-driven aggregation and separation bypass NEXUS_DWELL (no human signer to protect).

The chain's supply accounting tracks aggregated values exactly across the cascade.

In Practice

To illustrate concretely: every 10 seconds, a block is mined. The miner receives one COINBASE Parcel containing the block reward (newly issued FUNK per the emission schedule) plus 90% of the sum of all transaction fees in that block. The protocol creates one VAULT Parcel at VAULT_HOME containing 10% of those fees. Two Parcels per block, regardless of how many transactions were in the block.

Every Saturday at 23:00 UTC, the Menuha boundary triggers the chain's weekly separation. The protocol consolidates all VAULT Parcels accumulated over the past week —

approximately 60,480 small VAULT Parcels — into three new Parcels at the constitutional destinations FUND_HOME, RESERVE_HOME, and TITHE_HOME. One separation per week.

Every thirteenth Menuha (approximately every 90 days, the quarterly cadence), in addition to the standard weekly separation, the protocol executes the TITHE distribution: all TITHE Parcels accumulated over the past quarter (approximately 13 Menuha's worth of TITHE Parcels) are consumed and distributed to the constitutional charity destinations (Charity Pot, WFC, and others per the Governance Charter). The TITHE distribution piggybacks on the 13th Menuha — no separate quarterly-block cadence to track; the chain's natural weekly rhythm provides the slot.

User transactions stay small throughout. When a holder sends 100 FUNK to a recipient, the holder's SPEND transaction does not create per-transaction fee Parcels; the fee is declared in transaction metadata. The actual fee Parcels are aggregated at the block level (the miner's COINBASE + the VAULT Parcel) regardless of how many transactions were in the block. A SPEND is approximately 6,300 bytes whether the block contains one transaction or 332.

Protocol-driven aggregation and separation bypass NEXUS_DWELL because there is no human signer to protect. NEXUS_DWELL exists to give holders a 24-hour cancellation window against compromised-key transactions. Protocol-driven Parcels are created by deterministic consensus rules; they do not need a cancellation window because no human action initiated them.

HICP Indexing — Annual Real-Value Calibration

All EUR-denominated values in the Scriptum protocol are 2026-reference values, adjusted annually by the Harmonised Index of Consumer Prices (HICP) factor for the prior calendar year (per Scriptum Governance Charter §4.2). HICP indexing preserves real economic structure across the chain's century-scale lifetime: a €1,000 fee bracket in 2026 represents the same real purchasing power as the HICP-indexed equivalent in 2126.

EUR-denominated values subject to HICP indexing

The chain HICP-indexes the following 2026-reference values annually:

f_S progressive bracket boundaries (Tier 1 through Tier 10: €1, €1,000, €10,000, €50,000, €100,000, €200,000, €500,000, €1,000,000, €2,000,000, €10,000,000)

Dust threshold (€0.03)

Flat instruction fees: F_UNLOCK (€0.04), F_ACTIVATE (€0.01), F_REFRESH (€0.01)

Personal compensation caps (Charter §4.1): WOC €2,000,000, Founder €1,800,000, Curia €1,300,000

FUND_DISBURSEMENT_L1_CAP (€100,000 cumulative annual budget cycle; Charter §5.3)

All other EUR values referenced in the Whitepaper as architectural anchors are 2026-reference and subject to the same HICP-indexing rule.

The cumulative HICP factor

The chain maintains a single Tier 1 chain-state value: `cumulative_HICP_factor`, initialised to 1.00 at genesis. The cumulative factor compounds the annual HICP adjustments across all years since genesis:

$$\text{cumulative_HICP_factor} = \prod \text{ over all ratified years } N \text{ of } \max(1.00, \text{HICP_factor_year_}N)$$

The $\max(1.00, \dots)$ enforces Charter §4.2's monotonic non-decreasing rule: a year of negative HICP (deflation) contributes a factor of 1.00 — thresholds never decrease.

Application of the cumulative factor

At all fee calculation and cap check moments, the chain applies the cumulative HICP factor to the 2026-reference EUR value before further processing:

$$\text{effective_EUR} = \text{base_2026_EUR} \times \text{cumulative_HICP_factor}$$

For FUNK conversion, `V_A` is then applied:

$$\text{FUNK_amount} = \text{effective_EUR} / V_A$$

Example: an `f_S` Tier 1 boundary of €1,000 (2026-reference) becomes €1,000 × `cumulative_HICP_factor` in effective EUR, then divided by current `V_A` to produce the FUNK-equivalent threshold for the current block.

HICP Oracle and Annual Ratification

The chain knows the current cumulative HICP factor; it does not compute next year's adjustment. The annual HICP factor is observed externally and committed to chain state through a two-stage process: open observation followed by Council ratification.

The annual cycle:

1 January, year `N` — Observation Window opens. A 30-day window begins. During this window, any chain participant (validator nodes running `Ilmarinen`, miners running `Hayyānu`, or any other Dilithium-signing chain participant) may submit a `HICP_PROPOSE` transaction containing: their observed HICP factor for year `N-1` (the most recently complete calendar year), a source identifier (e.g., "Eurostat euro area HICP all-items annual"), and their Dilithium signature. There is no signer restriction; any signer may submit; the chain aggregates across all valid submissions.

31 January, year `N` — Observation Window closes. The chain stops accepting `HICP_PROPOSE` submissions. The aggregation function (median of valid submissions) processes all submissions during the window. If the window collected at least 3 distinct signer submissions, the aggregation produces a candidate HICP factor; chain state mutates to `pending_HICP_ratify = aggregated_value`, recorded with the `observation_close_block` and the `ratification_deadline_block`.

1 February through 31 March, year `N` — Countersign Window. The chain awaits Council ratification of the pending factor. Curia members' council apps observe chain state, surface the pending `HICP_RATIFY` notification, and allow Curia members to sign locally with their

Dilithium keys. 5-of-9 Curia signatures are required (L1 multisig per Charter §5.1). When 5-of-9 signatures collect, the HICP_RATIFY transaction broadcasts to chain; the chain validates signatures against the Curia Lock (Council Registry, see §Council Registry) and commits the cumulative HICP factor update.

31 March, year N — Countersign Window closes. If HICP_RATIFY committed during the window, the new cumulative factor takes effect for all subsequent fee calculations and cap checks. If HICP_RATIFY did not commit (insufficient Curia signatures), the candidate factor expires; the cumulative factor carries forward unchanged from the prior year. The chain continues to function with prior-year HICP applied; nothing breaks.

From 31 March onward (or earlier if HICP_RATIFY committed): the chain uses the updated cumulative HICP factor in all EUR conversions.

Failure modes:

No submissions or fewer than 3 distinct signers during Observation Window: no candidate; cumulative factor carries forward from prior year; governance investigates why observation didn't reach quorum

Submissions disagree widely: median aggregation suppresses outliers; if disagreement is severe enough that even median produces an obvious error, Council may decline to ratify and the candidate expires

Council declines to countersign: candidate expires; cumulative factor carries forward from prior year; the chain is informed by the Council's silence (not committing the HICP_RATIFY transaction)

Council can't reach 5-of-9 quorum: same as decline; candidate expires; cumulative factor carries forward

In all failure modes, the chain continues to function. The HICP monotonic non-decreasing rule is trivially preserved by carrying forward the prior cumulative factor (factor 1.00 contribution for that year).

Why two stages

The two-stage architecture (open observation + Council countersign) reflects honest separation of trust:

Observation is objective: the HICP factor is published publicly by Eurostat. The chain just needs to be informed of the official published value. Any chain participant who can read Eurostat's publication can submit. The aggregation function (median) suppresses honest mistakes and prevents single-observer manipulation.

Ratification is constitutional: the Council ratifies what the oracle observed — they can countersign or decline, but they cannot propose an arbitrary value. This bounded discretion is the protection against governance fabricating inflation adjustments. If the oracle output is corrupted, the Council can decline; if the oracle output is correct, the Council ratifies.

The combination ensures the chain's HICP indexing reflects the public Eurostat reality while remaining anchored to constitutional governance.

Source identification

The chain references Eurostat euro area HICP all-items annual figures as the authoritative source. The Eurostat publishes the prior-year annual HICP factor in mid-January with revisions through mid-February. The 30-day Observation Window covers the publication and initial revisions; the Countersign Window allows the Council to verify against finalised Eurostat data before ratifying.

HICP_PROPOSE submissions that reference a different source identifier (e.g., a national HICP variant, or HICP-CT excluding tax changes) are technically valid signed transactions but will diverge from majority submissions and be suppressed by median aggregation. Governance investigates persistent source discrepancies.

HICP Transaction Types

The chain defines two new transaction types for the annual HICP cycle.

HICP_PROPOSE. Submitted by any chain participant during the Observation Window (1 January – 31 January). Carries:

proposed_factor (numeric, HICP factor for the prior calendar year, e.g., 1.024 for 2.4% annual inflation)
source_identifier (canonical string, e.g., "EUROSTAT_HICP_ALL_ITEMS_ANNUAL_2026")
observed_at (millisecond timestamp of the observer's reading)
submitter_pubkey (the submitter's Dilithium pubkey; signed transaction)
signature (Dilithium3 signature over the transaction body)

The chain validates: signature against pubkey; submitted within the active Observation Window; well-formed numeric factor (typically $0.9 \leq \text{factor} \leq 1.2$ for sane values); source identifier well-formed string. Submissions accumulate in chain state as a multiset; the chain doesn't deduplicate by submitter (a single submitter may submit multiple observations — the median aggregation handles this). Fee: zero (chain primitive observation transaction; the same model as VA_PROPOSE).

HICP_RATIFY. Submitted during the Countersign Window (1 February – 31 March). Carries:

ratification_target (reference to the current pending_HICP_ratify candidate factor + observation_close_block)
multisig_signatures (5-of-9 Curia Dilithium signatures, verified against Curia Lock per Council Registry; see §Council Registry)

The chain validates: ratification_target matches current pending_HICP_ratify chain state; 5-of-9 Curia signatures collected from Curia Lock pubkeys; submitted within the active Countersign Window. On valid HICP_RATIFY: chain commits the cumulative HICP factor update:

$\text{cumulative_HICP_factor_new} = \text{cumulative_HICP_factor_prior} \times \max(1.00, \text{candidate_factor})$

Fee: zero (chain primitive governance transaction).

CAP_AMENDMENT_RATIFY. L4 multisig transaction (WOC + 3-of-5 Founders + 7-of-9 Curia) authorised only during the HICP Countersign Window (1 February – 31 March each calendar year). Carries new base values for FUND authorisation cap parameters: `monthly_cap_base_eur`, `annual_cap_base_eur`, `tier_small_base_eur`, `tier_medium_base_eur`, `tier_large_base_eur`. The chain validates: L4 signatures against constitutional locks (Council Registry); submitted within active Countersign Window; well-formed numeric base values. On valid CAP_AMENDMENT_RATIFY: chain commits new `fund_caps_base_2026` chain-state object; new values effective from ratification block onward. HICP indexing continues to compound on the new base. Zero fee (chain primitive governance transaction; same model as HICP_RATIFY).

Protocol-driven transitions (no signer required):

Observation Window Open: at the block height corresponding to 1 January year N, chain state mutates: `observation_window_active` = true; `observation_close_block` = (1 January year N block) + (30 days × `blocks_per_day`)

Observation Window Close + Aggregation: at `observation_close_block`, chain processes accumulated HICP_PROPOSE submissions: computes median of valid submissions; if at least 3 distinct signer pubkeys are present, `pending_HICP_ratify` = median; `countersign_deadline_block` = `observation_close_block` + (60 days × `blocks_per_day`); otherwise `pending_HICP_ratify` = null (carry-forward)

Countersign Window Close: at `countersign_deadline_block`, if `pending_HICP_ratify` is committed (HICP_RATIFY succeeded), the new cumulative factor takes effect; if `pending_HICP_ratify` expired without ratification, the cumulative factor carries forward unchanged

Transaction Sizes

The figures below reflect the universal Parcel structure and the bundle composition.

Bundle composition: version 1 byte + Dilithium3 pubkey ~1,952 bytes ≈ 1,953 bytes total (Kyber public key material and view-key derivation material are not in the bundle; they are maintained in the View-Key Registry as a separate chain-state object — see Three-Tree Key Architecture and Stealth Settlement). Per-transaction accounting: transaction base ~3,400 bytes (header + home_address + Dilithium3 signature + metadata); per input ~100 bytes (outpoint reference + commitment_opening); per output ~1,400 bytes (universal 16-field Parcel structure including kyber_ciphertext ~1,088 bytes + spend_commitment 64 bytes for stealth outputs, or domain-separated public constants for HOME-class outputs). A typical user SPEND consumes one input and creates two outputs (stealth payment + change): total ~6,300 bytes. Block-level overhead: ~2,800 bytes for two aggregate Parcels per block (miner + VAULT) created by protocol mechanics. Net available for user transactions at the 2 MB genesis block size: ~2,094,000 bytes, yielding ~332 typical SPENDs per block at sustained ~25 SPENDs per second.

Signed user transactions.

UNLOCK ~6,853 bytes user-signed (first-touch UNLOCK at a new HOME — transaction base ~3,400 + per input ~100 + universal output Parcel ~1,400 + HOME bundle reveal for Registry entry ~1,953; per additional stealth input add ~100 bytes commitment_opening)

UNLOCK ~4,900 bytes user-signed (subsequent UNLOCK at a registered HOME — transaction base ~3,400 + per input ~100 + universal output Parcel ~1,400; references HOME by address, no bundle reveal; per additional stealth input add ~100 bytes commitment_opening)

CANCEL ~3,500 bytes user-signed (signature path: Dilithium3 over the cancellation reference) or ~50 bytes (hash-preimage path: just S plus the dwelling-Parcel reference)

SPEND ~6,300 bytes user-signed: transaction base ~3,400 + one input ~100 + two outputs ~2,800 (stealth payment ~1,400 + change ~1,400, each carrying universal-structure kyber_ciphertext, spend_commitment, and the universal 16 Parcel fields). The Dilithium3 signature alone is ~3,293 bytes, accounting for over half of every transaction.

LOCK ~3,500 bytes user-signed, self-send to LOCKED (offset > 0 if a user-initiated time-lock is requested; the universal spendability rule applies)

BURN ~3,500 bytes user-signed, value sent to Gehenna

REFRESH ~4,900 bytes user-signed: consumes the existing ACTIVE Parcel at HOME and creates a fresh successor ACTIVE Parcel at HOME with reset activate_height and new expiry_height (transaction base ~3,400 + one input ~100 + successor Parcel ~1,400; HOME-class output, no kyber_ciphertext but full universal Parcel structure)

DONATE ~3,500 bytes user-signed, value to FUNDS + RESERVE split

INSCRIBE variable user-signed (~3,500 bytes baseline + inscribed content payload up to 1,572,864 bytes; the Parcel created references the content via content_hash and content_size and lives at the inscriber's HOME, ~200 bytes added to the Parcel set; fee is the continuous-formula fee per Inscription section, paid in FUNK at V_A)

Protocol-driven transitions.

EJECT 264 bytes protocol-driven, no signature (executes deterministically at creation_height + 8,640)

EXPIRE 232 bytes protocol-driven, automatic (fires when ACTIVE Parcel reaches creation_height + ACTIVE_WINDOW_BLOCKS without being consumed)

RESEAL 200 bytes protocol-driven, fires for OPEN VAULT at creation_height + OPEN_WINDOW_BLOCKS

VEST ~200 bytes protocol-driven, fires at each vesting milestone block height; reads the slot's Box state and creates either a LOCKED (CIRCULATING) Parcel at the keyholder's HOME or a re-locked LOCKED (TREASURE) Parcel at the slot HOME (see VEST in Protocol-Driven Transactions)

COINBASE 264 bytes miner reward, no signature

SWEEP variable protocol-driven (Menuha-time consolidation; size scales with number of swept stealth-address Parcels, each contributing 64 bytes commitment_opening)

Cycle size.

CYCLE_SIZE = UNLOCK + EJECT + SPEND $\approx$ 13,400 bytes (first-touch UNLOCK at a fresh HOME, full bundle revealed) to $\sim$ 11,500 bytes (registered HOME, bundle in Registry by address reference). Every SPEND carries the encrypted Sealed Sender Memo for the recipient and the universal-structure output Parcels.

Genesis Block Parameters

BLOCK_TIME 10 seconds protocol constant, fixed forever

BLOCK_SIZE_MAX 2 MB genesis setting, Curia variable

NEXUS 8,640 blocks = 24 hours, fixed forever

OR_MAX 13% accepted ceiling (not target). Blocks exceeding this orphan rate are rejected by consensus. We do not tolerate degradation by accident.

Settlements/sec $\sim$ 18 at 2 MB genesis (full settlement cycle = UNLOCK + EJECT + SPEND $\approx$ 11,500 bytes; $\sim$ 25 SPENDs/sec for raw SPEND-only throughput); scales linearly with BLOCK_SIZE_MAX

Storage/day max 17.28 GB if every block at 2 MB genesis is completely full (2 MB $\times$ 8,640 blocks/day). At genesis the chain is near-empty; actual storage approaches zero. Storage growth is adoption-driven and bounded by block size $\times$ fill rate $\times$ seconds per day, not by chain age. The transition from near-empty to fill-rate-bound takes years of organic adoption; specific GB/day projections at genesis are not meaningful.

At genesis the chain is nearly empty. Blocks are small. Storage is trivial. Coinbase fills the miner reward regardless of transaction volume. Empty blocks are not a problem — they are the natural state of an early chain. Satoshi mined empty blocks for months.

Block Size — The Single Governance Variable

Block size is the only parameter Curia ever needs to change. All other capacity parameters follow automatically.

Increase BLOCK_SIZE_MAX $\rightarrow$ more SPENDs/sec (linear)

$\rightarrow$ more storage/day (linear)

$\rightarrow$ higher orphan rate (capped at 13%)

$\rightarrow$ no other protocol changes

The block size ceiling at any bandwidth is:

$$\text{BLOCK_SIZE_MAX} = \text{OR} \times \text{bandwidth_bytes/sec} \times \text{BLOCK_TIME}$$

At 100Mbps: $= 0.13 \times 12,500,000 \times 10 = 16.25$ MB maximum

At 1Gbps: $= 0.13 \times 125,000,000 \times 10 = 162.5$ MB maximum

Curia begins at 2MB and increases as adoption demands. The governance question is always simply: do we need more SPENDs/sec? If yes, increase BLOCK_SIZE_MAX. Storage is the cost. That is the only tradeoff.

Miners determine block fullness by direct accounting. Each transaction has a deterministic serialized byte size known at construction time. A miner constructing a block selects transactions from their mempool one at a time, summing their sizes. When adding the next transaction would exceed BLOCK_SIZE_MAX (2 MB at genesis, governance-tunable) or MAX_TX_PER_BLOCK (8,192 transactions, structurally fixed), the miner stops adding and finalises the block. Block-level overhead (~2,800 bytes for the COINBASE + VAULT aggregate Parcels) is reserved upfront. Protocol-driven transitions (EJECT, EXPIRE, VEST, RESEAL, BOARD, etc.) take priority over user transactions per the priority queue rules (see Protocol-Driven Transitions and Limbo); user transactions fill remaining block capacity, typically sorted by fee-per-byte to maximise miner revenue. Mempool policy is operational implementation detail (not consensus); the consensus rule is simply that the finalised block must satisfy the size and transaction-count limits.

Protocol-Driven Transitions and Limbo

The chain has multiple protocol-driven transitions that fire deterministically at their scheduled triggers: EJECT (at unlock_height + NEXUS_DWELL_BLOCKS, deterministic from chain state — no randomness beacon), EXPIRE (at activate_height + 8,640), VEST (at TREASURE vesting milestones; protocol-driven Box-mediated transition), RESEAL (at OPEN VAULT window close), BOARD (at the Ragnarök boarding slot), COINBASE (at every block).

A block must include due protocol-driven transitions in priority order up to available block capacity. The priority order is: BOARD > EXPIRE > EJECT > RELEASE > RESEAL > COINBASE. Within each class, transitions are ordered by the block height of the originating event (oldest first).

SWEEP — Frau Holle’s cross-address consolidation — is wallet-driven and holder-signed, not protocol-driven. It does not appear in the priority queue. The wallet submits SWEEP transactions during Menuha windows like any other holder transaction; they compete for the user-transaction portion of block space. Aurum distributes its SWEEP submissions across the Menuha window to amortise mempool load.

Transitions exceeding block capacity extend their Limbo state to the next block.

Limbo is the architectural concept already defined for the brief interval between transition eligibility and actual block execution (see §[NEXUS] for the EJECT-side specification). Under normal load, Limbo is milliseconds — the next block always has room. Under congested load, Limbo extends to seconds or minutes as the queue drains across subsequent blocks. The transition’s eligibility is preserved; only its execution is queued.

This rule guarantees liveness under any load. Blocks remain valid as long as they make progress on the priority queue up to their capacity. Transitions never disappear; they wait. The chain never stalls.

For the holder, the visible effect is that the wallet's "AWAITING EJECT" Limbo status may persist beyond the typical milliseconds during congestion. CANCEL is invalid during Limbo regardless of duration (the dwell window has technically closed at the assigned eject slot). Aurum's reassurance notification at >10 seconds of Limbo (per the UX requirements) handles this naturally.

Pathological-overflow safety. If Limbo extension consistently grows over many consecutive blocks (a pathological case implying chain misconfiguration or attack), Curia may invoke the emergency block size variable to expand capacity until backlog clears. The chain continues to execute correctly even under sustained overflow; transitions drain in priority order, no transition is lost, no chain state corrupts.

Storage Formula

$\text{Storage/day} = \text{OR} \times \text{MBPS_bytes} \times 86,400$

At 100Mbps (13% orphan): $0.13 \times 12,500,000 \times 86,400 = 137 \text{ GB/day}$ maximum

At genesis (2 MB blocks): ~17.28 GB/day if every block is completely full; near zero actual at launch (empty chain)

Block time cancels from the storage formula too. Storage per day depends only on bandwidth, orphan rate, and seconds per day — nothing else.

The Scaling Path

Genesis (2 MB) → ~18 settlements/sec, up to ~17 GB/day storage if every block is full, orphan rate ~2%

Curia (4 MB) → ~36 settlements/sec, up to ~35 GB/day, orphan rate ~3%

Curia (8 MB) → ~72 settlements/sec, up to ~69 GB/day, orphan rate ~6%

Curia (16 MB) → ~144 settlements/sec, up to ~138 GB/day, orphan rate ~13% (100 Mbps bandwidth ceiling)

1 Gbps era (160 MB) → ~1,440 settlements/sec, up to ~1.38 TB/day

One parameter. One governance vote. The chain starts on a phone and ends in an industrial complex. That is success.

Horkos

For high-value or trust-sensitive interactions, Scriptum provides Horkos. Horkos enables bilateral commitment of funds, protocol-enforced conditional execution, third-party attestation where required, and deterministic settlement or automatic refund.

Use cases include delivery-versus-payment transactions, remote exchanges without prior trust, institutional over-the-counter agreements, and any settlement where the parties want cryptographic enforcement over custodial intermediation.

Within Horkos, funds are held under protocol control at an ESCROW address. No party — not the parties to the trade, not the attestor, not the Foundation, not Curia — can discretionarily move escrowed funds. Transfer occurs only upon protocol-driven resolution per the agreement signed at the start of the trade. Failure to perform results in deterministic routing per the agreement's pre-committed terms.

Horkos replaces traditional escrow structures without introducing custodial intermediaries. The escrow is the protocol itself.

Privacy Model

Scriptum is settlement infrastructure built on a deliberate prioritisation of ideas: safety, security, controlled transfer, usability, auditability, fairness, societal reroute. **Reasonable privacy is one of these properties. Unbreakable secrecy at every level is not.** The chain provides privacy in the bank-wire sense — ordinary commercial transactions are not subject to broadcast public surveillance — but does not commit to maximum stealth as a chain-level property.

This framing is a choice, not a default. Privacy beyond the chain's baseline is available to holders through compositions of existing primitives — multiple home addresses, stealth-on-receive, view-key delegation with the epoch-bounded and rotatable construction, and operationally-disciplined receive-address hygiene. Cryptographic anonymity for specific transactions is planned for a future Scriptum V2 specification; this document does not specify V2 extensions.

Scriptum is not a money laundromat. We are Scriptum — that which is written. The chain is the record, attributable, auditable, recoverable. Selective disclosure is the design pattern. Subpoena power is acknowledged as outside the chain's threat model. Users seeking unaccountable concealment of illegal activity should evaluate whether Scriptum is the right tool for them; users seeking institutional-grade settlement infrastructure with reasonable privacy and built-in audit will find the chain's properties match their needs.

This framing is deliberate. The chain has competing interests with stealth: auditability requires verifiable attribution; institutional positioning requires compliance compatibility; societal reroute (the Tithe-funded vaults) requires the chain to be a legitimate civic instrument; constitutional governance requires verifiable on-chain operations; fairness requires structural transparency on allocation and vesting. **The chain chooses these commitments and accepts that they constrain the stealth claim.** Holders who want stronger privacy postures can compose them from the available wallet-side primitives; the chain provides the primitives without claiming maximum stealth as the chain's purpose.

Scriptum is not a drug dealer's hideout. The architecture is auditable by design. Selective disclosure is the design pattern. Subpoena power is acknowledged as outside the chain's threat model. Users seeking unaccountable concealment of illegal activity should evaluate

whether Scriptum is the right tool for them; users seeking institutional-grade settlement infrastructure with reasonable privacy and built-in audit will find the chain's properties match their needs.

The Privacy Stack

Scriptum's privacy stack is built from four primitives, each providing a specific property and composing with the others into the chain's overall posture.

Stealth derivation per receive. Every SPEND derives a fresh one-time output address via Kyber/ML-KEM key encapsulation between the sender (who encapsulates an ephemeral shared secret) and the recipient's published Kyber view key. No persistent on-chain identifier links incoming payments to a recipient's published address. Every receive is a cryptographically distinct on-chain identity, derivable as the recipient's only by the recipient's view key. This is the chain's foundational pseudonymity primitive: receives are unattributable to a published identity by chain analysis alone.

Movement-side consolidation. UNLOCK consolidates value from many of a holder's stealth-derived addresses to the holder's HOME address, producing ACTIVE Parcel at HOME after the NEXUS dwell window. Frau Holle's Menuha-time cross-address sweep operates similarly across the holder's full address cluster, also producing output at HOME. Stealth-derived receive addresses become empty after consolidation; the holder's effective storage location is HOME for ACTIVE/operational use, with new receives continuing to land at fresh stealth-derived addresses until the next consolidation.

Multiple home addresses (baseline composition). A holder may operate multiple home addresses, each its own branch of the deterministic key tree derived from the seed, each with its own Dilithium signing key, Kyber/ML-KEM encapsulation key, and view-key generation. A holder can route value between their own home addresses via standard SPEND. This composition provides practical compartmentalisation between separate operational roles — a holder may use one home address for personal payments, another for business income, another for accumulating savings, all derived from the same seed. The chain provides the primitives (deterministic derivation, stealth output construction); the wallet provides the management surface (per-address labelling, per-address view-key derivation, routing controls). The cost is operational complexity in tracking which address holds which Parcels.

Sealed sender memo. Every SPEND carries an encrypted memo addressed to the recipient (using the recipient's Kyber view key as the encryption target). The memo carries the sender's published address by default, and may carry additional content within the size budget. Recipients decrypt the memo and know which address sent them value — preserving the address-level provenance property that recipients have on transparent chains today. External observers cannot read the memo without the recipient's view key.

The memo is the chain's per-transaction audit primitive: holders can selectively disclose memos to auditors, regulators, or counterparties to prove specific transactions without revealing their broader on-chain footprint. The memo is mandatory in form (every SPEND carries a well-formed encrypted memo) but optional in content (holders choose

what to put in the memo, within the minimum requirement of the sender's published address).

Architectural Note: Validity Versus Anonymity

The four primitives above provide pseudonymity comparable to ordinary banking records — strong protection against chain-only adversaries, with full holder-controlled audit. Counterparties and lawful investigators may obtain transaction visibility through view-key delegation while the general public does not. They do not provide cryptographic anonymity. The distinction matters for any reader trying to map Scriptum's privacy stack onto the deeper cryptographic literature.

Validity is mechanical. Every transaction's authority to spend is verified by signature and hash check — the spender produces a Dilithium3 signature over the transaction, the chain verifies the signature against the registered public key. This is ancient cryptography; hashes commit and signatures bind. No zero-knowledge machinery is required for the chain to enforce that only authorised holders move value.

Anonymity is what zero-knowledge buys. The primitive that hides which authorised credential among many was used — without revealing the credential itself — is the zk-proof. STARKs and SNARKs are the families; both prove "I know a witness satisfying this circuit" without disclosing the witness. Cryptographic anonymity for specific transactions in Scriptum is planned for a future Scriptum V2 specification, using STARK-based ring-membership proofs over fixed-denomination commitment trees to deliver credential-set unlinkability while preserving consensus-validatability. The consensus-rule specification for V2 cryptographic anonymity will be developed under the Foundation's research-and-standardisation track post-Genesis. The architectural reason cryptographic anonymity requires denomination tiers — and not arbitrary amounts — is structural: cryptographic anonymity is tractable when the primitive must hide exactly one variable (which leaf in the set), reducing the proof's witness to a single hidden choice; amounts must be a public label, leaving leaf identity as the only hidden variable. See the V2 forward-reference glossary entry.

Anonymity scales with denomination collision, not with cryptographic cleverness. STARKs hide which leaf in the tree was spent; the tree's anonymity set is determined by how many holders deposited at the same denomination. Without forced collision through fixed denominations, every depositor's amount would be its own near-empty tree, and STARKs would protect credential sets of size one. The seven denominations are infrastructure for collision; the cryptography rides on top of that collision.

This architectural truth is the design principle behind Scriptum's tiered privacy stack. The bank-wire tier (stealth + view keys + sealed memo) requires no zk machinery and has no anonymity-set cost. A future cryptographic-anonymity tier (see V2 forward-reference glossary entry) would accept the infrastructure cost (STARK prover, denomination decomposition, per-payment proof) in exchange for credential-set unlinkability. Holders who require cryptographic anonymity for specific transactions will route through the future V2 primitive when it is specified and shipped.

What the Chain Provides and Does Not Provide

What Scriptum provides:

Privacy against chain-only adversaries. Chain analytics firms, competitors, casual chain readers, surveillance services without subpoena power cannot link senders to recipients without view-key access, cannot identify a holder's address cluster across compartmentalised identities, cannot determine balances of named entities, and cannot see the topology of payment flows between specific parties.

Address-level recipient identification via decrypted memos. Recipients see who sent them value (in the form of the sender's published address per the memo) — the same property they have on transparent chains today, delivered cryptographically rather than via on-chain visibility.

Audit-grade compliance via selective disclosure. Holders can disclose individual memos for per-transaction audit (narrowest scope) or delegate a home address's view key for per-identity ecosystem audit (broader scope). Disclosure may be **range-bounded** by block-height range, so that the disclosed view-key cryptographically decrypts only outputs in that range, and **revocable forward in time** through master rotation, after which the old view-key no longer decrypts new outputs. Together the two refinements give holders precise control over the temporal scope of every disclosure they make. See §Selective Disclosure below for the construction.

Per-address compartmentalisation through multiple home addresses. Holders may operate multiple home addresses derived from the same seed, each its own ecosystem of stealth-derived receive addresses, each appearing on chain as an independent commercial counterparty without revealing the shared-seed origin. Routing between a holder's own home addresses via standard SPEND breaks on-chain linkability between the holder's operational compartments. The chain provides the primitives; the wallet provides the management surface.

What Scriptum does not provide:

Anonymity in the strong cryptographic sense. The chain is not designed to defeat state-level adversaries with subpoena power, who can compel KYC disclosure from CASPs, view-key disclosure from auditors, and identification through off-chain context. Scriptum does not pretend to provide this. It provides structural pseudonymity against chain-only adversaries plus full holder-controlled audit, and accepts that off-chain compulsion is outside the chain's threat model.

Cryptographic amount privacy at launch. Amounts are visible in plaintext on chain transactions. The chain's positioning as institution-grade settlement infrastructure accepts this trade-off; institutional users typically have audit obligations that require amount disclosure, and the chain's amount transparency simplifies that compliance. The architecture supports upgrade-ceremony amendment to add cryptographic amount commitments in a future Phoenix if real institutional demand materialises; this is not a launch commitment.

Symmetric receive-and-spend privacy. Stealth-on-receive protects the recipient at the moment of receipt; it does not protect the recipient retroactively against linkage that emerges later. The asymmetry is structural and worth naming honestly. When a holder spends value originally received at a stealth-derived address, the change/remainder returns to the holder's HOME address by default per CM-3.3.x. If an external observer has, by any means, previously associated HOME with the holder's real-world identity — through earlier multi-input UNLOCKS that consolidated stealth addresses to HOME, through off-chain identity leaks, through social-graph correlation, through subpoena, or through any other channel outside the chain's control — then the moment the holder spends from the stealth-received Parcel, that observer can retroactively link the original receive to the holder. Concretely: the holder receives 100M at a fresh stealth address; the receive looks unlinkable on chain at the moment it happens; the holder later spends 10 from it; the chain shows the 10 going to a stealth-derived destination (the recipient remains protected) and the ~99,999,990 of change going to the holder's HOME address; an observer who already knows that HOME belongs to the holder now sees, retroactively, that the 100M was a payment to the holder. The recipient of the 10 remains structurally private; the original sender's privacy in their send-to-the-holder is conditional on the holder's subsequent behaviour. This is the bank-wire-privacy property: HOME is the chain-visible identity of the holder to anyone who has ever linked it; transactions in and out of HOME are observable as a coherent profile over time. Holders who require stronger spender-side privacy — for example, the property that the original sender's privacy in sending-to-the-holder is preserved against retroactive correlation — must use wallet-level structural privacy (the address-tree topology described in the Aurum technical paper, §Super-Stealth Mode), where value received at stealth addresses is internally re-routed to deliberately specialised branches before any spend, keeping the spend-and-change pattern within the branch and never returning value to HOME. The chain provides the cryptographic primitives; complete spender-side unlinkability is a wallet-layer construction, not a default chain property.

Forensic-grade unlinkability. Pattern analysis with off-chain context, behavioural profiling, timing correlation across many transactions, and graph-clustering of inferred holder behaviour are all possible threats that the chain's primitives do not by themselves defeat. Holders seeking forensic-grade unlinkability must compose the chain's available primitives — multiple home addresses, disciplined receive-address hygiene, view-key non-disclosure — and accept the operational complexity that comes with maximum-privacy operation.

Adoption Is Privacy

The privacy property is set-anonymity with k scaling to active user base. With a small total user population, full-observation deanonymisation is feasible regardless of cryptographic primitives — distinctive timing rhythms, distinctive value sizes, distinctive interaction graphs leak attribution through pattern analysis. With sufficiently large active user base, the combinatorial space of plausible clusterings becomes intractable for chain-only adversaries. A new holder joining a million-holder chain inherits the anonymity set immediately. The same new holder joining a ten-holder chain has very little privacy.

This is honest about how the privacy model works in practice. Scriptum’s privacy strength is not a fixed cryptographic claim but a property that grows with adoption. The chain becomes more private the more it is used.

Ratatoskr — Chain-Traversal Enumeration

A holder’s wallet, given the holder’s key pair, can identify which Parcels on the chain belong to that holder by performing Kyber/ML-KEM decapsulation against each SPEND’s ephemeral encapsulation material and matching the derived candidate addresses against the SPEND’s outputs. This scan is the receive-side privacy property in action: only the holder’s view key can recognise the holder’s Parcels.

Ratatoskr is the named primitive for this scan operation. **It runs in the wallet (or in an archive node serving a holder’s authorised query), not as protocol consensus machinery.** The chain’s consensus does not have access to holder key pairs and therefore cannot identify which addresses belong to which holders; only parties with view-key access (the holder’s wallet, or auditors to whom the holder has explicitly delegated) can perform the Ratatoskr scan. Given a holder’s key pair as input, Ratatoskr traverses the chain and reports the address set derivable from those keys. The traversal is the same DH-against-ephemeral-key matching a wallet performs locally; the Ratatoskr name elevates it to a first-class architectural concept and specifies its interfaces.

Ratatoskr has one function: report. It does not consolidate, does not authorise, does not modify chain state. Wallets and auditors consume Ratatoskr’s reports for their own purposes downstream — wallets to populate balance displays and to construct cross-address SWEEP transactions during Menuha, auditors to obtain the holder’s complete address set without scanning the chain themselves.

The naming reuses the figure of Ratatoskr — the squirrel running up and down Yggdrasil carrying messages between the eagle at the top and the dragon at the roots — to name a primitive that traverses the chain reporting what it finds. The original Ratatoskr in the V_A architecture has been renamed and split into Hugin and Munin (see §16.4) freeing the name for this enumeration role.

Audit via Ratatoskr

A holder who delegates audit authority does so by providing the auditor with the view key (per the holder-controlled audit model). The auditor with the view key invokes Ratatoskr to obtain the holder’s complete address set without scanning the chain themselves. The audit becomes a single Ratatoskr invocation rather than a multi-day chain scan, regardless of chain age or holder transaction history depth. Combined with sealed sender memo decryption, the auditor reconstructs the holder’s complete sender-recipient-amount picture for every incoming and outgoing transaction.

Wallet Recovery via Ratatoskr

Routine wallet operation does not pay the scan cost — wallets maintain caches populated by Ratatoskr’s reports and updated incrementally as new blocks arrive. The full-scan cost

matters at one specific moment: wallet recovery from seed when the local cache is unavailable.

Two protocol-level optimisations make recovery operationally viable at mature chain scale.

View-tag specification. Each SPEND output carries a small precomputed hint (a view-tag, typically a single byte or a few bytes) that lets a scanner reject most non-matching outputs in a single byte comparison, reserving the full Kyber/ML-KEM decapsulation for the small fraction that pass the tag check. This reduces per-output scan cost by approximately 40x without weakening the privacy property — view-tags leak no information beyond what the address itself already leaks. Scriptum specifies view-tags as part of the stealth output format. This is a launch commitment, not deferred to future protocol evolution.

Remote-archive scan recovery. Wallets recovering from seed may transmit their view key (over an authenticated, encrypted channel) to an archive node. The archive performs the scan and returns a filtered result containing only the holder's addresses and Parcels. The wallet's recovery time becomes proportional to the holder's own activity rather than to the entire chain's. This is a substantial usability improvement at mature chain scale and does not introduce privacy concerns beyond the holder's deliberate disclosure of their view key to a chosen archive. Scriptum specifies remote-archive scan recovery as part of the Sanctum archive node specification (see §[Node Types]).

With both specifications in place, wallet recovery on a mature chain (year 99 of Phoenix 1, hypothetical full chain volume) takes hours rather than weeks. The privacy property is operationally viable at any chain scale.

Selective Disclosure

The View Tree's chain-visible footprint is the View-Key Registry, a chain-state object parallel to the Bundle Registry. The View-Key Registry stores per-HOME view-authority state — the sequence of generations a holder has used (advancing with each CANCEL_VIEWKEY rotation) and per-(generation, year) published epoch ML-KEM-768 public keys. Like the Bundle Registry, it is keyed by HOME address. Unlike the Bundle Registry, it is not consulted during signature verification; it is consulted only by senders constructing stealth outputs, by holders scanning the chain for their own receipts, and by auditors holding exported epoch secrets. The two registries are cryptographically and operationally independent. See Key Architecture — Spend, View, Recovery for the architectural framing.

A holder operating a home address possesses a 32-byte **master view secret** for that home address: a private value held only by the holder's wallet, derived from the holder's mnemonic via a wallet-specific path. The master view secret is the root from which all per-epoch view material derives. The chain never sees, derives, or computes any view-key secret.

A holder may delegate view-only access to a third party — an auditor, a tax preparer, a compliance recipient, a custodian, an heir, a legal discovery counterparty — by exporting a per-epoch view secret. The delegate gains the ability to decrypt the holder's stealth-derived

inbound outputs received during the disclosed epoch but cannot spend; spending requires the holder's Dilithium3 signing key, which is never derivable from any view secret.

The selective-disclosure architecture supports two operations: **annual epoch publication** and **master rotation**. Both are consensus-level transactions; both are surfaced to the holder by their wallet, typically as automatic background hygiene.

Epoch-Bounded View-Key Disclosure

Time on the chain is partitioned into UTC calendar years. Each block belongs to exactly one calendar year, determined unambiguously by its timestamp. An output's epoch is the epoch of the block that confirms it — the realisation epoch in tax-equivalent terms.

For each calendar year in which a holder intends to receive private outputs, the holder publishes an **epoch view pubkey** on chain via the PUBLISH_EPOCH_PUBKEY transaction. The pubkey is derived privately by the holder's wallet from the master view secret and the target epoch year; the corresponding secret stays with the holder. Senders send to a home address by reading the holder's published epoch pubkey for the intended inclusion epoch and encrypting the output to it.

When a holder needs to disclose visibility for a specific year — say, a 2025 tax audit — the holder exports the **epoch view secret** for 2025. The export consists of the Kyber768 secret key for the 2025 epoch keypair plus context (the home address, the epoch year, the generation identifier). The auditor's tool, given this export, scans Scriptum blocks whose timestamps fall in 2025 and attempts ML-KEM-768 decapsulation against each output's ciphertext using the exported secret. Successful decapsulations are the holder's 2025 inbound outputs; the auditor sees exactly the year's history.

The cryptography enforces the boundary:

- The auditor cannot derive any other year's secret. Per-epoch secrets are derived from the master via a one-way key derivation function (HKDF-SHA3-512) with the epoch year mixed into the derivation input; given any one epoch's secret, computing another epoch's secret would require inverting HKDF, which is infeasible under standard assumptions about SHA3-512.
- The auditor cannot derive the master view secret. Same HKDF one-way property.
- The auditor cannot decrypt outputs in years other than the disclosed one. Each year's outputs are encrypted under that year's epoch pubkey; ML-KEM-768's IND-CCA2 property means that secret keys for other years cannot decapsulate ciphertexts encrypted under the disclosed year's pubkey.

A holder may disclose multiple years to the same auditor (export multiple epoch secrets) or different years to different auditors (export each year's secret to one auditor). Each disclosure is bound to its specific year by the cryptography of the derivation, not by wallet policy or social trust.

Master Rotation

A holder may rotate their home address's view authority via the CANCEL_VIEWKEY transaction. Rotation advances the home address to a new view-authority generation: the holder's wallet generates a fresh 32-byte master view secret, derives the corresponding new master pubkey, and submits the rotation transaction signed by the home address's Dilithium3 spending key. The chain appends a new generation entry to the home address's view authority state.

After rotation:

- Outputs received at the home address **before** the rotation block remain decryptable under the **prior** master view secret. The holder retains the prior master in their wallet for historical decryption; any previously-exported epoch view secrets derived from the prior master continue to decrypt the outputs they could already decrypt.
- Outputs received at the home address **after** the rotation block are encrypted under the **new** generation's epoch pubkeys. The holder must publish a fresh epoch pubkey under the new generation before senders can construct private outputs for inclusion in any given epoch. Previously-exported epoch secrets cannot decrypt post-rotation outputs in the same year (the master-binding property ensures the new generation's epoch derivations are independent of all prior generations').

Rotation is the operational forward-revocation primitive. A holder cannot undisclose what they have already disclosed — the bytes have left their custody and the recipient retains them, and the cryptography of broadcast ciphertexts cannot be un-broadcast — but the holder can ensure that any previously-exported view-key sees no further activity. Rotation is forward-only; it does not retroactively invalidate any output already broadcast and confirmed.

Mid-year rotation produces a “two-segment year” for that epoch: outputs from the start of the year to the rotation block are encrypted under the old generation's epoch pubkey; outputs from the rotation block forward are encrypted under the new generation's epoch pubkey (once the holder has published it). A holder seeking full-year audit visibility after a mid-year rotation exports both segments' epoch secrets to the auditor. This is the unavoidable consequence of forward-only revocation, not a flaw in the architecture.

CANCEL_VIEWKEY is fee-free at consensus level. Self-protection against view-authority compromise is operational hygiene, not a chargeable service.

What the Holder Sees, What the Chain Sees, What the Delegate Sees

The chain sees: every home address's view-authority generations (list of master pubkeys, advancing on rotation) and every home address's published epoch pubkeys per generation. All public. The chain never sees any secret material.

The holder sees: their 32-byte master view secret (in their wallet). From it, they can derive any past or future epoch view secret on demand. They can scan any year's blocks against their own outputs. They retain prior generations' master secrets for historical decryption after rotation.

The delegate sees: exactly the epoch secrets the holder has exported. For each exported epoch secret, the delegate can decrypt the holder's inbound outputs in that epoch under the relevant generation. The delegate cannot derive any other epoch's secret, cannot derive the master, cannot derive secrets for other home addresses, and cannot decrypt outputs encrypted under a different generation's pubkey for the same epoch.

Boundary Honesty

The architecture is honest about what it does and does not provide. A holder who publishes their epoch pubkey for 2026 and gives an auditor the 2025 epoch secret has bounded the auditor's visibility to 2025. If the holder broadcasts a transaction in the last seconds of December 2025 UTC and the inclusion block lands a few seconds into 2026 UTC, the sender's wallet may have encrypted to the wrong epoch's pubkey; the output may be undecryptable to the recipient. This is the sender's problem and the chain does not engineer around it — no grace zones, no dual-encoding to current and next epoch, no boundary buffer. Wallets warn users near boundaries; the chain enforces the strict rule.

A holder who forgets to publish a new year's epoch pubkey becomes unreachable for private outputs in that year. Senders' wallets refuse to construct outputs to a home address with no published epoch pubkey for the inclusion epoch. The holder can come back online at any time, publish, and resume receiving. The discipline is the same as maintaining a forwarding address with a bank.

The strongest accurate public claim about the construction's selective-disclosure property is:

Possession of an epoch-bounded view key does not grant decryption capability outside that epoch and generation.

Claims stronger than this — retrospective revocation, perfect privacy, magical audit invisibility, universal deniability — are not supported by the architecture and are not made.

On-Chain Settlement vs Off-Exchange Trading

Centralised exchanges represent value via internal ledger entries, not on-chain ownership. When holders trade FUNK on a third-party exchange, ownership is tracked by the exchange — at best as a tokenised IOU, at worst as a bare promissory claim subject to the exchange's solvency. Trades on exchange ledgers do not pass through Scriptum's NEXUS protection because they are not on-chain value movements; the chain does not know about them.

Scriptum's Crowglass / Asbru mechanism (specified in the Aurum technical paper) enables on-chain peer-to-peer trading of CIRCULATING FUNK between holders directly via signed-intent SPENDs. Asbru-mediated trades are first-class on-chain transactions: they pass through NEXUS_DWELL, are cancellable during the NEXUS window, settle to genuine ownership, and have no counterparty risk beyond cryptographic verification of the trade construction. This makes Asbru-mediated trading structurally superior to exchange-mediated trading for safety-critical use: actual on-chain ownership transfer, NEXUS-cancellable, no exchange custody risk, no IOU representation.

For holders requiring high-volume liquidity or fiat on/off ramps, regulated exchanges remain a practical necessity. Holders should understand the distinction: on-exchange trading is convenient but lies outside the chain's protection guarantees; on-chain Asbru-mediated trading is the chain's native settlement layer with its full safety architecture intact.

Forward Reference to CM

The cryptographic specification of epoch-bounded view-key derivation, CANCEL_VIEWKEY, PUBLISH_EPOCH_PUBKEY, and the home address view-key state schema is in the Canonical Machine: CM-16 (Epoch-Bounded View Keys), CM-3.15 (CANCEL_VIEWKEY), CM-3.16 (PUBLISH_EPOCH_PUBKEY). The parameter values — KDF identifier, KEM identifier, HKDF salt, domain-separator bytes, seed sizes — are in SPX-21 (Selective Disclosure Parameters). The invariants that the construction must preserve and the fuzz target surfaces for parameter validation are in IA Part II.12 and Part IV.2. The WP describes the architecture and the holder-facing semantics; the corpus locks the byte-level rules and the testable correctness properties.

Wallet-Layer Encryption

Aurum's local address cache, transaction history, and view-key state are stored encrypted under keys derived from the holder's view key (in addition to the at-rest passphrase encryption protecting the entire wallet blob). This composition means partial exposure of wallet state — for instance via memory-scraping malware or cloud backup leak — does not directly yield the holder's address set; an attacker would need the view key to make the cache useful, at which point they would also have direct on-chain query capability and the cache adds no marginal value to them.

Wallet-layer encryption uses post-quantum-secure primitives consistent with the chain's Kavaca commitment. The symmetric encryption (AES-256-GCM or ChaCha20-Poly1305) provides 128-bit effective security against Grover-equipped quantum adversaries — well above any practical attack threshold. Key derivation uses Argon2id with parameters tuned to give meaningful resistance to brute-force attack on the holder's passphrase, with the explicit guidance that holders should use sufficiently high-entropy passphrases (or rely on the BIP39 seed as the underlying secret, with Argon2id stretching it) to remain strong against quantum-aided brute-force. Any asymmetric component of the wallet's encryption stack — for instance, key wrapping for cloud backup, sync between devices, or hardware-key integration — uses Kyber/ML-KEM rather than classical Diffie-Hellman or RSA.

Aurum supports a multi-tier sovereign recovery archive — the Lifeboat — for device loss, offline cold backup, opt-in trustee/inheritance recovery, and generational continuity across Phoenix migrations. The Lifeboat uses a post-quantum encryption stack consistent with Kavaca and supports optional enhancements including Shamir Secret Sharing for distributed recovery, geographic share distribution, and configurable deadman-switch protocols. The Lifeboat is what makes the chain's centennial Ragnarök architecture operationally complete at the holder level: a holder's recovery materials can survive a

century-scale horizon and enable heirs to board successor Phoenixes on the holder's behalf. Full specification of the Lifeboat — recovery tier mechanics, archive format, trustee architectures, deceased-holder boarding workflow — is in the Aurum technical paper.

The wallet-layer encryption is a defence-in-depth measure that complements the chain's stealth-derivation, sealed sender memos, and NEXUS-protected paths. Implementation details — Argon2id parameters, key derivation pipeline, secure-enclave integration, hardware-key handling, sync transport, recovery semantics — are specified in the Aurum technical paper. The principle the protocol commits to is: **everything the wallet encrypts is encrypted with primitives that survive store-now-decrypt-later attacks, consistent with Kavaca at the chain layer.**

Address Space and Cluster Behaviour

A holder's published address — HOME — is the canonical identifier they share with senders, created by their wallet at first launch from the holder's Dilithium and Kyber key pair. This address does not directly hold the holder's Parcels on a stealth-by-default chain. Sends to the holder land at fresh stealth-derived one-time addresses; the holder's wallet (with their view key, via local Ratatoskr scan) recognises these as theirs.

Over time, an active holder accumulates Parcels at many one-time stealth-derived addresses. Each UNLOCK consolidates value to HOME via the NEXUS dwell as a side effect of the operation: the consumed source addresses become empty husks (their bundles revealed but their Parcels gone), and value emerges at HOME as ACTIVE Scriptum ready for use. SPEND, LOCK, EXPIRE, and REFRESH all operate on the consolidated state at HOME.

For an active user — one who unlocks regularly enough that consolidating UNLOCKS occur naturally — the non-empty address count drifts up between transactions and gets pulled down at every multi-address UNLOCK. The count stays low through normal use; cross-address SWEEPs are unnecessary for active users. For an accumulator — one who receives often but rarely unlocks — the count grows until it crosses the cross-address threshold (an address count exceeding three, where address count means the number of distinct addresses currently containing the holder's Parcels as reported by the wallet's local Ratatoskr scan). The wallet (Aurum by default) constructs and submits a cross-address SWEEP transaction during the next Menuha that consumes eligible LOCKED Parcels across all of the holder's addresses and produces one consolidated LOCKED Parcel at HOME. The non-empty count drops to one.

In both cases the steady state immediately after consolidation is HOME holding the consolidated value. HOME is stable across consolidations — UNLOCK and Frau Holle both deliver to HOME — providing operational simplicity for the holder and predictable wallet behaviour.

UNLOCK and Frau Holle operate within a single home address's address space. A holder using a single home address has one address space, consolidated to that home address. A holder operating multiple home addresses has separate address spaces per home address, each consolidating independently to that home address. The chain's consolidation primitives respect home-address boundaries automatically because the holder's wallet

selects coins from within a single home address when constructing UNLOCK and SWEEP transactions.

A pathological case is structurally absorbed. A holder who deliberately fragments by repeatedly unlocking and re-locking small amounts at a single address creates large Parcel count at that address. This is operationally self-punishing — each cycle costs the holder f_U , f_A , and f_S — and does not break the chain. If the holder later receives at three other stealth addresses (crossing Frau Holle’s address-count threshold), the next Menuha sweep consumes all Parcels at all the holder’s addresses, including the heavy-fragmented one, into a single consolidated Parcel at Y. The protocol’s per-operation per-summed-value fee model means the sweep handles arbitrary input counts cleanly. The chain is robust to fragmentation in either direction: active users self-consolidate via use, pathological accumulators self-consolidate via Menuha.

Node Types — Genesis Requirements

Scriptum defines four node types with different storage and bandwidth requirements. All four are valid participants in the network. Only Archive Nodes (Sanctum) are required to retain full history.

Home Mining Node

Bandwidth: 100Mbps minimum

Storage: ~145 GB (Parcel set + 8,640 blocks = 1 NEXUS window)

Hardware: Any modern CPU

Role: Mines blocks, validates recent chain, pruned to NEXUS depth

Cost: ~€150 storage, standard home internet

Dedicated Mining Node

Bandwidth: 100Mbps+

Storage: 500 GB SSD

Hardware: GPU rig or high-core CPU

Role: Mines blocks, keeps 30 days history, faster validation

Cost: ~€500 storage, standard broadband

Full Validation Node

Bandwidth: 100Mbps+

Storage: 2TB+ recommended; near-zero growth at genesis (empty chain), up to 17.28 GB/day at saturated 2 MB blocks; growth scales with adoption and Curia block size votes

Hardware: Any CPU, server-grade preferred

Role: Validates complete chain, serves light clients

Storage growth at genesis depends on actual transaction volume, not just block size. Empty blocks at cold-start consume only header space.

Sanctum — Archive Node

Bandwidth: 1Gbps recommended

Storage: near-zero at genesis (empty chain); scales with adoption to a maximum of 17.28 GB/day at saturated 2 MB blocks; growth scales further with each Curia block size vote

Hardware: Server-grade, redundant storage

Role: Complete chain history from genesis to present

Operated by: Exchanges, explorers, institutions, Scriptum Foundation

Sanctum storage projections by adoption tier (block-size assumed at the bracket's column; growth driven by sustained block-fill rate over time):

Tier	Sustained fill	Block size	Storage/year	Cumulative @ this tier × 5 yrs	Hardware tier
Light	5–15%	2 MB	0.5–1.5 TB	2.5–7.5 TB	4×1TB NAS RAID-5 (~€200)
Moderate	30–50%	2 MB	3–6 TB	15–30 TB	4×4TB NAS RAID-5 (~€400)
Heavy	60–80%	4–8 MB	15–50 TB	75–250 TB	6×8TB NAS RAID-6 (~€800)
Institutional	80%+	16+ MB	100+ TB	500+ TB	Datacenter rack (~€2,000+/yr)
Mature	80%+	160 MB (1 Gbps era)	500+ TB	2.5+ PB	Institutional SAN (~€20,000+/yr)

The tier a Sanctum operator finds themselves in depends on the chain's actual adoption trajectory, not on the calendar. A chain that takes ten years to reach Moderate tier accumulates the same storage as one that reaches it in two years; the operator's hardware planning follows the trajectory rather than fixed time milestones.

Recommended storage architecture: ZFS RAIDZ2 (2-drive failure tolerance, self-healing). Consumer NAS operators (Synology, QNAP) can run Sanctum comfortably through the Moderate tier. Beyond that, storage requirements scale with chain activity, and Sanctum operation becomes an institutional role by design. Running a Sanctum archive at the Heavy

tier and beyond is a significant operational commitment — appropriate for exchanges, explorers, sovereign archivists, and research institutions that have natural incentives to maintain full history once Scriptum carries serious value. Sanctum is not free at scale. The burden grows in proportion to what it protects.

Storage cost halves approximately every 2-3 years historically. A Sanctum that costs €800 in hardware today will cost €200 equivalent in 10 years. The economics improve faster than the chain grows. Sanctum is not a burden. It is a privilege.

Sanctum is the permanent memory of the Scriptum chain. It stores every transaction ever confirmed, every Parcel lineage, all Lethe records, all Ragnarok migration manifests, and all Phoenix ARC records. Sanctum is not required for protocol operation. It is required for trust, compliance, and history.

At Ragnarok, the originating Phoenix Sanctum is frozen — permanent, immutable, readable forever. The successor Phoenix begins a new Sanctum at block 0. Every Phoenix leaves its Sanctum behind. Nothing is lost. Everything is remembered.

Sanctum nodes are incentivised by market forces: light nodes and applications pay small query fees to Sanctum operators. The protocol does not mandate payment but the market creates the incentive naturally.

Consensus and Finality

Chain Selection

The valid chain is the chain with the highest accumulated proof-of-work. Block 0 of every Phoenix is accepted by deterministic validation alone — it is verified against the ARC manifest, not by proof-of-work. Proof-of-work begins at block height 1.

Confirmations

Each transaction type requires a minimum number of confirmations before it is considered final:

UNLOCK → 3 confirmations

EJECT → 3 confirmations (protocol-driven)

CANCEL → 3 confirmations

SPEND → 10 confirmations

LOCK → 3 confirmations

BURN → 3 confirmations

EXPIRE → 3 confirmations (protocol-driven)

SWEEP → 3 confirmations (protocol-driven)

COINBASE → 100 confirmations

PACK → 3 confirmations

BOARD → 10 confirmations

HORKOS_AGREE → 3 confirmations

HORKOS_COMMIT → 3 confirmations

HORKOS_STAKE → 3 confirmations

HORKOS_CONFIRM → 3 confirmations

HORKOS_ATTEST → 3 confirmations

Horkos resolution UNLOCK → 3 confirmations (protocol-driven)

Horkos resolution EJECT → 3 confirmations (protocol-driven)

Horkos resolution SEND → 3 confirmations (protocol-driven)

CANCEL requires only 3 confirmations to remain accessible throughout the NEXUS window. SPEND and BOARD require 10 due to their irreversible and high-value nature. COINBASE outputs require 100 confirmations before they enter the spendable lifecycle.

DISEMBARK is not listed above. It is not a transaction — it is a protocol event executed at Block 0 of the successor Phoenix. No confirmation count applies. It is instantaneous and deterministic at the moment of Rebirth.

Fork Handling

Competing blocks may occur. Only one chain persists. Discarded transactions return to the mempool.

Terminal Block Height — RAGNAROK_HEIGHT

Ragnarök is enforced as a consensus-level terminal block height.

The Scriptum protocol defines a fixed block height, RAGNAROK_HEIGHT, at which the active Phoenix reaches its terminal state. This height is deterministically known from genesis parameters and cannot be altered by any participant, miner, or fork without departing from the Scriptum protocol.

RAGNAROK_HEIGHT = 315,360,000

No block with height > RAGNAROK_HEIGHT is valid under the Scriptum protocol. Any node receiving such a block must reject it. A chain extending beyond RAGNAROK_HEIGHT is not a continuation of Scriptum — it is a fork that has departed from the protocol definition.

Open Source and Community Review

Scriptum is open source from genesis. The code, the specification, and this whitepaper are public. The community is the first and most important line of review.

Critical issues identified post-launch are addressed through the hard fork mechanism defined in the Scriptum Governance Charter. A hard fork requires community consensus. It is not failure — it is governance in action. Bitcoin has forked. Ethereum has forked. Every serious chain has a mechanism for this. Scriptum is no different.

A formal independent security audit will be commissioned from the Reserve allocation (5% of total supply) when resources permit. Until then, the open source codebase and community review constitute the primary security assurance.

Network Behaviour and Mempool Policy

Nodes prioritize transaction processing using a two-factor mempool handicap model:

$$H = F1 \times F2$$

Lower H = higher priority. Two factors:

F1 — Transaction Type Factor

CANCEL: F1 = 1 (highest priority — safety action, always accessible, never penalised)

SPEND: F1 = 2 (low penalty — completion of value transfer)

UNLOCK: F1 = 3 (highest penalty among user-initiated actions — preparatory, churn-prone)

EJECT is not user-initiated. It is a protocol-driven transition that fires deterministically at `unlock_height + 8,640`. EJECT does not enter the mempool and is not subject to F1 prioritisation.

BURN, LOCK, and DONATE follow F1 = 2 (treated as outward value motion at parity with SPEND for prioritisation).

HORKOS_AGREE, HORKOS_COMMIT, HORKOS_STAKE, HORKOS_CONFIRM, HORKOS_ATTEST follow F1 = 2 (treated as outward value motion at parity with SPEND for prioritisation).

F2 — Value Tier Factor

F2 is assigned per transaction type based on amount and intent. Higher tier = lower F2 = higher priority.

Tier 10 (>€10M) → F2 = 1 (highest priority)

Tier 9 (€2M–€10M) → F2 = 2

Tier 8 (€1M–€2M) → F2 = 3

Tier 7 (€500k–€1M) → F2 = 4

Tier 6 (€200k–€500k) → F2 = 5

Tier 5 (€100k–€200k) → F2 = 6

Tier 4 (€50k–€100k) → F2 = 7

Tier 3 (€10k–€50k) → F2 = 8

Tier 2 (€1k–€10k) → F2 = 9

Tier 1 (€1–€1k) → F2 = 10 (lowest priority)

F2 assignment by transaction type:

SPEND → tier-based on amount (as above)

UNLOCK → tier-based on amount (preparatory, so F1 already penalises)

BURN → tier-based on amount (deliberate action, moderate priority)

LOCK → F2 = 1 (safety/sovereignty action, no penalty)

CANCEL → F2 = 1 (safety action; highest priority under honest network policy)

EJECT → F2 = 1 (protocol-driven, never deprioritised)

EXPIRE → F2 = 1 (protocol-driven, never deprioritised)

PACK → F2 = 1 (Arc procedure, user-initiated, time-sensitive)

Safety-critical actions — CANCEL and EJECT — are always assigned F2 = 1 regardless of value tier. They cannot be structurally deprioritised by any mempool implementation. This is non-negotiable: economic weight must never prevent a holder from cancelling an unwanted UNLOCK.

Mempool policy is non-consensus and may vary between nodes, but implementations are expected to follow these principles. Two principles are foundational to honest node behaviour: first, no fee level or value tier may delay CANCEL within the NEXUS; second, CANCEL must be includable in the next valid block under normal network conditions. CANCEL is always admitted to the mempool at maximum priority. These are core security properties of the NEXUS — not policy recommendations.

PART III — USE

Aurum — Wallet and User Experience

Aurum is the reference wallet for Scriptum. It implements the full CLASS/ROLE/STATE model, NEXUS management, all transaction types, UIS integration, the complete Arc procedure, the chain's privacy primitives (stealth-derived receive, sealed sender memo, view-key delegation with range-bounded and rotatable construction), and multi-home-address management for holders who want operational compartmentalisation.

Aurum's council mode. Aurum includes a council mode — the council app — accessible to council members through their Dilithium key. The council app coordinates off-chain transaction drafting and signature aggregation for constitutional multisig operations. Council members coordinate proposed transactions through the app, review them locally, sign with their own Dilithium key, and aggregate signatures from cooperating members. When threshold signatures are collected, the transaction broadcasts to the chain. The council app's key-gating ensures that compromise of a council member's everyday wallet does not also compromise their council authority — the council key lives separately. The council app is a genesis-critical component of the Scriptum ecosystem; specifications in the Aurum Technical Document. The council app also surfaces annual HICP_RATIFY ratifications during the Countersign Window (1 February – 31 March each year), allowing Curia members to review the median-aggregated HICP factor and countersign locally.

This Whitepaper describes the chain protocol Aurum operates against. The wallet's full architecture — including its UI design, deterministic key derivation, multi-home-address management, encrypted local cache, recovery semantics, REFRESH heuristics, Horkos attestor module if the holder chooses to operate as an attestor, and platform-specific implementation details — is specified in the **Aurum technical paper**, a standalone document parallel to this Whitepaper and to the Scriptum Governance Charter.

Aurum is open-source software, published on GitHub under Apache 2.0 and distributed from dfqr.org. Users download it voluntarily, install it on their own devices, and run it under their own control. The Scriptum Foundation does not push Aurum into App Store marketplaces, does not market it to specific jurisdictions, and does not target users by geography. The software is available; the choice to use it is the user's.

This is the same model under which Bitcoin Core, Monero, and every other self-custodial open-source wallet operates. The publication of open-source software on a website is not a regulated activity in any major jurisdiction. Users in every jurisdiction are responsible for their own compliance with their own local law.

Aurum is maintained and updated by the Foundation. The Foundation complies with the law of its jurisdiction of incorporation. It does not “launch in” other jurisdictions because there is nothing to launch — only software to publish. A resident of any country who downloads Aurum and runs it is exercising the same right they exercise when they download any other piece of open-source software: the right to run code on their own machine.

Core Flow

UNLOCK → NEXUS dwell (24h NEXUS) → EJECT → ACTIVE → SPEND / LOCK / BURN / REFRESH / WAIT

UX Requirements (Summary)

The Aurum technical paper specifies the wallet's full UX. The summary requirements that appear at the protocol layer:

Full CLASS/ROLE/STATE visualisation per Parcel

NEXUS dwell countdown timers per Parcel

CANCEL functionality throughout NEXUS dwell, surfacing both signature-based and (where configured) hash-preimage authorisation paths

Limbo state displayed explicitly: LIMBO — AWAITING EJECT · NO ACTION POSSIBLE. Aurum shows a reassurance notification if Limbo exceeds ~10 seconds (typical Limbo duration is milliseconds to under a minute).

REFRESH switch for ACTIVE Parcels — surfaces the time remaining in the ACTIVE window and offers a single-click extension. Aurum surfaces this prominently when an ACTIVE Parcel has less than 25% of its window remaining.

LOCK screen with coin selection and remainder preview

BURN screen with coin selection and remainder preview

Fee preview in both Scriptum and € at current V_A

UIS Protected Mode warning with additional confirmation

Arc procedure: enrollment, boarding timeline, straggler status, countdown

DONATE screen — dedicated action, sends to Chessed, receives Dana Parcel, fee = standard f_S per SPX-4 (DONATE is structurally SPEND with a vault destination), private by default

Direct vs Private SPEND mode selection on outward sends, with clear labelling of trade-offs (speed vs cancellation safety)

Multi-home-address management for holders who want operational compartmentalisation (see Aurum technical paper for the full surface)

Markets and Aurum

Aurum Market and any external exchange are off-chain systems used for price observation. They are not required for protocol operation. They do not feed into consensus.

Markets determine the external price of FUNK. The protocol does not depend on them. Price discovery — including the bootstrap path through Aurum Market's aggregation of Ásbú offer prices, external exchange listings, and aggregator data feeds (CoinGecko, etc.) —

is economically useful but protocol-irrelevant. The chain operates identically whether Scriptum trades at €1 or €1,000,000. Markets observe the protocol. The protocol does not observe markets.

V_A is designed to keep fees economically meaningful and predictable — not to track price with precision. The market sets the price of FUNK. The protocol uses a smoothed approximation for fee calibration only.

Scriptum as a Traded Asset

Scriptum is designed for deliberate value transfer. Store of value and tradability are not opposites. FUNK will be traded. It is expected, welcomed, and supported. The mechanisms that define Scriptum — the NEXUS, Menuha, the fee structure — do not inhibit trading. They stabilise it.

The Exchange Model

Exchanges interact with Scriptum through two natural tiers:

Buying → always instant

Exchange maintains an ACTIVE pool

Buyer sends EUR, receives FUNK immediately

Exchange manages the unlock schedule

No waiting for the buyer

Selling → planned, 24 hours ahead

Seller initiates UNLOCK

NEXUS runs for 24 hours

SPEND executes after EJECT

Exchange receives FUNK, pays EUR

This asymmetry is a feature. Sell pressure is always visible 24 hours in advance. The market can absorb it. Panic selling is structurally impossible — you cannot dump in milliseconds what takes 24 hours to unlock. Buy pressure is instant. The result is natural price stabilisation without price control.

What the NEXUS Prevents

Flash crashes → impossible. Selling requires 24h planning.

Bot-driven dumps → impossible. No instant execution.

Instantaneous automated draining → prevented. Any drain attempt must complete its observable NEXUS before value leaves the chain; speed is not a weapon.

Panic cascades → dampened. Market absorbs visible sell pressure.

What the NEXUS Does Not Prevent

Normal trading → fully supported

Exchange listing → fully supported

Price discovery → fully supported

Speculation → fully supported

Long-term holding → fully supported

Institutional custody → fully supported and preferred

Menuha and Volatility

Menuha — the Sunday rest — removes one day per week from SPEND execution. This is not an arbitrary restriction. It introduces a weekly rhythm to value movement. By rejecting SPEND, BURN, and DONATE on Sunday blocks, Menuha removes continuous execution as an attack surface. Strategies that depend on uninterrupted automated trading cannot operate across the weekly closure. The pause is the point.

Scriptum does not prevent speculation. It prevents the kind of speculation that destroys value rather than discovers it. High-frequency trading, bot-driven volatility, and flash dynamics are not compatible with the NEXUS. Everything else is welcome.

Menuha Sweep — Two Tiers (Goldmarie and Frau Holle)

Menuha is not only a pause in outward execution. It is the window during which Scriptum performs consolidation of accumulated fragmentation. The chain provides two complementary sweep mechanisms operating at different scopes.

Goldmarie is the chain primitive operating at the per-address scope. Frau Holle is the wallet-driven primitive operating at the cross-address scope. Together they handle the two complementary fragmentation modes: thin spread across many addresses (Frau Holle) and concentrated accumulation at few addresses (Goldmarie).

The naming follows the Frau Holle fairy tale. Goldmarie is the diligent helper — fires automatically at every Menuha, can't be opted out of, keeps every address from getting messy. Frau Holle is the divine housekeeper — the bigger event, the explicit consolidation, the holder's deliberate act.

Goldmarie — Chain Primitive (Tier 1)

Goldmarie is a protocol-driven Menuha-only transition that consolidates per-address fragmentation. At every Menuha block, for every address (HOME or stealth-derived) holding three or more LOCKED Parcels of the same (class, role), the chain consumes those Parcels and produces a single LOCKED Parcel at the same address with the summed

amount, preserved class and role, and binding rules inherited as in the existing SWEEP specification.

Scope. Goldmarie applies to Parcels of CLASS = CIRCULATING and CLASS = VAULT held at any address. TREASURE Parcels are excluded (they have their own VEST lifecycle). MARKER Parcels are excluded (they are PERMA_LOCKED). ESCROW-addressed Parcels are excluded (they carry Horkos trade metadata).

Authorisation. Goldmarie requires no authorisation. It is a consensus-enforced protocol-driven transition. The chain executes it without holder action, without wallet involvement, and without view-key access. The chain identifies eligible addresses by iterating its own state (per-address Parcel-count index, updated incrementally per transaction).

Fee. Zero. Goldmarie is housekeeping, consistent with the chain's fee philosophy that safety and operational hygiene primitives are free.

Block validity. A block at a Menuha height that omits any required Goldmarie consolidation is invalid and rejected by all nodes. Same inclusion discipline as EJECT, EXPIRE, RESEAL, and BOARD. The chain has no discretion; the consolidations fire.

In-place operation. Goldmarie does NOT move value across addresses. Consolidated Parcels remain at the same address as their consumed inputs. The mechanism reduces fragmentation in place — fewer distinct Parcels at the same address — without exposing cluster relationships. Privacy properties are preserved.

Per-address scope is intentional. Goldmarie handles per-address fragmentation because that's what the chain can see without view-key access. The chain knows every Parcel's owner_address (universal Parcel field); it knows how many Parcels of each (class, role) sit at each address; it can consolidate them in place without any view-key. The complementary cross-address scope (linking stealth addresses to a holder's HOME) requires view-key access and is handled by Frau Holle.

Frau Holle — Wallet-Driven Cross-Address Consolidation (Tier 2)

Frau Holle is the holder's explicit cluster-consolidation primitive. The wallet, using the holder's view key, identifies the holder's address set (stealth-derived receive addresses plus HOME) and constructs a multi-input SWEEP transaction consuming Parcels at all the holder's stealth addresses and producing a single LOCKED Parcel at the holder's HOME address. This is the deliberate act by which a holder exposes their cluster: bringing scattered stealth-address Parcels together at HOME for operational use.

Cross-address fuse. Frau Holle's trigger is the cross-address fuse: the holder's address count (the number of distinct stealth-derived addresses currently containing the holder's Parcels) exceeds three. The per-address fuse handled by earlier versions of Frau Holle has moved to Goldmarie; Frau Holle no longer evaluates per-address counts.

Wallet construction. The chain cannot identify which addresses belong to which holders without view-key access. Only the holder's wallet (with the holder's view key, via local Ratatoskr scan) can compute the address set and construct the multi-input SWEEP

transaction. The wallet signs the transaction with the relevant addresses' Dilithium keys; the chain validates signatures like any other transaction.

Aurum behaviour with reaffirmation opt-out. Aurum's default behaviour is automatic Frau Holle sweep at every Menuha when the cross-address fuse fires. The holder may opt out — per-Menuha or persistently — via wallet UI. Critically, persistent opt-out does not silence Aurum: at every Menuha when the fuse would otherwise fire, Aurum prompts the holder to confirm continued skip or opt back in. The mechanism prevents silent cluster growth — holders who toggled opt-out years ago are reminded each week that their fragmentation continues. The default action (if the holder doesn't respond to the prompt) is to sweep. Non-Aurum wallets follow their own policies; the chain validates signatures regardless of the wallet's policy.

Scope. Frau Holle applies to Parcels of CLASS = CIRCULATING held at the holder's stealth-derived addresses. VAULT, TREASURE, MARKER, and ESCROW Parcels are excluded for the same reasons as in Goldmarie. The output Parcel at HOME is LOCKED, CLASS = CIRCULATING.

Privacy implications. Frau Holle is the holder's explicit act of exposing their cluster. The chain learns that all the consumed stealth-derived addresses belong to the same holder (because they're consumed in the same transaction signing-to-HOME). This is the privacy cost the holder accepts by invoking Frau Holle. The reaffirmation opt-out ensures the holder makes this choice deliberately every Menuha, not by accident.

Fee. Zero. Frau Holle is housekeeping per the same philosophy as Goldmarie.

How the Two Tiers Interact

In a typical Menuha block, Goldmarie fires first (as a protocol-driven consensus rule) and consolidates any per-address fragmentation. Frau Holle fires next (as a wallet-constructed transaction) for holders whose address-count fuse trips after Goldmarie's per-address consolidation. The ordering is: Goldmarie reduces per-address counts; Frau Holle reduces cross-address counts.

For Alice who has 5 distinct stealth addresses each with 1 Parcel, plus HOME with 4 Parcels of the same (class, role): Goldmarie consolidates HOME's 4 Parcels into 1 LOCKED Parcel at HOME (per-address fuse fires at HOME). Frau Holle's address-count fuse evaluates Alice's cluster: 5 stealth addresses + HOME = 6 addresses with Parcels. Fuse fires (>3). Aurum prompts; default is sweep; Alice confirms (or doesn't respond). Frau Holle constructs a multi-input SWEEP consuming Parcels at the 5 stealth addresses, producing 1 LOCKED Parcel at HOME (which now has 1 consolidated Parcel from Goldmarie plus 1 consolidated Parcel from Frau Holle = 2 LOCKED Parcels of same (class, role)). Next Menuha: Goldmarie evaluates HOME again. If HOME has ≥ 3 Parcels of same (class, role), Goldmarie consolidates. Otherwise no action.

The two-tier architecture handles both fragmentation modes cleanly, with appropriate authority models for each: chain primitive for per-address (no view-key needed); wallet primitive for cross-address (view-key required for cluster identification).

Eligibility and Binding Rules (Both Tiers)

Goldmarie and Frau Holle share the following operational rules.

Consolidated Parcel preservation. The successor Parcel inherits class, role, and binding from the consumed inputs. Bindings must be consistent across consumed inputs (consensus-enforced: if any binding mismatch is detected, the consolidation is invalid and the block is rejected). The creation_height of the successor is $\max(\text{input creation_heights})$, preserving COINBASE_MATURITY clocks and other height-anchored properties.

Address consistency. Goldmarie consolidates within a single address; Frau Holle consolidates across the holder's stealth-derived addresses to HOME. Neither tier crosses holder boundaries; consensus enforces that all consumed inputs have authorisation traceable to the same holder.

Menuha-only. Both tiers fire only during Menuha Windows. The chain validates this at consensus level for Goldmarie (protocol-driven, fires only at Menuha blocks); the wallet enforces it for Frau Holle (Aurum's implementation constructs Frau Holle transactions only during Menuha Windows; the chain accepts Frau Holle transactions only during Menuha Windows per the SWEEP consensus rule).

Composite Settlement Primitives

Beyond direct SPEND, Scriptum provides two composite settlement primitives for multi-step trades where simple value transfer is insufficient. Both are multi-stage protocols where the chain enforces the on-chain portion of the trade while a third party or external system handles the off-chain or off-exchange portion. The two primitives are distinguished by their off-chain verification mechanism.

Horkos (described below) is the attester-driven primitive: a named third party (the attester) verifies the off-chain leg of the trade and signs YES or NO. Used for fiat wires, physical goods, cross-jurisdictional deliveries, and any trade where direct miner observation is impractical.

Ás-brú (described below) is the miner-driven primitive: miners running the Crowglass observation daemon query a named exchange's API to detect the off-chain transfer. Used for fast-rail same-exchange trades where the exchange's public API provides the verification signal.

Horkos

Horkos is Scriptum's bilateral settlement primitive. It enables two parties to settle a trade where one side — Scriptum — is on-chain and the other — X — may be off-chain (fiat, asset, custodial transfer, physical goods, services rendered). A third party named by the parties at commit time, whose public key is bound to the contract at Stage 1, attests to the off-chain delivery. The Scriptum side is enforced cryptographically; the off-chain side is attested. The protocol does not arbitrate trade fairness — A, B, and C negotiate terms directly and the protocol enforces what they agreed. The full Horkos parameter set including specific encoding, transaction formats, and validation rules is specified in SPX.

The name Horkos (Greek Ὅρκος) refers to the personification of sworn oath in Hesiod's Works and Days — son of Eris, punisher of perjurers. The fit is structural: a Horkos contract is a sworn agreement with cryptographic consequences for breach. The skin-in-the-game stake ($S \geq 0.1\%$ of P) and the deterministic protocol-driven resolution implement the oath's enforcement; the attestor is the witness whose verdict triggers the forfeit. Earlier specifications referred to this primitive as "Delayed Atomic Settlement" (DAS); the rename to Horkos unifies the naming convention with Scriptum's mythological vocabulary (Kavaca, Hayyānu, Ilmarinen, Bifröst).

Participants

A — the party committing Scriptum. A is selling Scriptum or otherwise placing Scriptum at risk in exchange for some off-chain deliverable from B.

B — the party delivering X (the counter-asset, fiat transfer, or other off-chain obligation). B places skin in the game in the form of a stake denominated in Scriptum; the stake is what B forfeits if B fails to perform.

C — the attestor. C is the third party that the parties have chosen to verify the off-chain leg of the trade. C verifies the off-chain delivery and signs HORKOS_ATTEST confirming YES (delivered) or NO (not delivered). C holds no funds; C is paid an attestation fee for honest work, regardless of attestation direction. The choice of C is the parties' own — counterparty risk for the attestor (liveness, honesty, competence) belongs to A and B, not to the protocol.

ESCROW — a protocol-controlled wallet address. No user holds its key. Funds at ESCROW addresses move only through protocol-driven sequences triggered by attestation outcome. Neither A, nor B, nor C, nor the Foundation, nor Curia can move escrowed funds discretionarily.

Negotiated Terms and Structural Floors

A Horkos contract is fully parameterised by what the parties agree:

P — A's principal, the Scriptum A is committing to the trade.

S — B's stake, denominated in Scriptum. The protocol enforces a structural floor: S must be at least 0.1% of P. Above the floor, the parties choose the stake amount that reflects their assessment of trade significance and counterparty trust.

z — the attestation fee A pays C for verification work. Negotiated by the parties; the protocol does not constrain the amount.

W — the lock-to-approval window length, in blocks. The protocol enforces two structural bounds: W must be at least 360 blocks (1 hour at 10-second block time), and W must not extend beyond the next Ragnarök. Within those bounds, parties choose what fits their trade — short windows for fast off-chain legs, long windows for complex or cross-jurisdictional deliveries. The 1-hour floor is the structural minimum that gives an honest attestor enough

time to verify the off-chain delivery without being structurally rushed; parties may negotiate longer freely.

The structural floors prevent zero-stake Horkos (which would be a SPEND with extra steps), prevent windows too short to permit honest attestation, and prevent contracts that would otherwise straddle the centennial Phoenix transition. Within those floors, the protocol provides primitives; the parties provide judgement. The protocol is the registrar of the agreement, not a party to it.

The Five-Stage Lifecycle

Horkos proceeds through five stages, each with explicit triggers and transitions.

Stage 1 — Agreement. A, B, and C each sign a multi-sig agreement transaction recording the negotiated terms (P, S, z, attesor identity, W). The protocol verifies all three signatures and validates the structural floors. If verification passes, the agreement is recorded on-chain. No funds have moved. The contract is in INTENT state.

Stage 2 — Load window. A 24-hour window opens from the moment the agreement is accepted. During this window, A loads P plus z plus the chain fees applicable to the load operations. B loads S plus chain fees. The chain tracks each party's loaded amount against the agreement and reflects loading status. If the 24-hour window closes with one or both parties incompletely loaded, the contract lapses. Loaded principals and stakes return to their respective loaders. Chain fees paid during loading are not refunded — the chain has performed real verification and tracking work, and the chain keeps its compensation for that work whether or not the contract ultimately locks.

Stage 3 — Final confirmation. With both A and B fully loaded, all three parties sign a second multi-sig confirmation transaction. This is the explicit “we are now committed” signature from each party. The protocol verifies all three signatures and transitions the contract from FUNDED to LOCKED. The safety is now off; the contract is binding.

Stage 4 — Lock-to-approval window. The window of length W runs. C may submit HORKOS_ATTEST (YES or NO) at any point during the window. B performs the off-chain delivery. The contract is in LOCKED state during this window — funds are escrowed, no party can move them, the protocol awaits an attestation event or a window expiry.

Stage 5 — Resolution. Triggered by C's HORKOS_ATTEST submission or by expiry of W. The protocol-driven UNLOCK → EJECT → SEND sequence executes the resolution per the outcome. Resolution executes through the chain's standard transaction primitives, not through Horkos-specific transfer logic. Because chain fees were paid at load (Stage 2), resolution is fee-free at the chain level.

Outcomes

The Stage 5 resolution routes funds based on the attestation outcome.

YES — C attests delivery occurred. The protocol releases A's principal P to B (the trade completes in B's favour); B's stake S returns to B (B performed honestly and recovers their collateral); z is paid to C. Everyone receives what they agreed to.

NO — C attests delivery did not occur. A's principal P returns to A (the trade does not complete; A keeps their Scriptum). B's stake S is slashed and routed via the standard Tithe split: 90% to the miner, 10% to the protocol Tithe (which then splits internally 48/32/10/10 across WFC, Funds, RESERVE, and Charity Pot per §15). z is paid to C — honest attestation work is compensated regardless of direction.

TIMEOUT — the lock-to-approval window expires with no attestation. A's principal P returns to A. A's z refunds to A — the attestation service was never rendered. B's stake S splits 50/50: half returns to B, half is routed via the standard Tithe split. C receives nothing — silence is not work.

In all three outcomes, the routing is consensus-enforced and protocol-driven. No party — not A, not B, not C, not the Foundation, not Curia — can alter the outcome once the attestation event has triggered the resolution sequence.

Menuha Behaviour

Only the agreement phase of Horkos is Menuha-prohibited: the Stage 1 agreement multi-sig, the Stage 2 load transactions (HORKOS_COMMIT and HORKOS_STAKE), and the Stage 3 final confirmation multi-sig commit new outward value to the contract and cannot be submitted during Menuha. A Horkos contract whose 24-hour load window would expire during Menuha has its deadline extended to post-Menuha. Stage 4 (HORKOS_ATTEST) and Stage 5 (resolution) are unaffected. HORKOS_ATTEST is informational — attestor C recording a delivery decision, with no value crossing to a counterparty — and is valid at any block including during Menuha. Stage 5 resolution is protocol-driven (UNLOCK → EJECT → SEND with no user signatures) and runs through Menuha by default.

A Horkos contract whose attestation finalises just before or during Menuha completes its resolution sequence on schedule. Only new agreements cannot be initiated during the rest day. The chain rests outward-value user activity; it does not rest the recording of delivery decisions or its own internal state machine.

Why This Design

Trustless bilateral settlement against off-chain delivery. Most cryptocurrency atomic-settlement primitives (HTLC, Lightning) work only for chain-to-chain swaps where both sides are cryptographic. Horkos extends atomic-settlement guarantees to bilateral trades where one side is off-chain — fiat payments, physical goods, services, real-estate transfers, art exchange — provided the off-chain side can be verified by an attestor.

Protocol as registrar, not arbiter. The chain does not decide how much principal A should commit, how much stake B should post, how much C should be paid, or how long the approval window should run. Those decisions belong to the parties. The chain enforces nominal structural floors (stake $\geq 0.1\%$ of principal as protocol minimum, window ≥ 1 hour, window $<$ remaining time to Ragnarök) and procedural rules (multi-sig at agreement

and confirmation, fees at load, protocol-driven resolution). The 0.1% stake floor is the consensus-level minimum; it is not a recommendation for any specific commercial use. Within those constraints, the parties shape the contract.

Skin in the game with negotiated proportionality. B's stake reflects what B and A together have decided about counterparty risk for this specific trade. The 0.1% protocol floor prevents zero-stake Horkos; commercial use cases will typically negotiate stakes substantially higher (5%, 10%, 25%, or more depending on the trust profile and the value at stake). Combined with public on-chain attestation outcomes, this creates a permanent on-chain record at each address — a verifiable trade history at protocol level rather than at platform level.

Fees at load, resolution free. The chain charges its fee when it does its work — verifying agreement, tracking loading, recording confirmation. Once the contract is locked, the chain has been compensated for handling it. Resolution executes through standard primitives at no further charge. Lapse before lock returns principals and stakes minus the fees already paid; the chain keeps what it earned by tracking the failed attempt.

Resolution through standard primitives. Stage 5 does not use Horkos-specific transfer transactions. It uses the chain's standard UNLOCK → EJECT → SEND pipeline, driven by the protocol rather than by user signatures. This means Horkos inherits all of the chain's standard properties — observability, auditability, Menuha-permeability for protocol-driven flow, Tithe routing for slashed funds — without requiring any Horkos-specific consensus rules at the resolution step.

Custody without a custodian. ESCROW addresses without private keys mean no party can divert escrowed funds. The Foundation cannot freeze a Horkos trade. Curia cannot reroute it. The chain's outcome is determined by the attestation per the agreement signed at Stage 1.

Complements exchange trading. Horkos is not designed to replace exchanges. Exchanges provide continuous liquid trading against an order book. Horkos provides deterministic bilateral settlement for large, pre-arranged, or cross-asset trades where custodial intermediation adds cost without adding value. Typical Horkos use cases: institutional OTC, large fiat-for-Scriptum purchases, peer-to-peer trades through Aurum Market, and any settlement where parties prefer cryptographic enforcement over venue execution.

Horkos Resolution Sequence

Horkos Resolution — fires at the conclusion of Stage 4 (the lock-to-approval window) when HORKOS_ATTEST has been submitted with verdict YES or NO, or when the window has expired without attestation (TIMEOUT). The resolution is not a single named transaction but a protocol-driven sequence using the chain's standard primitives — UNLOCK, EJECT, SEND — executed by the protocol rather than by user signatures. The escrowed Parcels are auto-UNLOCKED, EJECTED to their resolution-determined destinations, and SENT to the recipients per the outcome:

YES outcome — A's principal → B, B's stake → B, attestation fee z → C.

NO outcome — A's principal → A, B's stake → routed via standard Tithe split per §15 (90% miner, 10% Tithe allocated 48/32/10/10 across WFC/Funds/RESERVE/CP), attestation fee z → C.

TIMEOUT outcome — A's principal → A, attestation fee z refunded to A, B's stake split 50/50 (50% returned to B, 50% routed via standard Tithe split per §15), C receives nothing.

The resolution sequence is fee-free at the chain level — chain fees were paid at the load stage. The sequence inherits the standard properties of the chain primitives it uses: NEXUS observability for each SEND step, Tithe routing for slashed funds, on-chain auditability throughout. Consensus-enforced: any block omitting a required protocol-driven Horkos resolution operation at its scheduled height is invalid.

Ás-brú — Fast-Rail Exchange Bridge

Ás-brú is Scriptum's fast-rail bridge between Scriptum on chain and currencies held at centralised exchanges. It enables a holder of Scriptum to trade directly with a holder of fiat or another cryptocurrency at a named exchange, without an attestor, without exchange cooperation, and without leaving the Scriptum protocol layer. The chain enforces the Scriptum side cryptographically; miners observe the off-chain side via the exchange's public API and include the release transaction when the matching transfer is confirmed.

Ás-brú complements rather than replaces Scriptum's other settlement primitives. Horkos covers slow-rail and attestation-required trades — fiat wires, physical goods, cross-jurisdictional deliveries — where parties negotiate the window and engage a third-party attestor. Ás-brú occupies the fast-rail position: same-exchange internal transfers, fast settlement, no attestor required, miner consensus as the off-chain confirmation source. Direct SPEND handles transfers to known recipients across all sizes (Scriptum-to-Scriptum is a transfer, not a trade). Holders gain exchange-rail liquidity from day one through Ás-brú, regardless of whether Scriptum is formally listed at any specific venue.

The name *Ás-brú* refers to Bifröst, the bridge of the Æsir connecting Midgard to Asgard in Norse cosmology. The bridge spans two domains; it bears load while it stands; it has known limits. Ás-brú as a Scriptum primitive carries the same connotation: a reliable, narrow-purpose connector between the on-chain economy and exchange-rail liquidity, structurally bounded so the bridge does not exceed its load.

Participants

A — the party committing Scriptum. A is the Scriptum-side counterparty, locking principal P in protocol-controlled ESCROW for the duration of the trade.

B — the party transferring off-chain. B holds funds at the named exchange and transfers them to A's exchange address. B has no Scriptum-side commitment beyond receiving the released Scriptum at a stealth-derived address.

ESCROW — a protocol-controlled wallet address, derived deterministically from the trade parameters. No user holds its key. Funds at ESCROW addresses move only through

protocol-driven sequences: ASBRU_RELEASE on miner confirmation, ASBRU_TIMEOUT at window expiry.

Miners — the consensus-active subset of the network at the moment of trade execution. Miners run Crowglass — Scriptum's exchange-observation architecture — which queries the named exchange's public API for committed Ás-brú trade fingerprints. Crowglass is the protocol-level concept; concrete Crowglass implementations are exchange-specific. At launch, Aurum bundles Krakophone, the Crowglass implementation for Kraken. Additional Crowglass implementations may be developed as the Foundation's recommended Crowglass operational configuration expands post-incorporation. The Crowglass daemon is a standard component of the Ilmarinen node; miners participating in Ás-brú validation choose which exchanges they observe. The miner who builds a block within the trade's window includes ASBRU_RELEASE in that block if Crowglass reports a matching confirmed transfer; miners who find no match include nothing for this trade and proceed normally. A single miner's observation is consensus-sufficient: once ASBRU_RELEASE is included in a block, the chain accepts the state transition. Practical finality follows the standard confirmation-depth discipline that applies to any chain transition — the recipient typically waits six to ten additional blocks (sixty to one hundred seconds at ten-second block time) to amortise reorganisation risk before treating the trade as settled. The $W = 60$ -block window accommodates standard confirmation depth within the trade's lifecycle. Aurum surfaces the confirmation depth at Stage 4 per the user's chosen comfort level; see the Aurum Technical Document for the confirmation-depth UX specification. Miners earn the trade's chain fee for this work, in addition to standard block rewards.

The security model is observer diversity rather than curated trust. The protocol does not verify which exchange the trade references or whether the exchange is reputable; the protocol verifies only that ASBRU_RELEASE was validly included. The chain relies on rational miner behaviour — Crowglass-running miners observe the exchanges they choose to observe and reach block-level consensus through standard validation. A miner choosing to observe a fraudulent exchange wastes their effort because honest peers do not confirm the observation.

There is no attester C. The exchange is a passive data source, not a participant. The exchange does not sign anything for the protocol, holds no chain-registered keys, and bears no obligation to Scriptum. Ás-brú extracts a chain-verifiable signal from the exchange's public API through miner observation rather than through exchange cooperation.

Fixed Parameters and Trade Terms

Ás-brú is structurally simpler than Horkos. The protocol fixes the window; the parties specify the trade terms at commit time.

W — Window of trust (fixed). ASBRU_W is hard-coded to 60 blocks (approximately 10 minutes at 10-second block times). Parties cannot negotiate W. This fixes Ás-brú's positioning as fast-rail-only: trades whose off-chain leg cannot complete within 60 blocks should use Horkos, not Ás-brú. The fixed window also simplifies miner monitoring — every Ás-brú trade has the same lifecycle duration, and miners can budget their API polling cadence accordingly.

P — A’s principal. The Scriptum amount A commits to ESCROW. Constrained by DUST_THRESHOLD and ASBRU_MIN_VALUE (per SPX-6).

Exchange identifier. A canonical specifier (hashed for chain storage) of the exchange the parties agree to use for the off-chain leg. The protocol does not maintain an admitted-exchange registry and does not reject covenants by exchange identifier. Parties choose the exchange at commit time; miners running Crowglass against that exchange provide the observation; the chain enforces what was committed. The Foundation publishes a recommended Crowglass operational configuration listing exchanges considered operationally appropriate for Crowglass observers (Kraken at v1.0, more as operational experience accrues); the recommendation is informational, administered at Foundation-board cadence per GOV §11.3 and §12.1, and is not protocol-binding. Parties may name any exchange; miners may observe any exchange they choose. The chain’s consensus is over what miners actually included, not over whether the chain approves of their exchange choices.

Off-chain leg fingerprint. The trade parameters that miners use to identify the matching off-chain transfer: B’s exchange-side source identifier, A’s exchange-side destination identifier, transfer amount, currency, and a unique discriminator (memo, reference, or per-trade deposit address) sufficient to distinguish this trade’s transfer from any other transfer at the same exchange in the same window. The fingerprint is committed in the ASBRU_COMMIT transaction so the chain has an immutable record of what miners are looking for.

B’s destination. A stealth-derived Scriptum address where the Scriptum is delivered upon RELEASE.

There is no stake S. Ás-brú is a single-collateral primitive: A locks the Scriptum, B transfers off-chain. Asymmetry is intentional — the off-chain leg has no chain enforcement, so a stake from B would simply lock symmetric capital without adding cryptographic guarantees.

There is no attestor fee z. No third party is paid; miners are compensated through the standard chain fee on ASBRU transactions.

The Five-Stage Lifecycle

Ás-brú resolves through five stages:

Stage 1 — Off-chain coordination. A and B agree on terms outside the protocol: which exchange, what amount of Scriptum against what amount of off-chain currency, the relevant exchange addresses or accounts. Aurum’s UI surfaces a price oracle (queried from the same exchange’s public market data) to suggest a fair rate; final terms are agreed bilaterally. No chain interaction occurs at this stage.

Stage 2 — ASBRU_COMMIT. A submits ASBRU_COMMIT, locking principal P plus chain fees into ESCROW. The commit transaction encodes: the exchange identifier hash, the off-chain leg fingerprint, B’s stealth destination, and the commit-block height (W’s clock starts

here). Miners validate the commit like any other transaction; the Scriptum is locked from the moment the commit is confirmed.

Stage 3 — Off-chain transfer. B initiates the transfer at the named exchange. The transfer must complete and become visible on the exchange’s public API within W blocks of the commit. The protocol does not observe this stage directly; it occurs entirely off-chain on the exchange’s infrastructure.

Stage 4 — Miner observation. Each block built within the W window triggers the building miner to query the named exchange’s API for the committed fingerprint. The query returns either “match found” (the off-chain transfer is visible) or “no match” (transfer not yet observed). The miner who finds a match includes ASBRU_RELEASE in the block they build; miners who find no match include nothing for this trade and proceed normally. A single miner’s observation is consensus-sufficient: once ASBRU_RELEASE is included in a block, the chain accepts the state transition. Practical finality requires the standard confirmation-depth discipline; the recipient waits six to ten additional blocks before treating the trade as settled (the $W = 60$ -block window accommodates standard confirmation depth within the trade’s lifecycle).

Stage 5 — Resolution.

ASBRU_RELEASE routes the principal P from ESCROW to B’s stealth destination through the chain’s standard UNLOCK → EJECT → SEND pipeline, protocol-driven, no further user signature required. Chain fees accrue to the miner who included the release.

ASBRU_TIMEOUT fires at $\text{commit_block} + W$ if no ASBRU_RELEASE has been confirmed. The protocol returns P from ESCROW to A through the same standard pipeline. A bears the chain fees consumed at commit; the trade closes without prejudice.

The five-stage structure mirrors Horkos’s lifecycle but compressed. There is no separate attest stage because there is no attestor; the miner’s act of including ASBRU_RELEASE is the attestation, witnessed by the consensus they produce.

Menuha Interaction

Ás-brú follows the same Menuha rules as other value-transfer primitives. ASBRU_COMMIT is invalid during the Menuha Window (Saturday 23:00 UTC through Sunday 23:00 UTC, evaluated by Median Time Past per §Menuha) — new trades cannot be initiated during the rest day. ASBRU_RELEASE and ASBRU_TIMEOUT remain valid through Menuha — trades in flight at Saturday close resolve on schedule, even if the resolution block is mined during the rest day. The chain rests outward-value initiation; it does not rest the closure of decisions already in progress. This matches the Horkos rule and the protocol’s general posture toward Menuha.

Edge Cases and Failure Modes

Hostile miner refuses to include a confirmed release. A miner observes the off-chain transfer via the API but declines to include ASBRU_RELEASE in their block. The trade does not resolve in that block. The next miner builds the next block, queries the API, observes

the transfer, and includes the release. Worst case: a few blocks of delay, well within W's 60-block budget. The protocol's robustness against single-miner malice comes from the rotation of mining hashrate rather than from a multi-miner threshold.

Genuine API outage during W. The exchange's API is unavailable for all miners during the window. Miners observe nothing; ASBRU_TIMEOUT fires; A's Scriptum returns. If B did transfer, B has lost the off-chain leg without receiving Scriptum. Mitigation is reputational: B can re-initiate the trade once the API is healthy. Ás-brú does not pretend to atomicity across infrastructure failures it cannot detect; the timeout-and-refund path is the honest fallback.

Network partition affecting some miners but not others. Different miners see different API states. A miner who can reach the exchange includes the release; one who cannot, does not. The first miner to confirm wins; the partition is irrelevant to the trade's outcome.

Exchange changes API endpoints during W. The committed exchange identifier hash references a specific endpoint. If the endpoint changes mid-trade, miners querying the old endpoint receive errors. The trade times out as if the API were down. Exchange API changes are slow events (typically announced months in advance) and Aurum's per-exchange integration code handles version migration.

B initiates the transfer just before the W expiry. The transfer is in flight when W closes. Miners querying at the last block see no confirmed transfer; ASBRU_TIMEOUT fires; A's Scriptum returns. Defence is procedural: Aurum surfaces the W countdown prominently, and B's wallet refuses to initiate transfers within a configurable buffer of the deadline (default: refuse if less than 20 blocks remain).

Why This Design

Bilateral trust simplification. Ás-brú strips out the third-party attestor that Horkos requires. For trades that resolve in minutes via fast exchange rails, the attestor is structural overhead — the parties don't need a witness because the API is the witness. Removing the attestor eliminates the negotiation step (who is C?), the attestation fee (z), and the attestation-honesty-floor that justifies Horkos's 360-block window minimum. Two parties, fixed window, miner consensus as the confirmation source — simpler primitive, narrower use case, cleaner contract.

Fixed window discipline. ASBRU_W = 60 blocks, no negotiation. This forces Ás-brú into the role it is designed for: fast-rail, same-exchange internal transfers that complete within minutes. Trades that need longer windows must use Horkos with an attestor, which is the appropriate primitive for slower rails. Forbidding negotiation eliminates the failure mode where parties agree to a window too short for their rail or too long. The protocol's stance is that fast and slow are different products; Ás-brú is the fast one.

Miner-monitored, miner-paid. Miners already operate the heaviest infrastructure on the chain. Adding API integration to mining software is incremental work; the standard chain fee on ASBRU transactions compensates the additional cycles. No new participant class is

introduced — no oracles, no attestors, no monitoring registry. Ás-brú reuses mining as a side channel for off-chain observation, with payment aligned to the work performed.

Single-miner sufficiency, redundancy through rotation. Any miner who builds a block within W can confirm. The protocol does not require M -of- N attestations because the chain's existing consensus already provides honest-majority assumptions over the rotation of mining hashrate. A hostile miner refusing to include a release simply yields the work to the next miner; the trade resolves with at most a few blocks of delay. Single-miner sufficiency keeps the consensus rules simple.

Recommended exchange baseline, open at protocol level. The chain does not maintain an admitted-exchange registry. Ás-brú trades may reference any exchange the parties agree to; miners running Crowglass choose which exchanges they observe. The Foundation publishes a recommended Crowglass operational configuration — exchanges the Foundation considers operationally appropriate for Crowglass observers based on regulatory standing, asset segregation, API stability, audited operations — with Aurum shipping at launch with Krakophone — the Crowglass implementation for Kraken (MiCA-compliant, established, regulated, audited). Post-Foundation-incorporation, the Foundation may publish a recommended set including additional exchanges (and corresponding additional Crowglass implementations) as operational experience accrues. The recommendation is informational, administered at Foundation-board cadence per GOV §12.1 and §11.3, and is not protocol-binding. Aurum surfaces the recommended set as user-facing trade-construction defaults; users with different preferences may name any exchange, and a non-Aurum wallet may do the same. The open-protocol architecture (no protocol-level gate, no wallet-licensing key) is dictated by the Apache 2.0 licence and the chain's open-source commitment: a protocol-level gate would either provide no real protection (extractable key in open source) or recentralise the chain (Foundation gatekeeper).

Half-atomic, honestly named. Ás-brú does not claim cryptographic atomicity across both legs. The Scriptum side is fully enforced — escrow, miner-witnessed release, deterministic timeout. The off-chain side relies on the exchange's public API and miner observation, which together produce a strong but non-cryptographic signal. The protocol does not pretend otherwise. Holders trading via Ás-brú accept the asymmetry: cryptographic enforcement on what the chain controls, observational consensus on what it does not. Counterparties who require fully cryptographic atomicity should use Horkos with an attested rail.

Reputation backs the residual risk. Default behaviour by either party — A refusing legitimate trades repeatedly, B initiating transfers and not following through — accrues to permanent on-chain records at the relevant addresses. The reputation layer that Aurum Market established for direct peer-to-peer trades extends naturally to Ás-brú. Counterparty selection is the user's responsibility; the chain provides the public record from which selection can be informed.

Complements the settlement stack. Scriptum's settlement primitives form a graduated set: direct SPEND for instant pseudonymous payments, NEXUS-routed SPEND for cancellable

cold spends, Horkos for attested off-chain settlement, Ás-brú for fast-rail exchange bridges. Each fills a specific niche; no two are redundant. Ás-brú's contribution to the stack is exchange-rail liquidity without exchange cooperation — a property no other primitive provides, and one that makes FUNK practically tradeable from day one regardless of formal listing progress. Discovery and information surfaces (Kiva for counterparty finding, Aurum Market for price observation) sit above the settlement primitives; they are not themselves primitives.

Kiva — The Discovery Hall

Kiva is the opt-in discovery hall of the Scriptum ecosystem. It is a communal hall sitting above the settlement primitives — Horkos, Ás-brú, and direct SPEND — in which Aurum holders make themselves visible to one another, post structured Ás-brú offers, and find counterparties for bilateral trade. Kiva's concrete operational value is address pre-population: when two holders engage through Kiva, the counterparty's HOME address is handed off into whichever settlement primitive the parties choose, sparing both parties from manual address entry. Kiva does not execute trades, does not hold funds, does not become a counterparty. It is a discovery primitive, not a marketplace; the settlement primitives below it execute the trades. Aurum Market — the public price-and-offer data viewer — sits parallel to Kiva as the information surface; viewing prices and offers in Aurum Market does not require entering Kiva.

Without Kiva, two holders who do not already know each other have no protocol-defined way to find one another. The chain provides addresses but no directory; the wallets provide privacy but no presence; the settlement primitives provide bilateral negotiation but no introduction. Kiva fills exactly that gap and nothing more. Once two holders have found each other through Kiva, Kiva steps aside and the interaction becomes direct Aurum-to-Aurum settlement through whichever primitive the parties choose.

The name *Kiva* refers to the Pueblo ceremonial chamber — the underground room where the community gathers for shared business, distinct from private dwellings, distinct from the central plaza. The kiva is the place where members of a community become visible to one another by choice and for purpose, then return to their separate lives. Scriptum borrows the name and the architecture: a hall a holder enters by intent, in which their identity becomes visible by design, from which they leave when they're done, and which never holds anything they bring with them.

Participation Model

Kiva membership is opt-in and symmetric. A holder who has not joined Kiva is invisible to other holders in Kiva and cannot see other holders in Kiva. A holder who joins Kiva becomes visible to other holders and can see them. There is no asymmetric mode; visibility and observation are paired by construction.

The first time a holder enters Kiva, Aurum surfaces an explicit consent screen that states plainly what entering Kiva means: the holder's home Scriptum address is published to the discovery hall, other Aurum holders will see the address along with any tags and offers the holder posts, and the address's on-chain history — completed trades, defaults, publicly-

delegated view-key holdings if any — is verifiable by anyone in the hall. The consent is explicit; there is no default-on join.

Aurum wallets supporting multiple home addresses present a picker on first join. The user chooses which home address represents them in Kiva. Only one home address is active in Kiva at a time per Aurum instance. To switch addresses, the user leaves Kiva and re-enters under a different address; the previous address's accumulated Kiva history does not transfer to the new one. Each home address builds its own reputation independently.

Leaving Kiva is a single visible control. On leave, Aurum ceases broadcasting presence messages, withdraws all of the holder's active board offers, and removes itself from peers' hall views as their next presence-timeout pass executes. The address that left Kiva is not invisible to historical observation: on-chain trades concluded through Kiva remain on-chain, reputation already accrued remains, the fact that the address was once in Kiva is permanent. Leaving is a "now and going forward" operation, not an erasure.

Presence in the hall is equivalent to the holder's Ilmarinen node being reachable on the Scriptum P2P network. While the holder remains in Kiva, peers see them as present as long as their node is online; if the node disconnects, peers time the holder out within a defined window and re-discover them automatically when the node returns. Presence is a live network fact, not a per-day enrollment.

The Hall — Presence

The hall is the Kiva surface that displays who is currently present.

Each holder in Kiva broadcasts a signed presence record containing the holder's home address, zero or more tags drawn from the admitted enum (at launch: Buy and Sell), a millisecond timestamp anchored to chain time, and a Dilithium3 signature over the canonical bytes. The record is small and re-broadcast on a defined cadence. Peers verify the signature, verify timestamp freshness against their chain-time view, record the holder as present, and remove the holder from the hall view if no fresh record arrives within the timeout window.

Each row in the hall shows the holder's address, their tags, an inline on-chain reputation indicator (completed-trade count, default count, time-in-system), a freshness indicator, and a context action to open direct Aurum-to-Aurum contact. The hall is sortable by tag, by reputation, by time-in-Kiva. The hall is not searchable by free text; there is no profile to search against. Filters are limited to the tag enumeration.

The Board — Ás-brú Offers

The board is the second surface of Kiva. The board carries structured, signed, time-bounded Ás-brú offers.

At launch, the only structured offer type the board carries is the Ás-brú offer. This is by architectural intent. The natural first-day trade in a chain with no exchange listings is fiat-to-Scriptum or Scriptum-to-fiat settled through the Ás-brú exchange rail, because there are no other listing venues at launch, no P2P-fiat path that does not involve bilateral

negotiation, and no Scriptum-to-Scriptum trade useful to a counterparty who does not yet hold Scriptum. The board is, definitionally, the Ás-brú offerboard. Bilateral or otherwise-structured trades route through the hall's presence list and direct Aurum-to-Aurum contact, not through the board.

Each board offer carries the originator's home address, the trade direction (Buy or Sell from the originator's perspective), the Scriptum amount, the fiat amount and currency, the named exchange (typically from the Foundation's recommended Crowglass operational configuration; see §Exchange-Choice Architecture), an expiry timestamp constrained to at most 24 hours after origination, and the originator's Dilithium3 signature. There is no minimum lifetime; the originator may withdraw any offer at any moment by gossiping a signed withdrawal that propagates at gossip speed. Each home address may hold at most five active offers on the board simultaneously — a cap that prevents dust-spam without constraining honest two-sided pricing.

A counterparty accepts a board offer by opening direct Aurum-to-Aurum contact with the originator and signalling intent to take the offer. The originator's Aurum responds and the two wallets enter the Ás-brú trade flow specified in §Ás-brú. Accepted offers are automatically withdrawn from the board because the trade is now in progress; they are reposted only if the counterparty bails before commitment.

Handoff to Settlement

Kiva's role ends at the point two holders have found each other. From that point forward, the interaction is direct Aurum-to-Aurum, and settlement runs through one of the protocol's settlement primitives. Direct SPEND handles Scriptum-to-Scriptum transfers to a known recipient for arrangements not requiring a counter-asset. Horkos handles three-party attested off-chain settlement, with attestor selection at the bilateral negotiation stage between the two parties who found each other. Ás-brú handles two-party fiat-rail settlement through the exchanges the parties agree to use (per the Foundation's recommended operational configuration or otherwise); board offers are pre-structured Ás-brú trades and route directly into this primitive on acceptance. Holders who already know each other's HOME addresses can use Ás-brú or Horkos directly without entering Kiva; Kiva's address pre-population is convenience, not a gate.

The handoff is mechanical. Kiva's gossip surface provides a contact handshake (the two wallets exchange a session token over Kiva and then open a direct peer-to-peer connection over the Scriptum P2P network), and from the handshake onward Kiva carries no further information about the interaction. The trade — its terms, its execution, its outcome — happens between the two Aurum instances directly, without Kiva mediation.

Privacy Posture

Kiva is a public hall by intent. The architectural decision to identify holders by their home address — not by stealth address, not by ephemeral identity — is what makes the reputation surface work and what gives the hall its honesty. The cost of that decision is that everything happening in Kiva is observable to anyone with an Aurum.

This is plainly stated in Aurum’s UI at every join. Holders who require privacy of the fact-of-trading do not enter Kiva. The chain’s private settlement primitives — Aurum Market between known counterparties, Horkos with attestors known to both parties, Ás-brú with a counterparty obtained outside Kiva, Sealed Sender Memo, view-key delegation with the range-bounded and rotatable construction — remain fully available to holders who never join the hall. The privacy boundary in Aurum is exactly the threshold of Kiva: outside, every privacy primitive in the chain remains available; inside, the holder has chosen public visibility for the reputation benefit it provides.

Aurum does not provide an option to enter Kiva with a stealth address or under a pseudonym. Doing so would defeat the reputation surface that is Kiva’s entire point. A privacy-Kiva would be a different product with different properties and is out of scope.

Why This Design

Kiva exists because peer-to-peer settlement requires peer-to-peer discovery, and the chain by itself cannot provide it. Addresses on the chain are pseudonymous bit-strings; the P2P network is a transport layer for consensus, not a directory of who wants to trade. Without a discovery primitive, peer-to-peer trade in practice collapses into “trade with people you already know” — which works for a small early community and fails as the chain grows.

The design rests on three structural choices:

Opt-in symmetric visibility. A holder is either visible to the hall and sees the hall, or invisible and blind to it. There is no observer mode and no hidden browse. The symmetry keeps the hall honest by construction: if you want to see who is here, you accept that they see you.

Home-address identity. Reputation accrues to addresses on the Scriptum chain through the settlement primitives’ own machinery (completed trades, defaults). Kiva surfaces that reputation by requiring members to identify by home address rather than by ephemeral identity. Bad actors cannot refresh their identity to escape a record of defaults; good actors carry their reputation forward.

Kiva drops out at the handshake. The hall is a discovery primitive, not a marketplace. After two holders find each other, Kiva carries nothing further. The trade happens between the two wallets directly, settles through the existing protocol primitives, and Kiva learns no more about the outcome than any other Aurum holder learns by observing the chain.

Kiva is the minimum discovery primitive consistent with Scriptum’s no-custody, no-Foundation-service, peer-to-peer commitment. It provides exactly the missing piece between addresses and trade — presence and posted intent — and provides nothing else.

Price Discovery — The Bootstrap Path

The chain’s first observable Scriptum/EUR price is established at genesis through the Genesis First-Buy mechanism documented in §21.5: the 1% Genesis First-Buy pool is sold at €0.01/Scriptum to constitutional cohort and registered Genesis Sign-ups under the water-filling allocation algorithm. The first Genesis First-Buy delivery following first-buy

attestation marks the chain's first observable price point. Subsequent price discovery flows from this anchor through the venues described below.

Aurum Market is the public price-and-offer data viewer built into the Aurum wallet. It is not a trading layer. It is not an exchange. It operates outside the consensus protocol — the chain functions identically whether Aurum Market is used or not. Aurum Market aggregates the active Ás-brú offer board contents (from Kiva members who have posted offers), live prices from external exchanges (Kraken, Coinbase, and others as listings progress), and aggregator data feeds (CoinGecko and others). Holders can browse all of this without entering Kiva or registering with any service; viewing the market is a public, opt-in-free information surface.

Phase 1 — Aurum Market

Aurum Market is the price-and-offer data viewer embedded in the Aurum wallet. It is not an exchange. It is not a Foundation service. It is not a settlement primitive. Aurum Market displays the current Ás-brú offer board (posted by Kiva members), live external exchange prices, and aggregator data. Holders browse the market freely without any opt-in. Settlement happens through the protocol's settlement primitives — direct SPEND, Horkos, or Ás-brú — not within Aurum Market.

A holder who wishes to act on what they see in Aurum Market has two paths. If they already know the counterparty's HOME address (from a previous trade, from a personal relationship, from any out-of-band source), they can construct an Ás-brú or Horkos trade directly with the counterparty as input. If they need to find a counterparty or want the address-pre-population convenience, they enter Kiva — the opt-in discovery hall — where Kiva surfaces presence and reputation and pre-populates counterparty addresses into the settlement primitive's trade-construction flow. Aurum Market displays the offer's data (price, direction, amount, currency, exchange, expiry) but does NOT expose the originator's HOME address; that surfaces only when entering Kiva. The market is a public information layer; address-discovery is Kiva's role.

The Foundation provides no matching, no escrow, no arbitration, no dispute resolution, and operates no servers required for Aurum Market or Kiva to function. Turning off every Foundation asset would have no effect on either. Counterparty selection and counterparty performance are entirely the users' responsibility. The protocol cannot reverse trades. The Foundation provides no recourse. The settlement primitives (Ás-brú, Horkos, direct SPEND) execute as specified; Aurum Market shows you data and Kiva helps you find people, but the trade itself is between you and your counterparty.

Every Ás-brú or Horkos trade produces a signed on-chain record. Completed trades and any defaults are permanent public records against the counterparty addresses — a reputation layer emerges organically from the chain's own history, surfaced in Kiva for participants and in Aurum Market as part of the trade-history view.

Price discovery at scale happens through the community exchange listings described below and the global exchange phase that follows. Aurum Market provides the early bootstrap

information surface — Ás-brú offers, initial exchange prices as listings come online, aggregator data once aggregators index Scriptum.

Phase 2 — Community Exchange Listing

In parallel with the foundation exchange, Scriptum will seek listing on NonKYC.io — a proof-of-work focused exchange that lists new chains at minimal cost. This provides a second independent price feed and introduces Scriptum to the existing PoW mining community.

Phase 3 — Global Exchange Listing

Once Scriptum has established price history, community, and trading volume through Phases 1 and 2, the Foundation pursues listings on internationally credible exchanges to provide global access and broaden price discovery. The Foundation's listing strategy follows a tier discipline rather than a fixed venue plan.

The Foundation will pursue listings on exchanges that meet at least the following criteria: a credible regulatory perimeter (MiCA-licensed, MiCA-applied-for, or operating under equivalent jurisdictional authorisation), demonstrable security and proof-of-reserves practice, transparent corporate structure, and editorial standards on listings (i.e., the venue does not list arbitrary tokens for fees alone). Examples of exchanges that meet or are progressing toward these criteria as of writing include OKX, Bybit, Bitget, and KuCoin in the international tier, alongside the regulated incumbents (Coinbase, Kraken, Bitstamp) for the eventual institutional tier. The Foundation engages opportunistically rather than exclusively; multiple listings on the same tier are expected and welcomed, and listing decisions are not commitments to any single venue.

The Foundation will *not* pursue listings on venues whose primary business model is paid-for listings without editorial standards, or whose corporate structure has been the subject of regulatory disclaimer in their nominal jurisdiction. Naming such venues affirmatively in this whitepaper would itself signal the wrong tier; the Custody Reality companion document discusses concrete examples of unauthorised offshore platforms in the context of explaining custodial fragility, and that discussion is the appropriate place for those names.

No exchange fees are payable at launch. Price discovery is fully decentralised in Phase 1 (P2P inside Aurum), moves to community exchanges in Phase 2, and reaches global credible venues in Phase 3. The bootstrap path imposes no cost on the Foundation.

Operational Modes and Practical Use

Scriptum is not a single-use system. It is a structured environment in which value can be held, moved, or conditionally transferred under explicitly defined security regimes. Each mode represents a deliberate trade-off between immediacy and protection.

Secure Storage — The Default

All value in Scriptum is created and held in the LOCKED state. This is not an opt-in security mode. It is the default state of every Parcel, from the moment it enters existence. Mined coins are LOCKED. Received funds are LOCKED. Change outputs are LOCKED.

Movement of value is not instantaneous. Every transfer must pass through the Safety Control Window. Unauthorized transactions remain observable and cancellable for the full duration of the window.

Possession of a key does not imply immediate control over value. This is the core property that distinguishes Scriptum from every other chain.

This mode serves long-term capital storage, treasury and institutional holdings, and any situation where protection against key compromise matters more than transaction speed. It is also, incidentally, the only mode the protocol requires. Everything else is a user choice to reduce protection in exchange for immediacy.

Immediate Use — The ACTIVE State

Users may maintain a portion of funds in the ACTIVE state. ACTIVE funds can be transferred without passing through a new Safety Control Window. The mode exists for routine or low-value transactions — payments, small transfers, anything analogous to cash in a wallet.

ACTIVE is a deliberate reduction in protection. Funds in ACTIVE state are not covered by the cancellation window that protects LOCKED and NEXUS-dwelling Parcels. A user who keeps large value in ACTIVE state is making an explicit security trade-off.

ACTIVE Parcels automatically return to LOCKED after the ACTIVE window closes. Exposure is bounded by design — even a user who forgets a balance in ACTIVE state is returned to full protection within a day.

Scriptum separates liquidity from security by design. ACTIVE exists because users need it; its cost is an explicit, bounded, temporary reduction in the protocol's default protection.

Market Interaction and Liquidity

Scriptum interfaces with external markets for price discovery, liquidity provision, and entry or exit. Platforms such as Binance, Kraken, and others may facilitate trading. These venues provide efficient execution and global access that no on-chain settlement layer can replicate.

Exchanges operate outside the Scriptum settlement model. They introduce custodial and counterparty risk. A user's FUNK on an exchange is the exchange's obligation to deliver, not a Parcel under the user's key.

Scriptum does not attempt to replace exchanges. It provides a fundamentally different settlement layer beneath them. Users who want liquid continuous trading use exchanges. Users who want cryptographic settlement without custody use the chain directly.

System Coherence

These modes form a unified architecture. LOCKED provides maximum protection. ACTIVE provides immediate usability. The market layer provides liquidity and price discovery. Horkos provides conditional, trust-minimized settlement.

Each function is explicitly separated and cannot be conflated. A user's LOCKED balance is not mixed into the ACTIVE pool. Exchange holdings are not protocol state. Horkos escrow is not a governance instrument. Each mode does one thing clearly, with consequences the protocol makes visible.

Operational Principle

Scriptum enforces intentionality. Small value moves quickly because the user has chosen to accept speed over protection. Large value moves slowly because the user has chosen protection over speed. The protocol does not assume user intent. It requires it.

This is not paternalism. It is structure. Users who want speed can have speed — in the ACTIVE state, in measured amounts, with bounded exposure. Users who want security can have security — in the LOCKED state, with a cancellation window, with full visibility of any movement attempt.

Security is not a parameter. It is the default state.

PART IV — INSTITUTION

Governance — WOC, Founders, and Curia

In this document, "Council" is sometimes used as a generic shorthand for Scriptum's constitutional multisig bodies collectively. Where a specific authority matters for protocol validity or governance scope, the relevant body is named explicitly: the Warden of the Chain (WOC), the Founders Council, or the Curia Council, as applicable. Generic "Council" references should be read as collective shorthand only; authoritative governance references always name the specific body.

The Scriptum governance model is built on three constitutional roles: the Warden of the Chain (WOC), the Founders, and the Curia. These roles carry TREASURE allocations with no-cliff linear monthly vesting schedules and are instantiated at Genesis with their TREASURE Parcels pre-created at the slot HOMEs with offsets matching each monthly milestone. Full governance procedures are specified in the Scriptum Governance Charter; this section sets out the protocol-level commitments that the Charter operates within.

Constitutional Roles

WOC — 1 member. Warden of the Chain. Protocol guardian. Last resort. Permanent signature authority for chain protocol releases. Singular constitutional position.

WOC emergency authority — explicitly narrow scope. The WOC retains single-signature authority for two specific protocol-level actions: (a) protocol-release authorisation (L3) — chain upgrades require WOC's single signature to activate; without WOC's signature, no upgrade transitions from proposed to active state; (b) bounded protocol veto — within a 24-hour window after any Council Registry mutation transaction (REMOVE_SLOT_KEY, ADD_SLOT_KEY, WOC_REMOVE, WOC_ADD) is proposed on-chain (i.e., included in a block), the WOC may sign a veto transaction blocking the mutation from taking effect. After the 24-hour window expires without WOC veto, the mutation finalises. WOC has no unilateral operational disbursement authority — WOC cannot sign alone to move VAULT funds, cannot replace any other constitutional member alone, cannot bypass any class-driven authorisation rule. Emergency authority may direct operations, communications, and defensive actions but cannot alter consensus behaviour without the Level 4 process. Consensus rules — the protocol's deterministic state machine, fee schedules, state transitions, NEXUS dwell durations, vesting schedules, HICP indexing mechanics, and all other consensus-altering parameters — require the full L4 ceremony (WOC + 3-of-5 Founders + 7-of-9 Curia) to modify. WOC emergency authority is bounded to operational and procedural domains; the chain itself is unchanged by emergency action.

The chain is otherwise oblivious to operational emergencies. The chain has no 'emergency mode,' no 'fallback authority,' no 'override threshold.' Operational emergencies — capture, sustained absenteeism, factional dispute, urgent disbursement need — are handled by the council through normal multisig channels: REMOVE/ADD to replace inactive or compromised members per the threshold table; WOC's bounded veto where applicable; L4 governance for constitutional matters per the Governance Charter. The chain provides the

mechanism (multisig validation against thresholds) but not the policy (when to invoke a particular response). Policy lives in governance; mechanism lives in the chain.

Founders — 5 members. The architectural cohort. Operational executive authority. Distinct from the WOC.

Curia — 9 members. The cryptographic multi-sig governance body. Distinct from the WOC and from the Founders.

Total — 15 distinct constitutional position-holders. The cohorts do not overlap; no person holds positions in more than one cohort. The notation “1 / 5 / 9” refers to these three cohort sizes, summing to fifteen unique constitutional positions.

Governance levels and the Council Registry. Scriptum governance operates across four levels distinguished by scope and signature requirements. The chain's authority structure for VAULT-class Parcels (operational funds: FUND, RESERVE, TITHE, CHARITY_POT, WFC) routes through the Council Registry — a chain-state object holding three locks corresponding to the three constitutional bodies: WOC Lock (1 slot, the singular Warden), Founder Lock (5 slots), and Curia Lock (9 slots). Each lock's slots hold the Dilithium3 pubkeys of the current keyholders. When a transaction attempts to spend a VAULT-class Parcel, the chain reads the Parcel's role and amount, consults the appropriate locks per the role's hardcoded authorisation rule, and validates signatures against the relevant slot pubkeys. Without sufficient signatures from authorised slot pubkeys, the transaction fails — regardless of any other authorisation a transaction might carry.

Class-driven routing. The chain distinguishes authorisation paths by Parcel class. CIRCULATING-class Parcels (ordinary user value) use bundle-based authorisation per the standard Spend Tree mechanism — Dilithium signature against the HOME's bundle, with spend_commitment binding the Parcel to the authorised key. VAULT-class Parcels use lock-based authorisation per the Council Registry — class+role+amount determines which locks must sign and at what threshold. Both authorisation paths run through the chain's existing transaction-validation pipeline; the class determines which path applies. This routing is a Tier 1 protocol constant.

Heartbeat-dormancy per slot. The Council Registry tracks engagement at the slot level. Each slot carries a missed-vote counter alongside its pubkey. When a multisig transaction targeting a lock reaches threshold and is included in a block, the counter for each slot whose signature contributed resets to 0; slots whose signatures didn't appear increment their counter by 1. When a slot's counter exceeds 3 missed votes, the slot is marked dormant — it no longer counts toward the lock's effective N for threshold calculation. Effective M scales proportionally: $M_{\text{effective}} = \max(1, \text{floor}(M_{\text{nominal}} \times N_{\text{active}} / N_{\text{nominal}}))$. As slots return to active by signing again, counters reset and effective N rises. The mechanism is self-healing without governance intervention; chronic absentees naturally fade in their effective weight without permanently locking the council.

Slot composition and the bootstrap mechanism. At genesis, the Council Registry is initialised with 15 Dilithium pubkeys baked into the genesis block: one real WOC pubkey (controlled by the chain's founding WOC) and 14 placeholder pubkeys for the Founder and

Curia slots (also controlled by the founding WOC during the bootstrap period). Each non-WOC slot has a one-shot bootstrap flag (`woc_one_shot_used`). The WOC may sign a `WOC_INSTALL_SLOT` transaction — signed by WOC alone — that replaces a placeholder pubkey in a non-WOC slot with the real council member's pubkey, simultaneously setting that slot's flag to true permanently. Each non-WOC slot can be WOC-installed exactly once. After all 14 `WOC_INSTALL_SLOT` events, the chain's constitutional sovereignty is fully constituted; WOC's solo authority over slot composition is permanently exhausted.

Post-bootstrap slot mutations via `REMOVE/ADD`. After `WOC_INSTALL_SLOT` exhaustion (and at any point for slots already populated by real keyholders), slot mutations require multisig transactions. `REMOVE_SLOT_KEY` removes a pubkey from a slot, leaving it empty. `ADD_SLOT_KEY` installs a pubkey into an empty slot. Both transactions require thresholds appropriate to the slot's body (Curia, Founder, or WOC). The two operations may be sequenced to effect replacement: `REMOVE` the departing member's key, then `ADD` the replacement's key. Thresholds are asymmetric where appropriate: removal is generally lower-threshold (defang fast) than addition (deliberate empowerment).

Slot operation thresholds (Tier 1 protocol constants). Curia `REMOVE` requires WOC + 3-of-5 Founders + 5-of-9 Curia. Curia `ADD` requires WOC + 4-of-5 Founders + 7-of-9 Curia. Founder `REMOVE` requires WOC + 4-of-5 Founders + 5-of-9 Curia. Founder `ADD` requires WOC + 5-of-5 Founders + 7-of-9 Curia. WOC `REMOVE` and WOC `ADD` both require 5-of-5 Founders + 7-of-9 Curia, with WOC explicitly excluded from participating in their own removal or installation. All thresholds scale per the dormancy formula above.

Council coordination via the council app. Constitutional council members coordinate off-chain through the council app — an Aurum/Foundation deliverable that ships at genesis as a precondition for governance functioning. The app is key-gated by the council member's Dilithium key (without the key, the app does nothing). Council members draft proposed transactions in the app, review them locally, sign with their own Dilithium key, and aggregate signatures from cooperating members. When threshold signatures are collected, the transaction broadcasts to the chain. The chain validates received signatures against the relevant lock's slots; signature presence/absence per slot determines counter resets/increments per the heartbeat-dormancy rule. The council app's specific UX is specified in the Aurum Technical Document; the WP records only that the app must exist at genesis and operates by Dilithium key-gating.

WOC emergency authority is narrow. WOC retains single-signature authority for protocol-release activation (the L3 chain-upgrade signature; without WOC, no upgrade activates) and a bounded protocol veto: within a 24-hour window after any Council Registry mutation proposal is published on-chain, WOC may sign a veto transaction blocking the mutation. WOC has no unilateral operational disbursement authority — `VAULT`-class spends require the locks specified in the role's rules, regardless of WOC's participation. The chain is otherwise oblivious to operational emergencies. Operational emergencies — whatever form they take — are resolved by the council acting through normal multisig channels. The chain provides no emergency unilateral spending authority; if standard multisig cannot resolve an operational situation, the situation is a constitutional matter beyond the chain's authority structure.

Andre's personal key and the WOC key are separate. The WOC's slot in the Council Registry holds a Dilithium pubkey used exclusively for constitutional WOC authority (protocol release, vetoes, L4 participation, WOC_INSTALL_SLOT during bootstrap). Andre's personal Scriptum holdings (TREASURE allocation as WOC, WFC compensation, any other personal Scriptum) live at Andre's personal HOME address, authorised by Andre's separate personal Dilithium key. The two keys have entirely separate security postures: the personal key is everyday-device-resident for normal wallet operations; the WOC key lives in deep cold storage and is used only for the narrow ceremonial occasions when WOC authority is exercised. Compromise of one key does not compromise the other.

Persistent operational stall is a governance problem, not a chain problem. If the council cannot reach signature threshold on a needed operational decision, the decision does not happen. The chain does not have a 'fallback' threshold or 'emergency override' that bypasses the normal authority structure. Persistent failure to reach threshold is treated as a governance signal — the council members investigate why threshold isn't being met (chronic absence, factional dispute, capture/compromise) and respond via normal multisig channels: REMOVE/ADD to replace inactive members, WOC veto where applicable, L4 emergency action for constitutional matters. The Scriptum Governance Charter operationalises these responses; the chain provides the protocol-level commitment that the framework exists and the threshold mechanisms operate as specified.

Council Registry

The Council Registry is a Tier 1 chain-state object holding the constitutional authority structure for VAULT-class Parcel authorisation. It is parallel to the Bundle Registry (which holds bundle data for individual HOME addresses) and the SlotBox Registry (which holds TREASURE Box state for the vesting mechanism). The Council Registry holds three locks corresponding to the three constitutional bodies.

Lock structure. Each lock holds N slots, each containing per-slot state:

pubkey — Dilithium3 pubkey of the current keyholder, or the genesis placeholder pubkey for slots not yet WOC-installed

counter — u32, missed-vote counter; resets to 0 when slot's signature contributes to a passing transaction; increments by 1 when slot's signature is absent from a passing transaction targeting this lock

dormant — bool, true when counter > 3; false otherwise; auto-toggles based on counter

woc_one_shot_used — bool, set to true at genesis for WOC slot; initialised to false at genesis for non-WOC slots; set to true permanently when WOC_INSTALL_SLOT fires on that slot

The three locks at genesis:

WOC Lock: 1 slot, threshold 1 (single-signature authority for WOC actions). slot[0]: woc_pubkey (the real WOC pubkey baked into genesis); woc_one_shot_used = true (WOC cannot self-install).

Founder Lock: 5 slots, nominal threshold 3 (Founder routine threshold; specific operations have their own thresholds per the slot operation table). slot[0..4]: 5 placeholder pubkeys; each woc_one_shot_used = false.

Curia Lock: 9 slots, nominal threshold 5 (Curia routine threshold). slot[0..8]: 9 placeholder pubkeys; each woc_one_shot_used = false.

Effective threshold under dormancy. The lock's effective threshold scales with active slot count. For nominal (M, N), with $N_{\text{active}} = N - (\text{number of dormant slots})$:

$$M_{\text{effective}} = \max(1, \text{floor}(M \times N_{\text{active}} / N))$$

A lock with all slots active operates at nominal threshold. A lock with dormant slots operates at proportionally scaled threshold. The lock is self-healing: dormant slots returning to active raise N_{active} , raising $M_{\text{effective}}$ back toward nominal.

Mutation transactions. The Council Registry is mutated by signed transactions per locked thresholds:

WOC_INSTALL_SLOT (bootstrap mechanism, one-shot per slot): signed by WOC alone; target_slot.woc_one_shot_used must be false; sets target_slot.pubkey = new_pubkey AND target_slot.woc_one_shot_used = true.

REMOVE_SLOT_KEY (slot pubkey removal): signed per the slot operation thresholds; target_slot.pubkey set to empty marker; counter, dormant, and woc_one_shot_used preserved.

ADD_SLOT_KEY (slot pubkey installation): signed per the slot operation thresholds; target_slot.pubkey must be empty; sets target_slot.pubkey = new_pubkey; counter reset to 0.

WOC_REMOVE (WOC slot specifically): same mechanism as REMOVE_SLOT_KEY but with WOC-slot-specific threshold (5-of-5 Founders + 7-of-9 Curia, no WOC participation).

WOC_ADD (WOC slot specifically): same mechanism as ADD_SLOT_KEY but with WOC-slot-specific threshold (5-of-5 Founders + 7-of-9 Curia, no WOC participation).

Slot operation threshold table:

Curia REMOVE — WOC + 3-of-5 Founders + 5-of-9 Curia

Curia ADD — WOC + 4-of-5 Founders + 7-of-9 Curia

Founder REMOVE — WOC + 4-of-5 Founders + 5-of-9 Curia

Founder ADD — WOC + 5-of-5 Founders + 7-of-9 Curia

WOC REMOVE — 5-of-5 Founders + 7-of-9 Curia (no WOC)

WOC ADD — 5-of-5 Founders + 7-of-9 Curia (no WOC)

WOC_INSTALL_SLOT — WOC alone (one-shot per slot)

Protocol release — WOC single signature

Bounded protocol veto — WOC single signature within 24h veto window

All multisig thresholds scale per the dormancy formula above. The chain validates the effective threshold at transaction time using current slot active/dormant status.

Genesis bootstrap reality. At genesis, the founding WOC holds private keys for all 15 slots (1 real WOC key + 14 placeholders). During the bootstrap period, the founding WOC may sign WOC_INSTALL_SLOT transactions to transfer each non-WOC slot to its real keyholder as members are identified, vetted, and onboarded. The bootstrap period ends when all 14 non-WOC slots have been WOC-installed. The chain enforces the one-shot per slot; the founding WOC cannot re-claim slots after installation. There is no chain-enforced timeline for the bootstrap period; it may take months to years, governed by the practical pace of constitutional onboarding.

VAULT-Class Authorisation

VAULT-class Parcels (operational funds: FUND, RESERVE, TITHE, CHARITY_POT, WFC) use class-driven lock-based authorisation in addition to the standard layer-1 spend_commitment verification. The chain consults the Council Registry to determine which locks must sign and at what threshold, based on the Parcel's role and amount.

Authorisation by role:

FUND (operating budget at FUND_HOME): authority rests primarily with the Curia Lock. Disbursements are governed by per-transaction tier escalation plus cumulative monthly and annual caps. The values below are 2026-reference, HICP-indexed annually per Scriptum Governance Charter §4.2 via the HICP Indexing mechanism.

Per-transaction tier ladder:

Small ($\leq \text{€}1,000$): Curia 5-of-9 (L1).

Medium ($\leq \text{€}5,000$): Curia 5-of-9 + WOC.

Large ($\leq \text{€}10,000$): Curia 5-of-9 + WOC + Founder 3-of-5.

Extra-large ($> \text{€}10,000$): L3 (Founder 3-of-5 + Curia 5-of-9).

Cumulative envelopes:

Monthly cap: $\text{€}10,000$ cumulative across all FUND disbursements per calendar month.

Annual cap: $\text{€}100,000$ cumulative across all FUND disbursements per calendar year (Charter §5.3 FUND_DISBURSEMENT_L1_CAP).

Cap overage behaviour — L3 escalation. Any FUND transaction that would push monthly cumulative $> \text{€}10,000$ (HICP-indexed) requires L3 (3-of-5 Founders + 5-of-9 Curia) regardless of per-transaction tier. Any FUND transaction that would push annual cumulative $> \text{€}100,000$ (HICP-indexed) requires L3 regardless of per-transaction tier or month. Both ceilings reset deterministically: monthly each calendar month; annual each 1 January. Within the L3 escalation, the per-transaction tier still determines individual signature ceremony.

L4 override — always available. Any FUND disbursement at any amount may be authorised at any time via L4 (WOC + 3-of-5 Founders + 7-of-9 Curia). L4 override bypasses all cap structures and tier requirements. Use case: chain emergency, unbudgeted critical spend, missed budget cycle, or any genuinely exceptional operational need. L4 override is the constitutional escape valve; no FUND is ever trapped beyond reach. Each L4 override is

recorded on-chain with the full constitutional ceremony; visible record of exceptional weight.

Cap base-value amendments — L4 during HICP Countersign Window. The 2026-reference base values for monthly cap, annual cap, and per-tier boundaries are L4-amendable. L4 amendments are processed only during the HICP Countersign Window (1 February – 31 March each calendar year). The CAP_AMENDMENT_RATIFY transaction (L4: WOC + 3-of-5 Founders + 7-of-9 Curia) commits new base values. New base values take effect upon ratification (current calendar year onward); HICP indexing continues to compound on the new base. Years where no cap amendment is needed: no CAP_AMENDMENT_RATIFY transaction; caps remain at prior base values, HICP-indexed forward.

Cumulative tracker chain state. The chain maintains per-FUND-vault cumulative trackers in EUR: cumulative_month_eur, cumulative_year_eur. Each FUND_DISBURSE transaction adds the disbursed amount (converted from FUNK to EUR at current V_A) to both trackers. Monthly tracker resets at calendar-month boundaries; annual tracker resets at 1 January. The chain enforces signature-set verification + cumulative-cap checks at each FUND_DISBURSE; transactions that would push beyond cumulative caps without L3 escalation are rejected.

Why the tier ladder exists. Routine FUND operations ($\leq \text{€}1,000$) execute with L1 ceremony, keeping day-to-day spend administratively light. Larger disbursements require broader cohort participation, scaling ceremony to the operational weight. Monthly and annual caps bound Council routine action, requiring L3 escalation for atypically lumpy months or years. L4 override preserves access to FUND for genuinely exceptional needs without breaking the routine structure.

RESERVE (deep reserve at RESERVE_HOME): any access requires L4 (WOC + 3-of-5 Founders + 7-of-9 Curia) per Scriptum Governance Charter §5.4. There is no tier ladder; there is no L1, L2, or L3 access to RESERVE. RESERVE is the chain's strategic long-term reserve, deployable only in genuinely exceptional circumstances — catastrophic chain emergency, civilisational-scale uncertainty — and not for ordinary operation. Each RESERVE access is a constitutional moment, recorded on-chain with the full L4 ceremony as a visible record of exceptional weight.

TITHE (pre-quarterly tithe pool at TITHE_HOME): protocol-driven distribution only. No user authorisation required; the chain executes TITHE distribution at every 13th Menuha per the Fee Cascade architecture.

CHARITY_POT (charity distribution at CHARITY_POT_HOME): Curia 5-of-9 at any amount within the periodic allocation. The Curia governs Charity Pot allocations via L1 multisig.

WFC (compensation pool at WFC_HOME): protocol-driven distribution per the WFC Chain Primitive (see WFC Compensation Pool — Chain Primitive section). No user authorisation; chain executes WFC distribution quarterly with cap-overflow to Charity Pot.

Layer 1 verification for VAULT Parcels. The standard layer-1 check (spend_commitment + Dilithium signature against the HOME's bundle) applies uniformly to all Parcels including VAULT. For VAULT HOMES (FUND_HOME, RESERVE_HOME, etc.), the bundle's Dilithium pubkey is derived from a publicly-known seed (e.g., SHA3-512("scriptum.fund.bundle")),

SHA3-512("scriptum.reserve.bundle"), etc.). The corresponding private key is publicly derivable; anyone can sign for it. Layer 1 is trivially satisfiable for VAULT Parcels — it preserves the 'every Parcel is the same' verification uniformity without granting any meaningful authority. The real authority gate is layer 2 (class-driven lock-based authorisation via the Council Registry).

Layer 2 verification for VAULT Parcels. The chain reads class=VAULT, role, and amount from the Parcel being spent. The chain looks up the role's authorisation rule in the protocol's hardcoded rules table. The rule specifies which locks must sign and at what threshold (with amount-tier escalation). The chain validates signatures from the transaction against the relevant lock's slot pubkeys. Threshold satisfaction is computed per the effective threshold formula ($M_{\text{effective}} = \max(1, \text{floor}(M \times N_{\text{active}} / N))$) reflecting current slot dormancy state.

Why the public bundle key is intentional. Publishing the FUND_HOME bundle's private key would, in a normal Parcel architecture, be a catastrophic key-handling failure. Under Scriptum's class-driven authorisation architecture, it is intentional and harmless. The chain's authority structure for VAULT Parcels does not flow from the bundle. The chain enforces class-driven lock-based authorisation in addition to standard signature verification; without valid Council Registry signatures matching the role's rules, no transaction against FUND_HOME succeeds regardless of who signed the bundle key. The public bundle key satisfies the universal Parcel structure (every Parcel passes through standard verification uniformly) without granting any authority. The constitutional Parcels' authority is constitutional, not cryptographic at the HOME level.

Spam mitigation. Any actor can construct apparently-valid spend transactions against VAULT Parcels (layer 1 satisfiable trivially via the public bundle key). Such transactions fail at layer 2 (no valid Council Registry signatures). Mempool implementations may pre-filter VAULT-targeted transactions without Council Registry signatures to avoid wasting full validation effort; this is a mempool policy, not a consensus rule. Transaction fees apply to all spend attempts including VAULT spends, making spam economically expensive.

Decision Tiers — Four-Level Multi-Sig

All governance decisions are classified into four levels. The level determines the cohort participation and signature threshold required.

Level 1 — Curia Operational

Threshold: 5-of-9 Curia signatures.

Scope: routine Funds disbursements within budget, regular WFC compensation distributions, Charity Pot group-decision allocations, day-to-day operational decisions.

Founders and WOC do not participate in Level 1 decisions.

Level 2 — Founders Strategic

Threshold: 3-of-5 Founders signatures.

Scope: strategic operational decisions — public communications strategy, partnership commitments, executive hiring, product direction, Foundation board nominations.

Curia and WOC do not participate in Level 2 decisions.

Level 3 — Cross-Cohort Significant

Threshold: 3-of-5 Founders AND 5-of-9 Curia signatures (both gates required).

Scope: Charter amendments, large unbudgeted spending above the thresholds defined in Schedule A of the Governance Charter, membership changes (succession, removal-for-cause), parameter changes within protocol bounds.

The WOC does not participate unless the matter has Constitutional weight, in which case the decision proceeds to Level 4.

Level 4 — Constitutional

Threshold: WOC + 3-of-5 Founders + 7-of-9 Curia signatures (all three gates required).

Scope: Constitutional amendments, hard-fork protocol changes, fundamental architectural changes, revisions to FUND authorisation cap structure (monthly cap, annual cap, per-transaction tier boundaries) via CAP_AMENDMENT_RATIFY during the HICP Countersign Window including changes to the Tithe rate (90/10), changes to the Tithe split (48/32/10/10), changes to the existence of the four streams (WFC, Funds, RESERVE, CP), revisions to the cap-overflow destination rule, and **RESERVE access** (the chain's strategic long-term reserve, deployable only in genuinely exceptional circumstances — catastrophic chain emergency, civilisational-scale uncertainty — and not for ordinary operation).

Emergency Operations

Threshold: WOC emergency declaration + 5-of-9 Curia signatures.

Scope: time-critical matters where a Level 3 process would be too slow.

WOC acting alone is time-limited to 72 hours. After 72 hours the decision must be ratified by retroactive Level 3 (3-of-5 Founders AND 5-of-9 Curia) within 30 days of the original action. If ratification fails, the WOC action is automatically reversed by protocol where reversibility is technically feasible.

The WOC acting alone cannot: change the protocol consensus rules, change S_max, change RAGNAROK_HEIGHT, change the Tithe split, alter compensation caps, or remove other constitutional position-holders. These are Constitutional matters and require the full Level 4 process.

Decision Classification

Routine → Level 1 (Curia 5-of-9): routine Funds disbursements, regular compensation distributions, Charity Pot group allocations.

Strategic → Level 2 (Founders 3-of-5): partnerships, executive hiring, product direction, public communications.

Significant → Level 3 (Founders 3-of-5 AND Curia 5-of-9): Charter amendments, large unbudgeted spending, membership changes, parameter changes within protocol bounds.

Constitutional → Level 4 (WOC + Founders 3-of-5 + Curia 7-of-9): protocol fork, WOC succession, Constitutional amendments, changes to the Tithe architecture, RESERVE access; FUND cap structure amendments via CAP_AMENDMENT_RATIFY during HICP Countersign Window.

No-Quorum Rules

If signature thresholds cannot be reached, status quo prevails. The action does not execute. For Level 1 and Level 2 matters the vote may be called again at the next opportunity. For Level 3 matters the WOC may invoke emergency authority subject to the Emergency Operations rule above. For Level 4 Constitutional matters inertia is always correct — if 7 of 9 Curia and 3 of 5 Founders cannot agree alongside the WOC, the protocol does not change.

WOC Emergency Authority

The WOC holds emergency authority as a last resort. This activates when: (1) the chain faces an active threat and neither Curia nor Founders can act in time, or (2) both cohorts have failed to reach Level 3 thresholds and a decision must be made within hours rather than days.

WOC acting alone is time-limited to 72 hours. After 72 hours the decision must be ratified by retroactive Level 3 within 30 days. If ratification fails, the WOC action is automatically reversed by protocol where technically feasible.

The WOC acting alone cannot perform Constitutional changes. These remain Level 4 matters and require the full process regardless of urgency.

WOC Succession

The Scriptum Governance Charter — a separate document published with the protocol — specifies WOC succession, Founders cohort and Curia composition details, dispute resolution, and emergency procedures in operational detail. This whitepaper commits the protocol layer; the Charter commits the governance layer that operates within the protocol's hard caps. Requirement: the Charter MUST provide a delegated-backup protocol so that short-term WOC unavailability (days, not weeks) cannot block governance operations. Specific mechanism is deferred to the Charter.

Fountain Governance

The Fountain (Genesis Liquidity Reserve) is controlled as described below.

Auto-closure at block 864,000 → protocol-driven, no vote required

All Fountain votes are recorded on-chain as Dilithium3-signed transactions. Every vote is permanent, public, and auditable.

The Upgrade Ceremony

Every protocol change of consensus-altering scope is, technically, a hard fork. Nodes running pre-amendment software cannot validate post-amendment blocks. The architectural innovation Scriptum brings is not that it avoids hard forks; it is that it institutionalises them. What in other chains is an ad-hoc social-coordination event with uncertain outcomes — informal negotiation among developers, miners, exchanges, and users; deadline-pressure activation mechanisms; meaningful risk of permanent chain split — Scriptum performs as a structured public ceremony with defined gates, mandatory consultation, synchronised deployment, and explicit user exit rights.

The chain calls this a cut and splice. The rules cut at a defined boundary; the chain splices forward under the new rules without branching into parallel old and new chains continuing in parallel. The technical hard fork remains a hard fork at the consensus level — old software cannot validate new blocks. The ceremony is the institutional infrastructure that transforms it from a disruptive event into a routine procedure.

Cut and Splice, Not Branching

When Bitcoin or Ethereum hard-fork, the chain potentially splits into two chains. The new chain continues under the new rules; the old chain continues under the old rules. Both demand maintenance, both demand validation, both demand exchange listings, both demand wallet support. Communities fragment. CASPs face dual-integration requirements. Users hold balances on both sides until they decide which to abandon. The 2017 split that produced Bitcoin Cash and the 2016 split that produced Ethereum Classic illustrate the pattern: technically clean forks that produced two ongoing chains where one was intended.

Scriptum's Menuha-windowed upgrade produces no parallel old chain. The constitutional cohort declares which chain is canonical at Level 4 ratification. The Foundation publishes the new Aurum binary signed by the WOC's release key. Foundation-aligned infrastructure deploys the new software. Aurum installations auto-update during the Menuha window. CASPs integrated with the Foundation switch their integrations to the new chain. Miners follow the rewards. Operators who fail to upgrade find their nodes producing blocks that the upgraded network rejects; their hashpower is wasted; they upgrade or stop mining. The old-rule chain technically exists for as long as anyone runs it, but it has no constitutional endorsement, no Foundation infrastructure, no economic infrastructure, no user base. It is a chain in the sense that it is a sequence of cryptographically-bound blocks. It is not a chain in the sense that anyone uses it.

This property is structural rather than algorithmic. The technology cannot prevent operators from running old software. The institutions make the alternative chain economically inert. The cut-and-splice claim is a commitment that the Foundation, the constitutional cohort, and the institutional infrastructure will hold the canonical chain coherent through change. The technology is the substrate; the institutions are the chain.

The Ten Gates

The ceremony proceeds through a sequence of discrete gates, each of which is a multi-sig event recorded on-chain and attributable to its signers. The full procedural treatment is documented in the Governance Charter §8.2 and in the standalone Upgrade Ceremony exposition. The Whitepaper records the gates as a non-amendable procedural framework.

Gate 1 — Initiation. Level 4 multi-sig (WOC + 3-of-5 Founders + 7-of-9 Curia) signs to open the upgrade process. No specific change is yet proposed; the cohort declares it is entering deliberation.

Gate 2 — Specification input. The proposed change is documented in full and published on-chain in a structured proposal format.

Gate 3 — Specification lock. Level 4 multi-sig signs to lock the specification. After this signing the proposal is fixed; no edits or addendums are possible.

Gate 4 — Public consultation opens. Every Aurum wallet, every Sanctum archive, every connected node receives notification. The full specification is mirrored across Foundation public channels and published on the Foundation website. Constitutional position-holders are publicly accountable for the proposal they have just locked.

Gate 5 — Consultation window. A bounded period during which the community reads, debates, raises objections, and proposes amendments. Duration is set per proposal in the specification, between one and four Menuha cycles depending on consequentiality. The window is finite; ceremony stalls do not extend deliberation indefinitely.

Gate 6 — Amendment decision. Level 4 multi-sig signs to either accept consultation feedback (re-entering the ceremony from Gate 2) or confirm no changes are needed (proceeding to deployment). The cohort's reasoning is documented and published.

Gate 7 — Deployment lock. If no amendments, Level 4 multi-sig signs to commit the implementation. The new node software is published. The deployment timer begins, anchored to a specific future Menuha boundary.

Gate 8 — Final notice. Every wallet receives a second notification: protocol change confirmed, deployment scheduled. This is the last opportunity for users to exit before the change becomes binding.

Gate 9 — User exit window. Users have time before the upgrade fires to make their own decisions: continue and let their wallet auto-update, prepare to upgrade manually, sell holdings if they object to the change, withdraw to fiat through a CASP, or exit to a different chain entirely. This is the chain's Safety Control Window applied to its own changes — a window during which informed individual exit is genuinely possible.

Gate 10 — Fire at Menuha boundary. The change deploys at the start of the designated Menuha. The chain continues with new rules. Nodes that have not upgraded fall out of consensus until they update. Operations during the Menuha window run under the chain's normal Menuha rules (user-initiated outward transactions prohibited, protocol

housekeeping continues); the splice fires at the end of Menuha, and Monday operations resume under the new rules.

Procedural Commitments

In ratifying the architecture documented in this Whitepaper and in the Governance Charter, the constitutional cohort makes four explicit procedural commitments about how Level 4 protocol authority will be exercised.

First, the ceremony will not be skipped or accelerated for ordinary protocol changes. Genuine emergencies — critical bugs, active exploits — may invoke the WOC's emergency authority with retroactive Level 3 ratification per the Charter; the ceremony cannot be bypassed by re-classifying a substantive amendment as an emergency.

Second, public consultation periods are real periods of community input, not ceremonial pauses. Feedback received during Gate 5 must be considered at Gate 6, and the cohort's reasoning published.

Third, user notification through Aurum, Foundation channels, and dfqr.org is a procedural requirement, not a courtesy. Failure to notify users at Gates 4 and 8 constitutes a procedural breach of the ceremony.

Fourth, the user exit window between Gate 8 and Gate 10 is genuine. The deployment timing is fixed at Gate 7 and not accelerated thereafter; users who decide to exit have the full notified period to act.

Welcomed Forkability

Apache 2.0 license on all source. Anyone may fork the code and launch a derivative chain. The Foundation does not consider this a threat. What makes Scriptum what it is — the constitutional governance, the Foundation, the eight-year design history, the accumulated chain state, the people holding the fifteen constitutional positions, the trademark, the institutional trust — none of these travel with a code copy.

Forkability is what makes the architecture honest. There are no hidden mechanisms, no admin keys, no privileged paths that the open code does not reveal. Anyone with the technical competence may inspect the chain in full and may, if dissatisfied, build a derivative under different governance. This constrains the Foundation through the credible exit threat: governance that ceases to serve the chain's purpose may find its constituency departing for a fork that does. The strongest version of any tradition is the version that survives by being chosen freely, not by being imposed.

The chain's value is in what it is, not what it is written in. Scriptum welcomes scrutiny, derivation, and the existence of derivatives. The institutional commitments documented in this Whitepaper and in the Charter are what make the canonical chain canonical. The code is what makes the chain technically possible; the institutions are what make it institutionally durable.

Limits of Procedural Protection

The ceremony provides procedural protection against accidental or hasty constitutional changes. It does not provide architectural protection against deliberate constitutional misuse. Distributed-systems specification, in the tradition of Lamport’s work on consensus protocols and the formal-methods literature more generally, distinguishes between behaviour the system guarantees within its operating assumptions and behaviour outside those assumptions. A consensus protocol that tolerates f Byzantine actors out of $3f+1$ specifies precisely what happens up to that threshold; it does not specify what happens beyond, because beyond the threshold the system is no longer the system it was designed to be.

The same applies to Scriptum’s constitutional architecture. The protocol and the ceremony specify what happens when the constitutional cohort acts within model — exercising authority through the procedures and at the thresholds the architecture provides. They do not specify what happens when the cohort acts outside model: through coordinated bad-faith ratification of breaking changes, through institutional collapse, or through other fundamental failures of the governance assumptions the protocol relies on. The defence against constitutional failure is institutional — the ceremony, the consultation, the public attribution of every signature, and the open-source forkability of the chain itself — not architectural. This is a property the Whitepaper acknowledges honestly rather than overclaiming protection it cannot provide.

The Tithe — Protocol Revenue

Scriptum applies a protocol tithe of 10% on all f_S (transfer fees). The Tithe functions economically as a protocol-level earmarked levy enforced by consensus rules. It is not a state tax — no government authority imposes it, collects it, or controls its distribution. The Tithe is mandatory within canonical Scriptum operation: every miner-included transaction routes 10% to the Tithe stream per the universal fee split. The Tithe stream then allocates per the 48/32/10/10 split across WFC, Funds, RESERVE, and Charity Pot.

The Split — Stage One

Every fee-bearing transaction splits at the protocol level:

90% → miner (block-production reward to the miner including the transaction)

10% → Tithe (the chain’s protocol stream)

The 90/10 split applies uniformly to f_U , f_A , f_S , and $F_REFRESH$. There is no carve-out for flat instruction fees: every euro of fee revenue feeds the same two streams in the same proportion.

The 90% miner share preserves competitive proof-of-work economics across the chain’s century-scale lifetime. Bitcoin, Monero, Litecoin, and other major proof-of-work chains pay miners 100% of fees; Scriptum’s 90% is calibrated to remain attractive to mining hashrate while creating a meaningful 10% protocol stream. Reducing the miner share below 85%

would risk hashrate migration to competitor chains and weaken the chain's security budget. The 90/10 split is structurally bounded by the practical economics of competitive proof-of-work and is a Constitutional invariant subject to a structural floor of 85%.

The Tithe Allocation — Stage Two

WFC compensation caps — plain-language summary:

WFC compensation for constitutional position-holders is capped by rule, not by discretion. Annual caps are 2026-reference EUR values HICP-indexed: WOC €2M, each Founder €1.8M, each Curia €1.3M.

Once any position-holder reaches their annual cap, all subsequent WFC revenue that would accrue to that position-holder is diverted automatically to the Charity Pot. The chain enforces this; no human authority decides each year.

Amendments to the cap structure (base values, tier boundaries) require Constitutional process — L4 multi-sig (WOC + 3-of-5 Founders + 7-of-9 Curia) via CAP_AMENDMENT_RATIFY during the HICP Countersign Window. Routine governance cannot adjust caps.

This architecture prevents indefinite personal enrichment from protocol revenue while preserving predictable stewardship funding. Chain success at scale routes through to humanitarian benefit by structural design, not by promise.

The 10% Tithe is allocated across four streams in fixed proportions. The split is flat: the same percentages apply at every chain volume, from cold-start to maturity.

WFC — 48% of Tithe. The Warden / Founders / Curia compensation pool. Distributed across the constitutional position-holders by per-person share-units per the WFC Chain Primitive (see §WFC Compensation Pool — Chain Primitive). Subject to HICP-indexed personal compensation caps with excess routed to FUND via the CP tax mechanism.

Funds — 32% of Tithe. Protocol-defined stewardship allocation within the VAULT class. The protocol role is not itself a legal person; governance may direct such allocations to one or more suitable entities in accordance with the Charter and applicable law. The current intended recipient is the Scriptum Foundation upon incorporation at the Bootstrap T1 milestone; the protocol does not lock any specific legal entity at the consensus layer. Funds the operational work of the recipient: legal, audit, regulatory reporting, hosting, security retainers, professional services, infrastructure.

RESERVE — 10% of Tithe. Strategic long-term reserve, accumulated against exceptional circumstances. Released only by Constitutional governance decision (full process per the Governance Charter).

CP — 10% of Tithe. The Charity Pot — the chain's structural humanitarian allocation. Receives the flat 10% share of every Tithe euro at every chain volume, and additionally receives all cap-overflow from position-holder compensation per the cap-overflow rule below.

The four-stream split is a Constitutional invariant. The percentages within the split (48/32/10/10) and the existence of the four streams may be revised through Constitutional amendment; the existence of CP as a structural component of the Tithe is invariant.

WFC Compensation Pool — Chain Primitive

The WFC (Warden / Founders / Curia) compensation pool receives 48% of every Menuha's TITHE distribution. WFC funds ongoing compensation for the constitutional position-holders — distinct from TREASURE, which is one-time vested allocation at genesis. WFC is paid in FUNK; the Compensation Cap (CP) is a EUR-denominated value-ceiling mechanism that bounds the real value each position-holder receives per cadence.

Source: 48% of TITHE. At every 13th Menuha (the TITHE distribution cadence per the Fee Cascade), 48% of the TITHE distribution accumulates in the WFC pool at WFC_HOME. The remaining 52% of TITHE distributes to other allocations per the Fee Cascade.

Distribution cadence: quarterly with annual 1 January reset (Charter §3.4). The chain accumulates Tithe revenue across each calendar quarter. At quarter-end, the accumulated Tithe is split per the 48/32/10/10 allocation across WFC, Funds, RESERVE, and Charity Pot. The WFC slice is then distributed across the fifteen constitutional position-holders by the 10/9/5 share-units.

Distribution mechanic — calculate first, cap check second, overflow to Charity Pot. For each position-holder at quarter-end:

1. The chain computes the position-holder's FUNK share of the WFC pool per the 10/9/5 share-units (WOC 10%, each Founder 9%, each Curia 5%)
2. The chain converts the FUNK amount to EUR using current V_A and the cumulative HICP-indexed cap value (see HICP Indexing section)
3. The chain adds the EUR amount to the position-holder's running annual total
4. Cap check: if the running annual total remains below the position-holder's HICP-indexed cap (Charter §4.1), the full quarterly amount disburses in FUNK to the position-holder's HOME; if the running annual total crosses the cap during this quarter, the portion up to the cap disburses to the position-holder and the remainder routes to Charity Pot; if the running annual total already reached the cap in a prior quarter, the entire quarterly amount routes to Charity Pot
5. On 1 January of each calendar year, running annual totals reset to zero; HICP-indexed caps for the new year take effect via the HICP ratification mechanism

The calculate-first-then-check architecture is honest: the chain computes the proportional FUNK distribution, evaluates real-EUR value against the HICP-indexed cap, disburses within cap or overflows to Charity Pot. No 'tax' applies; the cap simply bounds the position-holder's annual real-EUR compensation and routes excess to humanitarian destination.

Per-person share-units (Tier 1 constant, not governance-tunable): 10 / 9 / 5. Per Scriptum Governance Charter §3.3: the WOC receives 10 share-units; each Founder receives 9 share-units; each Curia member receives 5 share-units. Total share-units sum to exactly 100:

WOC $1 \times 10 = 10$; Founders $5 \times 9 = 45$; Curia $9 \times 5 = 45$. Per-person share of WFC pool: WOC = 10%; each Founder = 9%; each Curia member = 5%. Per-person ratio: WOC receives $2 \times$ a Curia member's share; each Founder receives $1.8 \times$ a Curia member's share. The architecture establishes aggregate body parity (Founder body 45% in aggregate = Curia body 45% in aggregate) with per-person hierarchy reflecting constitutional weight.

Personal compensation caps (Charter §4.1, 2026-reference values, HICP-indexed annually per §HICP Indexing — Annual Real-Value Calibration): Warden of the Chain €2,000,000/year (1 person, cohort ceiling €2.0M); each Founder €1,800,000/year (5 persons, cohort ceiling €9.0M); each Curia €1,300,000/year (9 persons, cohort ceiling €11.7M); total ceiling across all 15: €22.7M/year. These are constitutional ceilings, not target compensations. At credible 'working chain' activity levels (€5M annual f_S per Charter §3.5), each position-holder receives substantially less than their cap: WOC $\approx$ €24,000/year (1.2% of cap); each Founder $\approx$ €21,600/year (1.2% of cap); each Curia member $\approx$ €12,000/year (0.9% of cap). The caps bind only at civilisational-scale chain activity (the WOC's €2M cap binds at approximately €417M annual f_S — corresponding to roughly €83 billion of on-chain settlement volume per year). The cap values are 2026-reference; they HICP-index annually per the chain's HICP Indexing mechanism. Each 1 January, the chain applies the cumulative HICP factor to the cap value, preserving real-EUR purchasing power across the chain's century-scale lifetime.

Cap-overflow: 100% to Charity Pot (Charter §4.4). WFC revenue that would exceed any position-holder's HICP-indexed annual cap routes 100% to the Charity Pot. Single rule, single destination, no cascade. There is no intermediate stop at Funds (Funds has its own dedicated 32% Tithe slice); there is no residual flow to RESERVE. Each position-holder's cap binds individually: if the WOC reaches cap in Q3 of a calendar year while the Founders and Curia are still well below their caps, only the WOC's overflow routes to Charity Pot — the others continue receiving their normal WFC distributions until each one's individual cap is reached. The destination — Charity Pot — is a Constitutional invariant.

The cap-overflow rule converts chain success into structural humanitarian commitment by design rather than discretion. The WFC pool always belongs to position-holders by share-units; the individual annual cap simply bounds each position-holder's annual real-EUR compensation; everything above the cap flows to Charity Pot under a single rule.

Underflow handling: prorated distribution. When the WFC pool is below total cap demand (typical during early chain when f_S is low), each position-holder receives their full ratio share (no cap binds yet) — they simply receive less than their CP would allow. No IOU mechanism; no future-payment accrual; the chain doesn't promise to make up underflow.

Conversion price: V_A at the distribution block. The cap-to-FUNK conversion uses V_A at the WFC distribution block. V_A is the chain's canonical fee-calculation price (signed $VA_PROPOSE$ aggregation; see V_A section). Using V_A is consistent with the rest of the chain's economic mechanics; V_A manipulation is economically irrational per the V_A architectural analysis.

Why the caps exist

The personal compensation caps reflect a deliberate constitutional commitment: meaningful but bounded compensation for constitutional service, with prosperity beyond the caps flowing to humanitarian benefit by structural design.

€1.3M/year per Curia member, €1.8M per Founder, and €2.0M for the WOC are calibrated as honest ceilings — enough that constitutional position-holders are compensated commensurate with their responsibilities and the chain's success; capped because no constitutional position-holder should extract unbounded value from a chain whose success rests on broad adoption.

If the chain reaches scale where caps bind, the overflow goes to Charity Pot. The architecture converts chain success into structural humanitarian commitment by design, not by discretion. The cap-overflow rule (Charter §4.4) is a single rule, single destination, no cascade: 100% to Charity Pot.

Underflow as bootstrap reality

In the early chain when annual f_S is far below the level that would saturate CP demand, position-holders receive prorated FUNK distributions (their full ratio share, but at low absolute value). This is the chain's honest economic reality during bootstrap. Foundation bootstrap funding (from Foundation TREASURE allocations or other sources) operates as an independent mechanism — outside the chain's WFC architecture — to provide bridge compensation during early years when WFC pool is too low to deliver meaningful real compensation. The chain's WFC primitive operates as specified; the Foundation's bootstrap discipline is an L1 governance decision separate from the chain primitive.

What Funds Pays For

Storage and hosting → Sanctum infrastructure

Security audit → commissioned from Funds

Bug bounty programme → funded from Funds

Website and maintenance → dfqr.org operations

Legal and regulatory → jurisdiction compliance

Secretariat → governance administration

What the RESERVE Holds

The RESERVE is the chain's savings account. It accumulates from the 10% Tithe slice automatically with every SPEND. It cannot be accessed without Constitutional governance — the highest authority threshold defined in the Governance Charter (WOC + 3-of-5 Founders + 7-of-9 Curia). It exists for genuine emergencies only — protocol-level threats that cannot be addressed through normal operations.

What the Charity Pot Holds

The Charity Pot accumulates from the 10% Tithe slice plus all WFC cap-overflow. Internal allocation is split: 42% individually directed (each position-holder personally allocates a defined fraction per the Governance Charter), and 58% group-decided via Curia 5-of-9 multi-sig. All disbursements are publicly disclosed on-chain. The Charity Pot is the chain's structural commitment that prosperity beyond constitutional architecture flows to humanitarian purpose.

Tithe at Genesis

At genesis the chain is nearly empty. The Tithe generates almost nothing. Genesis allocations and the Fountain mechanism (per §22) provide bridge liquidity for the chain's cold-start period. As adoption grows, the Tithe becomes the primary operational funding source. Genesis allocations are the seed. The Tithe is the harvest.

The Tithe rate (10% of f_S) is a protocol constant. It cannot be changed by Curia vote. It cannot be changed by Founders vote. It requires a Constitutional amendment — the highest governance threshold — to modify. It is designed to be permanent.

The Tithe is a protocol-level allocation, not a service fee. Miners pay no one. The protocol routes the Tithe into the WFC, Funds, RESERVE, and CP vaults, which are on-chain objects controlled through the four-level multi-sig structure defined in the Governance Charter. Every disbursement is an on-chain transaction, publicly visible, cryptographically authorised. The Foundation is a recipient of Curia-authorised disbursements from Funds; it is not a counterparty to any fee-paying user and provides no services to those whose fees feed the Tithe.

Fountain — Genesis Liquidity Provisioning

The protocol does not rely on pre-allocated liquidity or privileged issuance. However, during early network formation, the Fountain — a temporary and rule-bound mechanism — ensures functional market formation and price discovery without structural deadlock.

The Fountain is the concrete implementation of Genesis Liquidity Provisioning (GLP) during the initial network phase. The Fountain is instantiated at genesis and cannot be recreated.

Allocation

Name: Fountain (Genesis Liquidity Reserve — GLR)

Source: RESERVE. The Fountain is a reserved operational tranche within RESERVE, not an additional genesis allocation. Genesis allocations remain unchanged at 16% of S_{MAX} (5% TREASURE + 5% FUND + 5% RESERVE + 1% Genesis First-Buy).

Allocation: 0.25% of S_{MAX} = 105,000 FUNK. Released over the Fountain window from this tranche within RESERVE.

Duration: 864,000 blocks (approximately 100 days post-Genesis).

Reversion: any unused portion of the Fountain tranche at end of window remains in RESERVE under standard L4 lock; the Fountain does not “expire” funds, it simply closes the operational window.

Price

Floor: €0.01 per FUNK throughout the Fountain window; tracks market price upward. The Fountain matches the Genesis First-Buy price at the floor and tracks above-floor market price during the Fountain window. Concretely: the Fountain sells at max(€0.01, current market price). During bootstrap before robust market price discovery exists, the floor protects buyers from a 'no price' state. After market price discovery establishes a price above €0.01, the Fountain tracks the market to avoid subsidising Fountain buyers relative to market participants. The bootstrap fiduciary sets the daily Fountain price at authorisation time based on observed market price (Aurum Market aggregator and external exchange data) with €0.01 as the absolute floor. The Fountain does not maintain a chain-side price oracle; the price is set by the bootstrap fiduciary at each daily authorisation.

Mechanism — Horkos covenants via Aurum

Each Fountain sale is a Horkos covenant (per WP §Horkos / CM-7). The buyer initiates; the bootstrap fiduciary is Party A (seller); the bootstrap fiduciary or WOC serves as the named attester per covenant. The off-chain fiat leg is a wire transfer to the bootstrap fiduciary's client escrow account (client escrow account).

Aurum's Beginning (the onboarding section of the wallet) hosts a dedicated Fountain page. The page shows current Fountain status, the fixed price (€0.01/FUNK), the client escrow account wire instructions, and an “Initiate buy” button that constructs a HORKOS_AGREE with the bootstrap fiduciary's published Fountain address pre-filled as Party A. The buyer enters the FUNK amount, signs the covenant, wires fiat to the client escrow account, and waits for HORKOS_ATTEST. Standard Horkos lifecycle thereafter.

Daily Fountain release rate is managed off-chain by the bootstrap fiduciary (co-signed by WOC via CM-3.14 BOOTSTRAP authority): each day, the bootstrap fiduciary authorises a Fountain Parcel covering that day's expected demand. Buyers initiate Horkos covenants against the current standing Parcel. When the daily tranche depletes, Aurum shows “Fountain depleted today — resumes at next block-day boundary.” Annual operational pacing is documented in the Foundation's eventual annual report; during the Bootstrap Period (Genesis through Curia 5-of-9 seating), pacing is at the discretion of WOC + bootstrap fiduciary.

Eligibility — public

The Fountain is open to any FUNK-holding address. There is no registration requirement, no whitelist, no KYC at chain level. KYC and AML compliance occur at the bootstrap fiduciary's client escrow account when fiat arrives — the bootstrap fiduciary's professional

fiduciary obligations under applicable jurisdictional law handle the regulatory side. Buyers whose wire transfer is rejected by the client escrow account for KYC/AML reasons cannot complete the Fountain Horkos covenant; the covenant CANCEL or attestor refund proceeds per standard CM-7 rules.

Authority

Fountain Parcel releases (the OPEN_VAULT transactions that move FUNK from the RESERVE-held Fountain tranche into the bootstrap fiduciary's seller-side address for Horkos commitment) require multi-sig signing per CM-3.14.4. During the Fountain window (Genesis to block 864,000), which is entirely within the Bootstrap Period, the BOOTSTRAP authority applies: WOC + bootstrap fiduciary co-signature. This is a programmatic exception to the general rule that RESERVE access requires L4 — deliberately scoped to the Fountain window only.

After block 864,000, the Fountain window closes. Any further RESERVE access requires standard L4 procedure per CM-3.14.4 and GOV §5.4. The remaining Fountain tranche balance is no longer specially designated; it is simply part of the RESERVE general balance going forward.

Fiat proceeds

All Fountain fiat proceeds flow into the bootstrap fiduciary's client escrow account under standard professional client-account discipline. The proceeds are held in escrow until the Foundation incorporates (Bootstrap Period transition T1, target 6 to 12 months post-Genesis), at which point the bootstrap fiduciary transfers accumulated Fountain proceeds to the Foundation operating account. This mirrors the existing Genesis First-Buy fiat-proceeds handling per GOV §13.

Closure

At block 864,000, the Fountain window closes automatically. This is operational, not consensus — the CM-3.14 BOOTSTRAP authority for RESERVE OPEN_VAULT specifically scoped to Fountain ends when block 864,000 is reached. The chain enforces this through the BOOTSTRAP authority's time-window check (per CM-3.14.5). Any in-flight Horkos covenants initiated before block 864,000 continue through their normal lifecycle to completion or CANCEL; no new Fountain Horkos covenants can be initiated after block 864,000.

The Fountain is a chain-mechanism with a clear start, a clear pricing discipline, a clear end, and a clear architecture that does not depend on any institution that does not yet exist at Genesis. It is the chain's cold-start mechanism: bounded, public, mechanically simple, and operationally executable from the WOC and bootstrap fiduciary alone.

Exchange-Choice Architecture

Ás-brú trades reference an exchange identifier in their commit fingerprint. The architecture surrounding exchange choice — who decides which exchanges are

appropriate, who admits or rejects, how the recommendation is published and maintained — is open at protocol level. There is no chain-tracked admitted-exchange registry; there is no Curia governance vote over which exchanges qualify; covenants referencing arbitrary exchanges are not rejected at consensus.

This section specifies why the architecture is open, what supporting machinery the Foundation provides, and where the trust actually lives.

Open at protocol level. Ás-brú covenants reference an exchange-identifier hash. The protocol validates that the covenant is well-formed and that the parties signed; the protocol does not validate that the exchange exists, is reputable, or is approved. Miners running Crowglass observe the exchange APIs they choose to observe. The chain's consensus is over what miners actually included in their blocks, not over whether the chain approves of their exchange choices.

Why open. A protocol-level admitted-exchange gate has two failure modes, both worse than the open architecture:

- If the gate is a Foundation-controlled key (signing admission decisions), it provides no real protection because Aurum is Apache 2.0 open-source — the key would be extractable from the source by any sufficiently motivated reader. Honest users would trust the protection; adversaries would defeat it in minutes.
- If the gate is a chain-tracked registry curated by L3 governance, it provides real protection but recentralises the chain — Foundation governance becomes a gatekeeper for commercial activity, contradicting the open-protocol commitment and creating a single point of capture or coercion.

Neither outcome aligns with Scriptum's design commitments. The chain chose open-protocol architecture at Apache 2.0 licensing; this section reflects that choice consistently.

At launch, the only Crowglass implementation is Krakophone (Kraken observer); Aurum bundles Krakophone as its default Crowglass module. Post-Foundation-incorporation, the Foundation publishes a recommended Crowglass operational configuration listing exchanges considered operationally appropriate for Crowglass observation; the recommendation is informational, administered at Foundation-board cadence per GOV §11.3 and §12.1, and is not protocol-binding.

Genesis ships with Krakophone — the Crowglass implementation for Kraken — as Aurum's bundled observer. Kraken's selection at launch reflects MiCA-compliant, established, regulated, audited operations. The Foundation may add to the recommended Crowglass implementations post-incorporation. Removal from the recommended set follows the same Foundation-board procedure; Foundation removal of a recommendation does not retroactively invalidate trades that referenced the exchange before the recommendation changed.

Aurum surfaces recommendations as defaults. Aurum's trade-construction UI for Ás-brú offers the Foundation's recommended set as defaults for user convenience. A user who knows what they are doing may override the default and name a different exchange; a user

operating a non-Aurum wallet may do the same. The choice is the parties'; the protocol enforces what they agreed.

Where the trust lives. With protocol-level curation removed, trust in the exchange choice is the parties' own responsibility. The chain provides reputational signals — Kiva surfaces counterparty trade history; Aurum's recommendation reflects Foundation operational judgement — but does not enforce them. Parties who choose unreliable exchanges or who skip the Foundation's recommended set accept the operational risk that comes with their choice. The chain's view is that adults can make their own choices about commerce; the chain's role is to enforce what they agreed, not to curate what they should have agreed to.

The exchange identifier's specific encoding (hash format, fingerprint construction) is specified in CM-14 and SPX-16. The Foundation's recommendation publication is administered per GOV §12.

Protocol Hard Caps — Curia Capture Defence

The Fountain release schedule is subject to two hard caps enforced at the protocol level. These caps cannot be overridden by any governance vote — including a unanimous Curia vote. They are protocol rules, not governance rules.

Daily cap: MAX release $\leq 2\%$ of starting Fountain balance

per 8,640-block period (~24 hours)

At genesis: $105,000 \times 2\% = 2,100$ Scriptum maximum per day

10 day cap: MAX release $\leq 20\%$ of original Fountain allocation

per 86,400 block period (~10 days)

= 21,000 Scriptum per 10-day epoch, hard ceiling

Any transaction that exceeds either cap is rejected by all nodes. No governance decision can authorise a violation, the protocol rejects the transaction.

This eliminates the over-release attack vector at consensus level. Genesis liquidity is strictly subdominant to organic issuance.

What Fountain is Not

Fountain is not a market maker. It is not a price support mechanism. It is not a permanent liquidity provider. It is seed capital for price discovery — small enough to be honest, large enough to prevent deadlock. At €1 per Scriptum, 420,000 Scriptum represents €420,000. At any price, the NEXUS ensures it cannot be deployed instantly. It drips. That is the design.

The Donation Mechanism — Dana

Dana is the Sanskrit and Pali word for the act of giving — the virtue of generosity itself, recognised across Buddhist, Hindu, and Jain traditions. In Scriptum, Dana names both the donation mechanism and the token of recognition it produces.

Any holder may send Scriptum to the protocol-defined destination Chesed at any time. Chesed is a semantic label only — not a standalone vault. On confirmation, the donated amount is allocated to the Funds and RESERVE VAULTs at a 50/50 ratio as new SEALED VAULT Parcels.

On confirmation of a DONATE transaction, the protocol issues a Dana Parcel to the sending address:

CLASS = MARKER

ROLE = DANA

STATE = PERMA_LOCKED

ADDRESS = donor address

AMOUNT = 1 Nugget (display-only, non-economic)

The Dana is private by default. Only the holder knows it exists. They may choose to share it — or not. It is their record, their choice. The amount donated is not recorded in the marker. Only the act.

Like the Mayflower Parcel, the Dana is permanent and non-economic. It survives every Ragnarök unchanged. Every successor Phoenix carries the record of every holder who gave. The chain remembers.

Dana Parcels cannot be created, transferred, or faked. The protocol issues exactly one per confirmed DONATE transaction, at the sending address. No donation — no Dana.

Aurum Implementation

Aurum includes a dedicated DONATE screen. The holder enters an amount, reviews the destination (Chesed), confirms, and submits. Chesed is a semantic label only — not a standalone address. The donated amount is allocated as new Funds and RESERVE VAULT Parcels (SEALED state) at a ratio of 50/50.

On confirmation of the DONATE transaction, the Dana appears in the wallet quietly — no fanfare, no public announcement. The holder decides what to do with that knowledge.

The DONATE action is separate from all other transaction types. It cannot be triggered accidentally.

Other wallets may support this or not. Aurum defines the canonical experience. The mechanic is consensus-layer — any conforming wallet can implement DONATE and issue Dana.

Supply Accounting

Donated Scriptum is allocated to the Funds and RESERVE VAULTs at a 50/50 ratio as SEALED Parcels, and is excluded from free circulation under standard VAULT rules. Dana Parcels (1 Nugget each) are excluded from all supply calculations — not in S_{mined} , S_{net} , or S_{max} . Protocol overhead only, identical in accounting treatment to Mayflower Parcels.

PART V — GENESIS

Rebirth and the Founding Manifest

Genesis is a descriptive term referring to the first Rebirth event. It has no direct representation in consensus rules. All protocol behaviour is defined through Rebirth, block validation, and state instantiation.

The Scriptum protocol begins — and renews — through Rebirth. Rebirth is the instantiation of a new Phoenix at block height 0.

Block 0 — The Rebirth Block

Block height = 0 in every Phoenix

Exempt from standard block size limits

Coinbase reward = 0

Transaction fees = 0

Contains all DISEMBARK outputs derived from the ARC manifest

Contains all Mayflower Parcel re-instantiations

Contains all Constitutional and VAULT allocations

Standard block rules apply from height ≥ 1

The new Phoenix carries no history from the originating chain. The originating chain is frozen at RAGNAROK_HEIGHT, complete and immutable, left behind. The successor Phoenix starts at block height 0. Block 0 is the entire starting state of the new chain.

Block 0 — Deterministic Construction

Block 0 of the successor Phoenix has no miner. It requires no mining reward, no proof-of-work, and no trusted party to produce it.

Block 0 is fully deterministic from the ARC manifest. Any node that holds the ARC manifest from the originating Phoenix can independently construct the identical Block 0. All nodes verify it against the same manifest. Consensus is achieved not through mining but through deterministic agreement on a known input.

ARC manifest eligibility is determined entirely by protocol rules operating on the predecessor Phoenix. No party — including the Foundation, the Warden, Curia, or any other actor — selects which value crosses into the successor Phoenix. The ARC manifest is computed deterministically from chain state at the Ragnarök boundary block by applying the protocol's eligibility rules to every Parcel that successfully completed BOARDING during the migration window. Every node that maintains the predecessor Phoenix's state computes the same manifest. The successor Phoenix's Block 0 is constructed identically by every honest node. There is no central authority over the centennial transition.

No discretionary producer is trusted. Validity is determined entirely by deterministic reconstruction from the ARC manifest under canonical rules. No node gains an advantage by producing Block 0 first.

This is the only block in Scriptum that is not mined. Every subsequent block from height 1 follows standard proof-of-work rules.

DISEMBARK — Protocol-Driven Instantiation

DISEMBARK is a protocol-driven procedure executed entirely within Block 0. There is no user-initiated DISEMBARK. The protocol reads the ARC manifest from the originating Phoenix and instantiates all valid ARC Parcels directly as LOCKED outputs in the successor Phoenix. Holders find their Scriptum already waiting — LOCKED as always, ready to UNLOCK.

Rebirth Block 0 is exempt from ordinary block size limits. Its practical size depends on the number of ARC entries, their serialisation rules, and implementation constraints — all of which are specification concerns beyond the scope of this document. Rebirth is infrequent enough that elevated resource requirements at Block 0 are acceptable. Implementations may impose practical limits defined in the Scriptum Implementation Specification. Canonical serialisation and ordering rules will be defined in that document prior to mainnet.

ARC 0 — The Founding Manifest

ARC 0 is the initial manifest instantiated at the first Rebirth (Genesis). It contains:

Constitutional passengers — Warden, Founders, Curia (TREASURE, deterministic vesting)

Civil passengers — Aurum registrations (CIRCULATING, LOCKED)

Mayflower Parcels — one per founding participant (CLASS = MARKER, PERMA_LOCKED)

Cargo — Reserve, Funds, Treasure allocations (VAULT)

Chesed — the semantic name for the donation mechanism. Donated value flows to the Funds and RESERVE vaults at a 50/50 ratio as SEALED Parcels. Dana MARKER Parcels are issued in return to the sender as permanent private record.

Supply accounting for MARKER Parcels:

Mayflower Parcels (CLASS = MARKER, AMOUNT = 1 Nugget each) are instantiated by the protocol at Block 0. They are not created by coinbase (Block 0 coinbase reward = 0) and are not part of the economic supply. They are excluded from all supply calculations:

Not included in S_{mined}

Not included in S_{net}

Not counted against S_{max}

Not subject to the emission formula

MARKER Parcels exist outside the economic model entirely. They are protocol overhead — permanent identity records with no supply consequence. This applies equally to Mayflower Parcels (founding participants) and Dana Parcels (donors).

Allocation

84% Mining — distributed via block rewards across the life of Phoenix 1

5% Contributors — TREASURE: 2% WOC + 2% Founders (5 founders $\times$ 0.4%) + 1% Curia (9 members $\times$ $\sim$ 0.111%). Total 5% = 2,100,000 Scriptum across 15 distinct constitutional position-holders. Deterministic vesting per §21.6.

5% Ecosystem — VAULT, ROLE = Funds (Foundation operational allocation, sourced from Curia-authorized disbursements)

5% Reserve — VAULT, ROLE = RESERVE

1% Genesis First-Buy — distributed at genesis via a water-filling allocation algorithm. The Genesis First-Buy pool is 420,000 Scriptum (1% of S_{max}), offered to the constitutional cohort (1 WOC + 5 Founders + 9 Curia = 15 positions) and to FullArc Sign-ups and Genesis Sign-ups registered before the close of the sign-up window. The price is fixed at €0.01 per Scriptum. The per-person hard cap is 50,000 Scriptum (€500 maximum participation), preventing whale concentration in the first observable transaction.

The allocation algorithm is iterative water-filling with the per-person cap applied at each round. Let N be the count of registered participants and $v(n)$ the amount declared by participant n (capped at 50,000). Round 1 computes $V(\text{max1}) = \text{pool} / N$. All participants whose declared amount $v(n) \leq V(\text{max1})$ receive their full $v(n)$. The pool is reduced by what was allocated; N is reduced by the participants who received their full allocation. Round 2 computes $V(\text{max2}) = \text{remaining_pool} / \text{remaining_N}$ from the reduced state. The same logic applies. The algorithm continues until either all participants have received their full declared amount (in which case the pool may have a remainder, which flows to FUNDS), or a round closes with all remaining participants having $v(n) > V(\text{max_round})$ (in which case all remaining participants receive $V(\text{max_round})$ equally and the algorithm terminates with the pool fully distributed).

The water-filling property is provably fair: every participant either receives exactly the amount they declared (because their declaration was modest enough that the pool could

accommodate it) or receives the same amount as every other participant who did not receive their full declaration. No participant is unfairly truncated. No first-mover advantage exists because allocation runs once after the sign-up window closes, not first-come-first-served. The per-person cap of 50,000 prevents whale concentration in the rare case where N is very small.

The constitutional cohort participates on the same terms as every other participant. The WOC declares an amount and is allocated under the same algorithm as a Genesis Sign-up. There is no privileged tier. There is no founder rate. The WOC's distinction is being a constitutional position holder, not a buyer-priority status. The first observable Scriptum/EUR price is €0.01/Scriptum from the first Genesis First-Buy delivery onwards, irrespective of who buys.

Each first-buy is technically structured as a Horkos contract (per §Horkos) with the bootstrap fiduciary as the registered attester C. The participant's fiat is loaded into the bootstrap fiduciary's client escrow account; the protocol holds the Scriptum in a designated Genesis First-Buy escrow. Upon attestation YES (fiat received in trust), the protocol delivers the participant's allocated Scriptum via the standard NEXUS-routed delivery to the participant's stealth-derived one-time address. Upon attestation NO or TIMEOUT, the Scriptum returns to the Genesis pool and the fiat is refunded to the participant. The chain's bilateral settlement primitive bootstraps the chain's own genesis allocation.

Proceeds — the aggregate fiat raised at the close of the sign-up window — flow to the Genesis Bootstrap Fiduciary structure: held in the bootstrap fiduciary's client escrow account pending Foundation incorporation, then transferred to the Foundation upon successful registration in the chosen jurisdiction 6–12 months post-genesis. The Rechtsanwalt is paid in FUNK at €1.00/FUNK conversion rate for professional services rendered. Detailed legal mechanics, sign-up window timing, KYC requirements, and operational specifics are documented in the separate Scriptum Genesis Economics specification published alongside this Whitepaper.

Genesis allocations sum to 16% of $S_{\text{max}} = 6,720,000$ Scriptum (5% TREASURE + 5% Funds + 5% RESERVE + 1% First-Buy). S_{net} at Phoenix 1 block 1 = 6,720,000. The remaining 84% is distributed to miners across the Phoenix lifespan via the emission formula $r = 1 \times (1 - S_{\text{net}} / 42,000,000)$, with genesis emission $r_1 = 0.84$ Scriptum/block, asymptotically approaching the 42,000,000 cap by Ragnarök.

Vesting Schedules

Vesting is linear monthly from month 1 to month N, where N varies by cohort. There is no cliff. Each constitutional position-holder receives $1/N$ of their total TREASURE allocation per monthly milestone, beginning at month 1.

Vesting durations track the relative weight of each cohort's constitutional responsibility: WOC vests over the longest period (3 years), Founders over a medium period (2 years), and Curia over the shortest (1.5 years).

WOC — Warden of the Chain (Allocation: 2% of total supply = 840,000 Scriptum, 1 person)

Release = 1/36th of allocation per month, starting at month 1

Duration = 3 years (36 monthly releases)

Per-month release $\approx 23,333.33$ Scriptum

Notation = 0 / 36

Founders (Allocation: 2% of total supply = 840,000 Scriptum, distributed as 5 founders $\times$ 0.4% each = 168,000 Scriptum per founder)

Release = 1/24th of allocation per month per founder, starting at month 1

Duration = 2 years (24 monthly releases)

Per-month release per founder = 7,000 Scriptum

Notation = 0 / 24

Curia (Allocation: 1% of total supply = 420,000 Scriptum, distributed as 9 members $\times$ $\sim 0.111\%$ each $\approx 46,667$ Scriptum per Curia member)

Release = 1/18th of allocation per month per Curia member, starting at month 1

Duration = 1.5 years (18 monthly releases)

Per-month release per Curia member $\approx 2,592.59$ Scriptum

Notation = 0 / 18

No forfeiture. Constitutional position-holders who depart (voluntarily, through removal-for-cause, or through other circumstance) keep all TREASURE that has vested up to their departure date. Unvested TREASURE remains at the slot HOME; future monthly milestones continue to fire, with the Box mechanism delivering to whomever the slot's current Box keyholder is at each milestone. A vacated slot accumulates re-locked TREASURE Parcels (per the Box-empty branch: re-locked with offset ± 12 months) until the slot is filled with a replacement keyholder.

Pre-existence at genesis. TREASURE Parcels are created at genesis (creation_height = 0) at the slot HOMEs. For each slot, N Parcels are created where N is the vesting count for that cohort (WOC slot: 36 Parcels; each Founder slot: 24 Parcels; each Curia slot: 18 Parcels). Each Parcel has its offset set to the block height of its respective milestone (month 1 = block 259,200; month 2 = block 518,400; etc.). Total TREASURE Parcels at genesis = $1 \times 36 + 5 \times 24 + 9 \times 18 = 36 + 120 + 162 = 318$ TREASURE Parcels across 15 slot HOMEs.

All vesting is protocol-enforced via the VEST transition. The protocol reads the SlotBox Registry at each milestone block and delivers per the Box state (CIRCULATING LOCKED to keyholder if Box filled; re-locked TREASURE with offset ± 12 months if Box empty). No manual action is required from the holder. The arithmetic for Curia and WOC per-Parcel amounts uses $\text{floor}(\text{total_Nuggets} / N)$ for each milestone, with the final (Nth) Parcel absorbing the rounding remainder, ensuring exact Nugget-level totals.

TREASURE is held in personal capacity by the position-holders, not by the Scriptum Foundation. The Foundation holds its own institutional Scriptum position separately, sourced from Curia-authorized disbursements out of Funds (the 32% Tithe slice) for operating purposes.

PART VI — LONGEVITY

The Hard Ceiling — Supply, Scarcity, and the Answer to Everything

Scriptum has a fixed maximum circulating supply of 42,000,000 Scriptum. This number is not arbitrary. It is the answer to the ultimate question of life, the universe, and everything — as established by Douglas Adams in *The Hitchhiker's Guide to the Galaxy*. Unlike Bitcoin's 21 million, whose origin remains debated, Scriptum's supply cap has a clear and deliberate meaning. It signals the character of the project and the people behind it.

The 42M ceiling is approached asymptotically within each Phoenix. No single Phoenix formally reaches 42M. The Scriptum system continues through successive Phoenix cycles — each beginning clean, each approaching 42M anew. Individual Phoenix instances end at Ragnarök. The system does not.

The Coin and its Unit

The coin is Scriptum. Its ticker is FUNK. The base unit is the Nugget: one Scriptum is divisible into 100,000,000 Nuggets, giving Scriptum eight decimal places of precision — identical to Bitcoin's divisibility model.

1 Scriptum = 100,000,000 Nuggets (ticker: FUNK)

Aurum displays balances in Scriptum by default. Nuggets are shown when the transaction amount falls below 0.001 Scriptum.

The River of Forgetting — Lethe and the Lost Coins

Every blockchain that has ever existed has ignored a fundamental truth: that value does not only move. It disappears. Not through burning, which is deliberate and recorded. Not through theft, which is traceable. But through the quiet, invisible, unstoppable entropy of human existence. Keys are lost. Owners die. Addresses are mistyped. The coins remain on chain, visible, countable, forever unreachable.

Scriptum is the first protocol to name this honestly.

In Greek mythology, Lethe is the river of oblivion. The dead drink from it and forget everything. Scriptum designates all permanently unreachable coins as Lethe — Lost Scriptum. Three categories:

N1 — The Forgotten Address: coins sent to an address that never existed. No key was ever generated. No human ever controlled this address.

N2 — The Dead Man’s Wallet: the key existed, the holder is gone — through death, loss, or the destruction of the only record that mattered.

N3 — The Sleeper: the key exists, the holder is alive but silent. Indistinguishable from N2 at the protocol level. What cannot be proven is not protocol.

Two epistemic categories. The chain distinguishes what it can know in real time from what only Ragnarök reveals.

Known-Lethe (protocol-visible loss). Coins sent to provably-unreachable destinations — Gehenna burns, addresses with invalid bundle structure, malformed receive constructions — are recognisable from chain state alone. The chain accounts for these immediately: they enter S_{burned} (for burns) or are excluded from accounting (for malformed-address sends, which never enter S_{mined} to begin with). Known-Lethe is a real-time, deterministic ledger property.

Presumed-Lethe (private holdings of unknown status). Coins held at well-formed addresses where the chain has no signal of holder absence — categories N2 and N3 above — cannot be distinguished from active dormant holdings until Ragnarök. The chain counts these in S_{net} during the Phoenix lifetime exactly as it counts every other holder’s value. Only at Ragnarök, when holders must actively board the Arc to migrate to the successor Phoenix, does presumed-Lethe become known: addresses that fail to board are revealed as truly lost. The successor Phoenix begins with the known-boarded supply; the unboarded portion stays on the previous chain as permanent record. This is the structural function of Ragnarök: it converts presumed-Lethe into known-Lethe through the explicit boarding action.

Lost Scriptum is **explicitly and permanently excluded from protocol calculations to the extent the chain can identify it**. Known-Lethe is excluded immediately; presumed-Lethe is identified at Ragnarök through non-boarding and excluded from the successor chain. $S_{\text{net}} = S_{\text{mined}} - S_{\text{burned}}$. Scriptum does not pretend to know what it cannot know in real time, and resolves the unknowable exactly once per century via the Arc procedure.

Based on observed loss rates across comparable proof-of-work chains, Scriptum estimates that presumed-Lethe will represent approximately 15% of mined supply at the twenty-year horizon. By Year 100, accumulated presumed-Lethe could represent 30–40% or more of total mined supply. This is precisely why Ragnarök exists.

The Sacred Fire — Burn Economics and Gehenna

Burning in Scriptum is not destruction. It is a self-healing mechanism. When Scriptum is burned, S_{burned} increases, S_{net} falls, and the block reward rises. This attracts miners, increases hash rate, strengthens network security, and restores equilibrium without intervention. Burning returns energy to the network. Burning is the network breathing.

Early chain burning is economically rational. When circulating supply is small, burning signals confidence and stabilises early price discovery. As supply matures, burning

becomes progressively more expensive relative to its market impact and naturally self-limits. Rational economics prevents late-stage manipulation.

All burns are directed to Gehenna — the protocol-defined burn address, provably unspendable by mathematical construction. Gehenna takes its name from the Abrahamic tradition: the place of fire, of deliberate and permanent consumption. There is no key. There is no reserve. There is no recovery. What enters Gehenna is gone from spendable circulation permanently.

BURN is always partial. The holder specifies amount x . Remainder stays ACTIVE. S_{burned} increases by x Scriptum only.

The Mayflower Parcel

The Mayflower Parcel is a Genesis marker assigned to founding participants of the Scriptum system. It represents participation in the founding event (ARC 0) and serves as a permanent, non-economic proof of origin.

Structure:

CLASS = MARKER

ROLE = MAYFLOWER

STATE = PERMA_LOCKED

ADDRESS = participant address

AMOUNT = 1 Nugget (display-only denomination, non-economic)

Behavioural rules:

Non-spendable — no UNLOCK, SPEND, LOCK, or BURN

Excluded from all economic calculations and from S_{net}

Always included in ARC transitions automatically

Re-instantiated unchanged at every Rebirth

Exactly one per founding participant

Cannot be created after ARC 0

Mayflower and Dana Parcels are the only records in the Scriptum system that survive every Ragnarök unchanged. They do not carry value. They carry identity. The amount of 1 Nugget is a display-only denomination — it exists to satisfy the Parcel data model, not to represent economic value. Founding participants and donors alike are permanently inscribed in every successor Phoenix, for as long as the protocol runs.

The Centennial Renewal — Ragnarök, the Arc, and the Successor Phoenix

In Norse mythology, Ragnarök is the twilight of the gods. The world ends. And then — the world is reborn. Green and clean. The cycle complete.

Scriptum has its own Ragnarök.

At Year 100 plus one day, the protocol triggers the centennial renewal. A new Phoenix is born. Every holder must actively board the Arc within a defined migration window. This is not automatic. It is not passive. It requires intent, action, and a living key.

TREASURE — Automatic Boarding

TREASURE boarding authority is defined in Section 6.6. In summary: all Parcels with CLASS = TREASURE board automatically. Parcels that have completed vesting and converted to CLASS = CIRCULATING must be enrolled manually.

Protocol Enforcement

Ragnarök is not a soft convention. It is a hard consensus rule.

RAGNAROK_HEIGHT = 315,360,000

No block with height > RAGNAROK_HEIGHT is valid under the Scriptum protocol

All nodes must reject blocks beyond this height

The value of RAGNAROK_HEIGHT is fixed in genesis parameters

It cannot be changed without departing from the Scriptum protocol

A chain that continues beyond RAGNAROK_HEIGHT is not Scriptum. It is a fork. The Scriptum protocol does not recognise it.

The Arc Procedure — Enrollment and Boarding

Enrollment

A holder initiates migration by declaring the intent to board. Through Aurum, the holder selects a specific boarding time prior to Ragnarök. Enrollment is an explicit act. Unconfirmed participation does not exist.

Scheduled Boarding

Boarding occurs at a predetermined moment chosen by the holder. At this moment, control passes from the user to the protocol. There is no dependency on network conditions or user interaction at the time of execution. Boarding is deterministic and guaranteed.

Cancellation

Enrollment may be cancelled until CANCEL_THRESHOLD. Once this threshold is passed, the decision is final. No further changes are possible.

CANCEL_THRESHOLD = 10 blocks before the scheduled boarding slot

At CANCEL_THRESHOLD, the STRAGGLER_WINDOW also closes. These two events are simultaneous and deliberate: the last moment a holder can cancel is also the last moment stragglers are evaluated. After this point the protocol has full control.

PACKING Phase

At approximately 8,640 blocks (~24 hours) before the selected boarding slot, the holder initiates PACK: all Parcels associated with the enrolled wallet — regardless of state — are consolidated into a single PACKING Parcel with AMOUNT = SUM(all Parcels). This is the last user-initiated action in the boarding procedure. From this point the protocol has full control.

Incoming Parcels received after PACKING begins are classified as stragglers.

STRAGGLER_WINDOW

The straggler evaluation window closes at CANCEL_THRESHOLD — 10 blocks before the boarding slot. At this point the protocol evaluates all pending stragglers:

STRAGGLER_WINDOW closes = 10 blocks before boarding slot

Aurum displays a countdown timer for the straggler window. At closure, the holder is presented with a confirmation:

Include stragglers? [YES / NO]

Default: NO (countdown expires with no response = NO)

YES confirmed: PACKING Parcel and stragglers consumed, new consolidated PACKING Parcel created

NO confirmed or timeout: stragglers remain on the originating Phoenix, permanently excluded from migration

Only value that was enrolled and confirmed boards the Arc. Nothing boards silently.

BOARDING

At the selected boarding slot, the protocol executes BOARDING: consumes the PACKING Parcel (and optionally accepted stragglers) and creates a single ARC output with AMOUNT = SUM(all boarded value). This transition is deterministic, irreversible, and final.

Before boarding, value is gathered. At boarding, it is committed.

ANOINTED and CONDEMNED

At CANCEL_THRESHOLD — 10 blocks before the scheduled boarding slot — enrollment closes permanently. At that moment, every wallet is in one of two states:

ANOINTED → enrolled, straggler decision made, boarding confirmed

CONDEMNED → not enrolled, permanently excluded from this Arc

There is no middle ground. No late enrollment. No appeals. The decision is final.

At the boarding slot — 10 blocks after CANCEL_THRESHOLD — the protocol executes BOARD for every ANOINTED wallet simultaneously. This is consensus-enforced: any block that omits a required BOARD output is invalid and rejected by all nodes. The miner cannot exclude it. There is no transaction to censor — BOARD is part of block validation itself, identical in nature to EJECT and EXPIRE. All protocol-driven multi-output events — including BOARD and DISEMBARK — use deterministic canonical ordering defined in the Scriptum Implementation Specification. Implementations that diverge from canonical ordering produce invalid blocks.

A miner who attempts to build a block without the required BOARD outputs produces an invalid block. Other nodes reject it. His chain is orphaned. His hashrate is wasted. There is no attack surface.

After Boarding

The originating Phoenix continues to operate until Ragnarök. Any value that arrives in a boarded wallet after boarding is automatically classified as condemned — it remains valid on the originating Phoenix but is permanently excluded from migration. If the holder does not transfer condemned coins out via SPEND before Ragnarök, they will become permanently inaccessible at the Ragnarök event.

Wallets that were never enrolled (CONDEMNED at CANCEL_THRESHOLD) continue to operate normally on the originating Phoenix until Ragnarök. Their value remains valid and spendable until RAGNAROK_HEIGHT. At Ragnarök, no new blocks are produced. Their value is permanently frozen on the originating chain — visible, verifiable, and permanently unreachable in the successor Phoenix.

*****My name is Ozymandias, king of kings: Look on my works, ye Mighty, and despair.*****
****The coins remain. The chain remembers. Nothing else does.*

Ragnarök and Departure

At Ragnarök, the ledger is finalized. All ARC-state value is fixed. No further boarding is possible. The Arc departs. All boarded value is carried forward into the successor Phoenix. All non-migrated value remains permanently on the originating Phoenix.

The old chain does not cease to exist. It continues, frozen — a permanent archaeological record of every transaction, every burn, every Lethe coin, every block from genesis to Ragnarök.

ARRIVAL — Disembarkation in the Successor Phoenix

ARRIVAL is the process by which ARC-state value becomes available in the new Phoenix. It does not transfer value. It validates proof and reconstructs ownership.

DISEMBARK is a protocol-driven procedure. There is no user-initiated claim transaction. At Block 0 of the successor Phoenix, the protocol reads the complete ARC manifest and instantiates all valid ARC Parcels directly as LOCKED outputs. Holders find their Scriptum already waiting in the new Phoenix, locked and ready to unlock. No action required.

The originating Phoenix is left behind — frozen, immutable, permanently readable. The successor Phoenix begins at block height 0 with no inherited history. Block 0 is its entire starting state.

Finality

Boarding occurs once. Boarding is irreversible. There is no second boarding event, no recovery phase, no extension. What is not aboard remains behind.

The Number That Matters

The number of Scriptum that board the Arc at Ragnarök is the single most important metric in Scriptum's history. It will be known on the day of departure. It will never change. It is a permanent, immutable, public record of how well humanity looked after its wealth across one century. If 42 million board — the system worked perfectly. Every coin that does not board is a story.

Canonical Continuity and Forks

Scriptum does not prevent alternative chains. Any group may choose to continue the ledger beyond Ragnarök by modifying the protocol. Such a continuation is not part of Scriptum as defined by Scriptum. Scriptum's identity is bound to its finite lifecycle and explicit renewal.

The Oldest Truth

Every human civilisation has named the place where things go and do not return. The Greeks called it Lethe. The Norse called the ending Ragnarök. The Hebrews called the fire Gehenna. The Sumerians, four thousand years ago, called the underworld Irkalla — the land of no return.

Scriptum does not invent this. It inherits it. Value has always been lost. Keys have always been forgotten. Owners have always died. What Scriptum does — for the first time in the history of digital assets — is name these things honestly, exclude them from calculations they would corrupt, and design a mechanism that, once per century, washes the chain clean.

Scriptum is not designed as a transactional cryptocurrency in the classical sense built for speculation, high-frequency action, and everyday small purchases. It is a centennial protocol — designed to outlive everyone who builds it, everyone who mines it, and everyone who holds it, and to begin again for those who come after.

Ain Soph — System Philosophy

Scriptum is not:

- A high-throughput payment network
- A trading platform or speculative instrument
- A computational layer or smart contract environment
- A stablecoin or price-controlled instrument

Scriptum is:

- A controlled-value system optimised for secure storage of high-value assets — not optimised for transactional cryptocurrency use cases
- A deliberate value transfer mechanism — not optimised for transaction frequency
- A centennial protocol designed to outlast its creators
- Post-quantum from birth

PART VII — SECURITY & REGULATORY

The Security Model

- Scriptum does not eliminate theft. It transforms theft from an instantaneous, invisible act into a sustained, public operation that must persist across time.

Structural Safety vs. Economic Safety

Scriptum is designed as a system, not as a snapshot of current network conditions. Every claim in this section concerns Scriptum's architecture — the properties that hold from block zero through a century of operation, across rising and falling hashrate, through the ingress of institutional adoption and the shocks of attack. Day-one comparisons against mature chains miss the point. Scriptum's security is structural, and structural properties do not depreciate as the network grows.

At low hashrate, all proof-of-work systems are vulnerable to chain reorganisation. In Bitcoin, reorganisation directly translates into the ability to reverse settlements and extract value — the architecture equates a valid signature with final settlement, and a reorg moves final settlements. Every proof-of-work chain has faced this vulnerability at birth. Bitcoin did, in 2009. Every altcoin does. Scriptum does. The question is not whether the vulnerability exists; the question is what it enables.

In Scriptum, reorganisation alone is insufficient. Value transfer requires three conditions simultaneously: control of the chain, survival across the full 8,640-block Safety Control Window, and possession of the legitimate holder's Dilithium3 key material. Any one of the three is fatal to an attempted theft; two of the three is not enough. This is the second independent constraint on settlement that Scriptum adds.

Scriptum does not replace hashrate security — it adds a second independent dimension. At maturity, Scriptum inherits the same hashrate dynamics as any proof-of-work system, but with an additional constraint on settlement that Bitcoin does not have.

Normalizing the axis. “Safer than” is meaningless in isolation. Safer against what, for whom, at what cost? Without a shared axis, every comparison is rhetorical. Normalize to the only axis that matters to money — the protection of holder value under attack — and every other metric becomes a means to that end, not an end in itself. A holder does not buy hashrate. A holder buys the assurance that their value survives.

The Bitcoin model. Bitcoin's architecture equates authorization with execution. A valid signature is a final settlement. There is no window between the two. A compromised key is a lost coin — instantly, irreversibly, with no point of intervention. Hashrate is the only thing standing between an attacker and other people's money. Therefore, hashrate must be enormous. At current scale, Bitcoin secures value by continuously expending tens of billions of dollars per year in energy and hardware.

This is not a critique of Bitcoin. It is a description of what Bitcoin must spend to make its architecture safe. The spend is real, recurring, and load-bearing. If the hashrate falls materially, holder value becomes materially less safe. The two are directly coupled.

The Scriptum model. Scriptum separates authorization from execution by protocol. A valid signature does not settle value — it starts a clock. The Safety Control Window opens, the action becomes visible on chain, the holder can cancel, and protocol-enforced EXPIRE returns any unspent ACTIVE Parcel to LOCKED. Holder value is protected by two mechanisms working together: Dilithium3 ownership (the cryptographic dimension) and the NEXUS (the temporal dimension). Hashrate in Scriptum performs the same role it performs in every proof-of-work chain: establishing chain ordering and making censorship expensive. Hashrate alone does not guarantee settlement safety in Scriptum because the NEXUS adds an independent temporal constraint — but hashrate remains necessary to ensure the NEXUS can function honestly under adversarial conditions.

An attacker with unlimited hashrate and no key steals nothing — but may still degrade liveness by censoring user-initiated transactions. Protocol-driven transactions (EJECT, EXPIRE) cannot be censored; honest nodes reject any block omitting them. User-initiated transactions (UNLOCK, CANCEL, SPEND) can be delayed by a majority attacker, but not falsified or silently dropped. An attacker with a stolen key and no hashrate is visible for twenty-four hours before the attack can complete and can be cancelled by the legitimate holder within that window. Hashrate remains necessary to ensure the Safety Control Window can function honestly; it is no longer sufficient on its own to enable theft. Theft requires both conditions simultaneously, sustained across the full observation window.

Bitcoin spends billions of dollars a year on hashrate to protect what Scriptum protects by design.

The honest concession. At genesis, Scriptum's hashrate is low. An attacker with rented GPU time can successfully reorg a Scriptum block in the first weeks of the chain. We concede this openly. A reorg by itself, without possession of a legitimate holder's Dilithium3 private key, extracts no value from anyone. Every Parcel in NEXUS dwell returns to LOCKED on CANCEL or proceeds to its destination on EJECT — the chain's protocol-driven transitions are consensus-enforced and cannot be censored. Every Parcel in ACTIVE reaches its expiry and is returned to LOCKED by protocol-enforced EXPIRE, which honest nodes will not allow any block to omit. Every LOCKED Parcel requires a fresh 8,640-block NEXUS before it can move — visible throughout, cancellable throughout, observed throughout. A reorg alone is insufficient for theft.

We must be precise about what the NEXUS does and does not prevent. The NEXUS does not make theft impossible regardless of attacker resources. An attacker who possesses BOTH a legitimate holder's Dilithium3 private key AND sufficient sustained hashrate to suppress the holder's CANCEL across a full 24-hour NEXUS can extract value. What the NEXUS does is elevate the attack from an instant, hidden, irreversible event to a sustained, detectable, public operation. The attacker must maintain dominance for 8,640 consecutive blocks in plain sight of the entire network while the legitimate holder attempts to cancel. Every honest node sees the UNLOCK. Every wallet can flag it. Every exchange can freeze downstream operations. The attacker cannot hide. Scriptum does not promise impossibility; it promises visibility and time — the preconditions for intervention that no instantly-final chain can offer.

NEXUS protection is conditional on the user's CANCEL transaction reaching a confirmed block before the observation window expires. In adversarial conditions where a majority mining coalition actively censors CANCEL, the NEXUS can be defeated by sustained suppression across the full 8,640-block window. The censorship must be continuous: if even one honest block lands in the window carrying CANCEL, the attack fails. This is the same economic assumption underlying every proof-of-work chain — that sufficient honest mining participation exists for user-initiated transactions to be included in reasonable time. Scriptum does not eliminate this assumption. What Scriptum adds is a 24-hour public window in which the assumption can be tested: the legitimate holder sees that their CANCEL is not being included, the community observes the same, exchanges and counterparties can freeze downstream operations, and honest hash has time to respond. The attack is expensive, detectable, and on the clock. It is not silent.

Adoption and the permanent asymmetry. As Scriptum adoption grows, hashrate accumulates the same way it does on every successful proof-of-work chain — through the economic incentive of mining a chain with real value. At parity of adoption, Scriptum has Bitcoin's hashrate protection AND the NEXUS. Bitcoin has only the first. The asymmetry is permanent. Bitcoin cannot retrofit the NEXUS onto fifteen years of running commerce; no soft fork can introduce a mandatory 24-hour observation window onto a chain whose entire tooling — wallets, exchanges, custodians, on-chain contracts — assumes instantaneous finality. The NEXUS is an architectural decision that must be made at block zero. Scriptum made it. Bitcoin did not. Both chains will continue. Only one of them has both layers.

The structural claim. Scriptum is not claimed to be “safer than” Bitcoin on all axes. Bitcoin has battle-testing, deep liquidity, ecosystem maturity, and a fifteen-year track record that Scriptum cannot match at genesis and may not match for many years. Those are real advantages, earned through operation at scale. What Scriptum claims is structural: on the axis that matters to a holder — does my value survive the attack — the answer is yes, by design, from block zero, independent of hashrate. At maturity, that protection stacks with full PoW hashrate. That claim is precise, testable, and does not depend on any comparison that cannot be made rigorous.

A 51% mining attack gives the attacker block production advantage. It does not bypass protocol. Honest nodes reject any block that omits a required protocol-driven output, contains invalid state transitions, or includes forged signatures. A 51% attacker is constrained to producing valid blocks — and valid blocks must follow all consensus rules. The following sections enumerate precisely what such an attacker can and cannot do.

Bitcoin asks: what does it cost to attack the chain. Scriptum asks: what does the holder lose when the attack succeeds. The answers are not on the same axis.

What a 51% Attacker Can Do

Reorg recent blocks (within their sustained advantage window)

Censor user-initiated transactions — UNLOCK, CANCEL, SPEND — by excluding them from blocks

Reverse recent payments within the 10-confirmation window

What a 51% Attacker Cannot Do

Forge a Dilithium3 signature — classically or quantum. No key, no spend.

Steal coins without the private key — Parcel ownership is cryptographically enforced

Censor protocol-driven transactions — EJECT, EXPIRE, RELEASE, BOARD are consensus-enforced. Any block omitting them is invalid and orphaned by honest nodes.

Reverse a finalised SPEND and then use the restored value. Reversing a confirmed SPEND requires a reorg past the 10-confirmation finality window (§32.2). Even after a successful reorg, the restored ACTIVE Parcel remains cryptographically owned by the original holder — the attacker holds a state, not a key. Without the key, the restored ACTIVE Parcel runs to its expiry and is returned to LOCKED by protocol-enforced EXPIRE. To actually move the restored value, the attacker needs the Dilithium3 key AND must complete a fresh 8,640-block NEXUS — visible and cancellable for the full duration.

The Three Attack Scenarios

Scenario 1 — Reverse NEXUS dwell:

Result: Parcel returns to LOCKED. Attacker gains nothing. Holder loses time only.

Scenario 2 — Reverse ACTIVE:

Result: Parcel returns to LOCKED via EXPIRE (protocol-driven, cannot be censored). Attacker gains nothing. Holder loses time only.

Scenario 3 — Reverse SPEND (within 10-confirmation window):

Result: ACTIVE Parcel restored with original timer. But attacker still needs the private key to move it. Without the key, the timer runs to zero, EXPIRE fires, coins return to LOCKED. To use the ACTIVE state, attacker needs the key AND must go through the full NEXUS dwell again — another 24 hours, visible to all.

In every scenario, without the private key, the coins cannot be moved. The NEXUS combined with protocol-driven mandatory transactions means the worst realistic outcome of a mining attack without key compromise is delay — not loss.

The bank can be broken into. The safe remains locked.

The attacker who possesses both the key and sustained hashrate is the one remaining threat. That attack is not architecturally prevented; it is architecturally exposed. A 24-hour public window of sustained attacker dominance against a specific holder is an extraordinary event — visible, recordable, cancellable, and costly to the attacker whether it succeeds or fails. Scriptum does not promise that theft cannot happen. It promises that theft cannot happen silently.

Scriptum is open source. Every line of protocol code, every consensus rule, and every parameter defined in this whitepaper will be publicly available for review, challenge, and improvement.

The community is invited to audit the protocol. A bug bounty program will be established at launch, rewarding verified vulnerability disclosures in Scriptum from the Reserve allocation.

A formal independent security audit will be commissioned from the Reserve allocation when the chain has sufficient value to fund it responsibly. Until then, open-source community review is the primary security mechanism — the same model used by Bitcoin for its first years of existence.

Key Theft Scenario

An attacker obtains a user's Dilithium3 private key. In Bitcoin, Ethereum, or any chain with immediate execution, this the consequence is immediate: the attacker can drain the wallet before any observer has time to react.

In Scriptum, key theft is the beginning of a race the attacker loses by default. To move value, the attacker must submit UNLOCK. UNLOCK opens the Safety Control Window. The victim — or anyone watching the chain on the victim's behalf — can CANCEL the unlock for the full window duration. CANCEL is free, has maximum mempool priority, and is prioritised by honest mining policy at the NEXUS boundary (see §Mempool Policy).

For the attacker to succeed, they must prevent the victim from observing the UNLOCK and submitting CANCEL for the full window. Monitoring tools, delegated audit keys, and trusted parties make this window difficult to bypass. The theft of a key no longer means theft of value. The theft of a key means an observable alert.

Oracle Manipulation Scenario

An attacker attempts to manipulate V_A by corrupting oracle data feeds or coordinating V_P submissions across enough registered HOMEs to influence the aggregate. The attacker's goal is to make large SPEND transactions cheap (inflated V_A) or to force fee-bound economic instability (deflated V_A).

The Urðarbrunnr Integrity System blocks this through three independent layers. First, per-block V_A movement is bounded — no single block can move V_A more than a fixed percentage. Second, rolling-window V_A movement is bounded — V_A cannot drift beyond a fixed percentage over the UIS window. Third, oracle data itself is cross-checked: values must pass both a cross-source consistency filter and a statistical outlier filter before they enter V_A computation at all.

Beyond these filters, two separate warning mechanisms detect different manipulation signatures. The Hugin and Munin CUSUM detection pair raises an informational tampering alarm when the aggregated V_A consistently drifts from oracle-reference V_A — Hugin captures the per-block deviation, Munin accumulates it, and the alarm fires when the cumulative drift exceeds threshold. This is the signature of sustained manipulation. A

separate fast-price-divergence warning fires when instantaneous oracle-vs-V_A gap exceeds threshold — the signature of legitimate market events. Both are informational; neither affects execution or freezes the chain. Their purpose is transparency: making manipulation attempts economically irrational by making them publicly visible.

To manipulate V_A, an attacker must sustain distortion across multiple independent oracle sources, over time, while also controlling sufficient registered HOMEs (each independently authenticated and rate-limited) to influence the aggregate beyond the smoothing rule's tolerance. The required resource investment exceeds any plausible gain from the manipulation itself.

Timestamp Gaming Scenario

An attacker attempts to exploit the Menuha Window boundary by submitting outward-value transactions (SPEND, BURN, DONATE) with block timestamps that appear to fall just outside the restricted window.

Scriptum defends against timestamp gaming by using Median Time Past — the median of the last eleven block timestamps — rather than raw block timestamps to determine whether a block falls within Menuha. A single miner cannot shift MTP without controlling majority hashrate for multiple consecutive blocks. This is the same defense Bitcoin uses against timestamp manipulation, adapted to Menuha enforcement.

An attacker attempting to squeeze a SPEND in during Menuha-boundary blocks finds that MTP does not shift quickly enough to matter. The Menuha window is robust to single-miner timestamp games.

NIST Alignment Statement

Scriptum is designed to operate on a cryptographic foundation aligned with the post-quantum standards defined by the National Institute of Standards and Technology (NIST).

At its core, the system uses NIST-selected post-quantum primitives:

CRYSTALS-Dilithium3 (ML-DSA) — digital signatures. Active from genesis. Every user-authorized action on Scriptum is signed with Dilithium3 from genesis. Every address is secured against attacks from cryptographically relevant quantum computers. Protocol-driven transitions carry no signature — they are consensus-enforced and cannot be forged without breaking the consensus rules themselves.

CRYSTALS-Kyber (ML-KEM) — key encapsulation mechanism. Active from genesis. Enables stealth address derivation and holder-controlled audit keys.

Scriptum does not introduce proprietary cryptographic algorithms. It relies exclusively on publicly vetted, standardised, and peer-reviewed primitives implemented according to established best practices. Both Dilithium3 and Kyber were standardised by NIST in 2024 following years of open international review.

Crucially, Scriptum extends beyond cryptographic robustness by incorporating a protocol-level architecture that mitigates the impact of key compromise. Even in the event of key exposure, value transfer is subject to enforced temporal constraints — the Safety Control Window — and deterministic validation processes. This provides an additional layer of systemic security independent of the underlying cryptographic assumptions.

This dual approach — standards-based post-quantum cryptography combined with protocol-level temporal safeguards — ensures both formal alignment with emerging global cryptographic standards and structural resilience against future threat models.

In plain terms: even if the cryptography is broken, the NEXUS gives the holder time to respond. Scriptum does not rely on any single layer of security. It was designed to survive failure at any layer.

Regulatory Position

The legal classification of FUNK, the protocol unit, depends on the laws and regulatory guidance of the relevant jurisdiction. This document describes protocol architecture and intended operation; it does not assert a universal legal classification.

Scriptum (FUNK) is a protocol-native asset required for network operation. It provides no yield, no entitlement, and no profit claim. It is the medium through which fees are paid and miners are compensated for securing the Scriptum network.

Statement of intent:

The Scriptum foundation will seek legal clarity in all relevant jurisdictions commencing at genesis. We acknowledge that the regulatory classification of digital assets varies by jurisdiction and evolves continuously. We will invite qualified legal counsel in the EU (MiCA), UK (FCA), and US (SEC/CFTC) as part of the mainnet preparation process.

The Foundation is an intended future entity and does not exist at Genesis. All references to "the Foundation" throughout this document refer to the intended legal entity to be incorporated at the Bootstrap T1 milestone (target 6-12 months post-Genesis). Before incorporation, Foundation responsibilities are operationally distributed: protocol-level governance is exercised by the Curia multi-sig (a cryptographic structure that exists at Genesis), and public-facing infrastructure is operated by the Warden of the Chain personally. The Foundation's specific legal form (Stiftung, Verein, or other equivalent entity) and jurisdiction of incorporation are to be determined at or post-Genesis based on operational and regulatory considerations at that time.

Scriptum is a permissionless proof-of-work protocol with no issuer in the securities-law sense — no equity, no claim on profits, no central party controlling token supply or distribution. Bootstrap activities are performed by identifiable individuals and entities prior to Foundation formation. The Warden of the Chain personally operates pre-Foundation public-facing infrastructure; appointed counsel custodies pre-Foundation fiat proceeds; the Foundation incorporates at the Bootstrap T1 milestone (target 6-12 months post-Genesis). These bootstrap activities are operationally identifiable; the protocol itself remains issuer-less in the securities-law sense. The protocol has no central operator and no

custodial intermediary; it has no jurisdiction. Anyone, anywhere, can run a node, mine, hold, or transact, subject to their own local law.

The Scriptum Foundation, once incorporated at the Bootstrap T1 milestone (target 6-12 months post-Genesis), will be responsible for the public-facing infrastructure: the dfqr.org website, the Aurum wallet software, and governance coordination. The Foundation operates in compliance with the laws of its jurisdiction of incorporation and any jurisdictions where it actively conducts business. The Foundation does not offer Scriptum for sale, does not provide custodial services, and does not operate a centralised exchange.

Until incorporation, public-facing infrastructure is operated by the Warden personally with bootstrap fiduciary support per Bootstrap §14

Foundation legal form to be determined. The Foundation's specific legal form (Stiftung, Verein, or other equivalent entity) and jurisdiction of incorporation (Switzerland, Germany, or other) are to be determined at or post-genesis based on operational and regulatory considerations at that time. The chain's architecture does not depend on any specific legal form for the Foundation; the Foundation operates as a beneficiary of Curia-authorized disbursements once incorporated, regardless of legal form. Until incorporation, bootstrap functions (Genesis First-Buy proceeds custody, Fountain proceeds custody, bootstrap operational expense funding) are handled by an appointed legal counsel through a client escrow account under the applicable jurisdiction's professional fiduciary obligations.

Holders in any jurisdiction are responsible for their own compliance with applicable local law, including tax reporting, anti-money-laundering requirements, and any licensing obligations that may apply to their own activities (for example, operating a commercial exchange or providing custodial services).

The protocol is neutral. The Foundation is compliant where it sits. The users are responsible for themselves. This is the only honest framing for a decentralised system.

This whitepaper does not constitute financial advice, an offer of securities, or an invitation to invest. It is a technical and philosophical description of a protocol.

Invariants and Assurance

The Scriptum corpus consists of the following documents: the Whitepaper — the public-facing architectural document; the Canonical Machine — the chain's deterministic state-machine specification; the System Parameters Extended specification — the parameter and constant register; the Invariants and Assurance document — the chain's formal invariants and verification framework; the Economic and Adversarial Analysis — the economic model and adversary analysis; the Aurum Technical Document — the Aurum wallet architecture specification; the Governance Charter — the constitutional governance framework; the Highlights document — the executive summary and headline architecture; and the Constitution — the foundational principles document. Documents are versioned independently; cross-references between documents identify the canonical source for each architectural element.

A chain intended to last a hundred years cannot rely on craftsmanship alone to catch consensus bugs. Three real bugs were caught during Scriptum’s pre-genesis development by careful review of running code: a coinbase maturity bypass that would have allowed mined Scriptum to be spent before the 100-block confirmation discipline; a retarget direction inversion at sub-second block production that would have moved difficulty in the wrong direction during bootstrap and Phoenix renewal; and an arithmetic overflow in target adjustment that silently masked failure cases. Each was found before genesis. Each would have been a hard-fork problem to fix post-launch.

The IA document specifies five assurance practices:

Formal invariants. Mathematical properties the chain must always satisfy, stated as predicates over chain state. Examples: `s_mined` is monotonically non-decreasing; for every accepted block `N`, all transitions in `N` validate against state at height `N-1`; for any solution accepted at height `N`, the block timestamp satisfies both `block.timestamp > MTP(parent)` and `block.timestamp ≤ local_clock + FUTURE_TIMESTAMP_TOLERANCE_MS`; coinbase outputs match the protocol emission formula exactly. Each invariant has a precise statement; each statement is testable.

Property-based tests. For every formal invariant, a proptest-based test that exercises the invariant across randomised inputs. These run on every commit. The test suite must pass before any merge to the canonical branch.

Fuzzing harnesses. `cargo-fuzz` targets for every parser (message decoders), every state transition validator, every arithmetic operation involved in consensus (target adjustment, reward calculation, retargeting math). Continuous fuzzing runs across every parser surface.

Adversarial test scenarios. Simulated attacks with specified attacker capabilities, expected chain responses, and verified assertions. The current scenario catalogue includes timewarp manipulation, hashrate spike-and-leave, same-second timestamp flood under coarse-resolution emulation, mempool spam, reorg attempt at finality depth, and Sybil peer flooding. Each new attack-vector identified joins the catalogue and gains a regression test.

External audit. Pre-genesis, the chain undergoes at least one external security audit by a recognised firm. Audit scope, methodology, remediation requirements, and resulting reports are specified in IA. Audit findings that surface real consensus issues block genesis until resolved.

The three pre-genesis bugs caught by craftsmanship are documented in IA as retrospective case studies and have permanent regression tests in the assurance suite. Future bugs caught will join them. The chain’s assurance posture is observable, not claimed.

Conclusion — A System Built for Failure

Every custody system in history has been built on the assumption that nothing will go wrong. Scriptum is built on the assumption that something will.

Keys will be compromised. Mistakes will be made. Attackers will obtain valid signing capability. People will die with unspent holdings. These are not edge cases — they are the normal operation of any system at scale across a century.

Scriptum does not prevent failure. It contains it. Every mechanism in this document — the NEXUS, the CANCEL window, Limbo, EXPIRE, the Arc, Ragnarok — exists because the designers assumed something would go wrong and built a time buffer around it.

Scriptum is a time-controlled custody system with on-chain enforcement. It is the first system designed not to eliminate risk, but to make risk survivable.

Final Statement

Scriptum does not move value quickly.

It ensures value does not move when it should not.

Appendix A. Privacy Architecture

This appendix describes the Scriptum privacy architecture. It is part of the protocol at genesis. Every mechanism described here is live in block zero; nothing awaits activation.

The Three-Layer Model

Scriptum provides three composable security layers: cryptographic (Dilithium3 signatures, post-quantum), temporal (the Safety Control Window), and privacy (stealth addressing plus holder-controlled audit).

The three layers are designed to coexist. Privacy does not weaken the NEXUS. The NEXUS does not break privacy. The holder observes their own NEXUS through their view key — privately. The world sees an unlock occurring. It cannot see whose, or how much.

Stealth Wallets — How They Work

Every Scriptum wallet generates the full key bundle:

Dilithium3 spend key — signs transactions. Authorises movement.

Kyber view key — enables post-quantum key exchange. Derives shared secrets for stealth operation.

Together these form a stealth address. The stealth address is published. It never changes. But every transaction directed to it lands at a different one-time address derived from the shared secret.

Sending to a Stealth Address

The sender:

Obtains the recipient stealth address (Dilithium3 + Kyber keys)

Kyber/ML-KEM-based key encapsulation is a send-side operation. Only the sender performs the encapsulation. The recipient scans the chain with their view key to detect incoming stealth outputs by decapsulating each SPEND's ephemeral material; no handshake or back-channel communication is involved.

Picks a random ephemeral value r

Uses the recipient Kyber key and r to derive a shared secret via Kyber/ML-KEM key encapsulation

Derives a unique one-time destination address from that secret

Sends to that address

Publishes the Kyber KEM ciphertext on chain (public; only the recipient with the matching Kyber private key can decapsulate it to recover the shared secret).

The world sees a transaction to an unknown one-time address. It cannot link that address to the recipient. It cannot link it to any prior transaction. It cannot link sender to receiver.

Receiving — Chain Scanning

The recipient wallet continuously scans the chain. For each published r value it:

Derives the shared secret using its Kyber view key

Derives the corresponding one-time address

Checks whether funds arrived at that address

If yes: the wallet derives $ss_derivative = HKDF-SHA3-512(ss, "scriptum.spend-binding")$ from the KEM shared secret (per Stealth Settlement and Spend Authorisation), stores it locally associated with the Parcel, and recognises the Parcel as the holder's. The holder spends the Parcel later by signing with their HOME Dilithium3 key and presenting the $commitment_opening$ (the stored $ss_derivative$); no per-Parcel signing key exists.

No interaction between sender and recipient is required beyond the on-chain data.

The Audit Key

The holder may derive an audit key from their Kyber view key material. The recipient link — who sent, who received, how much — is visible only to the holder and to parties the holder explicitly authorises by sharing this key. No authority has inherent visibility. No regulator has automatic access. Disclosure is always and only the holder's decision.

The authority with the audit key can see:

All incoming transactions to the holder

Consumed inputs and output values

Transaction timing and amounts

The authority cannot see:

The real-world identity of the holder — pseudonymous addresses only

Outgoing transactions (spend key required — never shared)

Transactions of any other holder

The audit key grants read access only. It cannot sign, spend, unlock, or alter any transaction. The spend key never leaves the holder. An auditor with the audit key can see everything. They can do nothing. The mathematics enforces this separation — it is not a policy, it is a constraint.

Disclosure is voluntary. The holder decides who receives the audit key, when, and under what conditions. The protocol never compels or enables forced disclosure. Legal obligations are the holder's responsibility under their own jurisdiction. The chain does not know who you are. It cannot testify.

Identity is never exposed automatically by protocol. The chain knows addresses. It does not know people. Even the audit authority sees only what the mathematics allows — flows, not faces.

Protocol Integration

Stealth addressing is a live protocol feature from block zero. Every Scriptum address carries real Kyber key material. Every wallet performs Kyber/ML-KEM key encapsulation on send. Every recipient scans the chain with their view key. There is no activation event; the chain shipped complete.

Any future structural change to the privacy architecture requires the hard fork mechanism defined in the Governance Charter.

Appendix B. Aurum UX Reference

Aurum UX Reference

Implementation guidance for the Aurum reference wallet

This appendix defines Aurum-specific implementation requirements. These are not consensus rules. They are the canonical user experience model for the Scriptum reference wallet. Alternative wallet implementations may exist, but Aurum defines the standard against which user experience is measured.

Aurum displays a progressive warning system in parallel with all other wallet functions as Ragnarok approaches. Warnings are non-blocking. The wallet operates normally throughout. They are informational and time-sensitive.

Key heights:

ENROLLMENT_OPEN_HEIGHT = 288,873,600 (one year before Ragnarok)

RAGNAROK_HEIGHT = 315,360,000

A.1.1. Warning Levels

Level 1 — One Year Warning

Block 288,873,600. Enrollment opens.

THE ARC OPENS IN ONE YEAR

Enrollment is now available. Board before Ragnarok.

Level 2 — Six Month Warning

Block 297,116,800.

THE ARC DEPARTS IN SIX MONTHS

Enroll now. Boarding windows are filling.

Level 3 — Three Month Warning

Block 302,116,800.

THREE MONTHS TO RAGNAROK

Enroll immediately.

Level 4 — One Month Warning

Block 311,116,800.

RAGNAROK IN 30 DAYS

Final enrollment window. PACKING begins soon.

Level 5 — One Week Warning

Block 313,116,800.

SEVEN DAYS REMAINING

Enrollment closed. PACKING imminent.

Level 6 — PACKING Active

Triggered when wallet PACKING phase begins (~8,640 blocks before boarding slot).

PACKING IN PROGRESS

Your value is being consolidated. Do not send to this wallet.

Level 7 — Boarded

Triggered when BOARD transaction is confirmed.

YOU ARE ABOARD THE ARC

Value committed. Awaiting Ragnarok.

A.1.2. Warning Placement

All warning levels display as a persistent banner at the top of every Aurum screen. The banner is dismissible per session but reappears at each launch. It does not block wallet operation.

B.1. Boarding Status Display

Aurum maintains and displays a boarding status indicator at all times once ENROLLMENT_OPEN_HEIGHT is reached. Status transitions:

NOT ENROLLED -> enrollment available, no action taken

ENROLLED -> boarding slot selected, cancellation available

CANCELLED -> enrollment cancelled, re-enrollment available

PACKING -> consolidation in progress

STRAGGLER WINDOW -> 10 blocks before boarding, confirm or reject stragglers

ANOINTED -> enrollment confirmed, BOARD will execute automatically

CONDEMNED -> not enrolled, excluded from this Arc

BOARDED -> ARC state, awaiting Ragnarok

DEPARTED -> Ragnarok passed, value carried forward

The boarding status panel shows:

Current status

Selected boarding slot (block height and estimated date/time)

Countdown to boarding slot in blocks and estimated time

Countdown to Ragnarok

Total value enrolled in Scriptum and EUR

Straggler status if applicable

B.2. UIS Protected Mode

When the Urdarbrunnr Integrity System detects abnormal V_A conditions, Aurum activates Protected Mode:

PROTECTED MODE ACTIVE

Valuation anomaly detected. Fee estimates may be inaccurate.

Confirm to proceed or wait for V_A to normalize.

Protected Mode requires explicit user confirmation before any fee-bearing transaction (UNLOCK, SPEND) is submitted. CANCEL remains accessible without additional confirmation. Protected Mode deactivates automatically when UIS conditions normalize.

Extended Protected Mode (Prolonged Oracle Failure)

When oracle coverage remains below the minimum threshold for ORACLE_FAILURE_ESCALATION_BLOCKS (default ~7 days at 10-second blocks per SPX-8), Aurum enters Extended Protected Mode: a sustained banner warning that V_A is operating from a frozen historical value and that fees may not reflect current Scriptum-to-fiat market conditions. Aurum requires explicit re-confirmation for SPEND transactions above PROTECTED_MODE_SPEND_THRESHOLD (default €10,000 equivalent at frozen V_A). The Curia is notified via on-chain alert event and publishes a status assessment within 14 days. If oracle coverage does not recover within 30 days of escalation, the situation escalates to L4 constitutional governance for review of the oracle architecture itself. Throughout all stages of Extended Protected Mode, the chain continues operating with frozen V_A; consensus validation continues uninterrupted; the escalation rule is operational and governance-level, not consensus-level.

B.3. Parcel State Display

Aurum displays each Parcel with its full CLASS / ROLE / STATE attributes. Standard display format:

AMOUNT CLASS ROLE STATE

10.0000 CIRCULATING NONE LOCKED

5.0000 CIRCULATING NONE NEXUS [DWELL: 412 blocks until eject window]

2.5000 CIRCULATING NONE ACTIVE [Expires: 308 blocks]

50.0000 TREASURE FOUNDER LOCKED [Release: block 1,440,000]

1 Nugget MARKER MAYFLOWER PERMA_LOCKED

Limbo state display:

5.0000 CIRCULATING NONE LIMBO – AWAITING EJECT

B.4. Fee Preview

Before any fee-bearing transaction, Aurum displays a full fee breakdown in both Scriptum and EUR:

UNLOCK 10.0000 Scriptum

f_U 0.0004 Scriptum = EUR 0.04

f_A 0.0001 Scriptum = EUR 0.01 (deducted at EJECT)

Total 0.0005 Scriptum = EUR 0.05

V_A EUR 100,000 / Scriptum (current)

If Protected Mode is active, fee preview includes:

! V_A anomaly detected. Fees shown may differ at execution.

Colophon

Prepared by DrFunkenstein.

Apache 2.0 licence on all referenced code and source files.

Cross-references in this document align with corpus v16.